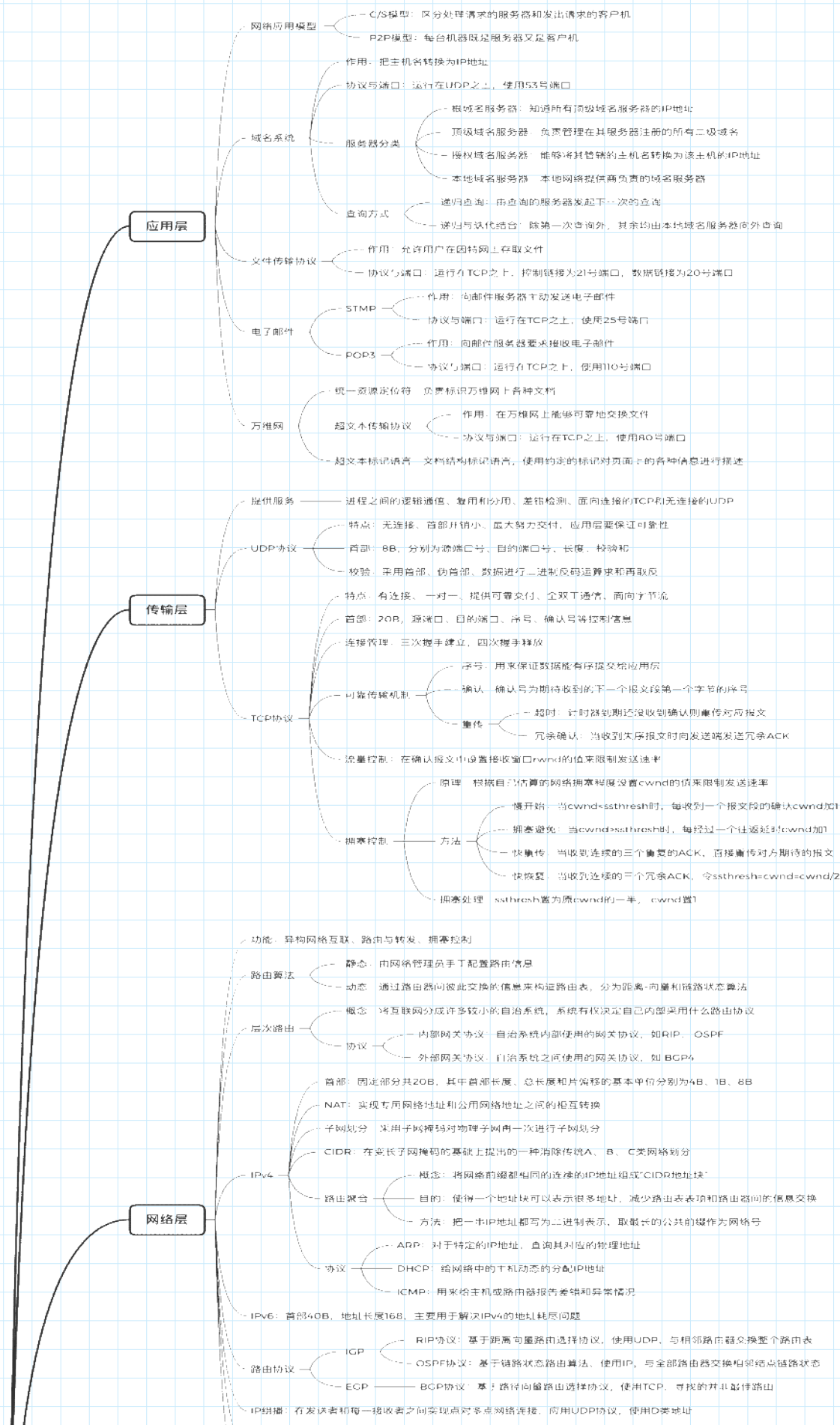




计算机网络

数据链路层

- 路由协议
 - OSPF协议：基于链路状态路由算法，使用IP，与全部路由器交换相邻点链路状态
 - EGP
 - BGP协议：基于路径向量路由选择协议，使用TCP，寻找的并非最佳路由
- IP组播：在发送者和每一接收者之间实现点对多点网络连接，应用UDP协议，使用D类地址
- 移动IP
 - 概念：移动节点以固定的网络IP地址，实现跨越不同网段的漫游功能
 - 组成：移动节点、本地代理、外部代理
- 路由器
 - 特点：可以用来连接异构网络，完成路由转发，能隔离冲突域和广播域，依照IP地址转发
 - 组成
 - 控制部分：路由选择处理机，根据路由协议构造与维护路由表
 - 分组转发部分：交换结构，一组输入端口和一组输出端口

- 功能：为网络层提供服务、链路管理、帧定界、帧同步与透明传输、流量控制和差错控制
- 组帧
 - 字符计数法：在帧头部使用一个计数字段来标明帧内字符数
 - 标志定界法
 - 概念：使用特定字符或比特模式定界帧的开始和结束
 - 字符填充：在数据中的特殊字符前面用转义字符填充
 - 比特填充：数据区每遇到连续5个“1”就填充一个“0”
 - 违规编码法：信号传输过程中采用违规的编码来表示帧的起始和终止（高电平和高电平，低电平和低电平）

- 差错控制
 - 检错编码：奇偶校验码、循环冗余码
 - 纠错编码：海明码

可靠传输：数据链路层通常使用确认和超时重传两种机制来保证可靠传输

- 流量控制
 - 停止等待协议：发送方每发送一帧都要等待接收方的应答信号才能发送下一帧
 - 后退N帧协议：发送方一次可发送N帧，按序接收，重传从最后一个确认开始
 - 选择重传协议：发送方一次可发送N帧，可以不按序接收，重传没有确认的帧

- 信道划分
 - 频分复用：将多路信号调制到不同频率载波上叠加形成一个复合信号
 - 时分复用：将物理信道按时间分为若干时间片，轮流给不同信号使用
 - 波分复用：在一根光纤中传输多种不同波长(频率)的光信号
 - 码分复用：用不同的编码来区分各路原始信号，例如CDMA技术

- 介质访问
 - ALOHA协议
 - 纯ALOHA：不检测直接发送，若无确认则等待重发
 - 时隙ALOHA：将时间划分为若干等长时隙，按时发送
 - CSMA协议
 - 1-坚持：闲则发送，忙则继续监听
 - 非坚持：闲则发送，忙则等待一个随机时间再听
 - p-坚持：闲则以概率p发送，1-p等待下一个时隙，忙则等待一个随机时间再听
 - CSMA/CD协议
 - 流程：先听后发，边听边发，冲突停发，随机重发
 - 碰撞解决：采用二进制指数退避算法来解决碰撞问题
 - CSMA/CA协议
 - 避免碰撞：预约信道，ACK帧，RTS/CTS帧
 - 解决碰撞：采用二进制指数退避算法来解决碰撞问题

令牌访问：令牌传递协议，只有得到令牌的机器才能发送数据，其他必须等待

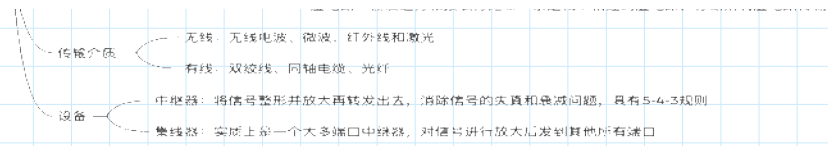
局域网：在一个较小地理范围内，将各种计算机等设备通过双绞线等介质连接

- 广域网
 - PPP协议：面向字节，采用字节填充方式，只支持全双工链路
 - HDLCL协议：面向比特，采用0比特插入法，帧类分为信息帧、监督帧和无编号帧

- 设备
 - 网桥
 - 特点：把两个或多个以太网通过网桥连接起来变成一个网段
 - 类型
 - 透明网桥：按照自学习算法填写转发表，按转发表转发
 - 源路由网桥：先发送发现帧，按返回结果转发
 - 交换机
 - 特点：实际就是一个多端口网桥
 - 交换方式
 - 直通式：帧在接收后只检查目的地址，几乎能马上就被转发出去
 - 存储转发：先将接收到的帧缓存在高速缓存器，检查数据正确性
 - 相同点：按MAC地址转发，都能隔离冲突域，不能隔离广播域

物理层

- 信道
 - 单工：只有一个方向的通信，而没有反方向的交互
 - 半双工：通信双方都可以发送、接收信息，但不能同时发送接收
 - 全双工：通信双方可以同时发送、接收信息
- 定理
 - $2W\log_2 V$
 - 奈氏准则：理想低通信道下的极限数据传输率
 - $W\log_2(1 + S/N)$
 - 香农定理：信道的极限数据传输速率
- 调制与编码
 - 调制：把数据转换为模拟信号的过程
 - 编码：把数据转换为数字信号的过程
 - 数字数据 → 数字信号：非归零码、曼彻斯特编码、差分曼彻斯特编码等
 - 数字数据 → 模拟信号：幅移键控、频移键控、相移键控、正交振幅调制
 - 模拟数据 → 数字信号：1抽样、2量化、3编码
 - 模拟数据 → 模拟信号：频分复用技术等
- 传输方式
 - 电路交换：两结点之间建立一条专用的物理通信路径，传输期间一直独占
 - 报文交换：无需建立专门连接，报文携带有目标地址等信息，采用存储转发方式
 - 分组交换：把报文分割成小的数据块，加上必要的控制信息进行传输
- 分组交换
 - 数据报：分组之间可能存在不同路径，各个分组分别转发
 - 虚电路：在发送方和接收方建立一条逻辑上相连的虚电路，分组沿着虚电路传输
- 传输介质
 - 无线：无线电波、微波、红外线和激光
 - 有线：双绞线、同轴电缆、光纤
- 中继器：将信号整形并放大再转发出去，消除信号的失真和衰减问题，具有5-4-3规则



第三章 数据链路层

星期日，三月 13，2022 5:17 下午

已剪辑自: <https://blog.csdn.net/donkey010/article/details/114698253>

概述和功能

基本概念

结点：主机、路由器

链路：网络中两个结点之间的**物理通道**，链路的传输介质主要有双绞线、光纤和微波。分为有线链路、无线链路。

数据链路：网络中两个结点之间的**逻辑通道**，把实现控制数据传输协议的硬件和软件加到链路上就构成数据链路。

帧：链路层的协议数据单元，封装**网络层**数据报。

功能概述

数据链路层在**物理层**提供服务的基础上**向网络层提供服务**，其最基本的服务是将源自网络层来的数据可靠地传输到 相邻节点的目标机网络层。其主要作用是加强物理层传输原始比特流的功能，将物理层提供的可能出错的物理连接 改造成为逻辑上无差错的数据链路，使之对网络层表现为一条无差错的链路。



- 为网络层提供服务。
 - 无确认无连接服务
 - 有确认无连接服务
 - 有确认面向连接服务（有连接一定有确认！）
- 链路管理，即连接的建立、维持、释放（用于面向连接的服务）
- 组帧
- 流量控制
- 差错控制（帧错 / 位错）

组装成帧和透明传输

组装成帧

封装成帧就是在一段数据的前后部分添加**首部**和**尾部**，这样就构成了一个**帧**。接收端在收到物理层上交的**比特流**后，就能根据**首部**和**尾部**的标记，从收到的**比特流**中识别帧的开始和结束。

首部和尾部包含许多的控制信息，他们的重要作用：**帧定界**（确定帧的界限）。

帧同步：**接收方**应当能从接收到的二进制比特流中区分出帧的起始和终止。

组帧的四种方法：1. 字符计数法，2. 字符（节）填充法，3. 零比特填充法，4. 违规编码法。



透明传输

透明传输是指不管所传数据是什么样的比特组合，都应当能够在链路上发送。因此，链路层就“看不见”有什么妨碍数据传输的东西。

当所传数据中的比特组合恰巧与某一个控制信息完全一样时，就必须采取适当的措施，使收方不会将这样的数据误认为是某种控制信息。这样才能保证数据链路层的传输是透明的。

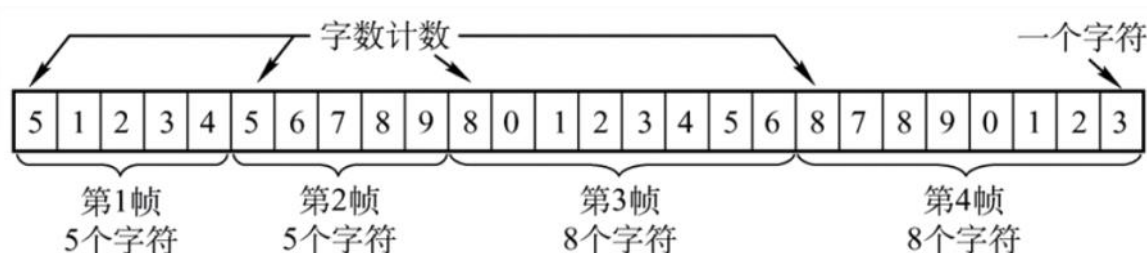


字符计数法

概念

帧首部使用一个计数字段（第一个**字节**，八位）来标明帧内字符数。

痛点：第一个帧计数字段发生错误，将导致整段数据组帧错误，造成不同步，相当于鸡蛋装在一个篮子里了。



字符（节）填充法

概念

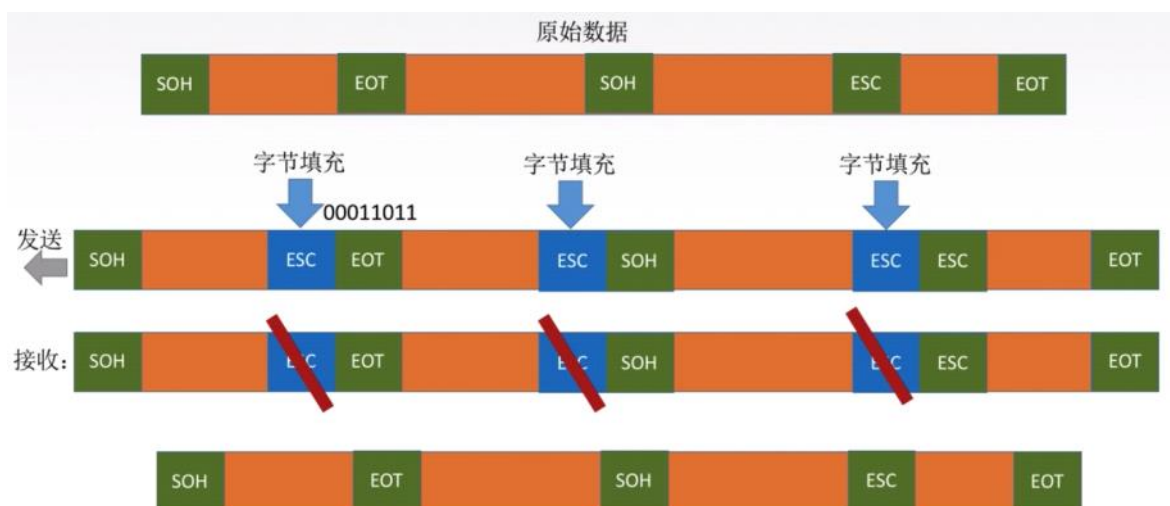
当传送的帧是由文本文件组成时（文本文件的字符都是从键盘上输入的，都是ASCII码）。不管从键盘上输入什么字符都可以放在帧里传过去，即透明传输。



当传送的帧是由非ASCII码的文本文件组成时（二进制代码的程序或图像等），就要采用字符填充方法实现透明传输。



实现过程



零比特填充法

操作

1. 在发送端，扫描整个信息字段，只要 连续5个1，就立即填入1个00
2. 在接收端收到一个帧时，先找到标志字段确定边界，再用硬件对比特流进行扫描。发现连续5个1时，就把后面的0删除。

原始数据

0110111111110111110010
0110111111**0**1110111111**0**0010

填充

0110111111**0**1110111111**0**0010

删除

011011111111101111110010

优点

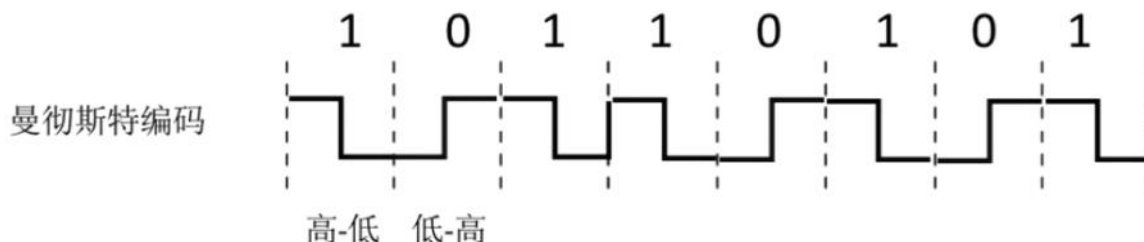
保证了透明传输在传送的比特流中可以传送任意比特组合，而不会引起对帧边界的判断错误。

违规编码法

概念

可以用“高一高”，“低一低”来定界帧的起始和终止。

由于字节计数法中Count字段的脆弱性（其值若有差错将导致灾难性后果）及字符填充实现上的复杂性和不兼容性，目前较普遍使用的帧同步法是**比特填充**和**违规编码法**。



差错控制

差错来源

概括来说，传输中的差错都是由于噪声引起的。

- 全局性噪声
 - 由于线路本身电气特性所产生的**随机噪声**（热噪声），是信道固有的，随机存在的。
 - 解决办法：提高信噪比来减少或避免干扰。（校对传感器下手）
- 局部性噪声
 - 外界特定的短暂原因所造成的**冲击噪声**，是产生差错的主要原因。
 - 解决办法：通常利用编码技术来解决。

差错分类

- 位错：比特位出错，1变成0，0变成1。

- 帧错
 - 丢失：收到 [#1]-[#3]
 - 重复：收到 [#1] - [#2]-[#2]-[#3]
 - 失序：收到 [#1]-[#3]-[#2]

差错控制

- 检错编码
 - 奇偶校验码
 - 循环冗余码CRC
- 纠错编码
 - 海明码

冗余编码：在数据发送之前，先按某种关系附加上一定的冗余位，构成一个符合某一规则的码字后再发送。当要发送的有效数据变化时，相应的冗余位也随之变化，使码字遵从不变的规则。接收端根据收到码字是否仍符合原规则，从而判断是否出错。



只有五本？拒收！



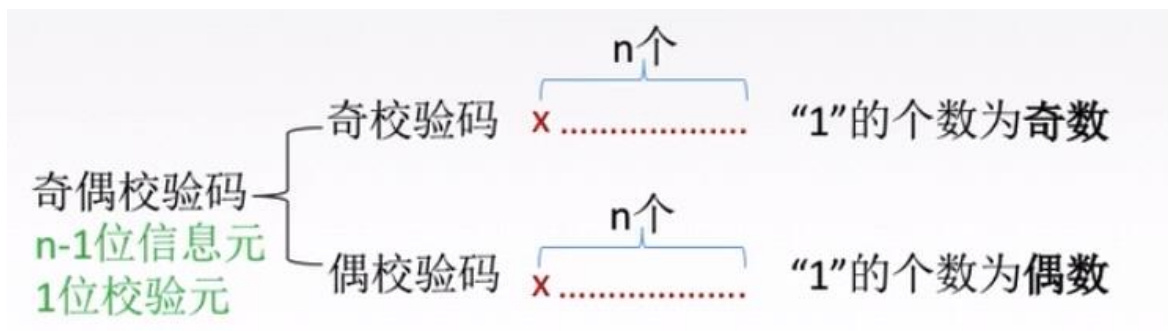
编码 VS 编码

数据链路层编码和物理层的数据编码与调制不同。物理层编码针对的是**单个比特**，解决传输过程中比特的同步等问题，如曼彻斯特编码。而数据链路层的编码针对的是一组**比特**，它通过冗余码的技术实现一组二进制比特串在传输过程是否出现了差错。

检错编码

奇偶校验码

奇偶校验码



特点：只能检查出奇数比特错误，检错能力为50%。

CRC循环冗余码

CRC循环冗余码

最终发送的数据：要发送的数据+帧检验序列FCS

发送端			接收端		
要传的数据	生成多项式	FCS帧检验序列/冗余码	接收到的数据	生成多项式	
5	÷ 2	= 2 1	6	÷ 2	= 3 0
最终发送数据：5+1=6			余数为0，判定无错，就接受。		

计算冗余码

1. **加0**：假设生成多项式G(x)的阶为r，则加r个0。
2. **模2除法**：数据加0后除以多项式，余数为冗余码/FCS/（CRC检验码的比特序列）

10011表示成多项式为

$$\boxed{X^4 + X^1 + X^0} = 1 \cdot X^4 + 0 \cdot X^3 + 0 \cdot X^2 + 1 \cdot X^1 + 1 \cdot X^0$$

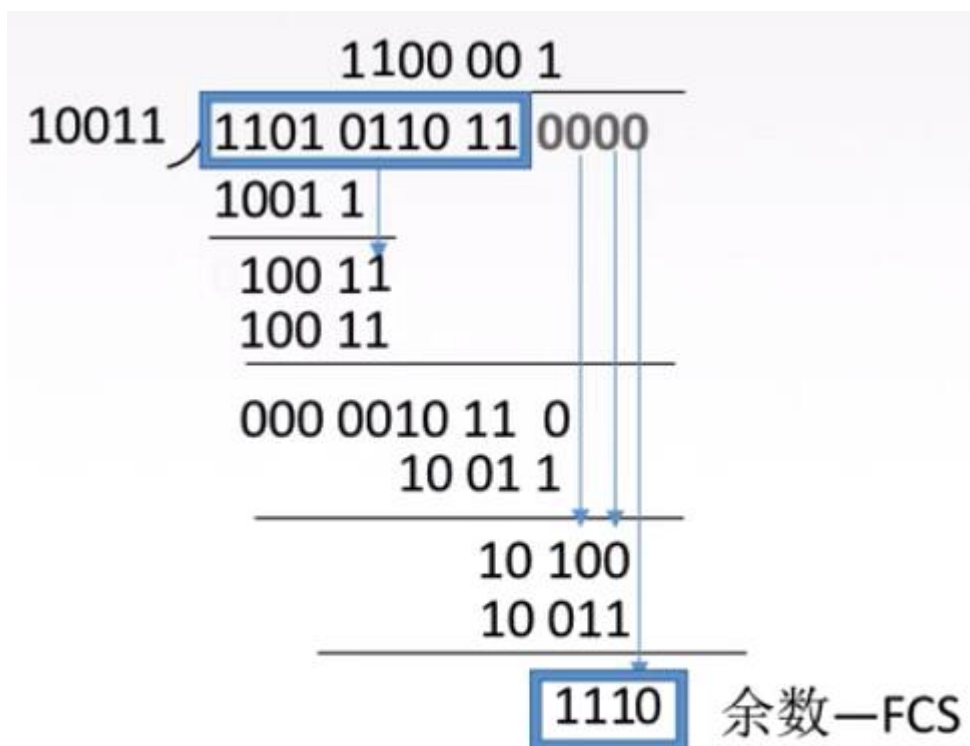
= $X^4 + X^1 + 1$

阶为4

异或：同0异1

TIPS：多项式N位，阶为N-1。

举例：要发送的数据是1101 0110 11，采用CRC校验，生成多项式是10011，那么最终发送的数据应该是？



最终发送的数据：11010110111110

接收端检错过程

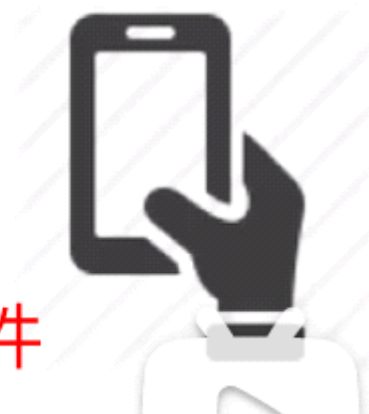
把收到的每一个帧都除以同样的除数，然后检查得到的余数R。

1. 余数为0，判定这个帧没有差错，接受。
2. 余数不为0，判定这个帧有差错（无法确定到位），丢弃。

FCS的生成以及接收端CRC检验都是由硬件实现，处理很迅速，因此不会延误数据的传输。



硬件



软件

在数据链路层仅仅使用循环冗余检验CRC差错检测技术，只能做到对帧的无差错接收，即“凡是接收端数据链路层接受的帧，我们都能以非常接近于1的概率认为这

些帧在传输过程中没有产生差错”。接收端丢弃的帧虽然曾收到了，但是最终还是因为有差错被丢弃。“凡是接收端数据链路层接收的帧均无差错”。

“可靠传输”：数据链路层发送端发送什么，接收端就收到什么。

链路层使用CRC检验，能够实现无比特差错的传输，但这还不是可靠传输。

纠错编码

海明码

海明码

发现双比特错，纠正单比特错。工作原理：动一发而牵全身。

工作流程



1. 确定检验码位数r

海明不等式： $2^r \geq k+r+1$ （r为冗余信息位，k为信息位）

要发送的数据：D=101101

数据的位数k=6，
满足不等式的最小r为4，
也就是D=101101的海明码应该有6+4=10位，
其中原数据6位，效验码4位。

2. 确定检验码和数据的位置

假设这4位校验码分别为P₁、P₂、P₃、P₄（放在2的几次方的位置），数据从左到右为D₁、D₂、…、D₆（按序把空填满）

数据位	1	2	3	4	5	6	7	8	9	10
代码	P ₁	P ₂	D ₁	P ₃	D ₂	D ₃	D ₄	P ₄	D ₅	D ₆
实际值										

3. 求出检验码的值

D=101101										
二进制	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010
数据位	1	2	3	4	5	6	7	8	9	10
代码	P ₁	P ₂	D ₁	P ₃	D ₂	D ₃	D ₄	P ₄	D ₅	D ₆
实际值	0	0	1	0	0	1	1	1	0	1

令所有要校验的位异或=0。

$$\begin{aligned}
 P_1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus D_5 &= 0 \Rightarrow P_1 = 0 \\
 P_2 \oplus D_1 \oplus D_3 \oplus D_4 \oplus D_6 &= 0 \Rightarrow P_2 = 0 \\
 P_3 \oplus D_2 \oplus D_3 \oplus D_4 &= 0 \Rightarrow P_3 = 0 \\
 P_4 \oplus D_5 \oplus D_6 &= 0 \Rightarrow P_4 = 1
 \end{aligned}$$

故101101的海明码为0010011101。

4. 检错并纠错

D=101101										
数据位	1	2	3	4	5	6	7	8	9	10
代码	P ₁	P ₂	D ₁	P ₃	D ₂	D ₃	D ₄	P ₄	D ₅	D ₆
实际值	0	0	1	0	1	1	1	1	0	1

故101101的海明码为0010011101。

假设第五位出错，因此接收到的数据位0010111101。

令所有要校验的位异或运算。

$$\begin{aligned}
 P_1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus D_5 &= 1 \\
 P_2 \oplus D_1 \oplus D_3 \oplus D_4 \oplus D_6 &= 0 \\
 P_3 \oplus D_2 \oplus D_3 \oplus D_4 &= 1 \\
 P_4 \oplus D_5 \oplus D_6 &= 0
 \end{aligned}$$

0101 → 5

二进制序列为0101，恰好对应十进制5，这样就找到了出错的位置，即出错位是第5位。

流量控制和可靠传输

流量控制过程

较高的发送速度和较低的接收能力的不匹配，会造成传输出错，因此流量控制也是数据链路层的一项重要工作。

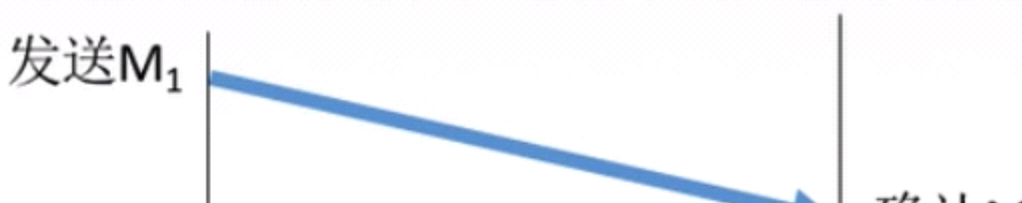
数据链路层的流量控制是点对点的，而传输层的流量控制是端到端的。

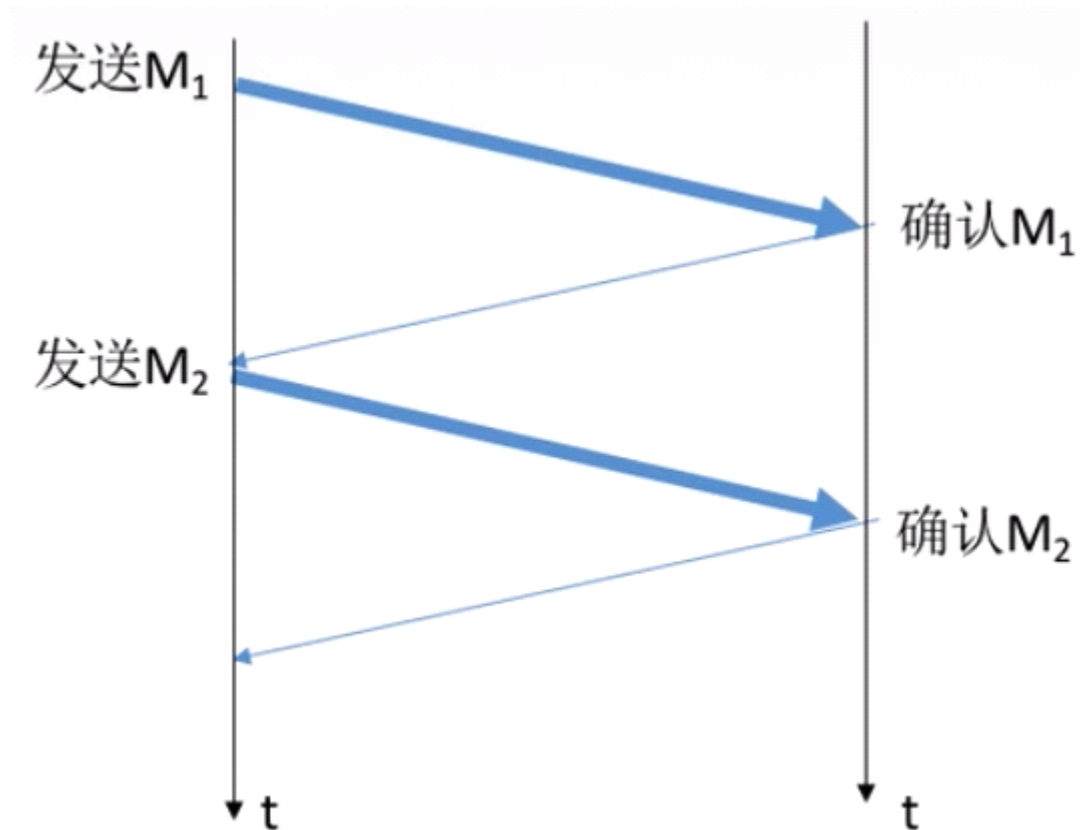
数据链路层流量控制手段：接收方收不下就不回复确认。

传输层流量控制手段：接收端给发送端一个窗口公告。

流量控制方法

- **停止-等待协议**：发送窗口大小=1，接收窗口大小=1
每发送完一个帧就停止发送，等待对方的确认，在收到确认后再发送下一个帧。





• 滑动窗口协议

- 后退N帧协议GBN: 发送窗口大小>1, 接收窗口大小=1
- 选择重传协议SR: 发送窗口大小>1, 接收窗口大小>1

发送窗口



停止-等待协议

背景

- 除了比特出差错, 底层信道还会出现丢包问题。
 - 丢包: 物理线路故障、设备故障、病毒攻击、路由信息错误等, 会导致数据包的丢失。
- 为了实现流量控制。

前提

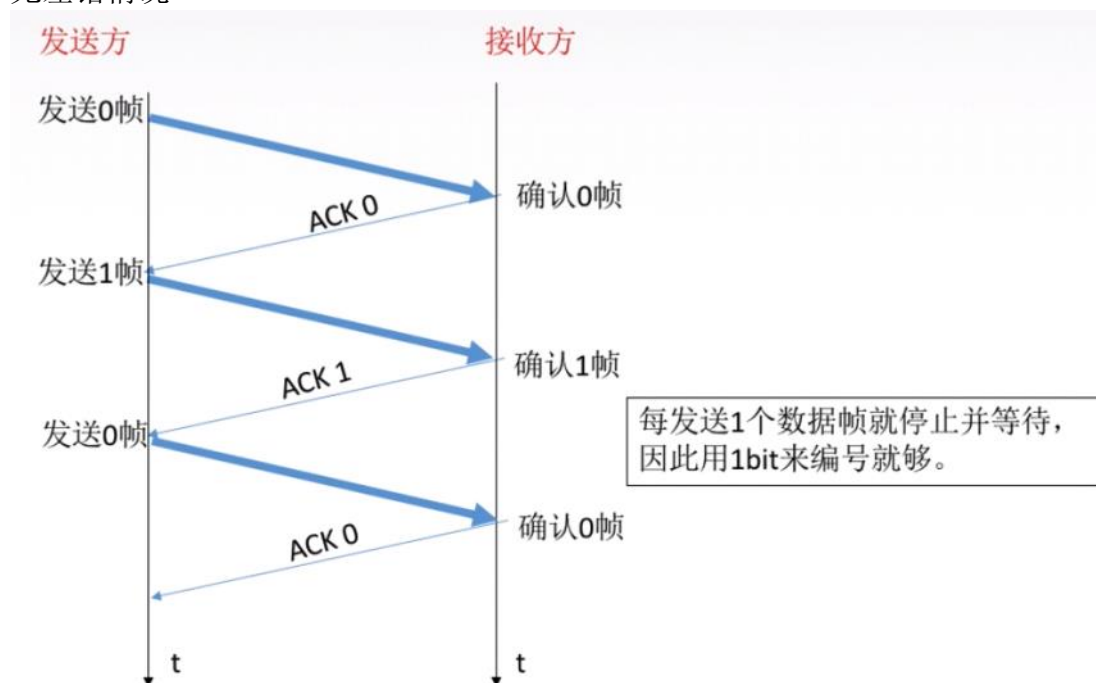
虽然现在常用全双工通信方式, 但为了讨论问题方便, 仅考虑一方发送数据 (发送方), 一方接收数据 (接收方)。

因为是在讨论可靠传输的原理，所以并不考虑数据是在哪一个层次上传送的。

“停止一等待”就是每发送完一个分组就停止发送，等待对方确认，在收到确认后
再发送下一个分组。

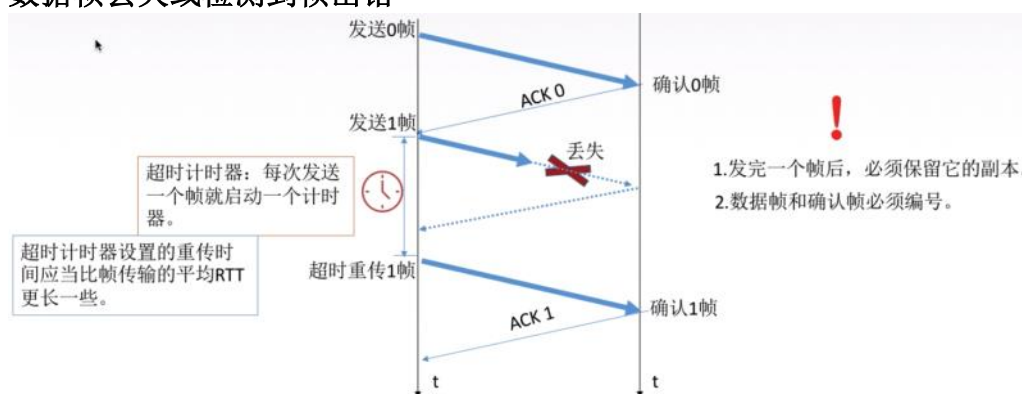
应用情况

- 无差错情况

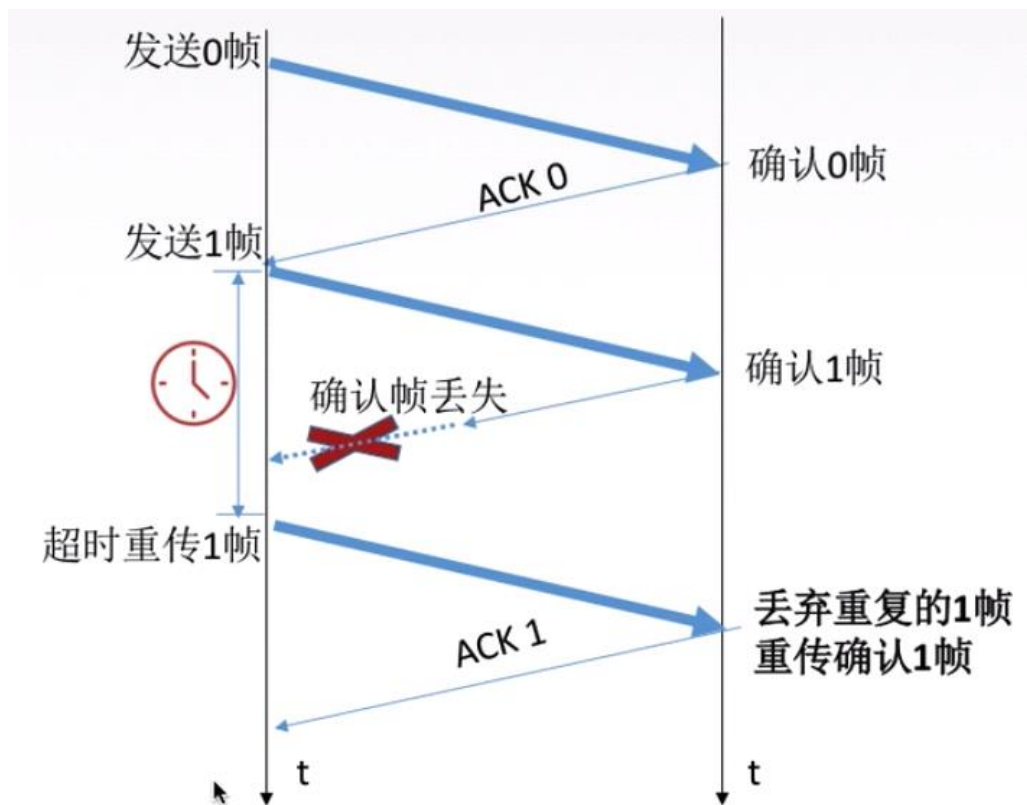


- 有差错情况

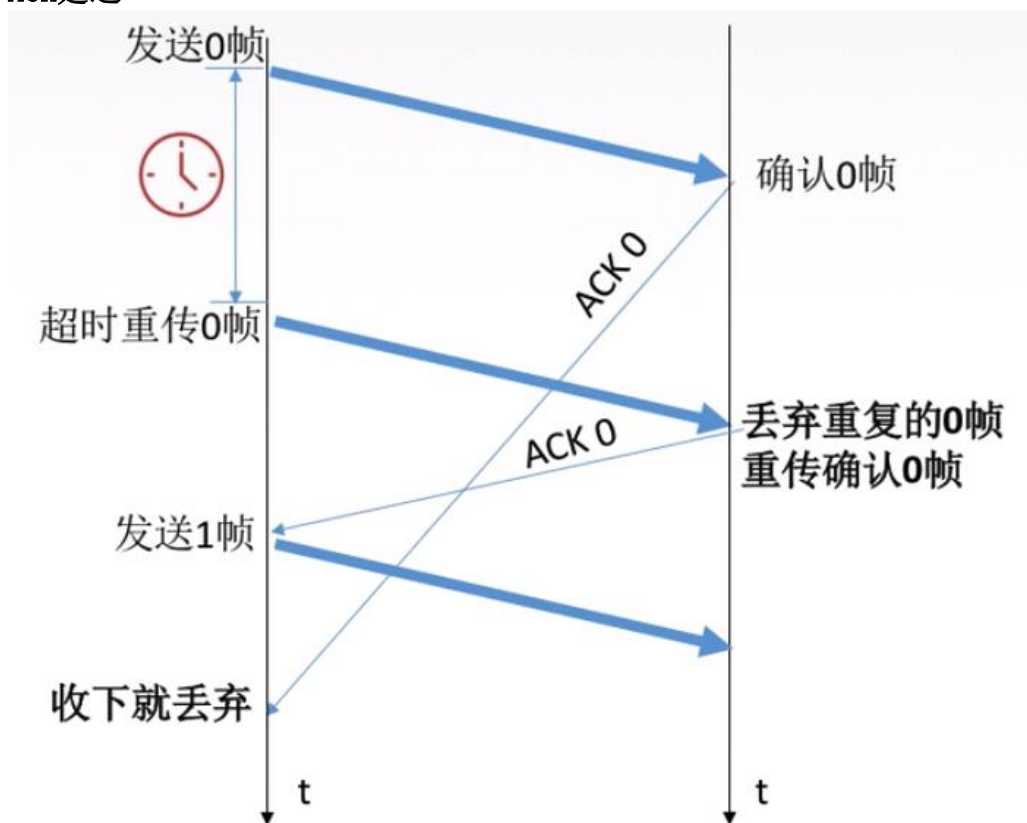
- 数据帧丢失或检测到帧出错



- ACK丢失



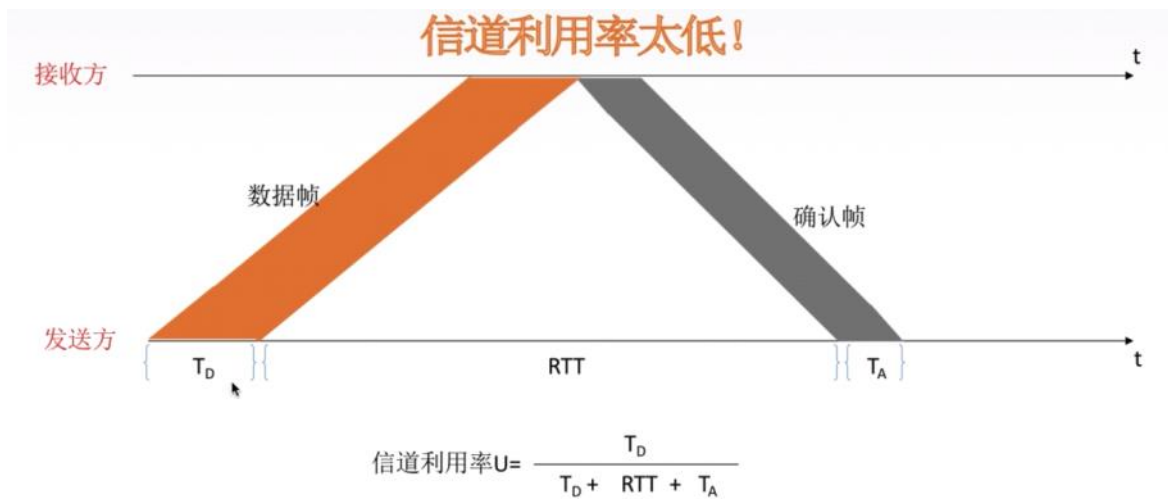
• ACK延迟



停等协议性能分析

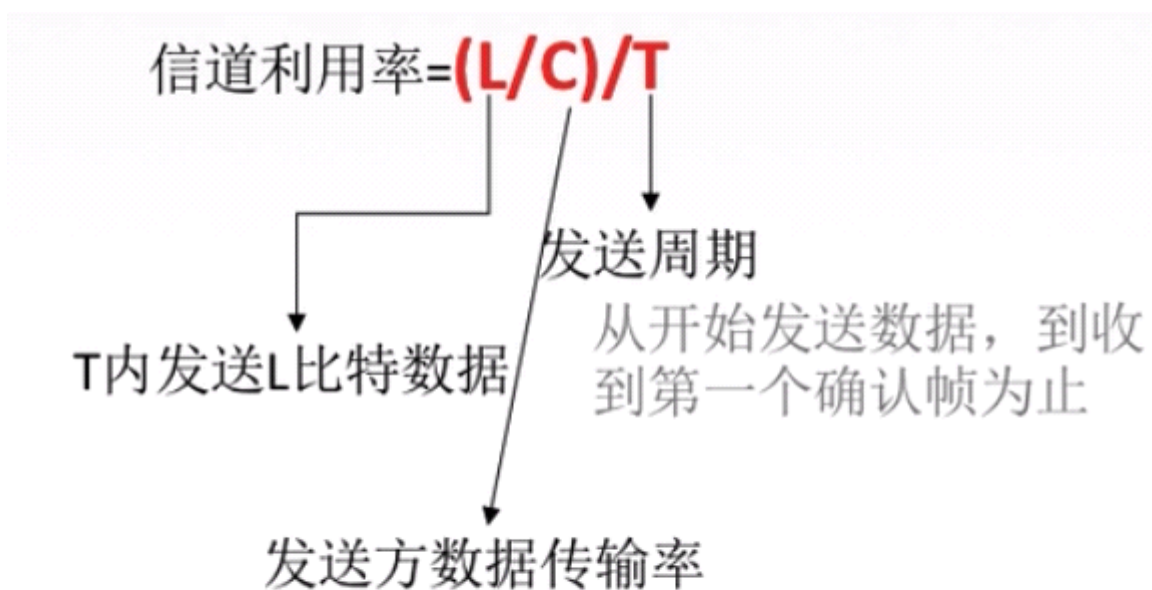
优点：简单

缺点：信道利用率太低



信道利用率

发送方在一个发送周期内，有效地发送数据所需要的时间占整个发送周期的比率。



信道吞吐率 = 信道利用率 * 发送方的发送速率

例题：一个信道的数据传输率为4kb/s，单向传播时延为30ms，如果使停止一等待协议的信道最大利用率达到80%，要求的数据帧长度至少为（ ）。

$$80\% = \frac{L/4}{L/4 + 2 \times 30\text{ms}}$$

$$= \frac{L}{L + 2 \times 30\text{ms} \times 4\text{kb/s}}$$

$$L = 960\text{bit}$$

后退N帧协议GBN

滑动窗口



发送方必须响应的三件事

1. 上层的调用

上层要发送数据时，发送方先检查发送窗口是否已满，如果**未**满，则产生一个帧并将其发送；如果**窗**口**已**满，发送方只需将数据返回给上层，暗示上层窗口已满。上层等一会再发送。（实际实现中，发送方可以缓存这些数据，窗口不满时再发送帧）。

2. 收到了一个ACK

GBN协议中，对n号帧的确认采用**累积确认**的方式，标明接收方已经收到n号帧和它之前的全部帧。

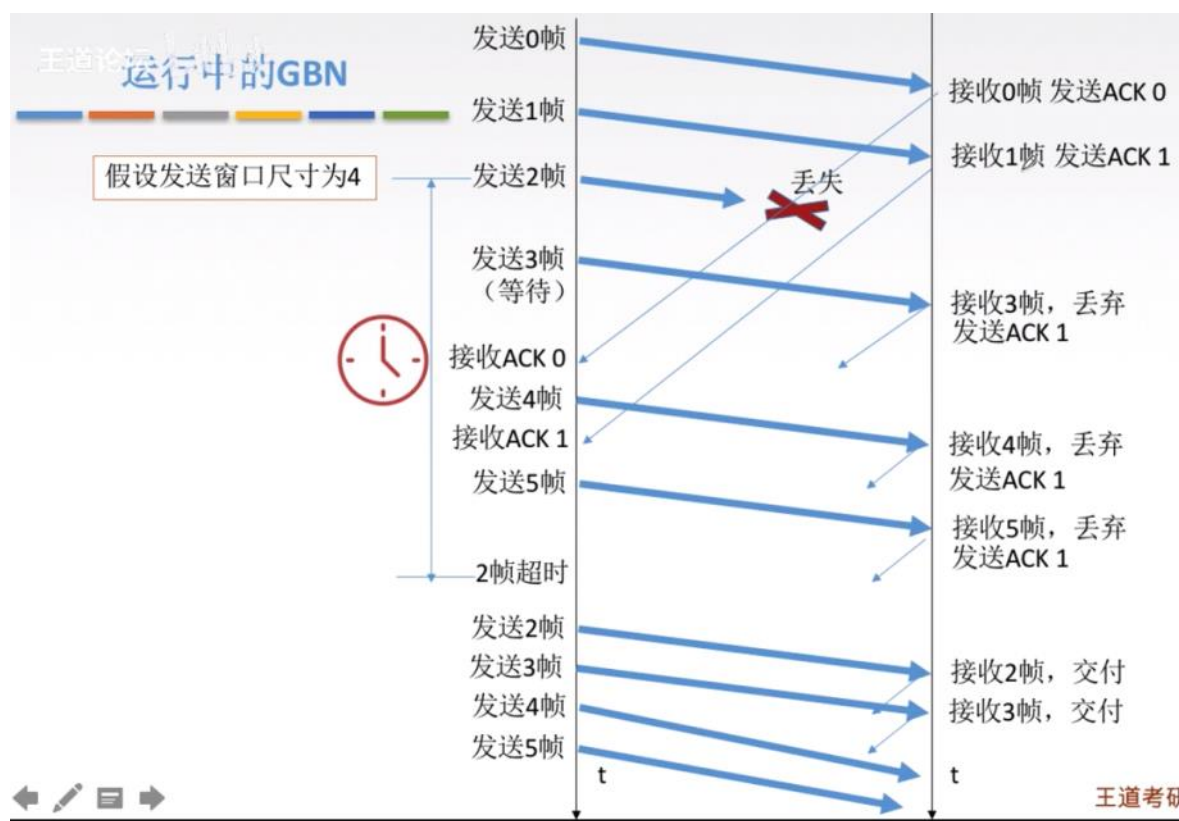
3. 超时事件

协议的名字为后退N帧 / 回退N帧，来源于出现丢失和时延过长帧时发送方的行为。就像在停等协议中一样，定时器将再次用于恢复数据帧或确认帧的丢失。如果出现超时，发送方重传所有已发送但未被确认的帧。

接收方要做的事

- 如果正确收到n号帧，并且按序，那么接收方为n帧发送一个ACK，并将该帧中的数据部分交付给上层。
- 其余情况都丢弃帧，并为最近按序接收的帧重新发送ACK。接收方无需缓存任何失序帧，只需要维护一个信息：expectedseqnum（下一个按序接收的帧序号）。

运行中的GBN



滑动窗口长度

若采用n个比特对帧编号，那么发送窗口的尺寸 W_t 应满足： $1 \leq W_t \leq 2^n - 1$ 。因为发送窗口尺寸过大，就会使得接收方无法区别新帧和旧帧。

总结

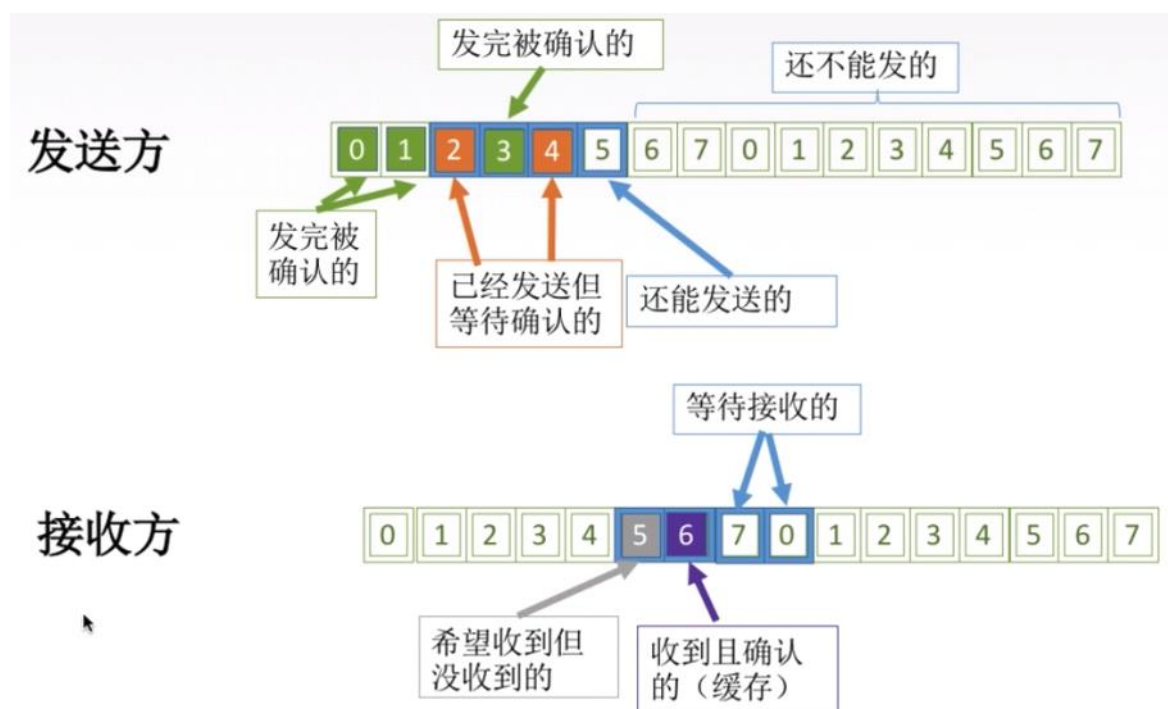
1. 累积确认（偶尔捎带确认）
2. 接收方只按顺序接收帧，不按序无情丢弃
3. 确认序列号最大的、按序到达的帧
4. 发送窗口最大为 $2^n - 1$ ，接收窗口大小为1

性能分析

- **优点：**因连续发送数据帧而提高了信道利用率。
- **缺点：**在重传时必须把原来已经正确传送的数据帧重传，是传送效率降低。
- **优化：**选择重传协议

选择重传协议SR

重传协议中的滑动窗口



发送方必须响应的三件事

1. 上层的调用

从上层收到数据后，SR发送方检查下一个可用于该帧的序号，如果序号位于发送窗口内，则发送数据帧；否则就像GBN一样，要么将数据**缓存**，要么**返回给上层**之后再传输。

2. 收到了一个ACK

如果收到ACK，加入该帧序号在窗口内，则SR发送方将那个被确认的帧标记为已接收。如果该帧序号是窗口的下界（最左边第一个窗口对应的序号），则窗口向前移动到具有最小序号的未确认帧处。如果窗口移动了并且有序号在窗口内的未发送帧，则发送这些帧。



3. 超时事件

每个帧都有自己的定时器，一个超时事件发生后**只重传一个帧**。

接收方要做的事

来者不拒（窗口内的帧）：SR接收方将**确认一个正确接收的帧**而不管其是否按序。失序的帧将被缓存，并返回给发送方一个该帧的确认帧【收谁确认谁】，直到所有帧（即序号更小的帧皆被收到为止，这时才可以将一批帧按序交付给上层，然后向

2. 只重传出错帧
3. 接收方有缓存
4. $W_{Tmax} = W_{Rmax} = 2^{(2n-1)}$

介质访问控制

传输数据使用的两种链路

- **点对点链路**

两个相邻节点通过一个链路相连，没有第三者。

应用：PPP协议，常用于广域网。



- **广播式链路**

所有主机共享通信介质。

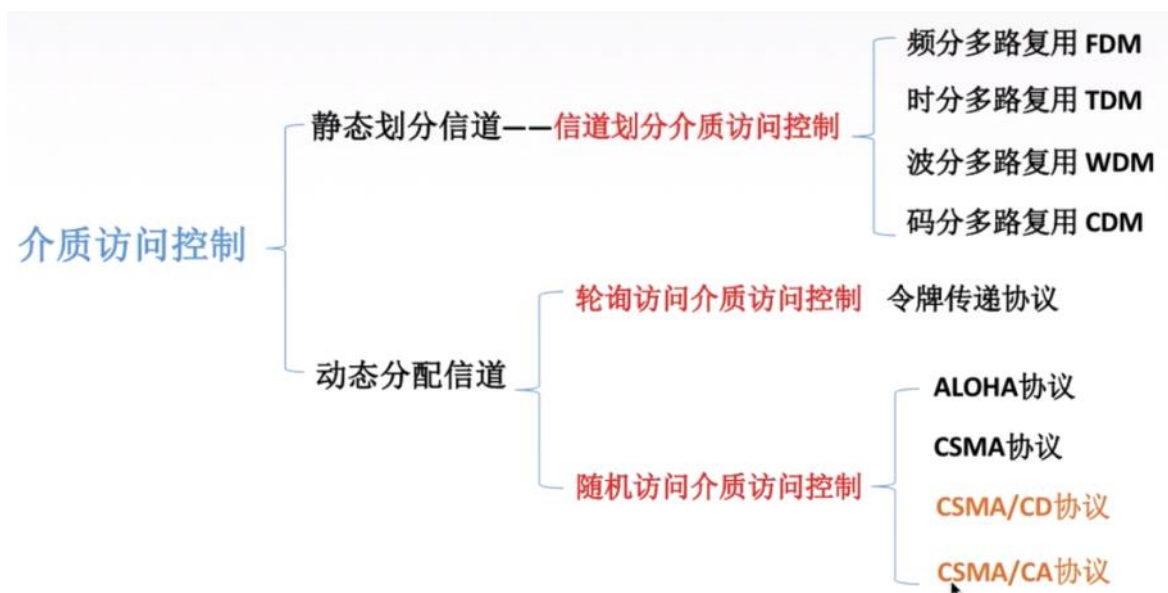
应用：早期的总线以太网、无线局域网，常用于局域网。

典型拓扑结构：总线型、星型（逻辑总线型）



介质访问控制

介质访问控制的内容就是，采取一定的措施，使得两对节点之间的通信不会发生相互干扰的情况。



协议对比

- 信道划分介质访问控制协议
 - 基于多路复用技术划分资源
 - 网络负载重：共享信道效率高，且公平
 - 网络负载轻：共享信道效率低
- 随机访问MAC协议
 - 用户根据意愿随机发送信息，发送信息时可独占信道带宽
 - 网络负载重：产生冲突开销
 - 网络负载轻：共享信道效率高，单个结点可利用信道全部带宽
- 轮询访问MAC协议/轮流协议/轮转访问MAC协议
 - 既要不产生冲突，又要发送时占全部带宽

静态划分信道

信道划分介质访问控制

将使用介质的每个设备与来自同一信道上的其他设备**通信隔离开**，把时域与频域资源合理地分配给网路上的设备。



多路复用技术

把多个信号组合在一条物理信道上进行传输，使得多个计算机或终端设备共享信道资源，提高信道利用率。



把一条广播信道，逻辑上分成几条用于两个节点之间通信的互不干扰的子信道，实际就是把广播信道转变为点对点信道。

控制方式

- 频分多路复用FDM
- 时分多路复用TDM
- 波分多路复用WDM
- 码分多路复用CDM

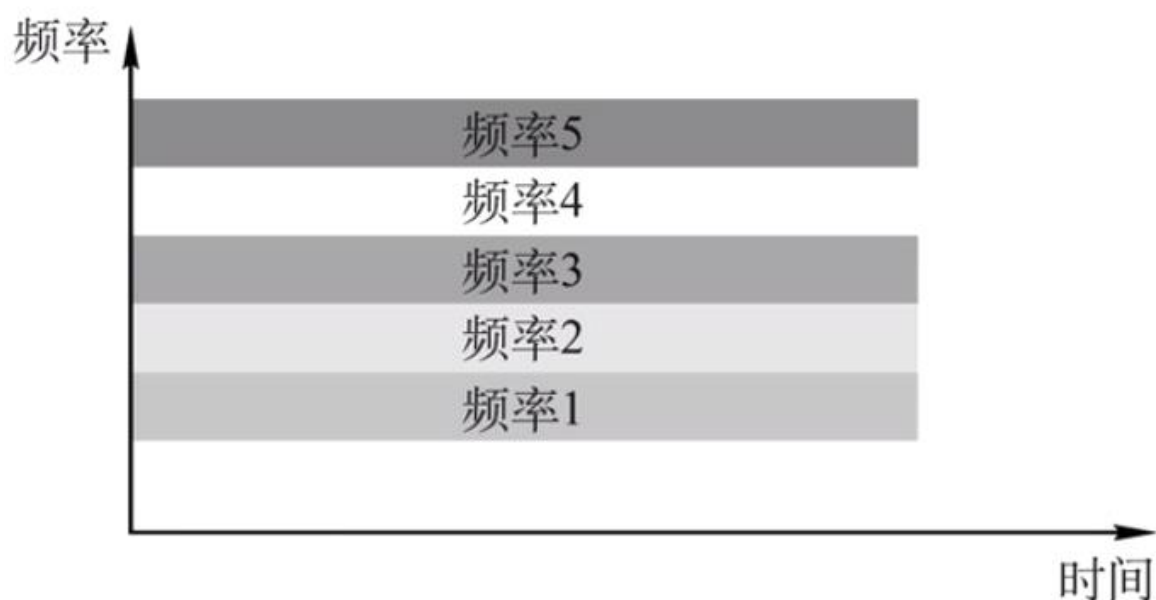
频分多路复用FDM

频分多路复用FDM

用户在分配到一定的频带后，在通信过程中自始至终都占用这个频带。频分复用的所有用户在同样的时间占用不同的带宽（频率带宽）资源。

优点：充分利用传输介质带宽，系统效率较高；由于技术比较成熟，实现也比较容易。

⊗ 频分复用



时分多路复用TDM

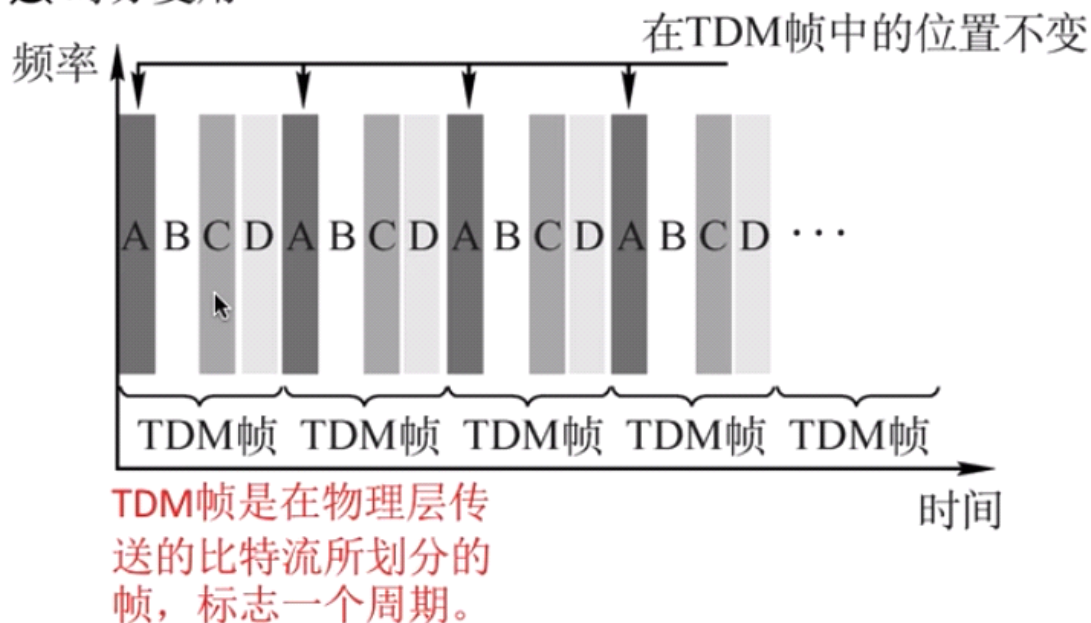
时分多路复用TDM

将时间划分为一段段等长的时分复用帧（TDM帧）。每一个时分复用的用户在每一个TDM帧中占用**固定序号的时隙**，所有用户轮流占用信道。

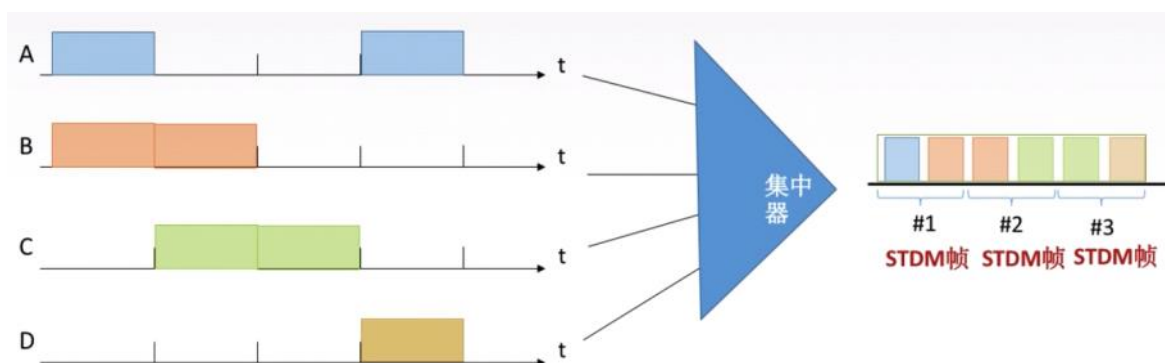
区别：

- 频分复用：相当于操作系统中的“并行”
- 时分复用：相当于操作系统中的“并发”

⊗ 时分复用



改进的时分复用：统计时分复用STDM

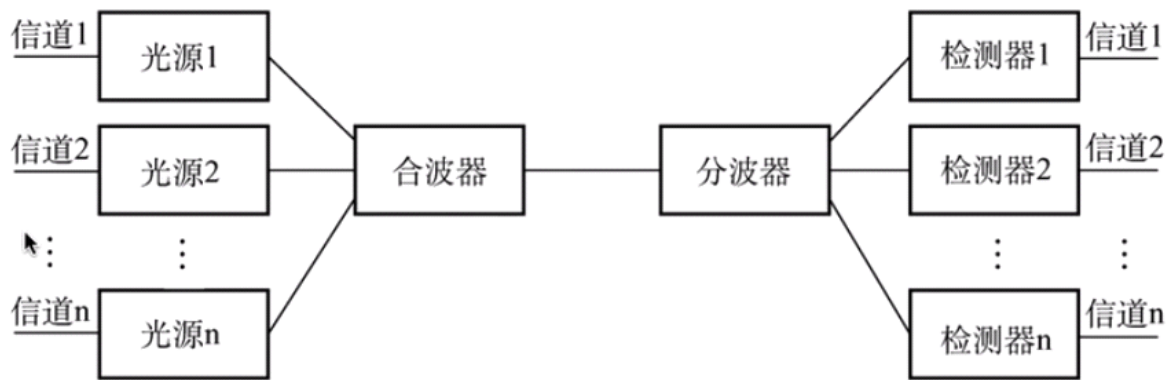


每一个STDM帧中的时隙数小于连接在集中器上的用户数。各用户有了数据就随时发往集中器的输入缓存，然后集中器按顺序依次扫描输入缓存，把缓存中的输入数据放入STDM帧中，一个STDM帧满了就发出。STDM帧不是固定分配时隙，而是按需动态分配时隙。

波分多路复用WDM

波分多路复用WDM

波分多路复用就是光的频分多路复用，在一根光纤中传输多种不同波长（频率）的光信号，由于波长（频率）不同，所以各路光信号互不干扰，最后再用波长分解复用器将各路波长分解出来。



码分多路复用CDM

码分多路复用CDM

码分多址（CDMA） 是码分复用的一种方式。

1个比特分为多个码片/芯片（chip），每一个站点被指定一个唯一的m位的芯片序列。

发送1时站点发送芯片序列，发送0时发送芯片序列反码（通常把0写成-1）。

如何不打架：多个站点同时发送数据的时候，要求各个站点芯片序列相互正交。

如何合并：各路数据在信道中被线性相加。

动态分配信道

动态分配信道

动态媒体接入控制/多点接入

特点：信道并非在用户通信时固定分配给用户。

分类：

- 随机访问介质访问控制
- 轮询访问介质访问控制

随机访问介质访问控制

随机访问介质访问控制

所有用户可随机发送信息。发送信息时占**全部带宽**。主要有以下四种协议。

ALOHA协议

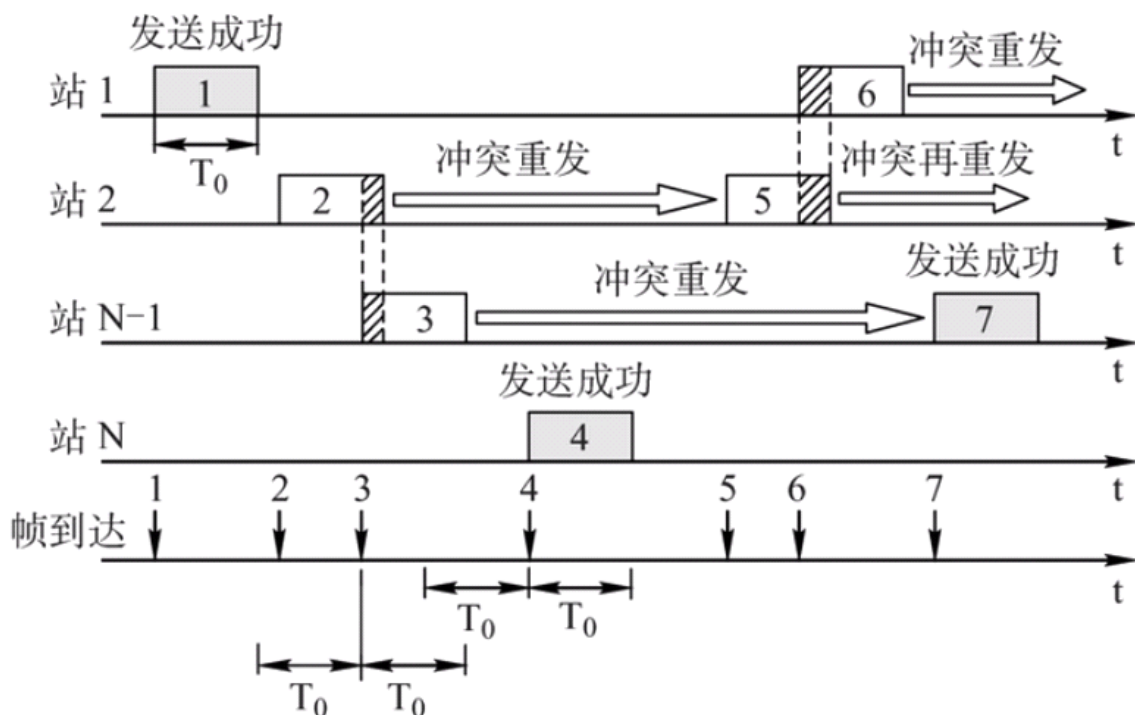
命名来源



分类：纯ALOHA协议、时隙ALOHA协议

纯ALOHA协议

不监听信道，不按时间槽发送，随机重发。想发就发



如何检测冲突

如果发送冲突，接收方就会检测出差错，然后不予确认，发送方在一定时间内收不到就判断发生冲突。

冲突如何解决

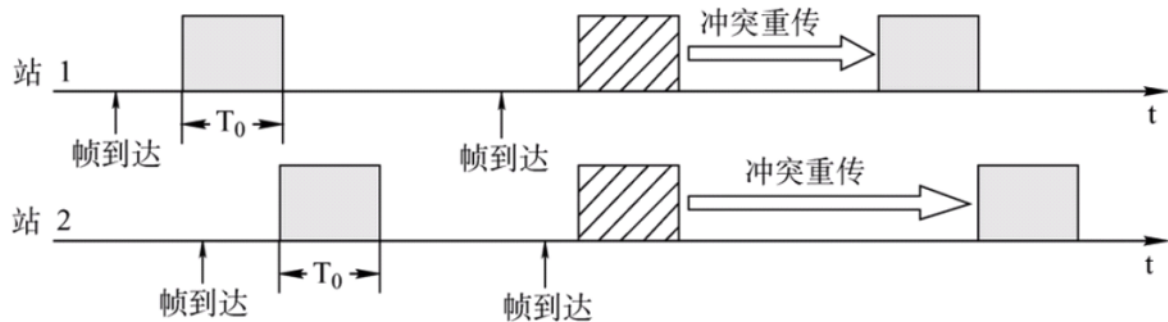
超时后等一随机时间再重传。

缺点：发生成功率低

时隙ALOHA协议

时隙ALOHA协议的思想：把时间分成若干个相同的时间片，若发生冲突，则必须等到下一个时间片开始时刻再发送。

所有用户在时间片开始时刻同步接入网络信道，若发生冲突，则必须等到下一个时间片开始时刻再发送。控制想发就发的随意性



区别

- 纯ALOHA比时隙ALOHA吞吐量更低，效率更低。
- 纯ALOHA想发就发，时隙ALOHA只有在时间片段开始时才能发。

CSMA协议

CSMA协议

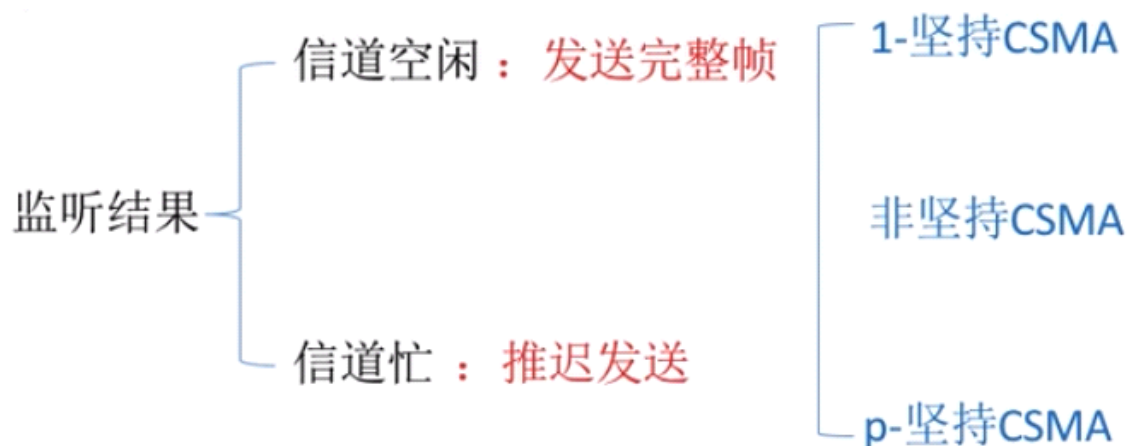
载波监听多路访问协议CSMA (carrier sense multiple access)

CS: 载波侦听 / 监听，每一个站在发送数据之前要检测一下总线上是否有其他计算机在发送数据。

MA: 多点接入，表示许多计算机以多点接入的方式连接在一根总线上。

当几个站同时在总线上发送数据时，总线上的信号**电压摆动值**将会增大（互相叠加）。但一个站检测到的信号电压摆动值超过一定门限值时，就认为总线上至少有两个站同时在发送数据，表明产生了碰撞，即发生了冲突。

协议思想：发送帧之前，**监听信道**。



坚持CSMA

坚持指的是对于监听信道忙之后的坚持。

坚持CSMA思想：

- 如果一个主机要发送消息，那么它先监听信道。
- 空闲则直接传输，不必等待。
- 忙则一直监听，直到空闲马上传输。
- 如果有冲突（一段时间内未收到肯定回复），则等待一个随机长的时间再监

听，重复上述过程。

优点：只要媒体空闲，站点就马上发送，避免了媒体利用率的损失。

缺点：假如有两个或两个以上的站点有数据要发送，冲突就不可避免。

非坚持CSMA

非坚持指的是对于监听信道忙之后就不继续监听。

非坚持CSMA思想

- 如果一个主机要发送消息，那么它先监听信道。
- 空闲则直接传输，不必等待。
- 忙则等待一个随机的时间之后再进行监听。

优点：采用随机的重发延迟时间可以减少冲突发生的可能性。

缺点：可能存在大家都在延迟等待过程中，使得媒体仍可能处于空闲状态，媒体使用率降低。

p-坚持CSMA

p-坚持指的是对于监听信道空闲的处理。

p-坚持CSMA思想

- 如果一个主机要发送消息，那么它先监听信道。
- 空闲则以p概率直接传输，不必等待；概率1-p等待到下一个时间槽再传输。
- 忙则等待一个随机的时间之后再进行监听。

优点：既能像非坚持算法那样减少冲突，又能像1-坚持算法那样减少媒体空闲时间的这种方案。

缺点：发生冲突后还是要坚持把数据帧发送完，造成了浪费。

对比总结

	1-坚持CSMA	非坚持CSMA	p-坚持CSMA
信道空闲	马上发	马上发	p概率马上发 1-p概率等到下一个时隙再发送
信道忙	继续坚持监听	放弃监听，等一个随机时间再监听	放弃监听，等一个随机时间再监听

CSMA/CD协议

CSMA/CD协议

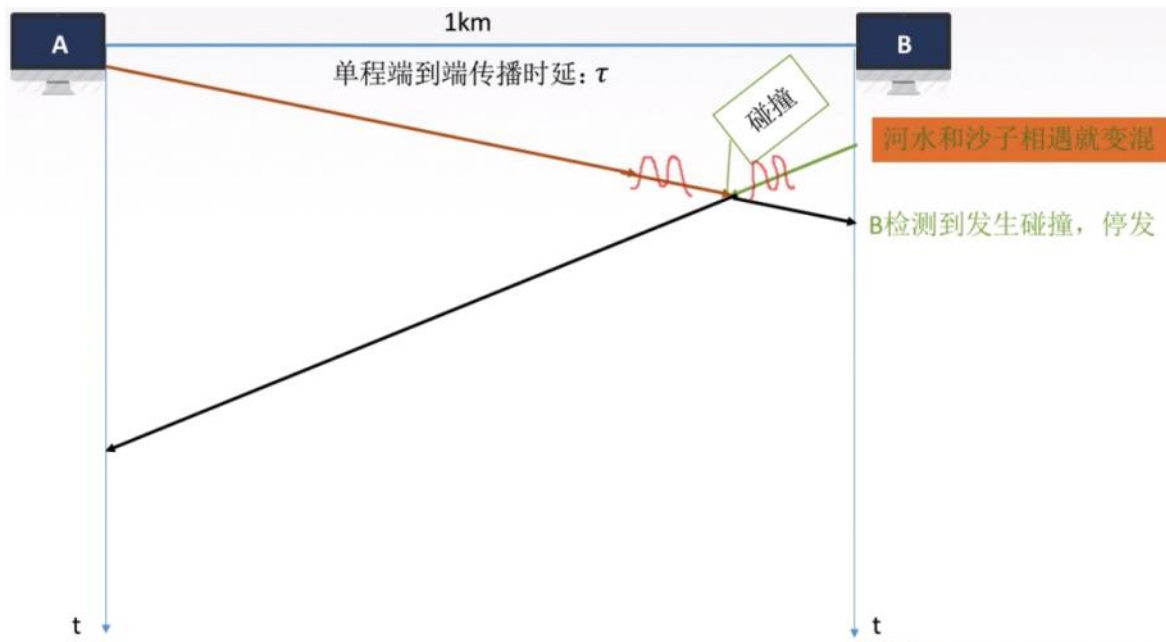
****载波监听多点接入/碰撞检测CSMA/CD (carrier sense multiple access with collision detection) ****

CS：载波侦听/监听，每一个站在发送数据之前以及发送数据时都要检测一下总线上是否有其他计算机在发送数据。

MA: 多点接入，表示许多计算机以多点接入的方式连接在一根总线上。总线型网络

CD: 碰撞检测（冲突检测），“边发送边监听”，适配器边发送数据边检测信道上信号电压的变化情况，以便判断自己在发送数据时其他站是否也在发送数据。半双工网络

传播时延对载波监听的影响



最迟多久能知道自己发送的数据没和别人碰撞

只要经过 2τ 的时间还没有检测到碰撞，就能肯定这次发送不会发生碰撞。

如何确定碰撞后的重传时机

使用截断二进制指数规避算法

1. 确定基本退避（推迟）时间为争用期 2τ 。
2. 定义参数 k ，它等于**重传次数**，但 k 不超过10，即 $k = \min[\text{重传次数}, 10]$ 。当重传次数不超过10时， k 等于重传次数；当重传次数大于10时， k 就不再增大而一直等于10。
3. 从离散的整数集合 $[0, 1, 2^k - 1]$ 中随机取出一个数 r ，重传所需要退避的时间就是 r 倍的基本退避时间，即 $2r\tau$ 。
4. 当重传达16次仍不能成功时，说明网络太拥挤，认为此帧永远无法正确发出，抛弃此帧并向高层报告出错。

第一次重传， $k=1$ ， r 从 $\{0, 1\}$ 选；

重传推迟时间为0或 2τ ，在这两个时间中随机选一个；

若再次碰撞，则在第二次重传时， $k=2$ ， r 从 $\{0, 1, 2, 3\}$ 选；

重传推迟时间为0或 2τ 或 4τ 或 6τ ，在这四个时间中随机选一个；

若再次碰撞，则第三次重传时， $k=3$ ， r 从 $\{0, 1, 2, 3, 4, 5, 6, 7\}$ 选.....

优点: 若连续多次发生冲突，就表明可能有较多的站参与争用信道。使用此算法可使重传需要推迟的平均时间随重传次数的增大而增大，因而减小发生碰撞的概率，

有利于整个系统的稳定。

最小帧长

最小帧长 = 总线传播时延 $\times$ 数据传输速率 $\times 2$

帧的传输时延至少要两倍于信号在总线中的传播时延。

$$\frac{\text{帧长 (bit)}}{\text{数据传输速率}} \geq 2\tau$$

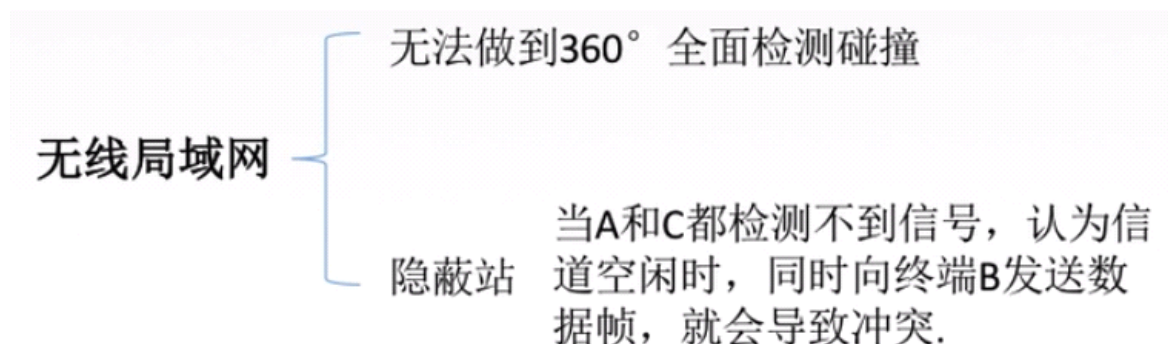
以太网规定最短帧长为64B，凡是长度小于64B的都是由于冲突而异常终止的无效帧。

CSMA/CA协议

CSMA/CA协议

载波监听多点接入/碰撞避免CSMA/CA (carrier sense multiple access with collision avoidance)

产生背景



工作原理

1. 发送数据前，先检测信道是否空闲。
2. 空闲则发出RTS (request to send), RTS包括发射端的地址、接收端的地址、下一份数据将持续发送的时间等信息；信道忙则等待。
3. 接收端收到RTS后，将响应CTS (clear to send)。
4. 发送端收到CTS后，开始发送数据帧（同时预约信道：发送方告知其他站点自己要传多久数据）。
5. 接收端收到数据帧后，将用CRC来检验数据是否正确，正确则响应ACK帧。
6. 发送方收到ACK就可以进行下一个数据帧的发送，若没有则一直重传至规定重发次数为止（采用二进制指数退避算法来确定随机的推迟时间）。

CSMA/CD和CSMA/CA比较

相同点

CSMA/CD与CSMA/CA机制都从属于CSMA的思路，其核心是先听再说。换言之，两个在接入信道之前都须要进行监听。当发现信道空闲后，才能进行接入。

不同点

1. **传输介质不同**：CSMA/CD用于总线式以太网【有线】，而CSMA/CA用于无线局域网【无线】。
2. **载波检测方式不同**：因传输介质不同，CSMA/CD与CSMA/CA的检测方式也不同。CSMA/CD通过电缆中电压的变化来检测，当数据发生碰撞时，电缆中的电压就会随着发生变化；而CSMA/CA采用能量检测（ED）、载波检测（CS）和能量载波混合检测三种检测信道空闲的方式。
3. **CSMA/CD检测冲突，CSMA/CA避免冲突**，二者出现冲突后都会进行有上限的重传。

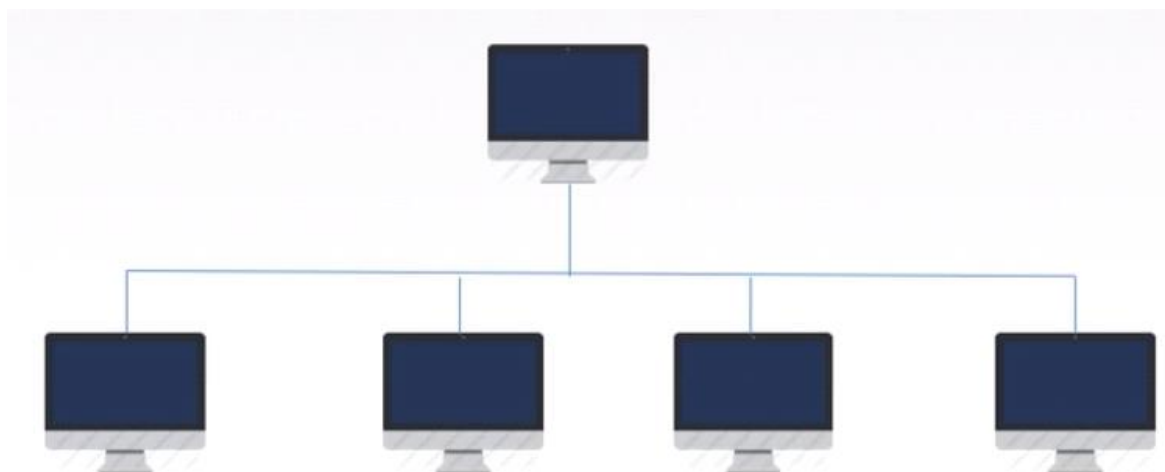
轮询访问介质访问控制

轮询协议

轮询协议

主节点轮流”邀请“从属节点发送数据。

存在问题：轮询开销、等待延迟、单点故障



令牌传递协议

令牌传递协议

令牌：一个特殊格式的MAC控制帧，不含任何信息。

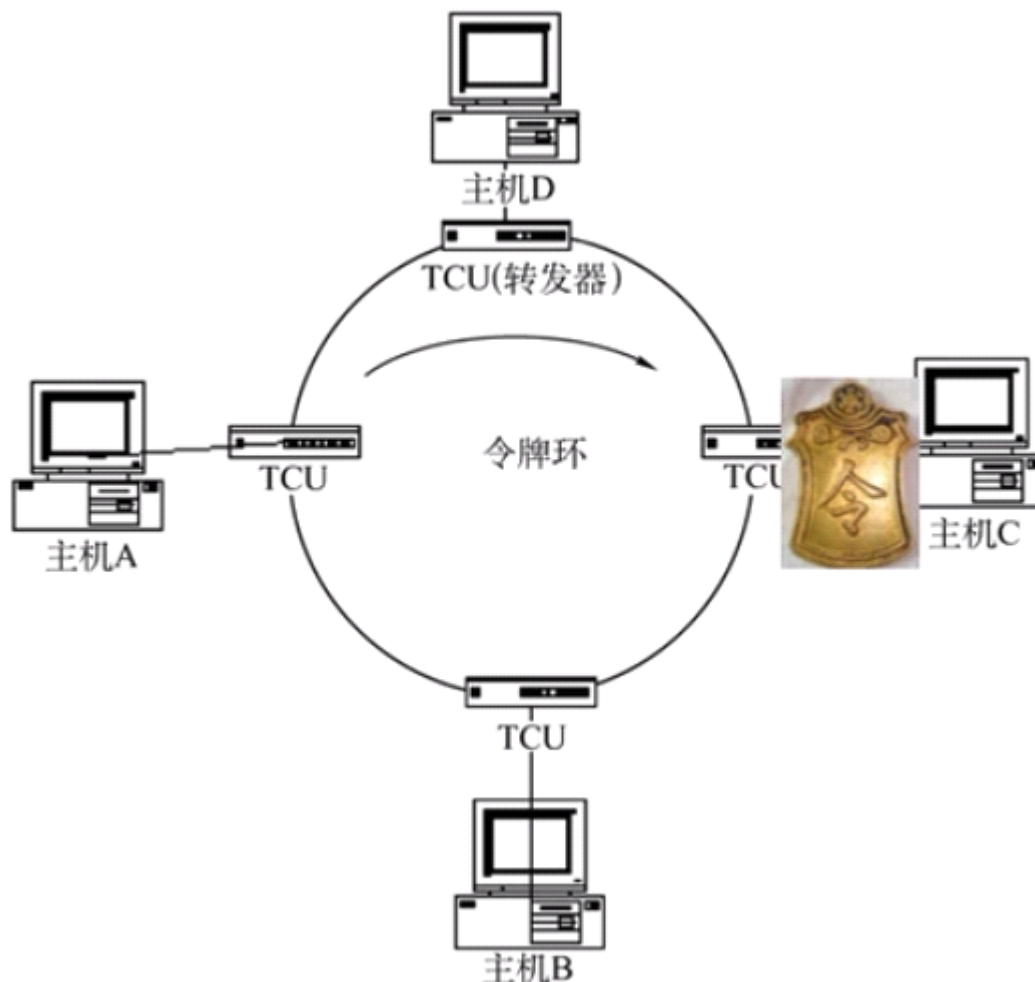
作用：控制信道的使用，确保同一时刻只有一个结点独占信道。

每个结点都可以在一定的时间内（令牌持有时间）获得发送数据的权利，并不是无限制地持有令牌。

存在问题：令牌开销、等待延迟、单点故障

应用于令牌环网（物理星型拓扑，逻辑环形拓扑）。

采用令牌传送方式的网络常用于负载较重、通信量较大的网络中。



局域网

局域网

局域网 (Local Area Network)

简称LAN，是指在**某一区域内**由多台计算机互联成的计算机组，使用广播信道。

特点

1. 覆盖的地理范围较小，只在一个相对独立的局部范围内联，如一座或集中的建筑群内。
2. 使用专门铺设的传输介质（双绞线、同轴电缆）进行联网，数据传输速率高（10Mb/s- 10Gb/s）。
3. 通信延迟时间短，误码率低，可靠性较高。
4. 各站为平等关系，共享传输信道。
5. 多采用分布式控制和广播式通信，能进行广播和组播。

决定局域网的主要要素为：**网络拓扑，传输介质与介质访问控制方法。**

4种拓扑结构

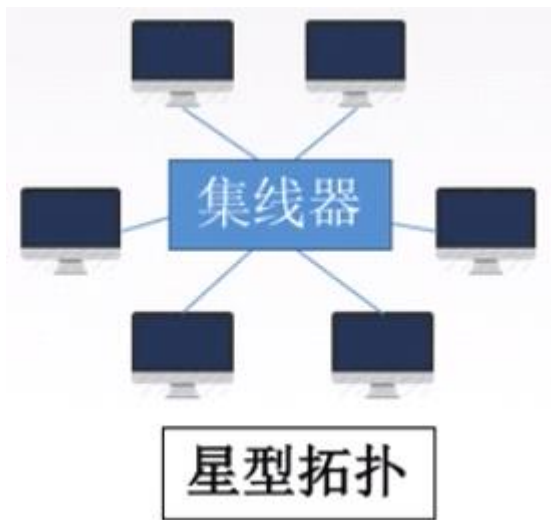
• 星型拓扑

• 优点

- 中心节点是控制中心，任意两个节点间的通信最多只需两步，传输

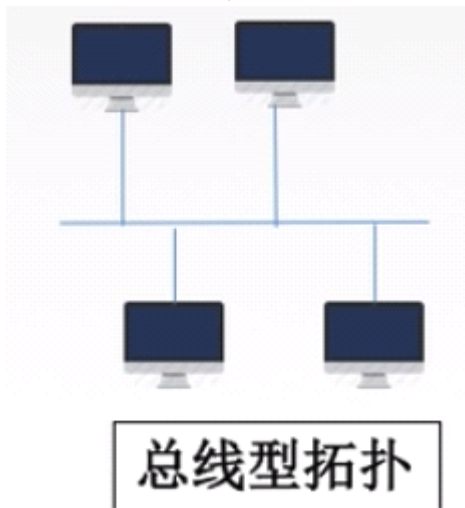
速度快。

- 网络构形简单、建网容易、便于控制和管理。
- 缺点
 - 但这种网络系统，网络可靠性低，网络共享能力差，有单点故障问题。



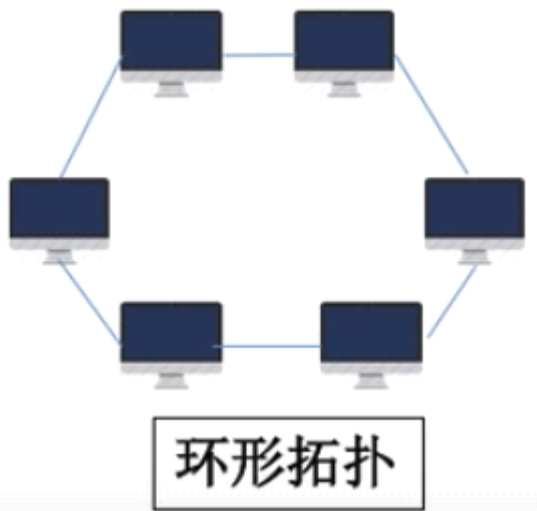
- 总线型拓扑（主流）

- 优点
 - 网络可靠性高、网络节点间响应速度快、共享资源能力强。
 - 设备投入量少、成本低、安装使用方便。
 - 当某个工作站节点出现故障时，对整个网络系统影响小。



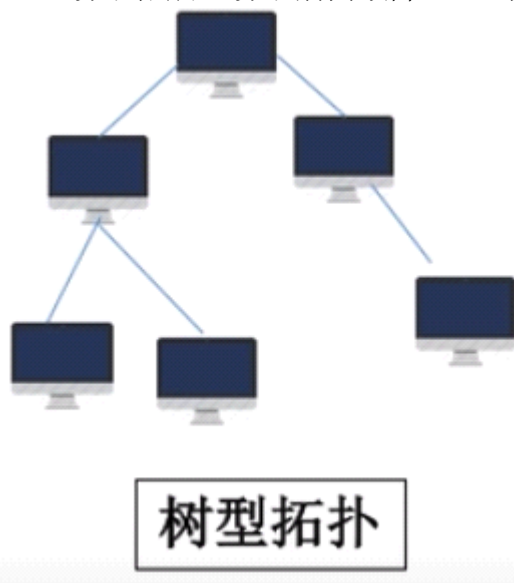
- 环型拓扑

- 优点
 - 系统中通信设备和线路比较节省。
- 缺点
 - 有单点故障问题，由于环路是封闭的，所以 不便于扩充。
 - 系统响应延时长，且信息传输效率相对较低。



- 树形拓扑

- 易于拓展，易于隔离故障， 也容易有单点故障。



介质访问控制方法

1. CSMA/CD

常用于总线型局域网，也用于树型网络。

1. 令牌总线

常用于总线型局域网，也用于树型网络。它是把总线型或树型网络中的各个工作站按一定顺序如按接口地址大小排列形成一个逻辑环。只有令牌持有者才能控制总线，才有发送信息的权力。

1. 令牌环

用于环形局域网，如令牌环网

局域网的分类

- 以太网 (Ethernet)

以太网以太网是应用最为广泛的局域网，包括标准以太网 (10Mbps)、快速以太网 (100Mbps)、千兆以太网 (1000 Mbps) 和10G以太网，它们都符合IEEE802.3系列标准规范。逻辑拓扑总线型，物理拓扑是星型或拓展星型。使

用CSMA/CD。

- **令牌环网** (Token Ring)
物理上采用了星形拓扑结构，逻辑上是环形拓扑结构，已是“明日黄花”。
- **FDDI网** (Fiber Distributed Data Interface)
物理上采用了双环拓扑结构，逻辑上是环形拓扑结构。
- **ATM网** (Asynchronous Transfer Mode)
较新型的单元交换技术，使用53字节固定长度的单元进行交换。
- **无线局域网** (Wireless Local Area Network; WLAN)
采用IEEE 802.11标准。

IEEE 802标准

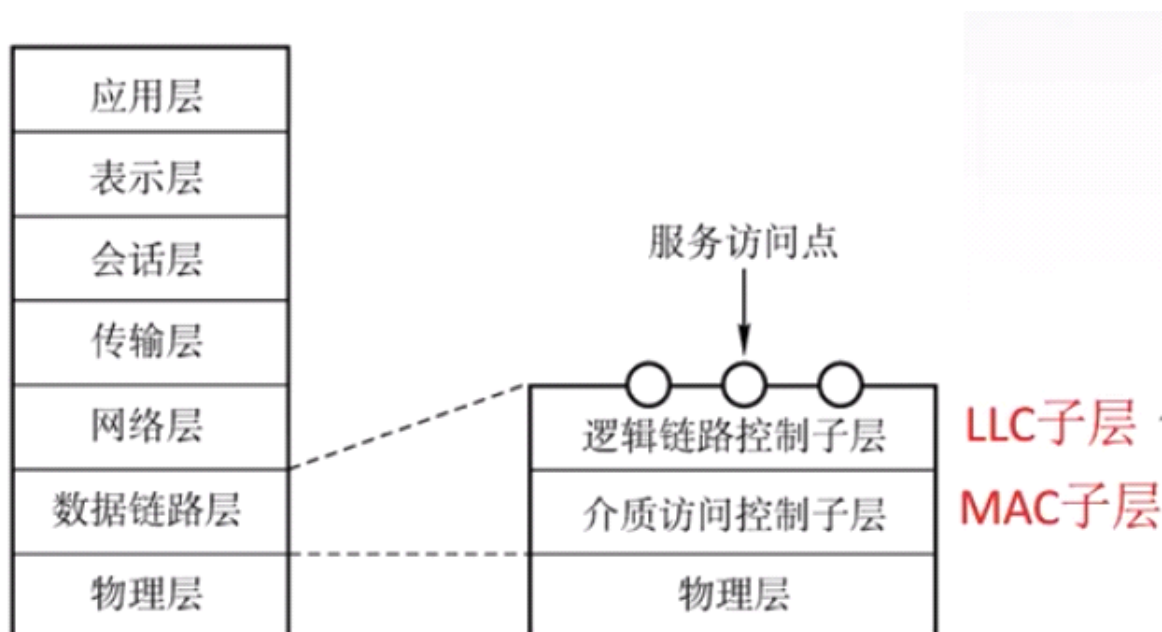
IEEE 802系列标准是IEEE 802 LAN/MAN标准委员会制定的局域网、城域网技术标准（1980年2月成立）。其中最广泛使用的有以太网、令牌环、无线局域网等。这一系列标准中的每一个子标准都由委员会中的一个专门工作组负责。

链路层的两个控制子层

IEEE 802标准所描述的局域网参考模型只对应OSI参考模型的数据链路层与物理层，它将数据链路层划分为逻辑链路层LLC子层和介质访问控制MAC子层。

LLC子层：LLC负责识别网络层协议，然后对它们进行封装。LLC 报头告诉数据链路层一旦帧被接收到时，应当对数据包做何处理。为网络层提供服务：无确认无连接、面向连接、带确认无连接、高速传送。

MAC子层：MAC子层的主要功能包括数据帧的封装 / 卸装，帧的寻址和识别，帧的接收与发送，链路的管理， 帧的差错控制等。MAC子层的存在屏蔽了不同物理链路种类的差异性。



以太网802.3

以太网

以太网 (Ethernet) 指的是由Xerox公司创建并由Xerox、Intel和DEC公司联合开发的基带总线局域网规范，是当今现有局域网采用的最通用的通信协议标准。以太网络使用CSMA/CD（载波监听多路访问及冲突检测）技术。以太网在局域网各种技术

中占统治性地位。

优势

1. 造价低廉（以太网网卡不到100块）
2. 是应用最广泛的局域网技术
3. 比令牌环网、ATM网便宜，简单
4. 满足网络速率要求：10Mb/s~10Gb/s

两个标准

- **DIX Ethernet V2**: 第一个局域网产品（以太网）规约。
- **IEEE 802.3**: IEEE 802委员会802.3工作组制定的第一个IEEE的以太网标准。（802.3局域网 AKA 也相当于以太网）

提供服务

==以太网只实现无差错接收，不实现可靠传输。 ==

无连接：发送方和接收方之间无“握手过程”。

不可靠：不对发送方的数据帧**编号**，接收方不向发送方进行**确认**，差错帧直接丢弃，差错纠正由高层负责。

传输介质和拓扑结构的发展

- 传输介质



- 拓扑结构

物理拓扑



使用集线器的以太网在逻辑上仍是一个总线网，各站共享逻辑上的总线，使用的还是CSMA/CD协议。

10BASE-T以太网

10BASE-T是传送基带信号的双绞线以太网，T表示采用双绞线，现10BASE-T采用的是无屏蔽双绞线（UTP），传输速率是10Mb/s。

- 物理上采用星型拓扑，逻辑上总线型，每段双绞线最长为100m。
- 采用曼彻斯特编码。
- 采用CSMA/CD介质访问控制。

适配器与MAC地址

适配器：计算机与外界有局域网的连接通过通信适配器的。



网络接口板

网络接口卡NIC（network interface card）

NOW，不再使用单独网卡。

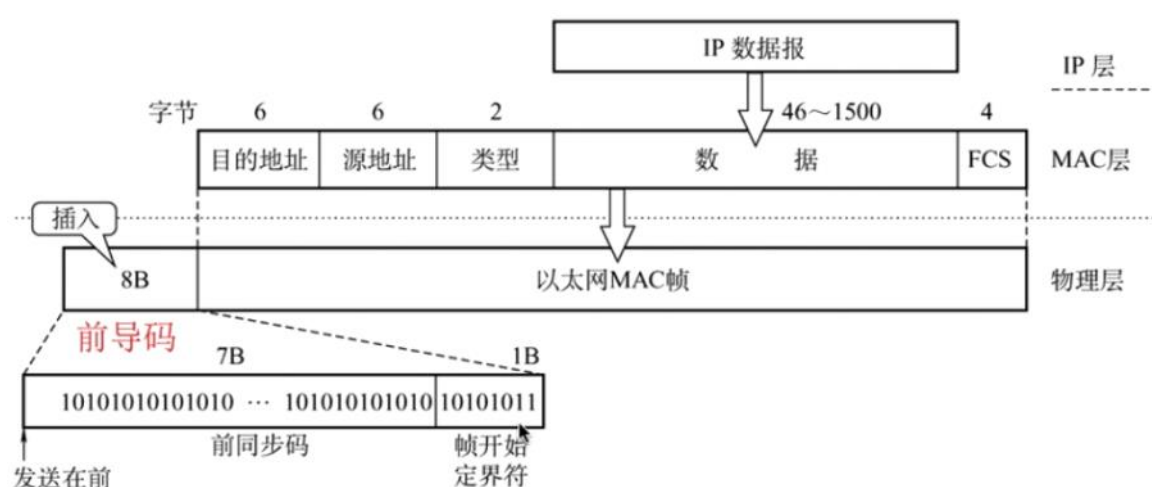
适配器上装有处理器和存储器（包括RAM和ROM）。

ROM上有计算机硬件地址**MAC地址**。

MAC地址：每个适配器有一个全球唯一的48位二进制地址，前24位代表厂家（由IEEE规定），后24位厂家自己指定。常用6个十六进制数表示，如02-60-sc-e4-b1-21。在局域网中，硬件地址又称为物理地址，或MAC地址。【实际上是标识符】

以太网MAC帧

最常用的MAC帧是以太网V2格式。



与IEEE 802.3的区别：

1. 第三个字段是长度/类型
2. 当长度/类型字段值小于0x0600时，数据字段必须装入LLC子层。

高速以太网

速率100Mb/s的以太网称为高速以太网。

1. 100BASE-T以太网

在双绞线上传送100Mb/s基带信号的星型拓扑以太网，仍使用IEEE802.3的CSMA/CD协议。

支持全双工和半双工，可在全双工方式下工作而无冲突。

2. 吉比特以太网

在光纤或双绞线上传送1Gb/s信号。

支持全双工和半双工，可在全双工方式下工作而无冲突。

3. 10吉比特

10吉比特以太网在光纤上传送10Gb/s信号。

只支持全双工，无争用问题。

无线局域网802.11

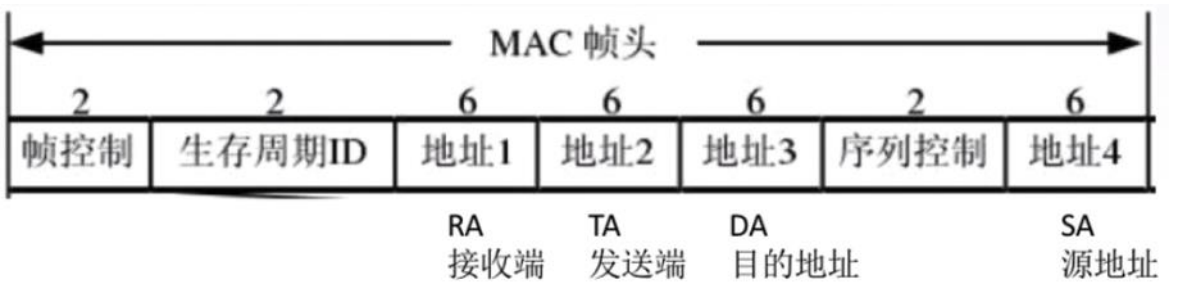
IEEE 802.11

IEEE 802.11是无线局域网通用的标准，它是由IEEE所定义的无线网络通信的标准。

- IEEE 802.11，1997年，原始标准（2Mbit/s，工作在2.4GHz）。
- IEEE 802.11a，1999年，物理层补充（54Mbit/s，工作在5.2GHz）。
- IEEE 802.11b，1999年，物理层补充（11Mbit/s工作在2.4GHz）。
- IEEE 802.11c，符合802.1D的媒体接入控制层桥接（MAC Layer Bridging）。
- IEEE 802.11d，根据各国无线电规定做的调整。
- IEEE 802.11e，对服务等级（Quality of Service, QoS）的支持。
- IEEE 802.11f，基站的互连性（IAPP, Inter-Access Point Protocol），2006年2月被IEEE批准撤销。
- IEEE 802.11g，2003年，物理层补充（54Mbit/s，工作在2.4GHz）。
- IEEE 802.11h，2004年，无线覆盖半径的调整，室内（indoor）和室外（outdoor）信道（5.2GHz频段）。

802.11标准帧格式

标准帧格式：MAC帧头格式

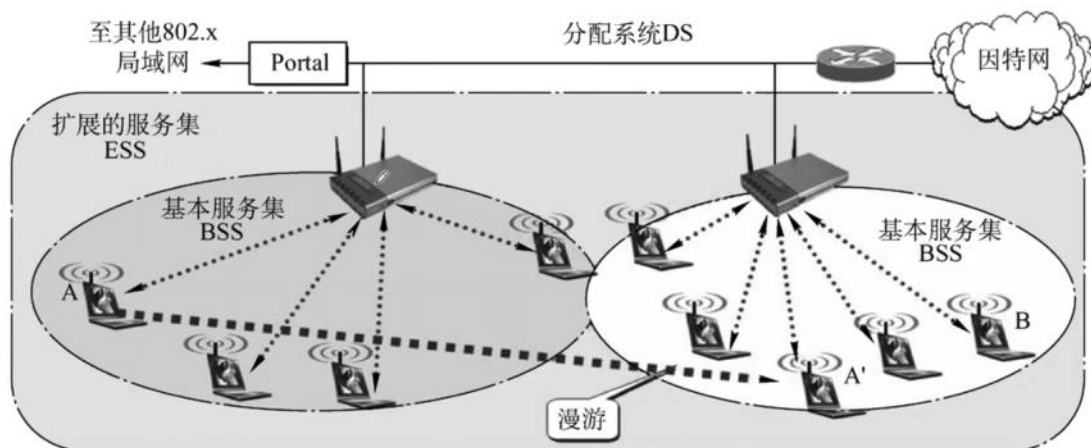


格式分类

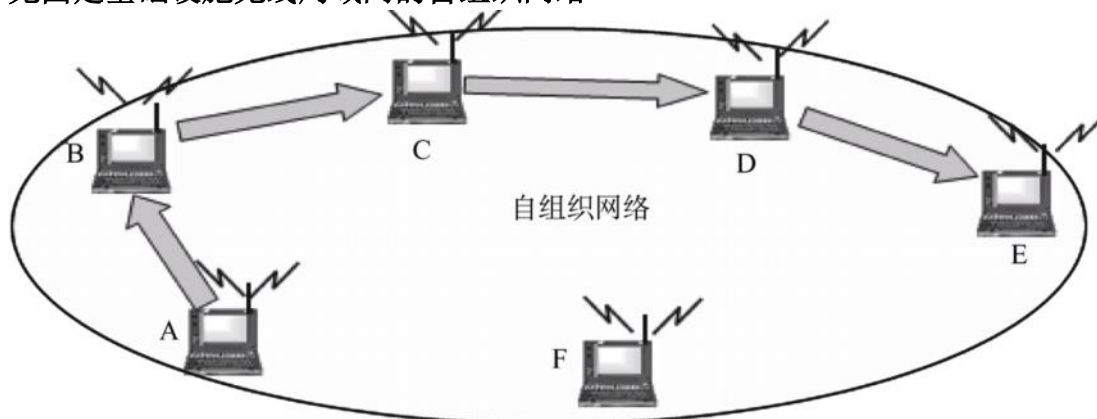
功能	To DS	From DS	Address1（接收端）	Address2（发送端）	Address3	Address4
IBSS	0	0	DA	SA	BSSID	未使用
To AP（基础结构型）	1	0	BSSID	SA	DA	未使用
From AP（基础结构型）	0	1	DA	BSSID	SA	未使用
WDS（无线分布式系统）	1	1	RA	TA	DA	SA

无线局域网的分类

- 有固定基础设施无线局域网



- 无固定基础设施无线局域网的自组织网络

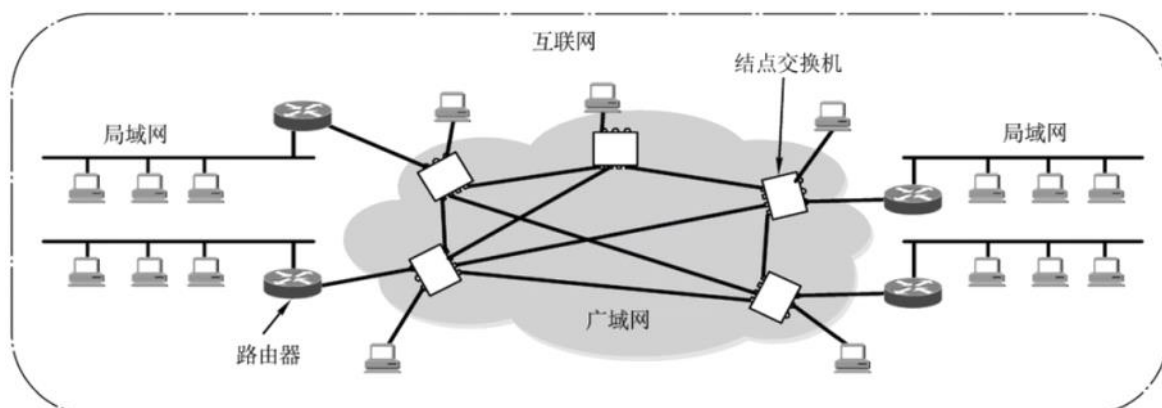


广域网

概念

广域网 (WAN, Wide Area Network), 通常跨接很大的物理范围, 所覆盖的范围从几十公里到几千公里, 它能连接多个城市或国家, 或横跨几个洲并能提供远距离通信, 形成国际性的远程网络。

广域网的通信子网主要使用分组交换技术。广域网的通信子网可以利用公用分组交换网、卫星通信网和无线分组交换网, 它将分布在不同地区的**局域网或计算机系统**互连起来, 达到资源共享的目的。如因特网 (Internet) 是世界范围内最大的广域网。



广域网常用链路层协议

- PPP协议
- HDLC协议

PPP协议

特点

点对点协议PPP（Point-to-Point Protocol）是目前使用最广泛的数据链路层协议，用户使用拨号电话接入因特网时一般都使用PPP协议。 只支持全双工链路。

协议应满足要求

- **简单**：对于链路层的帧，无需纠错，无需序号，无需流量控制。
- **封装成帧**：帧定界符
- **透明传输**：与帧定界符一样比特组合的数据应该如何处理：异步线路用字节填充，同步线路用比特填充。
- **多种网络层协议**：封装的'P数据报可以采用多种协议。
- **多种类型链路**：串行/并行，同步/异步，电/光 等。
- **差错检测**：错就丢弃。
- **检测连接状态**：链路是否正常工作。
- **最大传送单元**：数据部分最大长度MTU。
- **网络层地址协商**：知道通信双方的网络层地址。
- **数据压缩协商**：发送时需要把数据进行压缩。

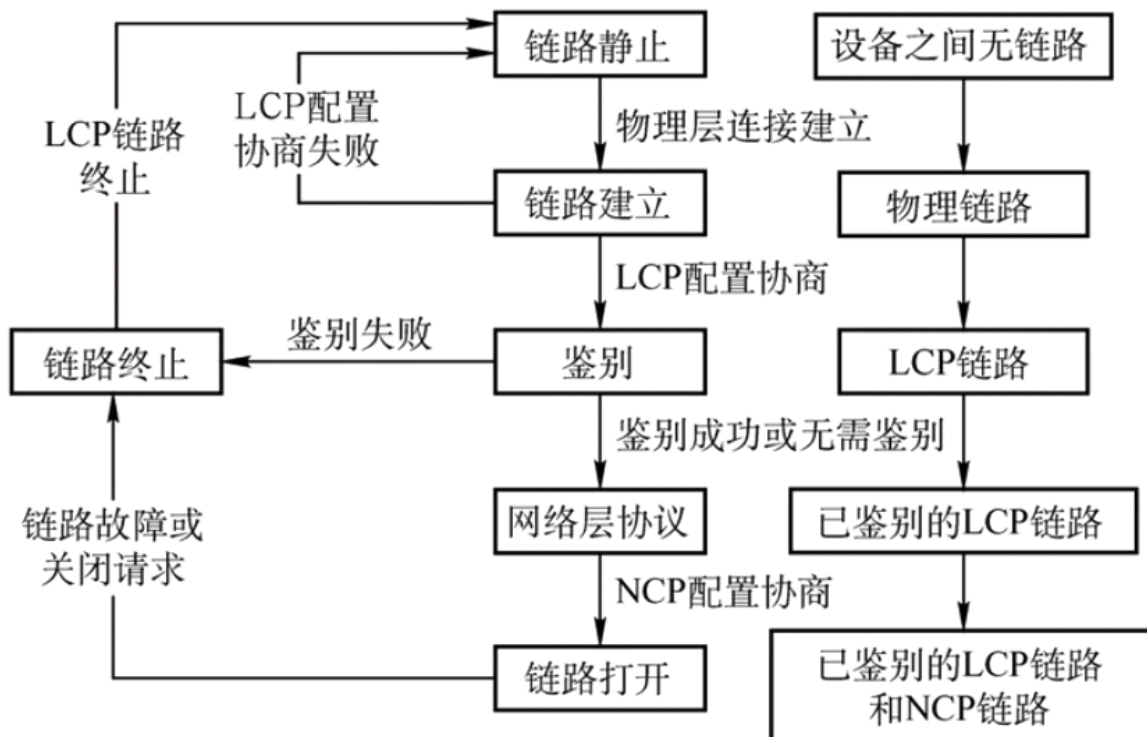
协议无需满足要求

- **纠错**
- **流量控制**
- **序号**
- **不支持多点线路**

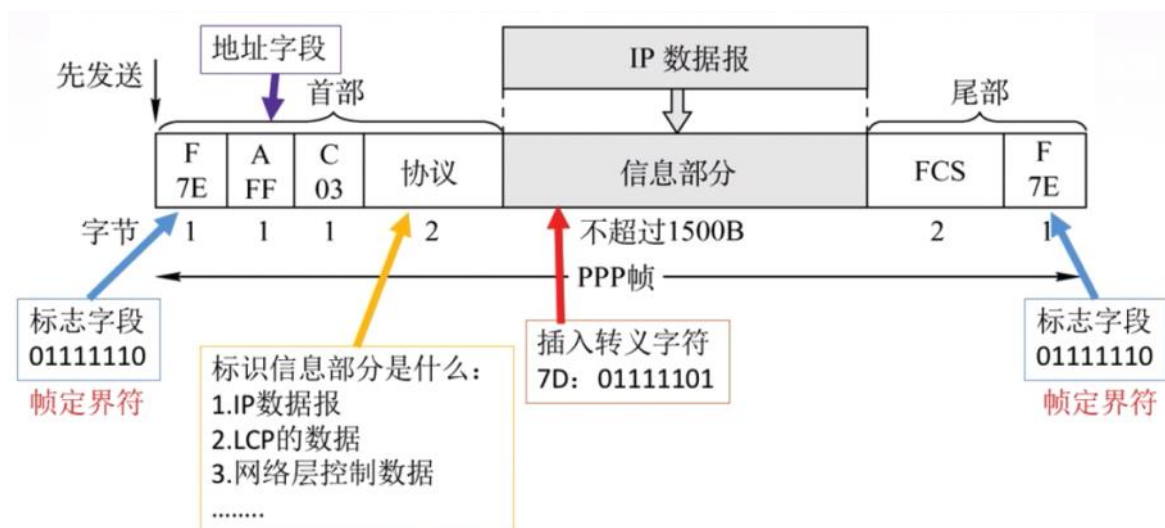
三个组成部分（需实现功能）

1. 一个将IP数据报封装到串行链路（同步串行 / 异步串行）的方法。
2. 链路控制协议LCP：建立并维护数据链路连接，实现身份验证功能。
3. 网络控制协议NCP：PPP可支持多种网络层协议，每个不同的网络层协议都要一个相应的NCP来配置，为网络层协议建立和配置逻辑连接。

协议状态图



协议的帧格式



HDLC协议

概念

高级数据链路控制（High-Level Data Link Control或简称H D LC），是一个在同步网上传输数据、面向比特的数据链路层协议，它是由国际标准化组织（ISO）根据IBM公司的SDLC(SynchronousData Link Control)协议扩展开发而成的。数据报文可透明传输，用于实现透明传输的“0比特插入法”易于硬件实现。采用全双工通信。

所有帧采用CRC检验，对信息帧进行顺序编号，可防止漏收或重份，传输可靠性高。

HDLC的站

主站：主要功能是发送命令（包括数据信息）帧、接收响应帧，并负责对整个链路的控制系统的初启、流程的控制、差错检测或恢复等。

从站：主要功能是接收由主站发来的命令帧，向主站发送响应帧，并且配合主站参与差错恢复等链路控制。

复合站：主要功能是既能发送，又能接收命令帧和响应帧，并且负责整个链路的控制。

三种数据操作方式

1. 正常响应方式
2. 异步平衡方式
3. 异步响应方式

HDLC帧格式

- **信息帧：**(I) 第1位为0，用来传输数据信息，或使用捎带技术对数据进行确认。
- **监督帧：**(S) 10，用于流量控制和差错控制，执行对信息帧的确认、请求重发和请求暂停发送等功能。
- **无编号帧：**(U) 11，用于提供对链路的建立、拆除等多种控制功能。



PPP协议 VS HDLC协议

相同点

- HDLC、PPP都只支持全双工链路。
- 都可以实现透明传输。
- 都可以实现差错检测，但不纠正差错。

不同点

- **帧格式：**PPP面向字节，HDLC面向比特
- **协议字段：**PPP有2个字节，HDLC没有
- **编号与确认机制：**PPP没有，HDLC有
- **传输可靠性：**PPP不可靠，HDLC可靠（HDLC少用主要是避免资源浪费，减低传输延迟）

链路层设备

物理层扩展以太网

光纤

使用光纤

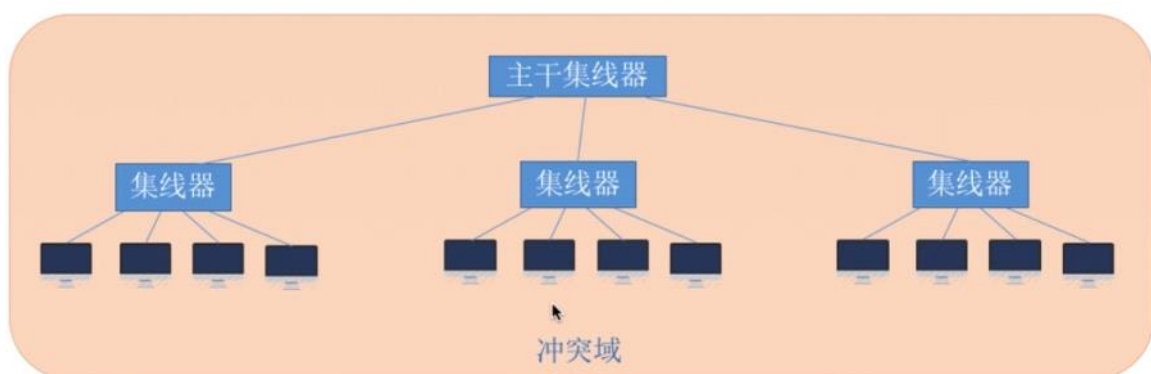


主干集线器

使用集线器

好处：① 跨域通信，② 扩大以太网覆盖的地理范围

缺点：通信效率变低、发生冲突概率变高



优化：使用链路层扩展以太网

链路层扩展以太网

网桥

作用

网桥根据**MAC帧的目的地址**对帧进行转发和过滤。当网桥收到一个帧时，并不向所有接口转发此帧，而是先检查此帧的目的MAC地址，然后再确定将该帧转发到哪一个接口，或者是把它丢弃（即过滤）。



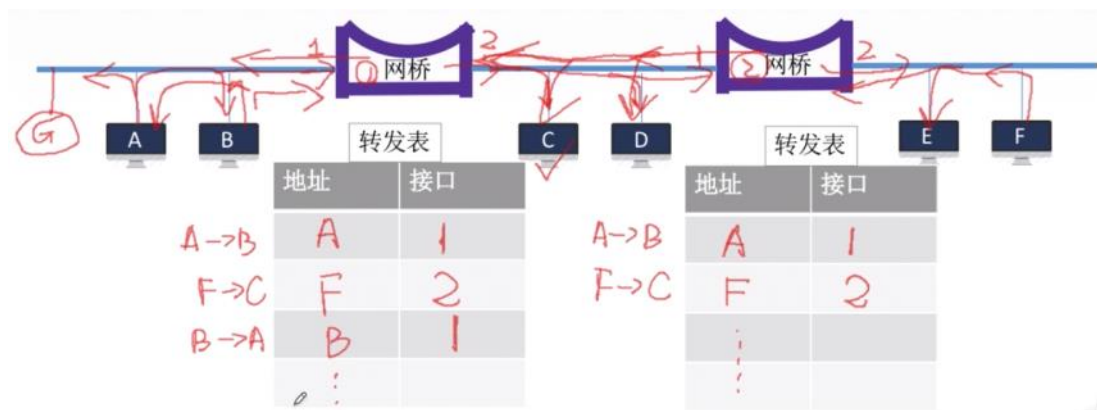
网段：一般指一个**计算机网络**中使用同一物理层设备（传输介质，中继器，集线器等）能够直接通讯的那一部分。

优点

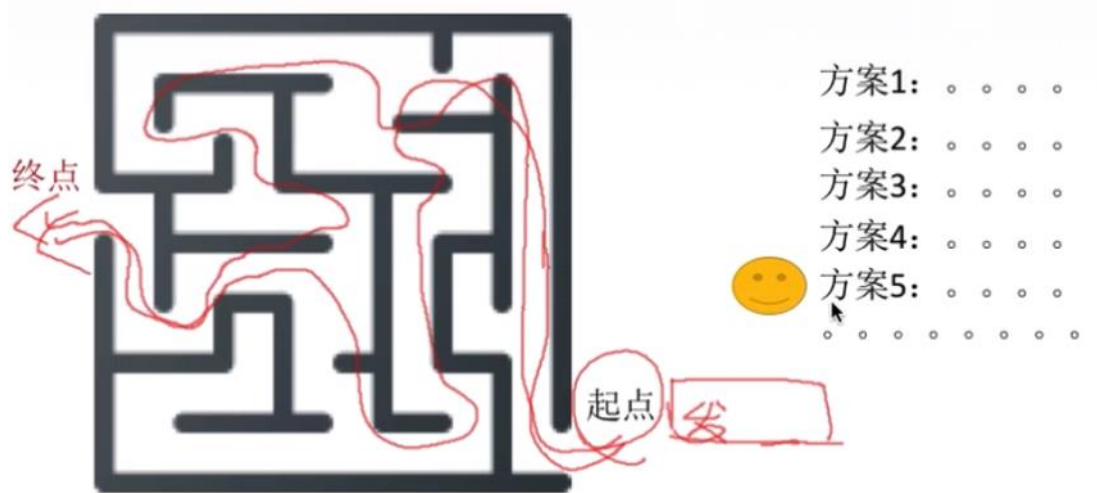
1. 过滤通信量，增大吞吐量。
2. 扩大了物理范围。
3. 提高了可靠性。
4. 可互连不同物理层、不同MAC子层和不同速率的以太网。

分类

- **透明网桥**：“透明”指以太网上的站点并不知道所发送的帧将经过哪几个网桥，是一种即插即用设备——自学习。

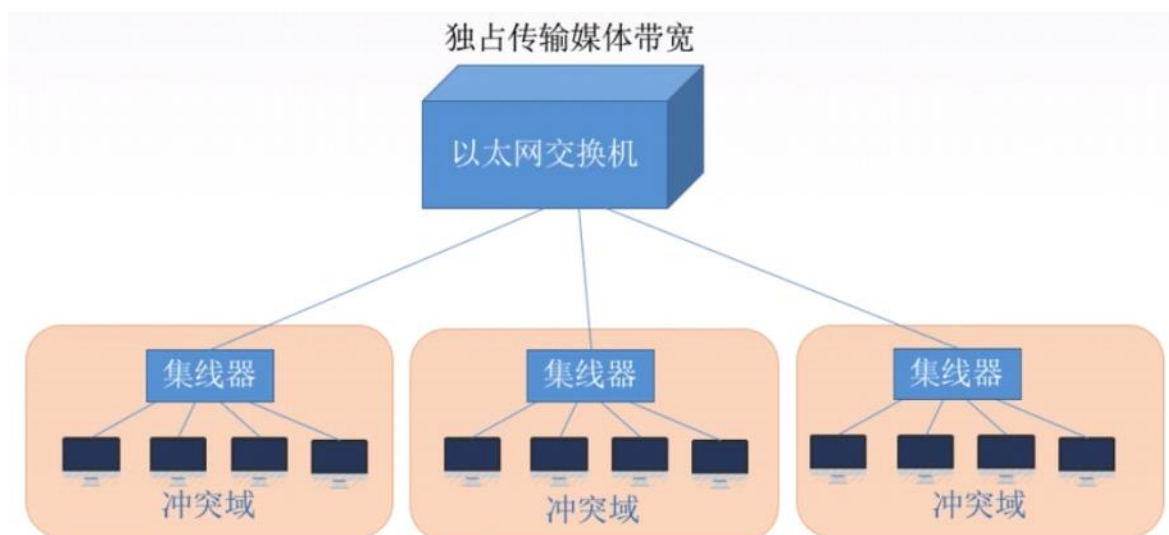


- **源路由网桥：**在发送帧时，把详细的最佳路由信息（路由最少/时间最短）放在帧的首部中。
方法：源站以广播方式向欲通信的目的站发送一个发现帧。



交换机

多接口网桥 - 以太网交换机



两种交换方式

- **直通式交换机**
 - 查完目的地址（6B）就立刻转发。
 - 优点：延迟小
 - 缺点：可靠性低，无法支持具有不同速率的端口的交换。

- 存储转发式交换机

- 将帧放入高速缓存，并检查否正确，正确则转发，错误则丢弃。
- 优点：可靠性高，可以支持具有不同速率的端口的交换。
- 缺点：延迟大

冲突域和广播域

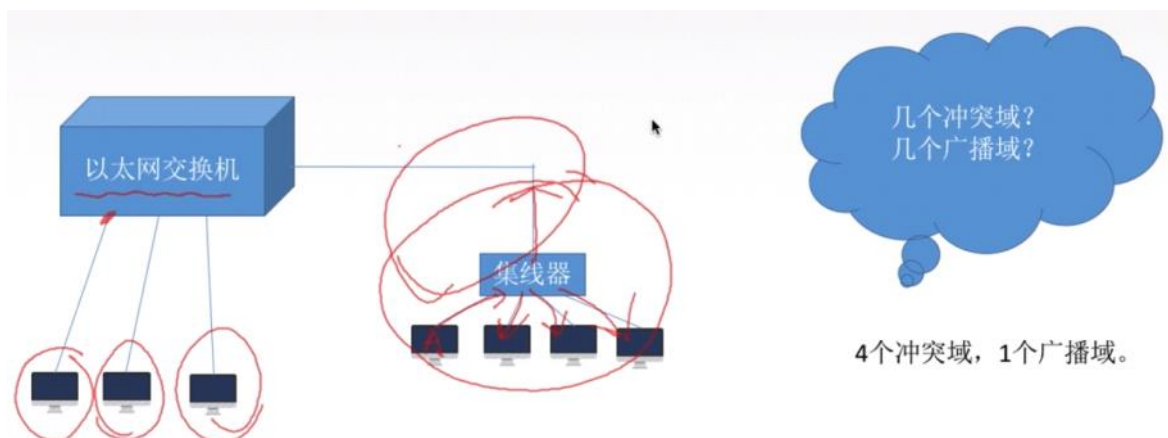
概念

冲突域：在同一个冲突域中的每一个节点都能收到所有被发送的帧。简单的说就是同一时间内只能有一台设备发送信息的范围。

广播域：网络中能接收任一设备发出的广播帧的所有设备的集合。简单的说如果站点发出一个广播信号，所有能接收收到这个信号的设备 范围称为一个广播域。

	能否隔离冲突域	能否隔离广播域
物理层设备【傻瓜】 (中继器、集线器)	×	×
链路层设备【路人】 (网桥、交换机)	√	×
网络层设备【大佬】 (路由器)	√	√

练习



第六章 应用层

星期日, 三月 13, 2022 5:19 下午

已剪辑自: <https://blog.csdn.net/donkey010/article/details/114684828>

6.1 网络应用模型

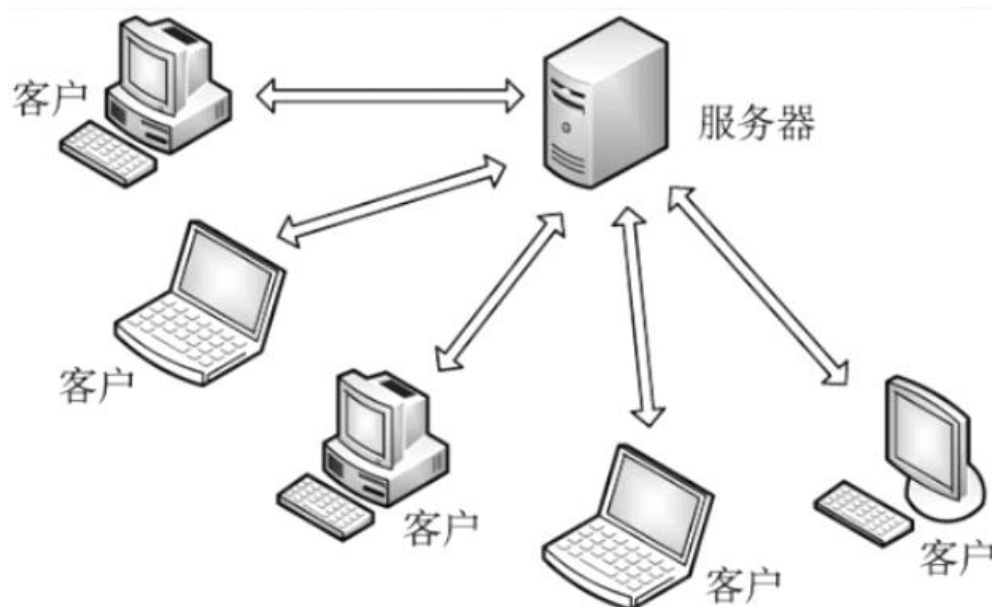
6.1.1 客户/服务器模型

概述

在客户/服务器(Client/Server, C/S)模型中, 有一个总是打开的主机称为服务器, 它服务于许多来自其他称为客户机的主机请求

客户/服务器模型最主要的特征是: 客户是服务请求方, 服务器是服务提供方。

常见的使用客户/服务器模型的应用包括 Web、文件传输 (FTP)、[远程登录](#)和电子邮件等。



服务器: 提供计算服务的设备。

1. 永久提供服务
2. 永久性访问地址/域名

客户机: 请求计算服务的主机。

1. 与服务器通信, 使用服务器提供的服务
2. 间歇性接入网络
3. 可能使用动态IP地址
4. 不与其他客户机直接通信

客户/服务器模型的主要特点

1. 网络中各计算机的地位不平等, 服务器可以通过对用户权限的限制来达到管理客户机的目的, 使它们不能随意存储/删除数据, 或进行其他受限的网络活动。整个网络的管理工作由少数服务器担当, 故网络的管理非常集中和方便。
2. 客户机相互之间不直接通信。例如, 在 Web 应用中两个浏览器并不直接通信。
3. 可扩展性不佳。受服务器硬件和网络带宽的限制, 服务器支持的客户机数有

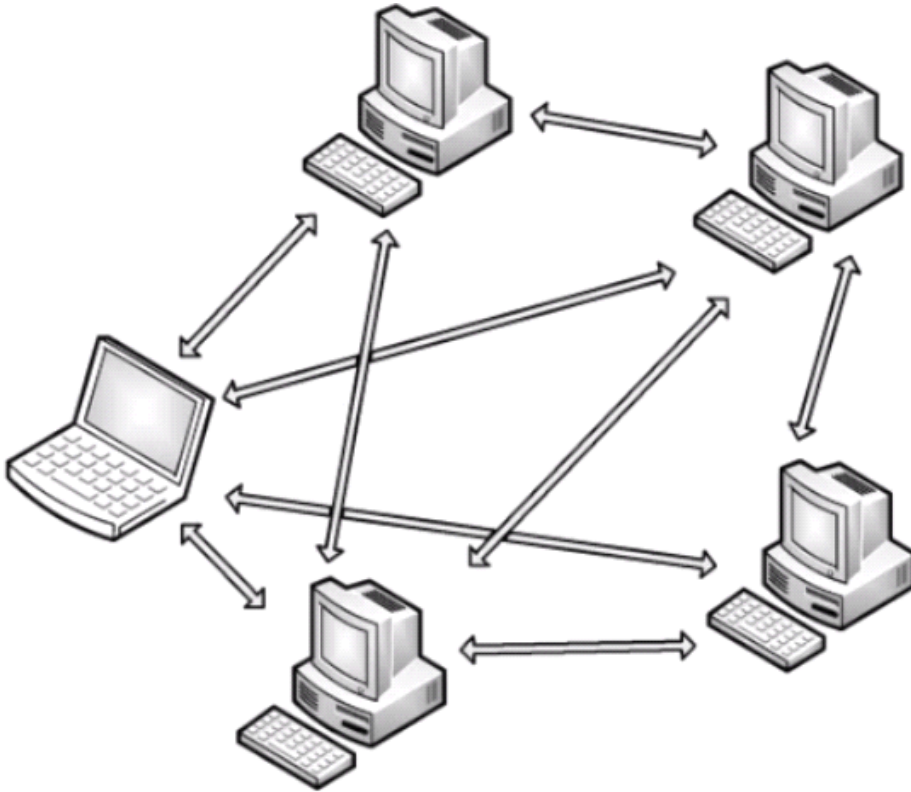
限。

6.1.2 P2P模型

概述

在 P2P模型中，各计算机没有固定的客户和服务划分。相反，任意一对计算机一一称为对等方(Peer)，直接相互通信。

P2P模型从本质上来看仍然是使用客户/服务器方式，每个结点既作为客户访问其他结点的资源，也作为服务器提供资源给其他结点访问。当前比较流行的P2P应用如PPlive、Bittorrent和电驴等。



特点

- 不存在永远在线的服务器
- 每个主机既可以提供服务，也可以请求服务
- 任意端系统/节点之间可以直接通讯
- 节点间歇性接入网络
- 节点可能改变IP地址
- 可扩展性好
- 网络健壮性强

P2P & C/S 比较

优点

1. 减轻了服务器的计算压力，消除了对某个服务器的完全依赖，可以将任务分配到各个结点上，因此大大提高了系统效率和资源利用率。
 - 例如，播放流媒体时对服务器的压力过大，而通过P2P模型，可以利用大量的客户机来提供服务。
2. 多个客户机之间可以直接共享文档。
3. 可扩展性好，传统服务器有响应和带宽的限制，因此只能接受一定数量的请求。
4. 网络健壮性强，单个结点的失效也不会影响其他部分的结点。

缺点：在获取服务的同时，还要给其他结点提供服务，因此会占用较多的内存，影响整机速度。

6.2 域名解析系统DNS

概述

域名系统 DNS (Domain Name System) 是因特网使用的命名系统，用来把便于人们记忆的含有特定含义的主机名 (如. www.cskaoyan.com) 转换为便于机器处理的IP地址。相对于IP地址，人们更喜欢使用具有特定含义的字符串来标识因特网上的计算机。DNS 系统采用客户/服务器模型，其协议运行在UDP 之上，使用 53 号端口。

DNS分为3个部分：层次域名空间、域名服务器和解析器。

6.2.1 层次域名空间

概述

因特网采用层次树状结构的命名方法。采用这种命名方法，任何一个连接在因特网上的主机或路由器，都有一个唯一的层次结构的名称，即域名 (Domain Name)。
“域” (Domain) 是名字空间中一个可被管理的划分。

每一个域名都是由标号序列组成，而各标号之间用点 (“.”) 隔开。



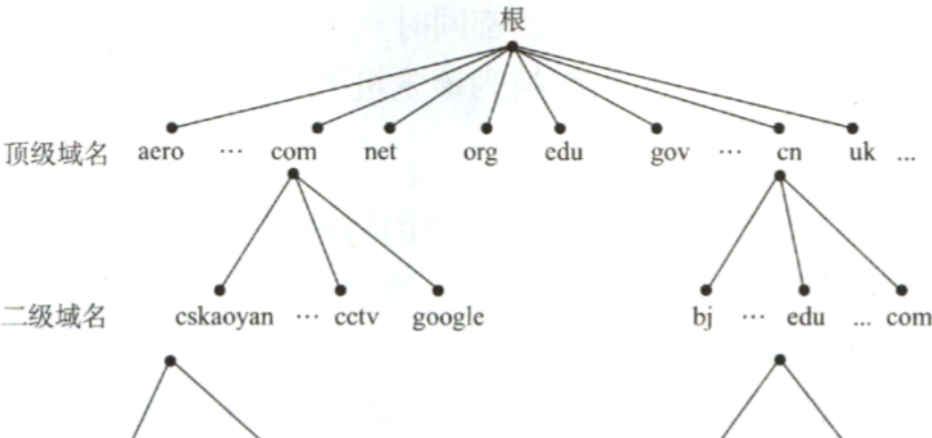
域名中使用标号注意点

- 1. 标号中的英文不区分大小写。
- 2. 标号中除连字符 (-) 外不能使用其他的标点符号。
- 3. 每一个标号不超过63个字符，多标号组成的完整域名最长不超过255个字符。
- 4. 级别最低的域名写在最左边，而级别最高的顶级域名 写在最右边。

顶级域名

顶级域名 (Top Level Domain, TLD) 有以下三大类

- 1. 国家顶级域名 nTLD。国家和某些地区的域名，如 “.cn” 表示中国， “.us” 表示美国， “.uk” 表示英国， “.hk” 表示中国香港特区。
- 2. 通用顶级域名 gTLD。常见的有 “.com” (公司企业)、 “.net” (网络服务机构)、 “.org” (非营利性的组织) 和 “.gov” (美国的政府部门) 等。
- 3. 基础结构域名。这种顶级域名只有一个，即arpa，用于反向域名解析， 因此又称为反向域名。



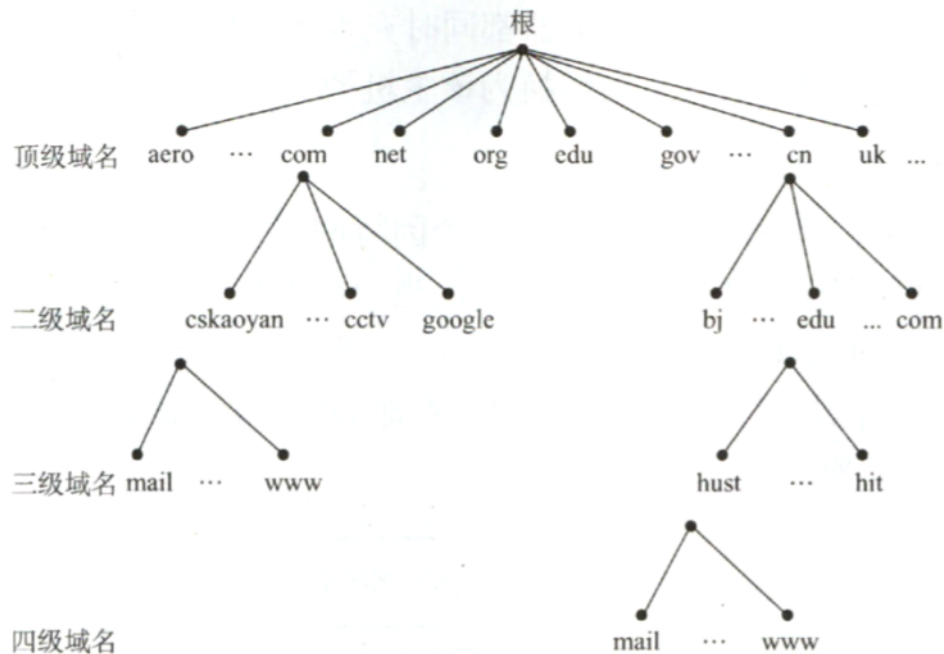


图 6-4 域名空间的树状结构

国家顶级域名下注册的二级域名均由该国家自行确定。如类别域名（ac, corn, edu, gov, mii, net, org 等），行政区域名（用于我国各省、自治区、直辖市 bj, js）等。

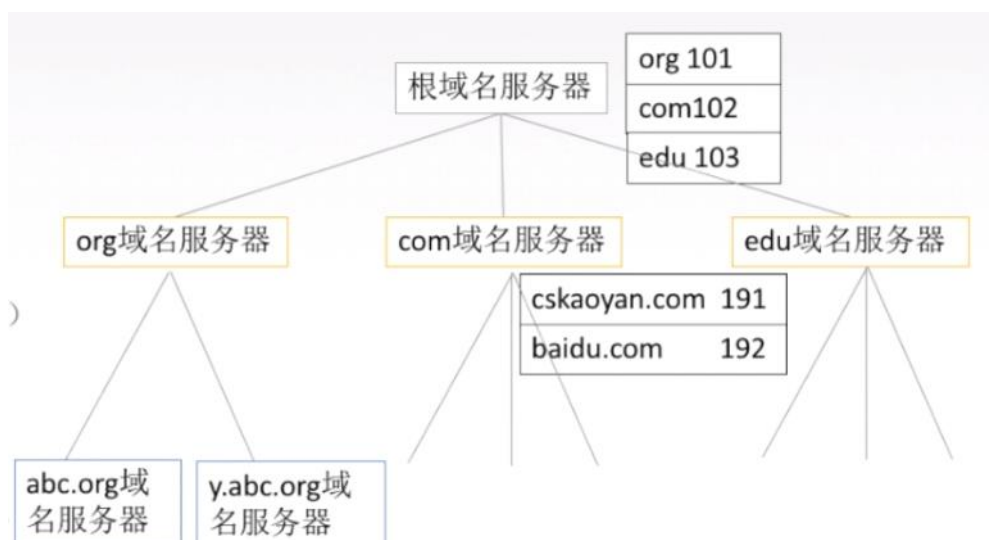
在域名系统中，每个域分别由不同的组织进行管理。每个组织都可以将它的域再分成一定数目的子域，并将这些子域委托给其他组织去管理。例如，管理CN域的中国将 EDU.CN 子域授权给中国教育和科研计算机网 CERNET 来管理。

6.2.2 域名服务器

概述

因特网的域名系统被设计成一个联机分布式的数据库系统，并采用客户/服务器模型。域名到IP地址的解析是由运行在域名服务器上的程序完成的，每一个域名服务器不但能够进行一些域名到IP地址的解析，而且还必须具有连向其他域名服务器的信息。

DNS 使用了大量的域名服务器，它们以层次方式组织。没有一台域名服务器具有因特网上所有主机的映射，相反，该映射分布在所有的 DNS 服务器上。采用分布式设计的DNS系统，是一个在因特网上实现分布式数据库的精彩范例。



域名服务器类型

1. 根域名服务器

- 根域名服务器是最高层次的域名服务器，所有的根域名服务器都知道所有的顶级域名服务器的IP地址。根域名服务器用来管辖顶级域（如.com），通常它并不直接把待查询的域名直接转换成IP地址，而是告诉本地域名服务器下一步应当找哪一个顶级域名服务器进行查询。

2. 顶级域名服务器

- 这些域名服务器负责管理在该顶级域名服务器注册的所有二级域名。当收到DNS查询请求时，就给出相应的回答。

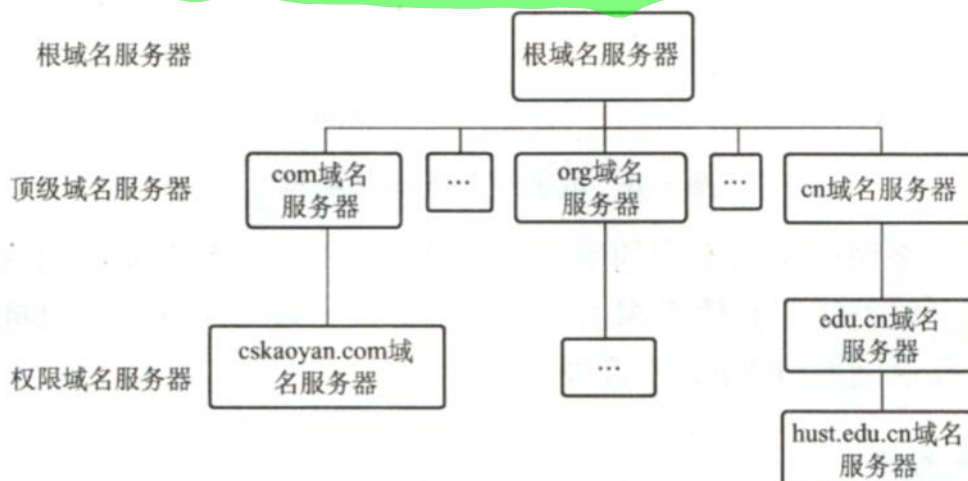
3. 授权域名服务器（权限域名服务器）

- 每一个主机都必须在授权域名服务器处登记。授权域名服务器总是能够将其管辖的主机名转换为该主机的时址。

4. 本地域名服务器

- 每一个因特网服务提供者ISP，或一个大学，甚至一个大学里的系，都可以拥有一个本地域名服务器。当一个主机发出DNS查询请求时，这个查询请求报文就发送给该主机的本地域名服务器。

DNS域名服务器层次结构



6.2.3 域名解析过程

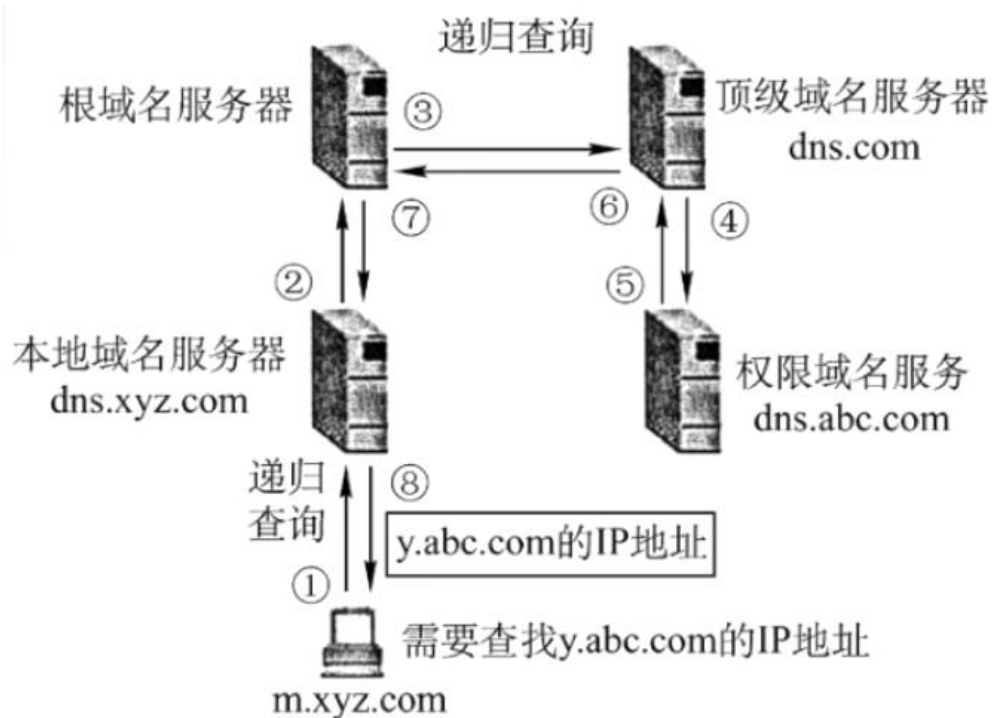
概述

域名解析是指把域名映射成为IP地址或把IP地址映射成为域名的过程。前者称为**正向解析**，后者称为**反向解析**。当客户端需要域名解析时，通过本机的DNS客户端构造一个DNS请求报文，以UDP数据报方式发往本地域名服务器。

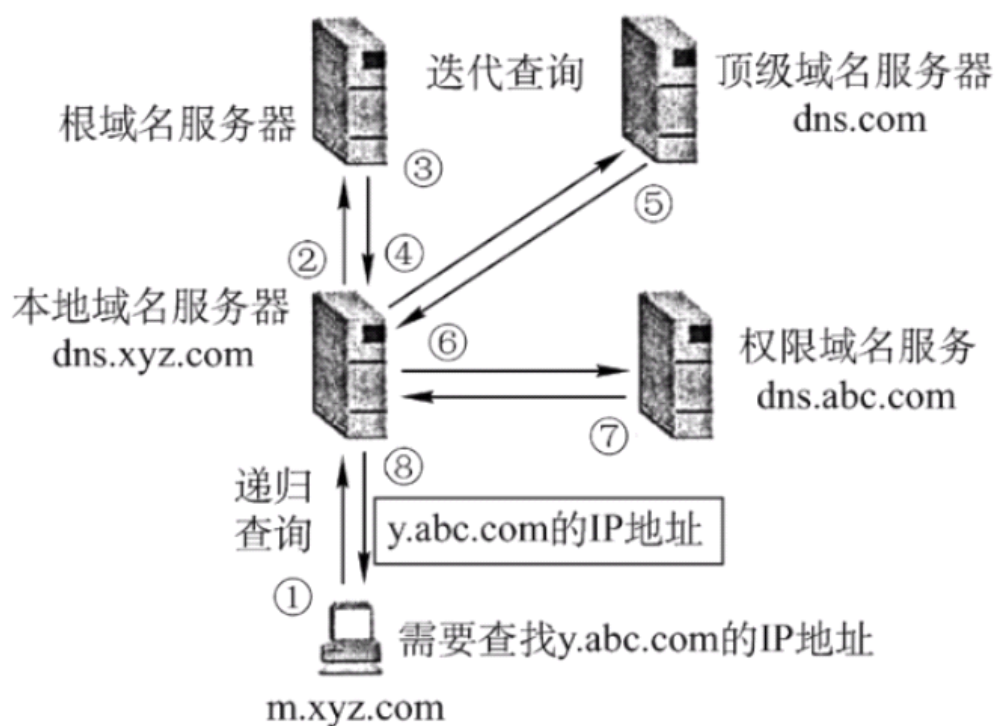
域名解析有两种方式：递归查询和递归与迭代相结合的查询。

两种域名解析方式工作原理

1、**递归查询（比较少用）**：主机向本地域名服务器的查询采用的是递归查询。



2、递归与迭代查询相结合方式：本地域名服务器向根域名服务器的查询采用迭代查询。



为了提高DNS的查询效率，并减少因特网上的DNS查询报文数量，在域名服务器中广泛地使用了高速缓存。与此同时，因为主机名和IP地址之间的映射不是永久的，所以DNS服务器将在一段时间后丢弃高速缓存中的信息。

6.3 文件传输协议FTP

6.3.1 FTP的工作原理

概述

文件传输协议FTP (File Transfer Protocol) 是因特网上使用得最广泛的文件传

送协议。FTP提供交互式的访问，允许客户指明文件的类型与格式，并允许文件具有存取权限。它屏蔽了各计算机系统的细节，因而适合于在异构网络中任意计算机之间传送文件。

提供功能

- 1. 提供不同种类主机系统（硬、软件体系等都可以不同）之间的文件传输能力。
- 2. 以用户权限管理的方式提供用户对远程 FTP 服务器上的文件管理能力。
- 3. 以匿名FTP的方式提供公用文件共享的能力。

服务组成

FTP采用客户/服务器的工作方式，它使用TCP可靠的传输服务。一个FTP服务器进程可同时为多个客户进程提供服务。

FTP 的服务器进程由两大部分组成：一个主进程，负责接收新的请求；若干个从属进程，负责处理单个请求。

工作步骤

- 1. 打开熟知端口 21（控制端口），使客户进程能够连接上。
- 2. 等待客户进程发连接请求。
- 3. 启动从属进程来处理客户进程发来的请求。主进程与从属进程并发执行，从属进程对客户进程的请求处理完毕后即终止。
- 4. 回到等待状态，继续接收其他客户进程 的请求。

FTP 服务器必须在整个会话期间保留用户的状态信息。特别是服务器必须把指定的用户账户与控制连接联系起来，服务器必须追踪用户在远程目录树上的当前位置。

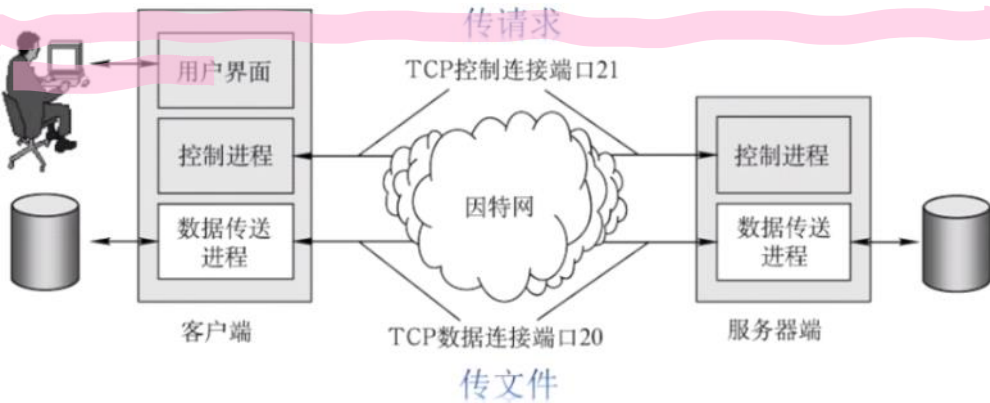
6.3.2 控制连接与数据连接

工作原理

互连网中有很大一部分FTP服务器被称为“匿名”（Anonymous）FTP服务器。这类服务器的目的是向公众提供文件拷贝服务，不要求用户事先在该服务器进行登记注册，也不用取得FTP服务器的授权。 Anonymous（匿名文件传输）能够使用户与远程主机建立连接并以匿名身份从远程主机上拷贝文件，而不必是该远程主机的注册用户。用户使用特殊的用户名“anonymous”登陆FTP服务，就可访问远程主机上公开的文件。

概述

FTP 在工作时使用两个并行的TCP连接，一个是控制连接(端口号21)，一个是数据连接(端口号20)。使用两个不同的端口号可使协议更加简单和更容易实现。



控制连接

服务器监听在21号端口，等待客户连接，建立在这个端口上的连接称为控制连接，

控制连接用来传输控制信息(如连接请求、传送请求等)。并且控制信息都是以7位ASCII格式传送的。

FTP 客户发出的传送请求，通过控制连接发送给服务器端的控制进程，但控制连接并不用来传送文件。在传输文件时还可以使用控制连接（例如，客户在传输中途发一个中止传输的命令），因此控制连接在整个会话期间一直保持打开状态。

数据连接

服务器端的控制进程在接收到FTP客户发送来的文件传输请求后就创建“数据传送进程”和“数据连接”。数据连接用来连接客户端和服务器的数据传送进程，数据传送进程实际完成文件的传送，在传送完毕后关闭“数据传送连接”并结束运行。

因为FTP使用了一个分离的控制连接，所以也称FTP的控制信息是带外（Out-of-band）传送的。

特点

- 控制连接始终保持
- 数据连接保持一会
- 是否使用TCP 20端口建立数据连接与传输模式有关
 - 主动方式使用下CP 20端口
 - 被动方式由服务器和客户端自行协商 决定（端口>1024）

FTP传输模式

文本模式：ASCII模式，以文本序列传输数据；

二进制模式：Binary模式，以二进制序列传输数据。

6.4 电子邮件

6.4.1 电子邮件系统的组成结构

概述

电子邮件是一种异步通信方式，通信时 不需要双方同时在场 。 电子邮件把邮件发送到收件人使用的邮件服务器，并放在其中的收件人邮 箱中，收件人可以随时上网到自己使用的邮件服务器进行读取。

组成构件

一个电子邮件系统应具有三个最主要的组成构件：用户代理(MserAgent)、 邮件服务器和电子邮件使用的协议， 如 SMTP、POP3（或IMAP）等。



1、用户代理UA

- 用户与电子邮件系统的接口，用户代理使用户能够通过一个很友好的接口来发送和接收邮件，用户代理至少应当具有撰写、显示和邮件处理的功能。
- 通常情况下，用户代理就是一个运行在PC上的程序，常见的有

Outlook、Foxmail 和 Thunderbird 等。

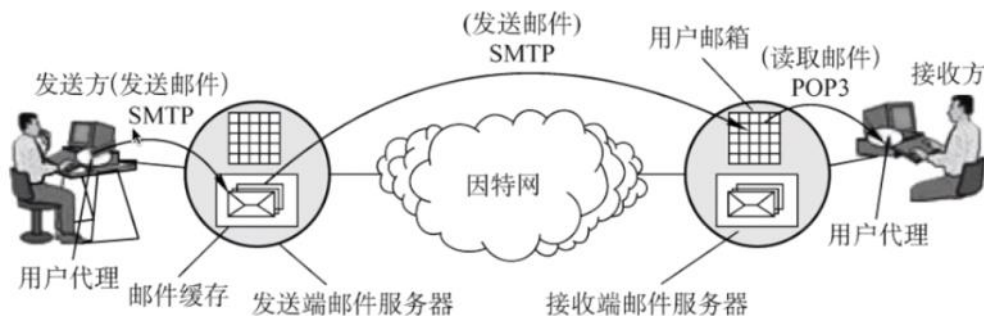
2、邮件服务器

- 组成了电子邮件系统的核心。邮件服务器的功能是发送和接收邮件，同时还要向发信人报告邮件传送的情况（已交付、被拒绝、丢失等）。
- 邮件服务器采用客户/服务器方式工作，但它能够同时充当客户和服务器。

3、邮件发送协议和读取协议

- **邮件发送协议：**用于用户代理向邮件服务器发送邮件或在邮件服务器之间发送邮件，通常使用的是SMTP。
- **邮件读取协议：**用于用户代理从邮件服务器读取邮件，如POP3（邮局协议的第3个版本）。

电子邮件发送接收过程



电子邮件的收发过程：

1. 发信人调用用户代理来撰写和编辑要发送的邮件。用户代理用SMTP协议把邮件传送给发送方邮件服务器。
2. 发送方邮件服务器将邮件放入邮件缓存队列中，等待发送。
3. 运行在发送方邮件服务器的SMTP客户进程，发现在邮件缓存中有待发送的邮件，就向运行在接收方邮件服务器的SMTP服务器进程发起建立TCP连接。
4. TCP连接建立后，SMTP客户进程开始向远程的SMTP服务器进程发送邮件。当所有的待发送邮件发完了，SMTP就关闭所建立的TCP连接。
5. 运行在接收方邮件服务器中的SMTP服务器进程收到邮件后，将邮件放入收信人的用户邮箱中，等待收信人在方便时进行读取。
6. 收信人在打算收信时，调用用户代理，使用POP3（或IMAP）协议将自己的邮件从接收方邮件服务器的用户邮箱中的取回（如果邮箱中有来信的话）。

6.4.2 电子邮件格式与 MIME

电子邮件格式

一个电子邮件分为信封和内容两大部分。**邮件内容又分为首部 and 主体两部分。**[RFC 822]规定了邮件的首部格式，而邮件的主体部分则让用户自由撰写。

邮件内容的首部包含一些首部行，每个首部行由一个关键字后跟冒号再后跟值组成。有些关键字是必需的，有些则是可选的。最重要的关键字是：To和Subject。

- **To**
 - 这是必需的关键字。后面填入一个或多个收件人的电子邮件地址。
 - 规定格式：收件人邮箱名@邮箱所在主机的域名。（收信人邮箱名，即用户名，在对应邮件服务器上必须是唯一的，这也就保证了这个邮件地址在整个因特网上是唯一的。）
- **Subject**
 - 可选的关键字。它是邮件的主题，反映了邮件的主要内容。

```

From:hoopdog@hust.edu.cn
To:abc@cskaoyan.com
Subject:Say hello to Internet
blahblah...
...

```

} 首部

} 主体

MIME (多用途网际邮件扩充)

由于SMTP只能传送一定长度的ASCII码,许多其他非英语国家的文字(如中文、俄文,甚至带重音符号的法文或德文)就无法传送,且无法传送可执行文件及其他二进制对象,因此提出了多用途网络邮件扩充(Multipurpose Internet Mail Extensions, MIME)。

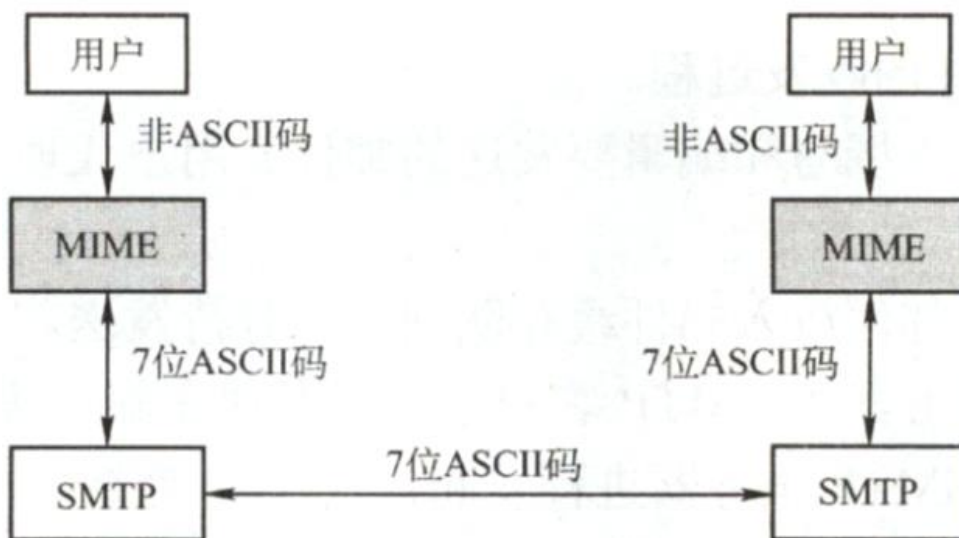


图 6-10 SMTP 与 MIME 的关系

MIME 主要包括内容:

1. 5个新的邮件首部字段,包括MIME版本、内容描述、内容标识、内容传送编码和内容类型。
2. 定义了许多邮件内容的格式,对多媒体电子邮件的表示方法进行了标准化。
3. 定义了传送编码,可对任何内容格式进行转换,而不会被邮件系统改变。

通用因特网邮件扩充MIME,使电子邮件系统可以支持声音、图像、视频、多种国家语言等等,使得传输内容丰富多彩。

6.4.3 SMTP协议&POP3协议

SMTP协议

概述

简单邮件传输协议 (Simple Mail Transfer Protocol, SMTP) 是一种提供可靠且有效的电子邮件传输的协议,控制两个相互通信的SMTP进程交换信息。由于SMTP使

用客户/服务器方式， 因此负责发送邮件的SMTP进程就是SMTP客户， 而负责接收邮件的SMTP进程就是SMTP服务器。**SMTP用的是TCP连接， 端口号25。**

通信三个阶段

1、连接建立

1. 发件人的邮件发送到发送方邮件服务器的邮件缓存后，SMTP客户就每隔一定时间对邮件缓存扫描一次。
2. 如发现有邮件，就使用SMTP的熟知端口号（25）与接收方邮件服务器的SMTP服务器建立TCP连接。
3. 在连接建立后，接收方SMTP服务器要发出 “220 Service ready”（服务就绪）。
4. 然后SMTP客户向SMTP服务器发送HELO命令，附上发送方的主机名。



SMTP不使用中间的邮件服务器。TCP连接总是在发送方和接收方这两个邮件服务器之间直接建立，而不管它们相隔有多远。当接收方的邮件服务器因故障暂时不能建立连接时，发送方的邮件服务器只能等待 一段时间后再次尝试连接。

2、邮件发送

1. 邮件的传送从MAIL命令开始，MAIL命令后面有发件人的地址。
2. 若SMTP服务器已准备好接收邮件， 则回答 “250 OK”。
3. 接着SMTP客户端发送一个或多个RCPT（收件人 recipient 的缩写）命令， 格式为 RCPT TO: <收件人地址>。
4. 每发送一个RCPT命令，都应当有相应的信息从SMTP服务器返回，如 “250 OK “ 或 “550 No such user here”（无此用户）。
5. 获得OK的回答之后，客户端就使用DATA命令，表示要开始传输邮件的内容了。
6. SMTP服务器回复信息是 ” 354 Start mail input; end with . ”（是“回车换行”的意思，两个回车，中间一个点表示邮件内容的结束）。

A: MAIL FROM: <wangdao@163.com>

B: 250 OK / B: 451 (452、500...) SMTP服务器是否已经准备好接收邮件

A: RCPT TO: <mooc@163.com> 可以有多个RCPT命令

B: 250 OK / B: 550 No such user here SMTP服务器确定是否有这个用户

A: DATA 要开始传输邮件的内容了

B: 354 start mail input; end with <CR><LF>.<CR><LF> SMTP服务器同意传输

A: Date.... 开始传输邮件内容

B: 250 OK 接收结束

RCPT命令的作用：弄清接收方系统是否已做好接收邮件的准备，然后才发送邮件。不至于发送了很长的邮件以后才知道是地址错误，以避免浪费通信资源。

3、连接释放

- 邮件发送完毕后， SMTP客户应发送QUIT命令。SMTP服务器返回的信息是 “221”（服务关闭），表示SMTP同意释放TCP连接。邮件传送的全部过程就结束了。

POP3协议

概述

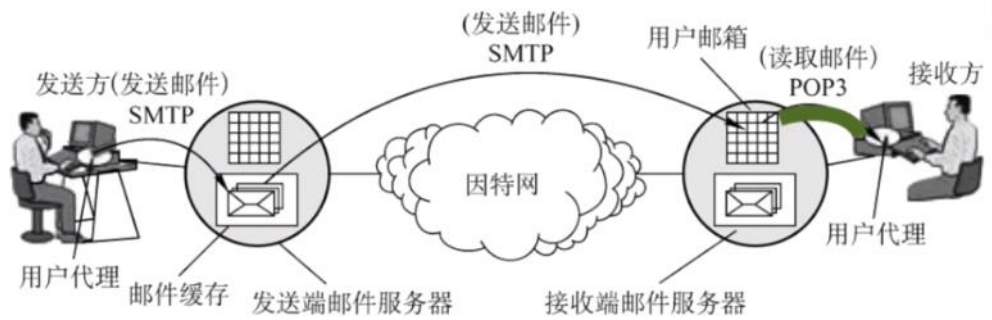
邮局协议（Post Office Protocol, POP）是一个非常简单、但功能有限的邮件读取协议，现在使用的是它的第3个版本POP3。POP3采用的是“拉”（Pull）的通信方式，当用户读取邮件时，用户代理向邮件服务器发出请求，“拉”取用户邮箱中

的邮件。

POP 也使用客户/服务器的工作方式，在传输层使用TCP协议，端口号110。在接收方计算机中的用户代理必须运行POP客户程序，而在接收方的邮件服务器上则运行POP服务器程序。

POP有两种工作方式：“下载并保留” 和 “下载并删除模式”。

- “下载并保留” 模式：用户从邮件服务器上读取了邮件之后，邮件依然会保存在邮件服务器上，用户下次可以再次从服务器上读取该邮件。
- “下载并删除” 模式：邮件一旦被读取之后，就被从邮件服务器上删除了，用户不能再次从服务器上读取了。

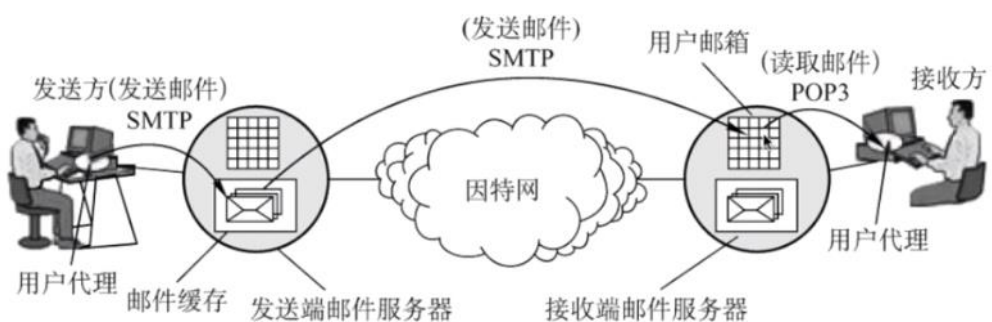


IMAP

概述

国际报文存取协议IMAP比POP协议复杂。当用户PC上的IMAP客户程序打开IMAP服务器的邮箱时，用户可以看到邮箱的首部，若用户需要打开某个邮件，该邮件才上传到用户的计算机上。

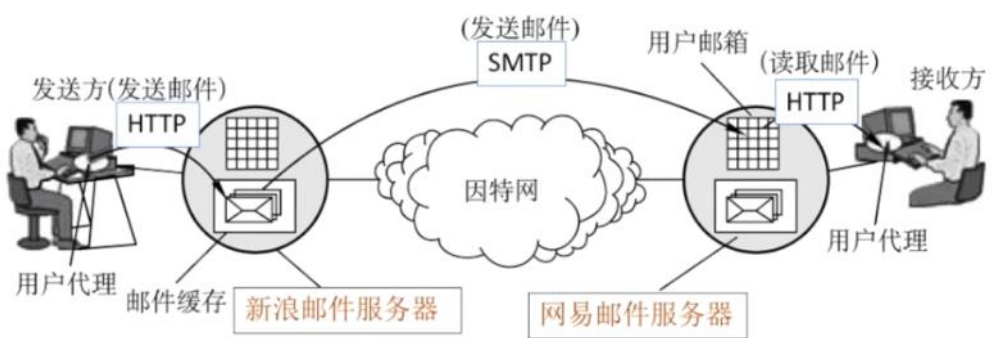
IMAP可以让用户在不同的地方使用不同的计算机随时上网阅读处理邮件，还允许只读取邮件中的某一个部分（先看正文，有WiFi的时候再下载附件）。



基于万维网的电子邮件

概述

如Hotmail、Gmail等。这种电子邮件的特点是用户浏览器与Hotmail或Gmail的邮件服务器之间的发送或接收邮件是使用HTTP协议，而仅在不同邮件服务器之间传送邮件才使用SMTP协议。



6.5 万维网 WWW

6.5.1 WWW 的概念与组成结构

概述

万维网WWW (World Wide Web) 是一个资料空间。在这个空间中：一样有用的事物，称为一样“资源”；并且由一个全域“统一资源定位符”(URL)标识。这些资源通过超文本传输协议(HTTP)传送给使用者，而后者通过点击链接来获取资源。

万维网内核部分构成标准

- 1、**统一资源定位符 (URL)**：负责标识万维网上的各种文档，并使每个文档在整个万维网的范围内具有唯一的标识符URL。
- 2、**超文本传输协议 (HTTP)**：它是一个应用层协议，使用TCP连接进行可靠的传输，是万维网客户程序和服务器程序之间交互所必须严格遵守的协议。
- 3、**超文本标记语言 (HTML)**：是一种文档结构的标记语言，使用一些约定的标记对页面上的各种信息(包括文字、声音、图像、视频等)、格式进行描述。

统一资源定位符

统一资源定位符 (Uniform Resource Locator, URL) 是对可以从因特网上得到的资源的位置和访问方法的一种简洁的表示。URL相当于一个文件名在网络范围的扩展。

统一资源定位符URL **唯一标识** **资源 (文字、视频、音频...)**

URL的一般形式: <协议>://<主机>:<端口>/<路径>

- 协议：常见的有 http、ftp等。
- 主机：存放资源的主机在因特网中的域名，也可以是IP地址。
- 端口 & 路径：有时可以省略。

在URL中不区分大小写。

万维网工作模式

万维网以客户/服务器方式工作。浏览器是在用户计算机上的万维网客户程序，而万维网文档所驻留的计算机则运行服务器程序，这个计算机称万维网服务器。客户程序向服务器程序发出请求，服务器程序向客户程序送回客户所要的文档。

万维网工作流程

1. Web用户使用浏览器 (指定URL) 与Web服务器建立连接，并发送浏览请求。
2. Web服务器把URL转换为文件路径，并返回信息给Web浏览器。
3. 通信完成，关闭连接。

万维网是无数个网络站点和网页的集合，它们在一起构成了因特网最主要的部分 (因特网也包括电子邮件、Msenet以及新闻组)。

6.5.2 超文本传输协议 HTTP

概述

HTTP 协议定义了浏览器 (万维网客户进程) 怎样向万维网服务器请求万维网文档，以及服务器怎样把文档传送给浏览器。

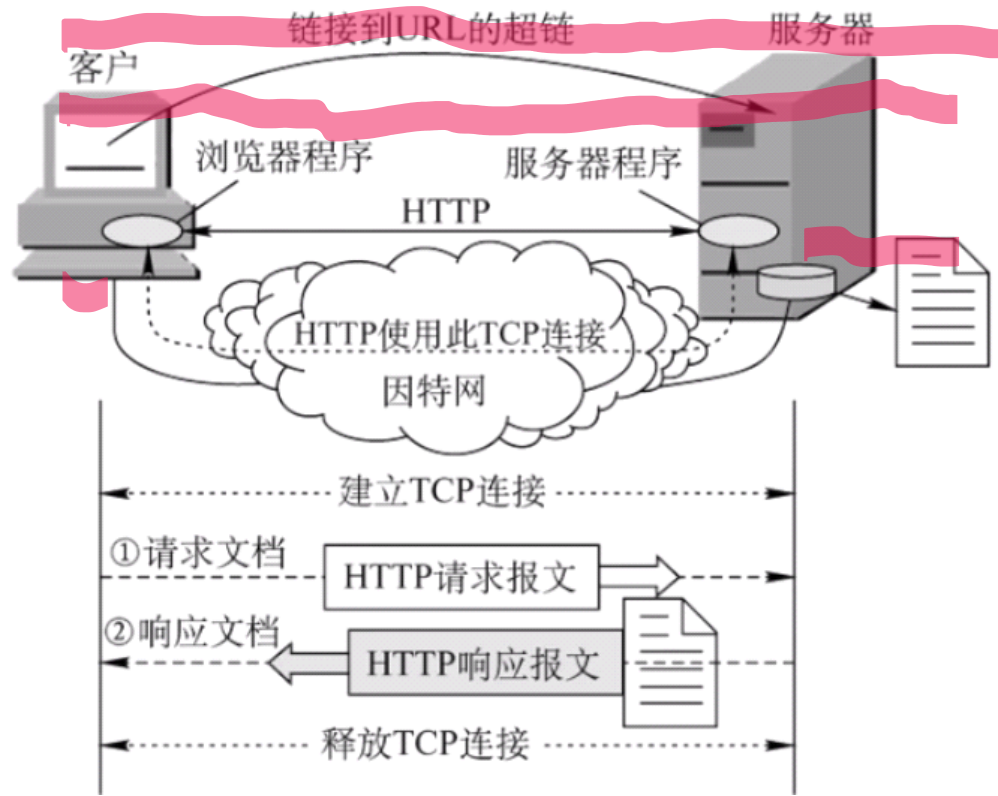
从层次的角度看，HTTP是面向事务的 (Transaction - oriented) 应用层协议，它规定了在浏览器和服务器的请求和响应的格式和规则，它是万维网上能够可靠

地交换文件(包括文本、声音、图像等各种多媒体文件)的重要基础。

HTTP的操作过程

- 1. 每个万维网站点都有一个服务器进程， 它不断地监听TCP的端口80（默认），当监听到连接请求后便与浏览器建立连接。
- 2. TCP连接建立后，浏览器就向服务器发送请求获取某一 Web页面的HTTP请求。
- 3. 服务器收到HTTP请求后， 将构建所请求的Web页必需的信息， 并通过HTTP响应返回给浏览器。
- 4. 浏览器再将信息进行解释， 然后将Web页显示给用户。最后， TCP连接释放。

万维网的工作过程



在浏览器和服务端之间的请求和响应的交互，必须按照规定的格式和遵循一定的规则，这些格式和规则就是HTTP。

HTTP有两类报文：请求报文（从Web客户端向Web服务器发送服务请求）和响应报文（从Web服务器对Web客户端请求的回答）。

HTTP协议特点

1、HTTP协议是无状态的。

- HTTP的无状态特性简化了服务器的设计，使服务器更容易支持大量并发的HTTP请求。
- 在实际应用中，通常使用Cookie加数据库的方式来跟踪用户的活动（如记录用户最近浏览的商品等）。
- Cookie是存储在用户主机中的文本文件，用于Web服务识别用户。

2、HTTP采用TCP作为运输层协议，保证了数据的可靠传输。

3、HTTP 既可以使用非持久连接，也可以使用持久连接。

- 对于非持久连接，每一个网页元素对象的传输都需要单独建立一个TCP连接。同时，请求一个万维网文档所需的时间是该文档的传输时间加上两倍往返时间RTT（一个RTT用于TCP连接，另一个RTT用于请求和接收文档）。



HTML 超文本标记语言
URL 统一资源定位器

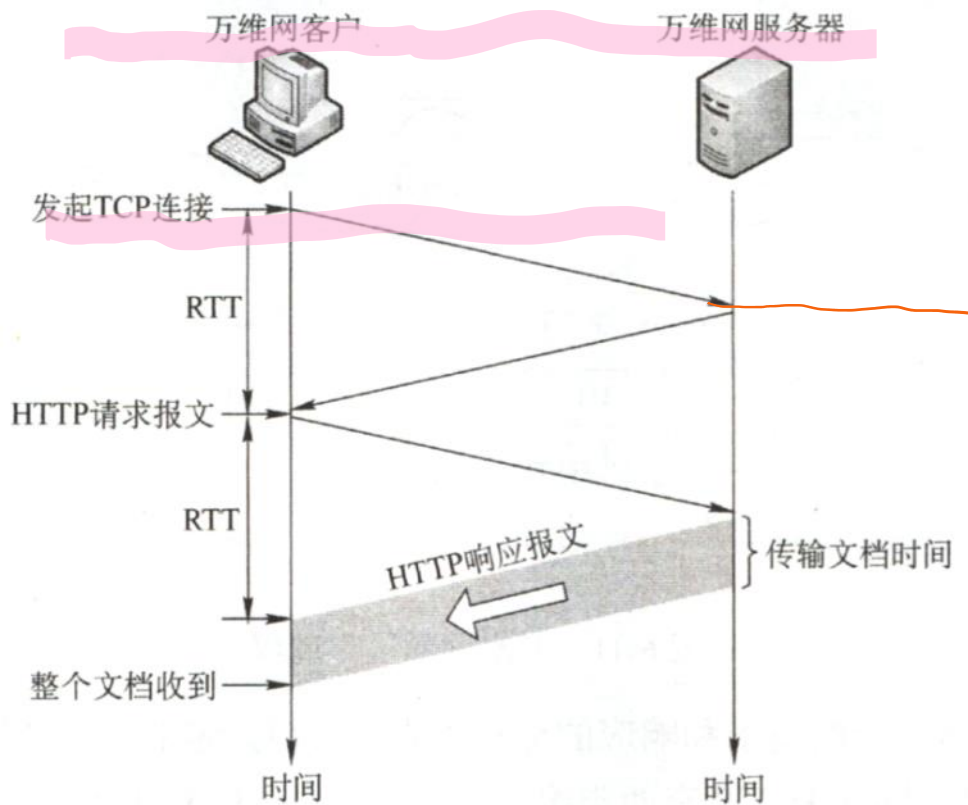


图 6-12 请求一个万维网文档所需的时间

- 对于持久连接，万维网服务器在发送响应后仍然保持这条连接，使同一个客户和服务端可以继续在这条连接上传送后续的HTTP请求和响应报文。持久连接又分为非流水线和流水线两种方式。

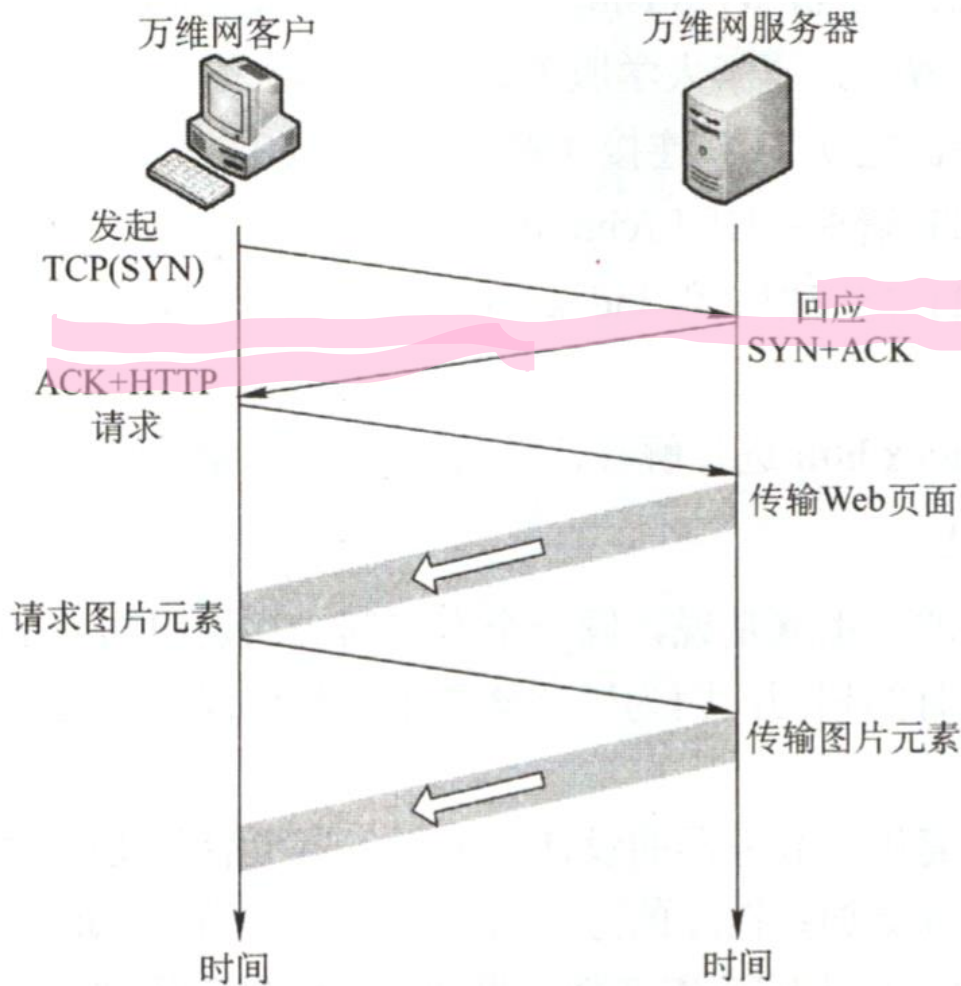


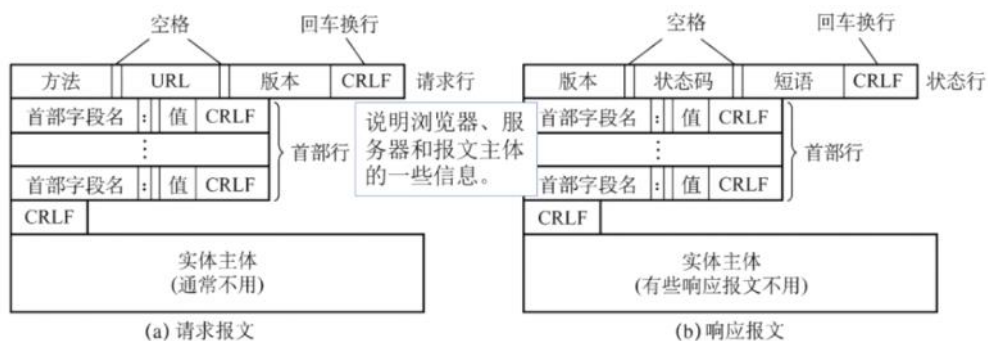
图 6-13 使用持久连接（非流水线）

HTTP报文结构

HTTP是面向文本的（Text-Oriented），因此在报文中的每个字段都是一些ASCII码串，并且每个字段的长度都是不确定的。

有两类HTTP报文：

- 请求报文：从客户向服务器发送的请求报文。
- 响应报文：从服务器到客户的回答。



HTTP请求报文和响应报文都是由三个部分组成。这两种报文格式的区别就是开始行不同。

HTTP请求报文常用方法

表 6-1 HTTP 请求报文中常用的几个方法

方法（操作）	意 义
GET	请求读取由 URL 所标志的信息
HEAD	请求读取由 URL 所标志的信息的首部
POST	给服务器添加信息（例如，注释）
CONNECT	用于代理服务器

常用应用层协议总结

表 6-2 常见应用层协议总结

应用程序	FTP 数据链接	FTP 控制链接	TELNET	SMTP	DNS	TFTP	HTTP	POP3	SNMP
使用协议	TCP	TCP	TCP	TCP	UDP	UDP	TCP	TCP	UDP
熟知端口号	20	21	23	25	53	69	80	110	161

6.6 小结

客户进程端口号与服务器进程端口号关系

通常我们所说的熟知端口号是指应用层协议在服务器端的默认端口号。而客户端进程的端口号是由客户端进程任意指定的(临时的)。

当客户进程向服务器进程发出建立连接请求时，要寻找连接服务器进程的熟知端口号，同时还要告诉服务器进程自己的临时端口号，以用于建立连接。接着，服务器进程就用自己的熟知端口号与客户进程所提供的端口号建立连接。

互联网、因特网和万维网的区别

互联网(internet)泛指由多个[计算机网络](#)，按照一定的通信协议，相互连接而成一个大型计 算机网络。

因特网 (Internet)：是指在ARPA网基础上发展而来的世界上最大的全球性互联网络。因特网和其他类似的由计算机相互连接而成的大型网络系统，都可算是“互联网”，因特网只是互联网中最大的一个。

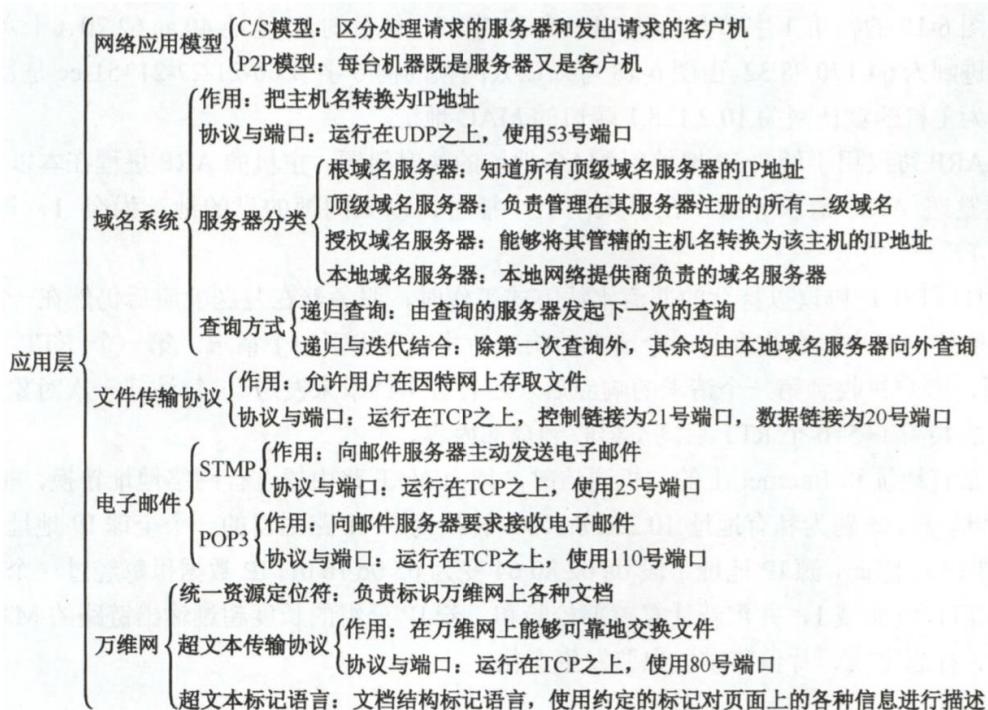
万维网：是无数个网络站点和网页的集合，它们在一起构成了因特网最主要的部分(因特网也包括电子邮件、 Msenet以及新闻组)。

DNS高速缓存是什么

每个域名服务器都维护一个高速缓存，存放最近用过的名字以及从何处获得名字映射信息的记录。可大大减轻根域名服务器的负荷，使因特网上的DNS查询请求和回答报文的数量大为减少。

为保持高速缓存中的内容正确，域名服务器应为每项内容设置计时器，并处理超过合理时间的项(如每个项目只存放两天)。当权限域名服务器回答一个查询请求时，在响应中，都指明绑定有效存在的时间值。增加此时间值可减少网络开销，而减少此时间值可以提高域名转换的准确性。

知识架构



第四章 网络层

星期日，三月 13，2022 5:20 下午

已剪辑自: <https://blog.csdn.net/donkey010/article/details/114695904>

第四章：网络层

4.1 网络层功能

概述

网络层的功能：① 异构网络互联；② 路由选择与分组转发；③ 拥塞控制。

4.1.1 异构网络互联

概述

网络的互联是指将两个以上的计算机网络，通过一定的方法，用一种或多种通信处理设备（即中间设备）相互联接起来，以构成更大的网络系统。

中继系统

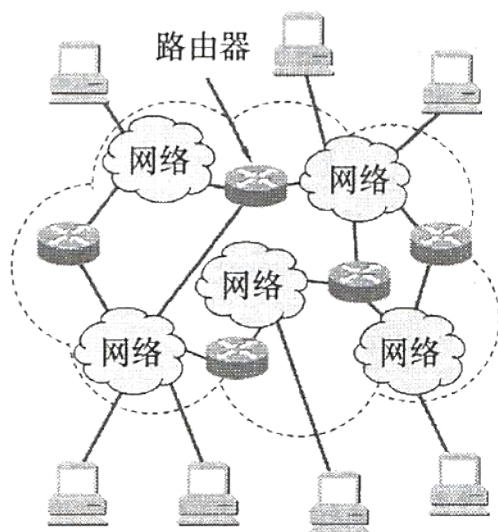
中间设备又称为中间系统或中继系统。根据中继系统所在层次，有以下四种不同的中继系统：

1. 物理层中继系统：中继器，集线器（Hub）。
2. 数据链路层中继系统：网桥或交换机。
3. 网络层中继系统：路由器。
4. 网络层以上的中继系统：网关。 网络层以上

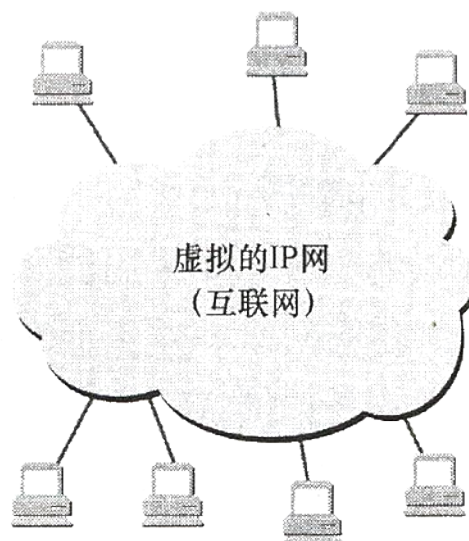
虚拟互联网络

网络互联通常是指用路由器进行网络互联和路由选择。路由器就是一台专用计算机，用于在互联网中进行路由选择。

TCP/IP体系在网络互联上采用的做法是在网络层（即IP层）采用了标准化协议，但相互联接的网络则可以是异构的。图(a)表示有许多计算机网络通过一些路由器进行互联。由于参加互联的计算机网络都使用相同的网际协议IP（Internet Protocol），因此可以把互联以后的计算机网络看成是如图(b)所示的一个虚拟互联网络。



(a) 实际的互联网络



(b) 虚拟的 IP 网

虚拟互联网络也就是逻辑互联网络，它的意思就是互联起来的各种物理网络的异构性本来是客观存在的，但是通过使用IP协议就可以使这些性能各异的网络在网络层上看起来好像是一个统一的网络。这种使用IP协议的虚拟互联网络可简称为IP网络。

使用虚拟互联网络优点：当互联网上的主机进行通信时，就好像在一个网络上通信一样，而看不见互联的具体的网络异构细节(如具体的编址方案、路由选择协议等)。

4.1.2 路由与转发

概述

路由器主要完成两个功能：一个是路由选择(确定哪一条路径)，另一个是分组转发(当一个分组到达时所采取的动作)。

前者是根据特定的路由选择协议构造出路由表，同时经常或定期地和相邻路由器交换路由信息而不断地更新和维护路由表。

后者处理通过路由器的数据流，关键操作是转发表查询、转发以及相关的队列管理和任务调度等。

路由选择

指按照复杂的分布式算法，根据从各相邻路由器所得到的关于整个网络拓扑的变化情况，动态地改变所选择的路由。

分组转发

指路由器根据转发表将用户的IP数据报从合适的端口转发出去。

路由表是根据路由选择算法得出的，而转发表是从路由表得出的。转发表的结构应当使查找过程最优化，路由表则需要对网络拓扑变化的计算最优化。在讨论路由选择的原理时，往往不去区分转发表和路由表，而是笼统地使用路由表这一名词。

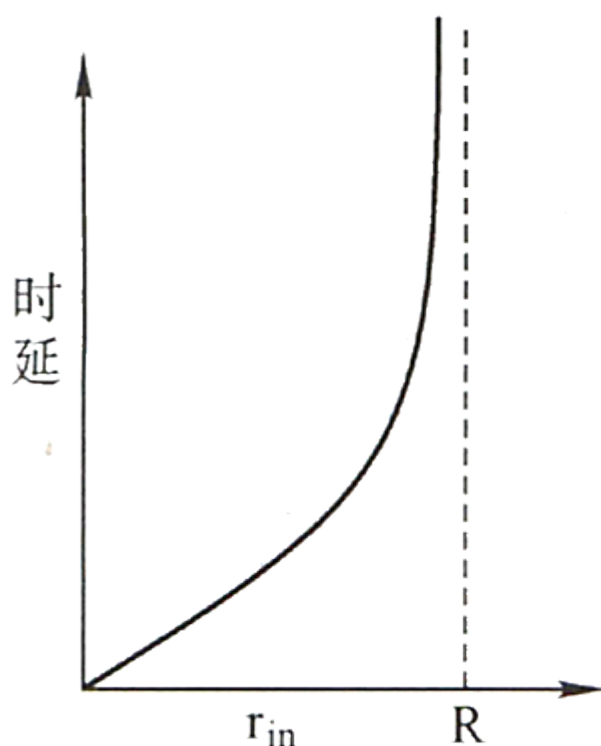
4.1.3 拥塞控制

概述

在通信子网中，由于出现过量的分组而引起网络性能下降的现象称为拥塞。

分组发送速率与时延的关系

考虑一种情况，某个路由器所在链路的带宽为 R B/s，如果IP分组只从它的某一个端口进入，其速率为 r_{in} B/s。当 $r_{in}=R$ 时，可能看起来是件“好事”，因为链路带宽被充分利用。但是当分组到达路由器的速率接近 R 时，平均时延急剧增加，并且会有大量的分组被丢弃（路由器端口的缓冲区是有限的），整个网络的吞吐量会骤降，源与目的地之间的平均时延也会变得近乎无穷大。



判断网络是否进入拥塞状态的方法是观察网络的吞吐量与网络负载的关系：

- 如果随着网络负载的增加，网络的吞吐量明显小于正常的吞吐量，那么网络就可能进入了**轻度拥塞**状态；
- 如果网络的吞吐量随着网络负载的增大反而下降，网络就可能进入**拥塞状态**；
- 如果网络的负载继续增大，而网络的吞吐量下降到零，网络就可能进入了**死锁状态**。

拥塞控制

拥塞控制主要解决的问题是如何获取网络中发生拥塞的信息，从而利用这些信息进行控制，以避免由于拥塞而出现分组的丢失以及严重拥塞而产生网络死锁的现象。

拥塞控制是确保子网能够承载所达到的流量，这是一个全局性的过程，涉及各方面的行为：主机、路由器及路由器内部的转发处理过程等。单一的增加资源并不能解决拥塞。

拥塞控制方法有两种：

1. **开环控制**：在设计网络时事先将有关发生拥塞的因素考虑周到，力求网络在工作时不产生拥塞。它是一种静态的预防方法。一旦整个系统启动并运行起来，中途就不再需要修改。开环控制手段可包括确定何时可接收新流量、何时可丢弃分组及丢弃哪些分组，确定何种调度决策等。所有这些手段的共性是：在做决定时不考虑当前网络的状态。
2. **闭环控制**：事先不考虑有关发生拥塞的各种因素，采用监测网络系统去监视，及时检测到哪里发生拥塞，然后将拥塞信息传到合适的地方，以便调整网络系统的运行，并解决出现的问题。闭环控制是基于反馈环路的概念，它是一种动态的方法。

流量控制和拥塞控制的区别

流量控制：往往是指在发送端和接收端之间的点对点通信量的控制。流量控制所要做的就是抑制发送端发送数据的速率，以便使接收端来得及接收。

拥塞控制：必须确保通信子网能够传送待传送的数据，是一个全局性的问题，涉及网络中所有的主机、路由器以及导致网络传输能力下降的所有因素。

4.2 路由算法

概述

路由算法：① 静态路由与动态路由；② 距离-向量路由算法；③ 链路状态路由算法；④ 层次路由。

4.2.1 静态路由与动态路由

概述

路由器转发分组是通过路由表转发的，而路由表是通过各种算法得到的。从能否随网络的通信量或拓扑自适应地进行调整变化来划分，路由算法可分为两大类：静态路由算法 & 动态路由算法。

静态路由算法

概念：又称为非自适应路由算法，指由网络管理员手工配置的路由信息。当网络的拓扑结构或链路的状态发生变化时，网络管理员需要手工去修改路由表中相关的静态路由信息。

优点：简便、可靠，在负荷稳定、拓扑变化不大的网络中运行效果很好，广泛用于高度安全性的军事网络和较小的商业网络。

缺点：路由更新慢，大型和复杂的网络环境不适用。一方面，网络管理员难以全面地了解整个网络的拓扑结构；另一方面，当网络的拓扑结构和链路状态发生变化时，路由器中的静态路由信息需要大范围地调整，这一工作的难度和复杂程度非常高。

动态路由算法

概念：又称为自适应路由算法，指路由器上的路由表项是通过相互联接的路由器之间彼此交换信息，然后按照一定的算法优化出来的，而这些路由信息是在一定时间间隔里不断更新，以适应不断变化的网络，以随时获得最优的寻路效果。

优点：路由更新快，适用大型网络，及时响应链路费用或网络拓扑变化，能改善网络的性能并有助于流量控制。

缺点：算法复杂，会增加网络的负担，有时因对动态变化的反应太快而引起振荡，或反应太慢而影响网络路由的一致性。

常用的动态路由算法可分为两类：距离-向量路由算法和链路状态路由算法。

- 分散性距离向量路由算法
使用RIP协议，路由器只掌握物理相连的邻居及链路费用。
- 全局性-链路状态路由算法
使用OSPF协议，所有路由器掌握完整的网络拓扑和链路费用信息。

4.2.2 距离-向量路由算法

概述

在距离-向量路由算法中，所有的结点都定期地将它们的整个路由选择表传送给所有与之直接相邻的结点。这种路由选择表包含：① 每条路径的目的地(另一结点)；② 路径的代价(也称距离)。

注意：这里的距离是一个抽象的概念，如RIP就将距离定义为“跳数”。跳数指从源端口到达目的端口所经过的路由个数，经过一个路由器跳数加1。

算法触发条件

在这种算法中，所有的结点都必须参与距离向量交换，以保证路由的有效性和一致性，也就是说，所有的结点都监听从其他结点传送来的路由选择更新信息，并在下列情况下更新它们的路由选择表：

- 被通告一条新的路由，该路由在本结点的路由表中不存在，此时本地系统加入这条新的路由。
- 发送来的路由信息中有一条到达某个目的地的路由，该路由比当前使用的路由有较短的距离(较小的代价)。此种情况下，就用经过发送路由信息的结点的新路由替换路由表中达到那个目的地的现有路由。

算法优劣

距离-向量路由算法的实质是迭代计算一条路由中的站段数或延迟时间，从而得到到达一个目标的最短(最小代价)通路。它要求每个结点在每次更新中都将其全部路由表发送给其所有相邻结点。显然，更新报文的大小与通信子网的结点数成正比，大的通信子网将导致很大的更新报文。由于更新报文发给直接邻接的结点，所以所有的结点都将参加路由选择信息交换。基于这些原因，在通信子网上传送的路由选择信息的数量很容易变得非常大。

最常见的距离-向量路由算法是RIP算法，它采用“跳数”作为距离的度量。



4.2.3 链路状态路由算法

概述

链路状态路由算法要求每个参与该算法的结点都有完全的网络拓扑信息，它们执行下述两项任务：

1. **主动测试所有邻接结点的状态。**（两个共享一条链接的结点是相邻结点，它们连接到同一条链路，或者连接到同一广播型物理网络。）
2. **定期地将链路状态传播给所有其他的结点（或称路由结点）。**典型的链路状态算法是OSPF算法。

算法主要特征

- 1、向本自治系统中所有路由器发送信息，这里使用的方式是泛洪法，即路由器通过所有端口向所有相邻的路由器发送信息。而每一个相邻路由器又将此信息发往其所有相邻路由器（但不再发送给刚刚发来信息的那个路由器）。
- 2、发送的信息就是与路由器相邻的所有路由器的链路状态，但这只是路由器所知道的部分信息。所谓“链路状态”就是说明本路由器都和哪些路由器相邻以及该链路的“度量”。对于OSPF算法，链路状态的“度量”主要用来表示费用、距离、时延、带宽等。
- 3、只有当链路状态发生变化时，路由器才向所有路由器发送此消息。

由于一个路由器的链路状态只涉及相邻路由器的连通状态，因而与整个互联网的规模并无直接关系，因此链路状态路由算法可以用于大型的或路由信息变化聚敛的互联网环境。

算法优点

- 1、每个路由结点都使用同样的原始状态数据独立地计算路径，而不依赖中间结点的计算。
- 2、链路状态报文不加改变地传播，易于查找故障。当一个结点从所有其他结点接收到了报文时，它可以在本地立即计算正确的通路，保证一步会聚。
- 3、由于链路状态报文仅运载来自单个结点关于直接链路的信息，其大小与网络中的路由结点数目无关，因此，链路状态算法比距离-向量算法有更好的规模可伸展性。

与链路状态路由算法比较

在距离-向量路由算法中，每个结点仅与它的直接邻居交谈，它为它的邻居提供了从自己到网络中所有其他结点的最低费用估计。

在链路状态路由算法中，每个结点通过广播的方式与所有其他结点交谈，但它仅告诉它们与它直接相连的链路费用。相较之下，距离-向量路由算法有可能遇到路由环路等问题。

4.2.4 层次路由

背景

当网络规模扩大时，路由器的路由表成比例地增大。这不仅消耗越来越多的路由器缓冲区空间，而且需要用更多CPU时间来扫描路由表，用更多的带宽来交换路由状态信息。因此路由选择必须按照层次的方式进行。

层次路由分类

1、内部网关协议（IGP）

一个自治系统内部所使用的路由选择协议，也称为域内路由选择，具体的协议有RIP和OSPF等。

2、外部网关协议（EGP）

自治系统之间所使用的路由选择协议，也称为域间路由选择，用于不同自治系统的路由器之间交换路由信息，并负责为分组在不同自治系统之间选择最优的路径。具体的协议有BGP。

优点

当使用层次路由时，OSPF将一个自治系统再划分为若干个区域（Area），每个路由器都知道第在本区域内如何把分组路由到目的地的细节，但不用知道其他区域的内部结构。

采用分层次划分区域的方法虽然使交换信息的种类增多了，同时也使OSPF协议更加复杂了。但这样做却能使每一个区域内部交换路由信息的通信量大大减小，因而使OSPF协议能够用于规模很大的自治系统中。

4.3 IPv4

4.3.1 IPv4 分组

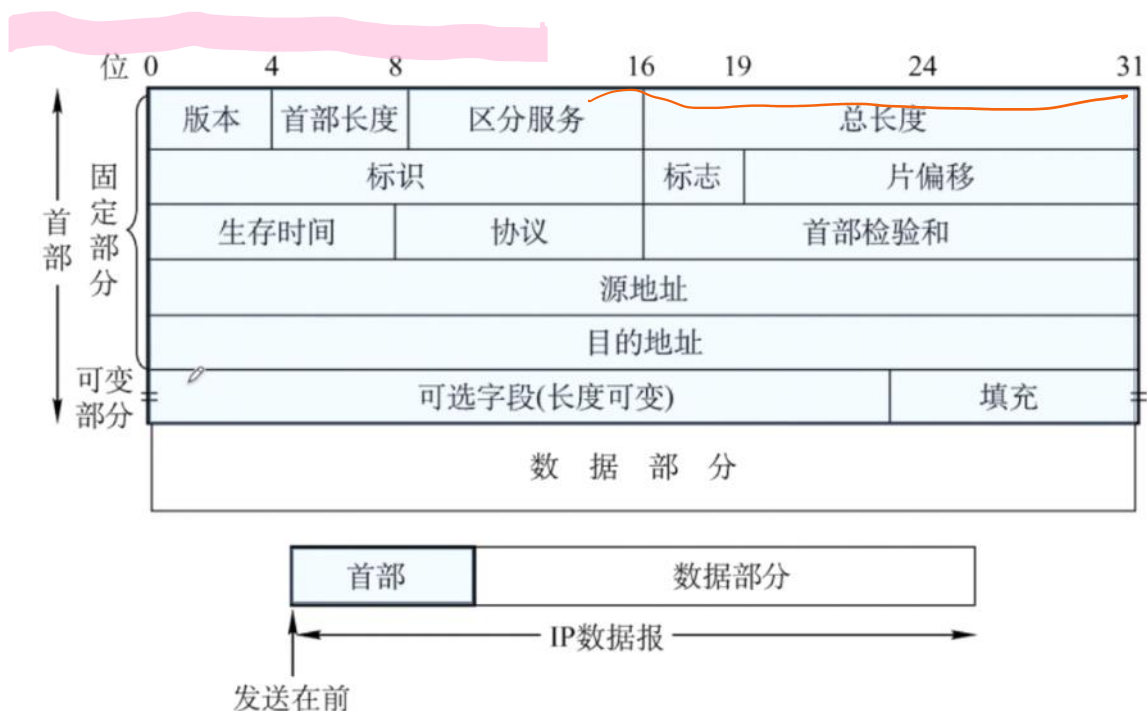
概述

IPv4，即现在普遍使用的IP协议(版本为4)。IP协议定义数据传送的基本单元——IP分组及其确切的数据格式。IP协议也包括一套规则，指明分组如何处理、错误怎样控制。特别是，IP协议还包含非可靠投递的思想，以及与此关联的分组路由选择的思想。

IP数据报格式

一个IP分组由首部和数据两部分组成。首部的前一部分是固定长度的，共20字节，是所有IP分组必须具有的。在首部固定部分的后面是一些可选字段，其长度是可变的，用来提供错误检测及安全等机制。





IP首部重要字段含义：

- **版本**：指IP协议的版本，目前广泛使用的版本号为4。
- **首部长度**：占4位。以32位为单位，最大值为60字节（15×4字节）。最常用的首部长度是20字节，此时不使用任何选项（即可选字段）。
- **总长度**：占16位。指首部和数据之和的长度，单位为字节，因此数据报的最大长度为 $2^{16} - 1 = 65535$ 字节。以太网帧的最大传送单元 MTU为1500字节，因此当一个IP数据报封装成帧时，数据报的总长度（首部加数据）一定不能超过下面数据链路层的MTU值。
- **标识**：占16位。它是一个计数器，每产生一个数据报就加1，并赋值给标识字段。但它并不是“序号”（因为IP是无连接服务）。当一个数据报长度超过网络的MTU时，必须分片，此时每个数据报片都复制一次标识号以便能正确重装成原来的数据报。
- **标志**：占3位。标志字段的最低位为MF，MF=1表示后面还有分片；MF=0表示最后一个分片。标志字段中间的一位是DF，只有当DF=0时才允许分片。
- **片偏移**：占13位。它指出较长的分组在分片后，某片在原分组中的相对位置。片偏移以8个字节为偏移单位，也即每个分片的长度一定是8字节（64位）的整数倍。
- **生存时间TTL**：占8位。即数据报在网络中可通过的路由器数的最大值，标识分组在网络中的寿命，以确保分组不会永远在网络中循环。路由器在转发分组之前，先把TTL减1。若TTL被减为0，则该分组必须丢弃。
- **协议**：占8位。指出此分组携带的数据是使用何种协议，即分组的数据部分应交给哪个传输层协议，如TCP，UDP等。其中值为6表示TCP，值为17表示UDP。

协议名	ICMP	IGMP	TCP	EGP	IGP	UDP	IPv6	ESP	OSPF
字段值	1	2	6	8	9	17	41	50	89

- **首部检验和**：占16位。IP数据报的首部检验和只校验分组的首部，而不校验数据部分。
- **源地址**：占4字节，标识发送方的IP地址。
- **目的地址**：占4字节，标识接收方的IP地址。
- **可选字段**：0~40B，用来支持排错、测量以及安全等措施。
- **填充**：全0，把首部补成4B的整数倍。

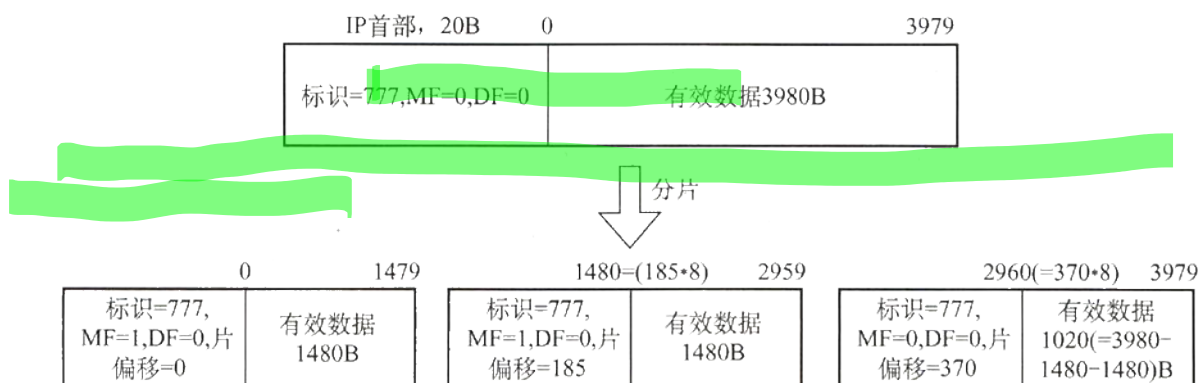
IP数据报分片

一个链路层数据报能承载的最大数据量称为**最大传送单元(MTU)**。因为IP数据报被封装在链路层数据报中，故链路层的MTU严格地限制着IP数据报的长度，而且在IP数据报的源与目的地路径上的各段链路可能使用不同的链路层协议，有不同的MTU。

当IP数据报的总长度大于链路MTU时，就需要将数据报中的数据分装在两个或更多个较小的IP数据报中，这些较小的数据报叫做片。

片在目的地的网络层被重新组装。目的主机使用IP首部中的标识、标志和片偏移字段来完成对片的重组。

大于MTU则需要分片。



IP分片例子

① 创建一个长 4000字节的IP数据报（首部20字节，数据部分3980字节），源主机为该数据报加上一个标识号777。

② 该数据报到达一个路由器，要转发到一条MTU为1500字节的链路上，故需要将该原始数据报中3980字节数据分配到3个独立的片中（每个片也是一个IP数据报），形成的每个数据报（即片）都具有原始数据报的标识号777。

IP首部中的标志位有3个比特，但只有后2个比特有意义，分别是 **MF位 (More Fragment)** 和 **DF位 (Don't Fragment)**。

只有当 DF=0 时，该IP数据报才可以被分片。MF则用来告知目的主机该IP数据报是否为原始数据报的最后一个片。当MF=1时，表示相应的原始数据报还有后续的片；当MF=0时，表示该数据报是相应原始数据报的最后一个片。目的主机在对片进行重组时，使用片偏移字段来确定片应放在原始IP数据报的哪个位置。

可以看出，由于偏移值的单位是8字节，所以除了最后一个片外，其他所有片中的有效数据载荷都是8的倍数。

③ 当目的主机收到来自同一发送主机的一批数据报时，它可以通过检查数据报的标识号以确定哪些数据报是属于同一个原始数据报的片。

网络层转发分组的流程

网络层的路由器执行的分组转发算法如下：

1. 从数据报的首部提取目的主机的IP地址D，得出目的网络地址为N。
2. 若网络N与此路由器直接相连，则把数据报直接交付目的主机D，这就叫路由器的直接交付；否则是间接交付，执行步骤3。

3. 若路由表中有目的地址为D的特定主机路由(对特定的目的主机指明一个特定的路由, 通常是为了控制或测试网络, 或出于安全考虑才采用的), 则把数据报传送给路由表中所指明的下一跳路由器; 否则, 执行步骤4。
4. 若路由表中有到达网络N的路由, 则把数据报传送给路由表指明的下一跳路由器; 否则, 执行步骤5。
5. 若路由表中有一个默认路由, 则把数据报传送给路由表中所指明的默认路由器; 否则, 执行步骤6。
6. 报告转发分组出错。

4.3.2 IPv4地址与NAT

IPv4地址

连接在 Internet 中的每一台主机(或路由器)都分配一个32比特的全球唯一的标识符, 即IP地址。传统的IP地址是分类的地址, 分为A、B、C、D、E五类。

无论哪类IP地址, 都由网络号和主机号两部分组成。即IP地址::={<网络号>, <主机号>}。其中网络号标志主机(或路由器)所连接到的网络。一个网络号在整个因特网范围内必须是唯一的。主机号标志该主机(或路由器)。一个主机号在它前面的网络号所指明的网络范围内必须是唯一的。由此可见, 一个IP地址在整个因特网范围内是唯一的。

分类的IP地址

在各类IP地址中, 有一些IP地址用于表示特殊用途, 不用于做主机IP地址:

NetID 网络号	HostID 主机号	作为IP分组 源地址	作为IP分组目 的地址	用途
全0	全0	可以	不可以	本网范围内表示主机, 路由表中用于表示默认路由(表示整个Internet网络)
全0	特定值	不可以	可以	表示本网内某个特定主机
全1	全1	不可以	可以	本网广播地址(路由器不转发)
特定值	全0	不可以	不可以	网络地址, 表示一个网络
特定值	全1	不可以	可以	直接广播地址, 对特定网络上的所有主机进行广播
127	任何数 (非全0/1)	可以	可以	用于本地软件换回测试, 称为环回地址

- ? 主机号全为0表示本网络本身。例如 202.98.174.0
- 主机号全为1表示本网络的广播地址, 又称直接广播地址。例如 202.98.174.255
- 127.0.0.0 网络保留作为环路自检(Loopback Test)地址, 此地址表示任意主本身, 目的地址为环回地址的IP数据报永远不会出现在任何网络上。
- 32位全为0, 即 0.0.0.0 表示本网络上的本主机。
- 32位全为1, 即 255.255.255.255 表示整个TCP/IP网络的广播地址, 又称受限广播地址。

实际使用时由于路由器对广播域的隔离, 255.255.255.255 等效为本网络的广播地

址。

网络类别	最大可用网络数	第一个可用的网络号	最后一个可用的网络号	每个网络中的最大主机数
A	2^7-2	1	126	$2^{24}-2$
B	$2^{14}-1$	128.1	191.255	$2^{16}-2$
C	$2^{21}-1$	192.0.1	223.255.255	2^8-2

三种类别的IP地址的使用范围

- A类地址可用的网络数为 2^7-2 ，减2的原因是：① 网络号字段全为0的IP地址是保留地址，意思是“本网络”；② 网络号为127为环回测试地址。
- B类地址可用网络数为 $2^{14}-1$ ，减1的原因是：128.0 这个网络号是不可指派的。
- C类地址可用网络数为 $2^{21}-1$ ，减1的原因是：网络号为 192.0.0 的网络是不可指派的。

IP地址重要特点

1、每个IP地址都由网络号和主机号两部分组成， 因此IP地址是一种分等级的地址结构。

分等级的好处是：① IP地址管理机构在分配IP地址时只分配网络号(第一级)，而主机号(第二级)则由得到该网络的单位自行分配，方便了IP地址的管理；② 路由器仅根据目的主机所连接的网络号来转发分组(而不考虑目标主机号)，从而减小了路由表所占的存储空间。

2、IP地址是标志一个主机(或路由器) 和一条链路的接口。

当一个主机同时连接到两个网络时，该主机就必须同时具有两个相应的IP地址， 每个IP地址的网络号必须与所在网络的网络号相同，且这两个IP地址的网络号是不同的。因此IP网络上的一个路由器必然至少应当具有两个IP地址（路由器每个端口必须至少分配一个IP地址）。

3、用转发器或桥接器(网桥等)连接起来的若干个LAN仍然是同一个网络(同一个广播域)，因此该LAN中所有主机的IP地址的网络号必须相同，但主机号必须不同。

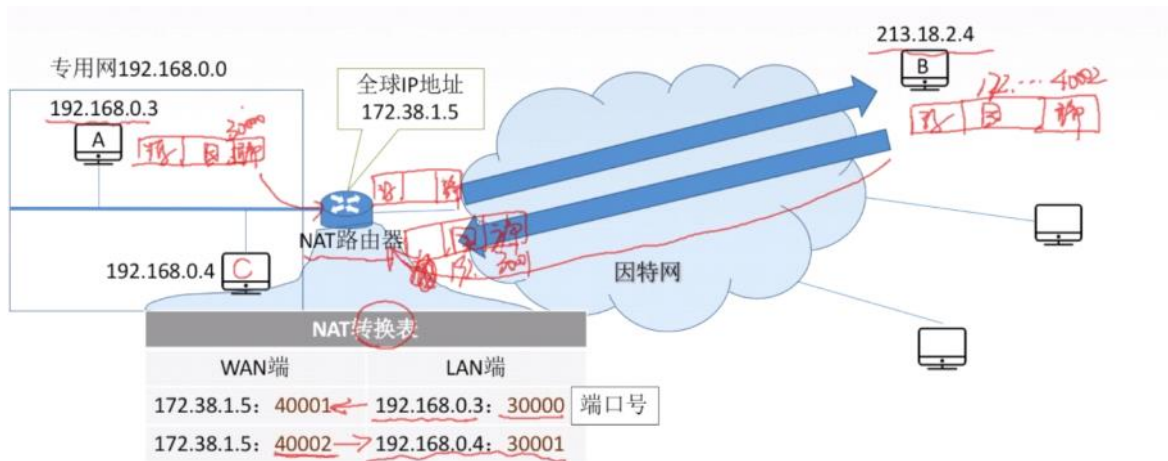
4、在IP地址中，所有分配到网络号的网络(无论是LAN或WAN)都是平等的。

5、在同一个局域网上的主机或路由器的IP地址中的网络号必须是一样的。路由器总是具有两个或两个以上的IP地址，路由器的每一个端口都有一个不同网络号的IP地址。

网络地址转换

网络地址转换NAT (Network Address Translation)，是通过在路由器上安装NAT软件，将专用网络地址(如企业内部 Intranet)转换为公用地址(如Internet)，从而对外隐藏了内部管理的IP地址，安装了NAT软件的路由器叫**NAT路由器**，它至少有一个有效的**外部全球IP地址**。

当使用本地地址的主机在和外界通信时， NAT路由器使用**NAT转换表**将本地地址转换成全球地址，或将全球地址转换成本地地址。 NAT转换表中存放着{本地IP地址：端口}到{全球IP地址：端口}的映射。通过{IP地址：端口}这样的映射方式，就可以让多个私有IP地址映射到同一个全球IP地址 。



优点：① 它使得整个专用网只需要一个全球IP地址就可以与因特网连通，由于专用网本地IP地址是可重用的，所以NAT大大节省了IP地址的消耗。② 它隐藏了内部网络结构，从而降低了内部网络受到攻击的风险。

私有IP地址

为了网络安全，划出了部分IP地址为私有IP地址。**私有IP地址只用于LAN，不用于WAN连接**(因此私有IP地址不能直接用于 Internet，必须通过网关利用NAT把私有IP地址转换为 Internet 中合法的全球IP地址后才能用于 Internet)，并且允许私有IP地址被LAN重复使用。这有效地解决了IP地址不足的问题。

地址类别	地址范围	网段个数
A类	10.0.0.0~10.255.255.255	1
B类	172.16.0.0~172.31.255.255	16
C类	192.168.0.0~192.168.255.255	256

私有IP地址

- A类：1个A类网段，即10.0.0.0~10.255.255.255
- B类：16个B类网段，即 172.16.0.0~172.31.255.255
- C类：256个C类网段，即 192.168.0.0~192.168.255.255

在因特网中的所有路由器，对目的地址是私有地址的数据报一律不进行转发。这种采用私有IP地址的互联网络称为专用互联网或本地互联网。**私有IP地址也称可重用地址。**

示例

假设一个宿舍办理了 2Mb/s 的电信宽带，那么这个宿舍就获得了一个全球IP地址 (如138.76.29.7)，而宿舍内4台主机使用私有地址 (如 192.168.0.0 网段)。宿舍的网关路由器应该开启NAT功能，当路由器从LAN端口收到源IP及源端口号为 192.168.0.2:2233 的数据报时，就将其映射成 138.76.29.7:5001，然后从WAN端口发送到因特网上。当路由器从WAN端口收到目的IP及目的端口号为 138.76.29.7:

5060 的数据报时，就将其映射成 192.168.0.3:1234，然后从LAN端口发送给相应的本地主机。这样，只需要一个全球地址，就可以让多台主机同时访问英特网。

NAT 转换表	
WAN 端	LAN 端
138.76.29.7:5001	192.168.0.2:2233
138.76.29.7:5060	192.168.0.3:1234
...	...

一个典型的NAT转换表

注意：普通路由器在转发 IP 数据报时，不改变其源IP地址和目的IP地址。而NAT路由器在转发IP数据报时，一定要更换其IP地址(转换源 IP地址或目的IP地址)。普通路由器仅工作在网络层，而NAT路由器转发数据报时需要查看和转换传输层的端口号。

4.3.3 子网划分与子网掩码、CIDR

子网划分

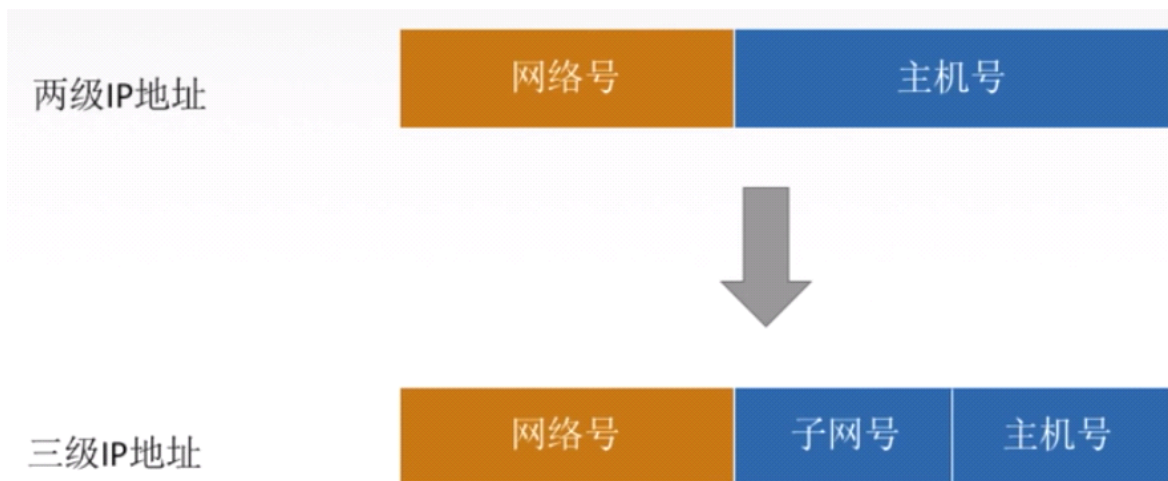
背景

两级IP地址的缺点：

- ① IP地址空间的利用率有时很低；
- ② 给每一个物理网络分配一个网络号会使路由表变得太大因而使网络性能变坏；
- ③ 两级的IP地址不够灵活。

概述

从1985年起，在IP地址中又增加了一个“子网号字段”，使两级IP地址变成为三级IP地址。这种做法叫做划分子网。划分子网已成为因特网的正式标准协议。



划分子网的基本思路

- 划分子网纯属一个单位内部的事情。单位对外仍然表现为没有划分子网的网络。
- 从主机号借用若干个比特作为子网号，当然主机号也就相应减少了相同的比特。
 - 三级IP地址的结构：IP地址={<网络号>，<子网号>，<主机号>}。
- 凡是从其他网络发送给本单位某个主机的IP数据报，仍然是根据IP数据报的目的网络号，先找到连接在本单位网络上的路由器。然后此路由器在收到IP数据报后，再按目的网络号和子网号找到目的子网。最后就把IP数据报直接交付给目的主机。

注意：

1. 划分子网只是把IP地址的主机号这部分进行再划分，而不改变地址原来的网络号。因此，从一个IP地址本身或IP数据报的首部，并无法判断源主机或目的主机所连接的网络是否进行了子网划分。
2. [RFC 950]规定，对分类的IPv4地址进行子网划分时，子网号不能为全1或全0。但随着CIDR的广泛使用，现在全1和全0的子网号也可以使用，但一定要谨慎使用，要弄清你的路由器所用的路由选择软件是否支持全0或全1的子网号这种用法。
3. 不论是分类的IPv4地址还是CIDR，其子网中的主机号为全0或全1的地址都不能被指派。**子网中的主机号全0的地址为子网的网络号，主机号全1的地址为子网的广播地址。**

子网掩码

概述

子网掩码是一个与IP地址相对应的长32bit的二进制串，它由一串1和跟随的一串0组成。其中，1对应于IP地址中的网络号及子网号，而0对应于主机号。计算机只需将IP地址和其对应的子网掩码逐位相“与”（逻辑AND运算），就可得出相应子网的网络地址。

现在的因特网标准规定：所有的网络都必须使用子网掩码。如果一个网络没有划分子网，那么就采用默认子网掩码。

由于子网掩码是一个网络或一个子网的重要属性，所以路由器在相互之间交换路由信息时，必须把自己所在网络(或子网)的子网掩码告诉对方。路由表中的每一个条

目，除了要给出目的网络地址和下一跳地址外，还需要同时给出该目的网络的子网掩码。

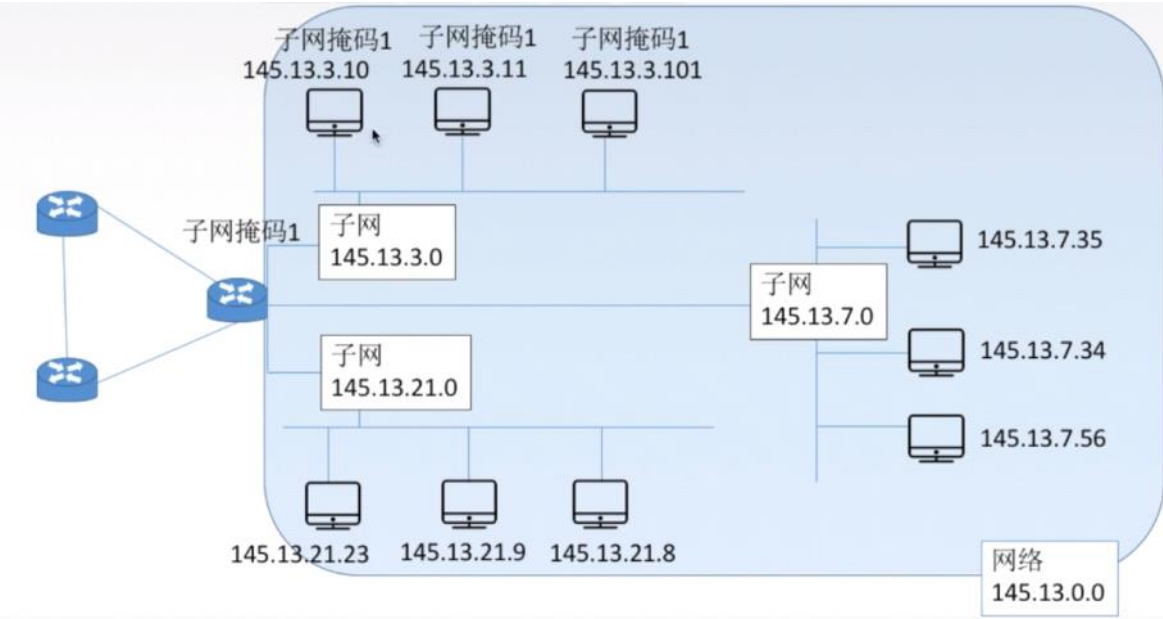
两级IP地址	145	13	3	10
两级IP地址的子网掩码	11111111	11111111	00000000	00000000
三级IP地址	145	13	3	10
三级IP地址的子网掩码	11111111	11111111	11111111	00000000

子网掩码与IP地址逐位相与，就得到子网网络地址

使用子网掩码情况

1. 一个主机在设置IP地址信息的同时，必须设置子网掩码。
2. 同属于一个子网的所有主机以及路由器的相应端口，必须设置相同的子网掩码。
3. 路由器的路由表中，所包含的信息其主要内容必须有：目的网络地址、子网掩码、下一跳地址。

使用子网掩码路由器分组转发算法



使用子网时分组的转发

1. 从收到的分组的首部提取目的IP地址，记为D。
2. 先判断是否为直接交付。对路由器直接相连的网络逐个进行检查：用各网络的子网掩码和D逐位相“与”，看结果是否和相应的网络地址匹配。若匹配，则将分组直接交付，否则就是间接交付，执行步骤3。

3. 若路由表中有目的地址为D的特定主机路由，则将分组传送给路由表中所指明的下一跳路由器；否则，执行步骤4。
4. 对路由表中的每一行(目的网络地址，子网掩码，下一跳地址)中的子网掩码和D逐位相“与”，其结果为N。若N与该行的目的网络地址匹配，则将分组传送给该行指明的下一跳路由器；否则，执行步骤5。
5. 若路由表中有一个默认路由，则将分组传送给路由表中所指明的默认路由器；否则，执行步骤6。
6. 报告转发分组出错。

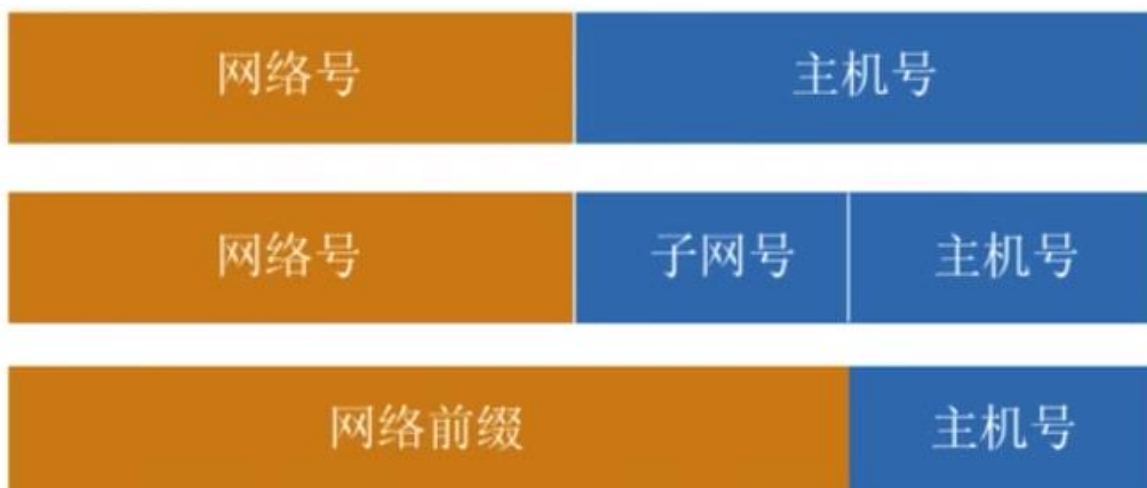
无分类编址CIDR

概述

无分类域间路由选择(CIDR)是在变长子网掩码的基础上提出的一种消除传统A、B、C类网络划分，并且可以在软件的支持下实现超网构造的一种IP地址的划分方法。

主要特点

- 1、消除了传统的A、B、C类地址以及划分子网的概念，因而可以更加有效地分配IPv4的地址空间。



- CIDR使用“网络前缀”的概念代替子网的概念。这样IP地址的无分类两级编址为 $IP := \{ \langle \text{网络前缀} \rangle, \langle \text{主机号} \rangle \}$ 。
 - CIDR使用“斜线记法”(或称**CIDR记法**: IP地址后加上“/”，然后写上网络前缀(可以任意长度)的位数，如128.14.32.0/20)。
 - CIDR 虽然不使用子网，但仍然使用“掩码”这一名词。“CIDR不使用子网”是指CIDR并没有在32位地址中指明若干位作为子网字段。但分配到一个CIDR地址块的组织，仍可以在本组织内根据需要划分出一些子网。
- 2、融合子网地址和子网掩码，将网络前缀都相同的连续的IP地址组成“CIDR地址块”，方便子网划分。

128.14.35.7/20是某CIDR地址块中的一个地址

二进制: 10000000 00001110 00100011 00000111

最小地址: 10000000 00001110 00100000 00000000
128.14.32.0

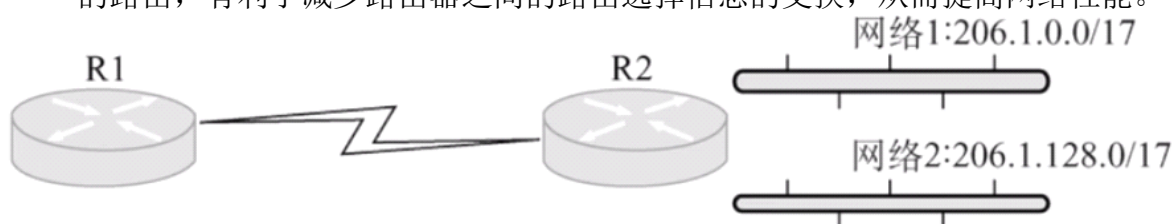
最大地址: 10000000 00001110 00101111 11111111
128.14.47.255

地址块: 128.14.32.0/20 “/20地址块”

地址掩码 (子网掩码):

11111111 11111111 11110000 00000000

- 一个CIDR地址块可以表示很多地址, 这种地址的聚合称为**路由聚合**, 或称为构成超网。路由聚合使得路由表中的一个项目可以表示很多个原来传统分类地址的路由, 有利于减少路由器之间的路由选择信息的交换, 从而提高网络性能。



路由聚合

- CIDR地址块中的地址数一定是2的整数次幂, 实际可指派的地址数通常为 $2^N - 2$, N 表示主机号的位数, 主机号全0代表网络号, 主机号全1为广播地址。网络前缀越短, 其地址块所包含的地址数就越多。而在三级结构的IP地址中, 划分子网是使网络前缀变长。
- CIDR的优点在于网络前缀长度的灵活性。由于上层网络的前缀长度较短, 因此相应的路由表的项目较少。而内部又可采用延长网络前缀的方法来灵活的划分子网。

最长前缀匹配

最长前缀匹配 (最佳匹配): 使用CIDR时, 路由表中的每个项目由 “网络前缀” 和 “下一跳地址” 组成。在查找路由表时可能会得到不止一个匹配结果。应当从匹配结果中选择具有最长网络前缀的路由, 因为网络前缀越长, 其地址块就越小, 因而路由就越具体。

CIDR查找路由表的方法: 为了进行更加有效的查找最长前缀匹配, 通常是将无分类编址的路由表存放在一种层次的数据结构中, 然后自上而下地按层次进行查找。这里最常用的数据结构就是二叉线索。

4.3.4 ARP协议、DHCP协议与ICMP协议

IP地址与硬件地址

概述

IP地址是网络层使用的地址，它是分层次等级的。硬件地址是数据链路层使用的地址(例如MAC地址)，它是平面式的。在网络层及网络层之上使用IP地址，IP地址放在IP数据报的首部，而MAC地址放在MAC帧的首部。通过数据封装，把IP数据报分组封装成为MAC帧之后，数据链路层看不见数据报分组中的IP地址。

由于路由器的隔离，IP网络中无法通过广播方式依靠MAC地址来完成跨网络的寻址，因此在IP网络的网络层只使用IP地址来完成寻址。

寻址过程

1. 每个路由器依据其路由表(依靠静态路由或动态路由协议生成)选择到目标网络(即主机号全为0的网络地址)需要转发到的下一跳(路由器的物理端口号或下一网络地址)。
2. 而IP分组通过多次路由转发到达目标网络后，则改为在目标网络LAN中通过数据链路层的MAC地址以广播方式寻址。这样可以提高路由选择的效率。

特点

1. 在IP层抽象的互联网上只能看到IP数据报。
2. 虽然在IP数据报首部中有完整的源IP地址和目的IP地址，但路由器只根据目的IP地址的网络号进行路由选择。
3. 在局域网的链路层，只能看见MAC帧。而通过路由器转发IP分组时，此IP分组在每个网络中都被路由器解封装和重新封装。因此IP数据报在被路由器转发时其数据链路层封装所使用的MAC地址是在不断改变的。这也决定了无法使用MAC地址跨网络通信。
4. 尽管互联在一起的网络的硬件地址体系各不相同，但IP层抽象的互联网却屏蔽了下层这些很复杂的细节。只要在网络层上讨论问题，就能够使用统一的、抽象的IP地址研究主机与主机或路由器之间的通信。

注意：路由器由于互联多个网络，因此它不仅有多多个IP地址，也有多个硬件地址。

地址解析协议ARP

概述

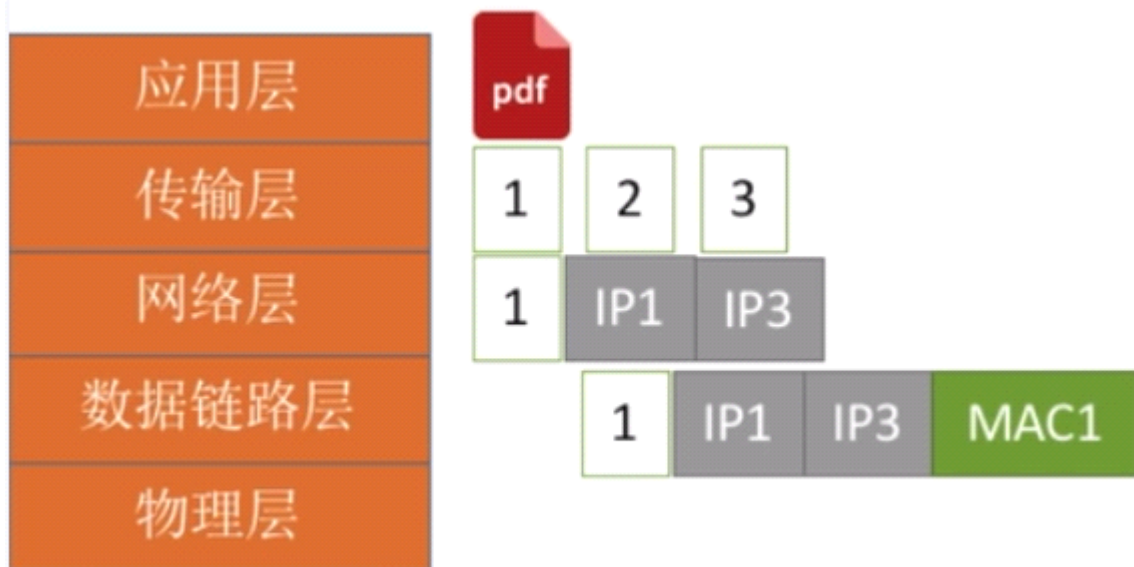
无论网络层使用什么协议，在实际网络的链路上传送数据帧时，最终必须使用硬件地址。所以需要一种方法来完成IP地址到MAC地址的映射，这就是**地址解析协议ARP(Address Resolution Protocol)**。

每个主机都设有一个ARP高速缓存，存放本局域网上各主机和路由器的IP地址到MAC地址的映射表，称ARP表。使用ARP协议来动态维护此ARP表。



ARP工作原理

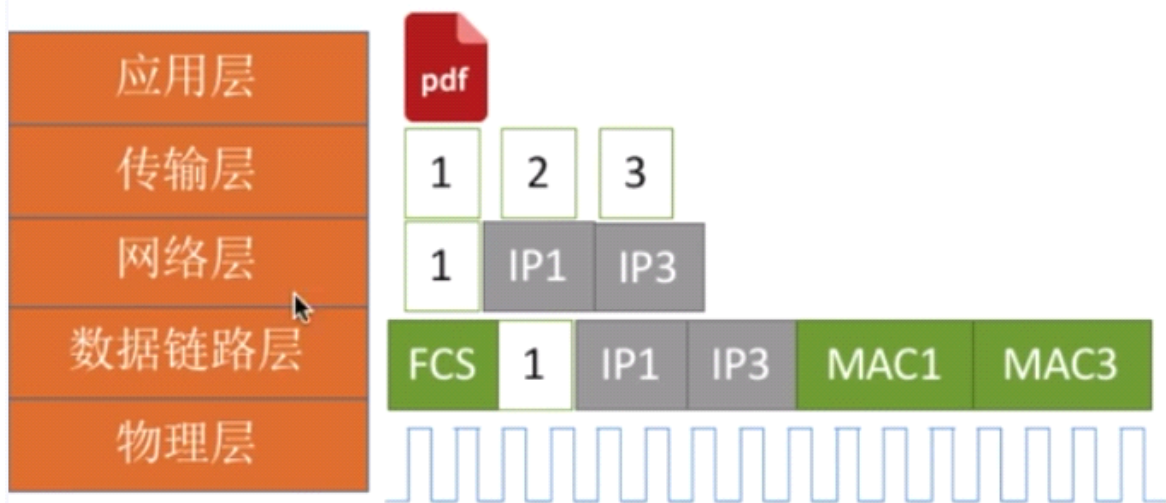
- 当主机A欲向本局域网上的某个主机B发送IP数据报时，就先在其ARP高速缓存中查看有无主机B的IP地址。
 - 有：查出其对应的硬件地址，再将此硬件地址写入MAC帧，然后通过局域网将该MAC帧发往此硬件地址。
 - 没有：通过使用目的MAC地址为 FF-FF-FF-FF-FF-FF 的帧来封装并广播 ARP请求分组，可以使同一个局域网里的所有主机收到ARP请求。



- 当主机B收到该ARP请求后，就会向主机A单播一个ARP响应分组，分组中包含主机B的IP与MAC地址的映射关系。



- 主机A在收到后将此映射写入ARP缓存中（10-20min更新一次），然后按查询到的硬件地址发送MAC帧。



注意：

① ARP是解决同一个局域网上的主机或路由器的IP地址和硬件地址的映射问题。如果所要找的主机和源主机不在同一个局域网上，那么就要通过ARP协议找到一个位于本局域网上某个路由器的硬件地址，然后把分组发送给这个路由器，让这个路由器把分组转发给下一个网络。剩下的工作就由下一个网络来做。

② 尽管ARP请求分组是广播发送的，但是ARP响应分组是普通的单播，即从一个源地址发送到一个目的地址。

ARP协议4种典型情况

1. 发送方是主机，要把IP数据报发送到本网络上的另一个主机。这时用ARP找到目的主机的硬件地址；
2. 发送方是主机，要把IP数据报发送到另一个网络上的一个主机。这时用ARP找到本网络上的一个路由器的硬件地址，剩下的工作由这个路由器来完成；
3. 发送方是路由器，要把IP数据报转发到本网络上的一个主机。这时用ARP找到目的主机的硬件地址；
4. 发送方是路由器，要把IP数据报转发到另一个网络上的一个主机。这时用ARP找到本网络上的一个路由器的硬件地址，剩下的工作由找到的这个路由器完成。

从IP地址到硬件地址的解析是自动进行的，主机的用户对这种地址解析过程是无感知的。只要主机或路由器要和本网络上的另一个已知IP地址的主机或路由器进行通信，ARP协议就会自动地将这个IP地址解析为链路层所需要的硬件地址。

动态主机配置协议DHCP

概述

动态主机配置协议（Dynamic Host Configuration Protocol, DHCP）常用于给主机动态地分配IP地址，它提供了即插即用联网的机制，这种机制允许一台计算机加入新的网络和获取IP地址而不用手工参与。DHCP是应用层协议，它是基于UDP的。

DHCP协议工作原理

1. 需要IP地址的主机在启动时就向DHCP服务器广播发送发现报文，这时该主机就成为DHCP客户。
2. 本地网络上所有主机都能收到此广播报文，但只有DHCP服务器才回答此广播报文。
3. DHCP服务器先在其数据库中查找该计算机的配置信息。若找到，则返回找到的信息。若找不到，则从服务器的IP地址池中取一个地址分配给该计算机。DHCP服务器的回答报文叫做提供报文。

DHCP使用客户/服务器方式。

DHCP服务器聚合客户端交换过程

1. DHCP客户机广播“DHCP发现”消息，试图找到网络中的DHCP服务器，服务器获得一个IP地址。
2. DHCP服务器收到“DHCP发现”消息后，就向网络中广播“DHCP提供”消息，其中包括提供DHCP客户机的IP地址和相关配置信息。
3. DHCP客户机收到“DHCP提供”消息，如果接受DHCP服务器所提供的相关参

数，则通过广播“DHCP请求”消息向DHCP服务器请求提供IP地址。

4. DHCP服务器广播“DHCP确认”消息，将IP地址分配给DHCP客户机。

DHCP协议允许网络上配置多台DHCP服务器，当DHCP客户机发出DHCP请求时，有可能收到多个应答消息。这时，DHCP客户机只会挑选其中一个，通常挑选最先到达的。

DHCP服务器分配给DHCP客户的IP地址是临时的，因此DHCP客户只能在一段有限的时间内使用这个分配到的IP地址。DHCP称这段时间为租用期。租用期的数值应由DHCP服务器自己决定，DHCP客户也可在自己发送的报文中提出对租用期的要求。

DHCP协议的客户端和服务端需要通过广播方式来进行交互，原因是在DHCP协议执行期间，客户端和服务端都没有标识自己身份的IP地址，因此不可能通过单播的形式进行交互。采用UDP而不采用TCP的原因也很明显，TCP协议需要建立连接，如果连对方的IP地址都不知道，更不可能通过双方的套接字建立连接了。

网际控制报文协议ICMP

概述

为了提高IP数据报交付成功的机会，在网络层使用了网际控制报文协议（Internet Control Message Protocol, ICMP）来允许主机或路由器报告差错和异常情况。ICMP报文作为IP层数据报的数据，加上数据报的首部，组成IP数据报发送出去。ICMP协议是IP层协议。

ICMP报文的种类有两种，即ICMP差错报告报文和ICMP询问报文。

ICMP差错报告报文

ICMP差错报告报文用于目标主机或到目标主机路径上的路由器向源主机报告差错和异常情况。

发送ICMP差错报告报文五种类型：

1. **终点不可达：**当路由器或主机不能交付数据报时就向源点发送终点不可达报文。
2. **源点抑制：**当路由器或主机由于拥塞而丢弃数据报时，就向源点发送源点抑制报文，使源点知道应当把数据报的发送速率放慢。
3. **时间超过：**当路由器收到生存时间TTL为零的数据报时，除丢弃该数据报外，还要向源点发送时间超过报文。当终点在预先规定的时间内不能收到一个数据报的全部数据报片时，就把已收到的数据报片都丢弃，并向源点发送时间超过报文。
4. **参数问题：**当路由器或目的主机收到的数据报的首部中有的字段的值不正确时，就丢弃该数据报，并向源点发送参数问题报文。
5. **改变路由(重定向)：**路由器把改变路由报文发送给主机，让主机知道下次应将数据报发送给另外的路由器(可通过更好的路由)。

不应发送ICMP差错报告报文的几种情况：

1. 对ICMP差错报告报文不再发送ICMP差错报告报文。
2. 对第一个分片的数据报片的所有后续数据报片都不发送ICMP差错报告报文。
3. 对具有组播地址的数据报都不发送ICMP差错报告报文。
4. 对具有特殊地址（如127.0.0.0或0.0.0.0）的数据报不发送ICMP差错报告报文。

ICMP询问报文

ICMP询问报文有四种类型，最常用的是前两类。

1. 回送请求和回答报文

- 主机或路由器向特定目的主机发出的询问，收到此报文的主机必须给源主机或路由器发送ICMP回答报文。试目的站是否可达以及了解其相关状态。

2. 时间戳请求和回答报文

- 请某个主机或路由器回答当前的日期和时间。用来进行时钟同步和测量时间。

3. 掩码地址请求和回答报文

4. 路由器询问和通告报文

ICMP应用

ICMP的两个常见的应用是分组网间探测PING和traceroute。

PING (Packet Internet Groper)：用来测试两个主机之间的连通性，使用的是ICMP回送请求和回答报文。

traceroute：NUIX中的名字，在Windows中是 `tracert`，可以用来跟踪分组经过的路由，使用的是ICMP时间超过报文。

注意：PING工作在应用层，它直接使用网络层的ICMP协议，而没有使用传输层的TCP或UDP协议。`traceroute/tracert`工作在网络层。

4.4 IPv6

4.4.1 IPv6主要特点

概述

要解决IP地址耗尽的问题的措施有以下三种：

1. 采用无类别编址CIDR，使IP地址的分配更加合理。
2. 采用网络地址转换NAT方法以节省全球IP地址。
3. 采用具有更大地址空间的新版本的IP协议IPv6。

其中前两种方法只是延长了IPv4地址分配结束的时间，只有第三种方法从根本上解决了IP地址的耗尽问题。

IPv6主要特点

1. 更大的地址空间。IPv6将地址从IPv4的32位增大到了128位。IPv6的字节数（16字节）是IPv4字节数（4字节）的平方。
2. 扩展的地址层次结构。
3. 灵活的首部格式。
4. 改进的选项。
5. 允许协议继续扩充。

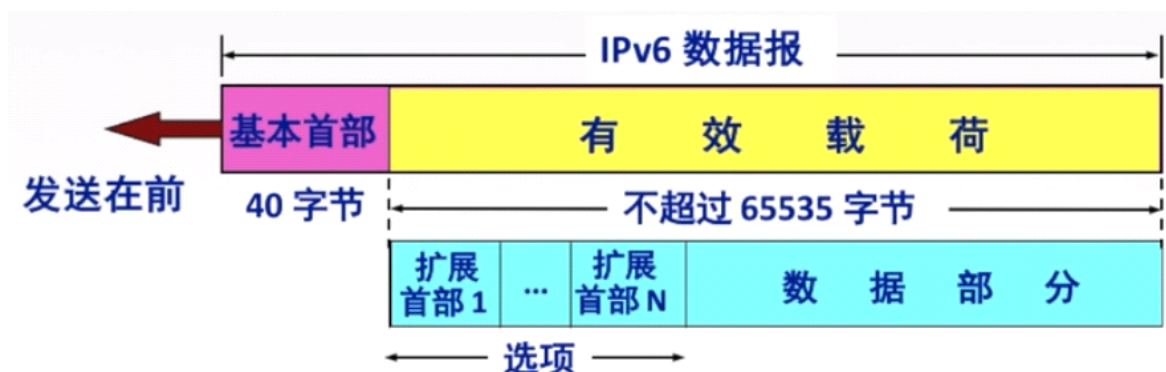
6. 支持即插即用(即自动配置)。
7. 支持资源的预分配。
8. IPv6只有在包的源结点才能分片，是端到端的，传输路径中的路由器不能分片，所以从一般的意义上来讲，IPv6不允许分片（不允许类似IPv4在路由分片）。
9. IPv6首部长度的必须是8B的整数倍，而IPv4首部是4B的整数倍。
10. 增加了安全性。身份验证和保密功能是IPv6的关键特征。

IPv6主要优势

1. **IPv6有比IPv4长得多的地址。** IPv6的地址用16个字节表示，地址空间是IPv4的 $2^{128-32} = 2^{96}$ 倍，从长远的未来来说，这些地址是绝对够用的。
2. **改进首部格式，更好地支持选项。** 一些从前是必要的段现在变成了可选，此外，表示选项的方式的改变还能加快分组的处理速度。
3. **简化了IP分组头，能更快速处理/转发数据报。** 它包含8个域（IPv4是12个域）。这一改变使得路由器能够更快的处理分组，从而可以改善吞吐率。
4. **支持QOS算法**（Quality of Service, QOS，指一个网络能够利用各种基础技术，为指定的网络通信提供更好的服务能力，是网络的一种安全机制，是用来解决网络延迟和阻塞等问题的一种技术）。

4.4.2 IPv6 地址

IPv6数据报



固定40B基本首部



IPv6数据报首部字段

版本：指明了协议版本，总是6。

优先级：区分数据报的类别和优先级。

流标签：“流”是互联网络上从特定源点到特定终点的一系列数据报。所有属于同一个流的数据报都具有同样的流标签。

下一个首部：标识下一个扩展首部或上层协议首部。

跳数限制：相当于IPv4的TTL。

IPv6 & IPv4 区别

1. IPv6将地址从32位 (4B) 扩大到128位 (16B)，更大的地址空间。
2. IPv6将IPv4的**校验和字段彻底移除**，以减少每跳的处理时间。
3. IPv6将IPv4的可选字段移出首部，变成了扩展首部，成为灵活的首部格式，路由器通常不对扩展首部进行检查，大大提高了路由器的处理效率。
4. IPv6支持即插即用（即自动配置），不需要DHCP协议。
5. IPv6首部长度必须是8B的整数倍，IPv4首部是4B的整数倍。
6. IPv6只能在主机处分片，IPv4可以在路由器和主机处分片。
7. ICMPv6：附加报文类型“分组过大”。
8. IPv6支持资源的预分配，支持实时视像等要求，保证一定的带宽和时延的应用。
9. IPv6取消了协议字段，改成下一个首部字段。
10. IPv6取消了总长度字段，改用有效载荷长度字段。
11. IPv6取消了服务类型字段。

IPv6地址表示形式

- 一般形式

- 冒号十六进制记法：4BF5:AA12:0216:FEBC:BA5F:039A:BE9A:2170。
- 压缩形式
 - 4BF5:0000:0000:0000:BA5F:039A:000A:2176 -> 4BF5:0:0:0:BA5F:39A:A:2176。
- 零压缩：一连串连续的0可以被一对冒号取代。如 FF05:0:0:0:0:0:0:B3 -> FF05::B3。
- 双冒号表示法在一个地址中仅可出现一次。

注意：IPv4 地址通常使用点分十进制表示法。

IPv6基本地址类型

- **单播**：就是传统的点对点通信，即一对一通信，可做源地址+目的地址。
- **多播**：一点对多点的通信，分组被交付到一组计算机的每一个，可做目的地址。
- **任播**：IPv6增加的一种类型，任播的目的站是一组计算机，但数据报在交付时只交付其中的一个，通常是距离最近的一个，属于一对多中的一个通信，可做目的地址。

IPv6向IPv4过渡的策略

双栈协议：双协议栈技术就是指在一台设备上同时启用IPv4协议栈和IPv6协议栈。这样的话，这台设备既能和IPv4网络通信，又能和IPv6网络通信。如果这台设备是一个路由器，那么这台路由器的不同接口上，分别配置了IPv4地址和IPv6地址，并很可能分别连接了IPv4网络和IPv6网络。如果这台设备是一个计算机，那么它将同时拥有IPv4地址和IPv6地址，并具备同时处理这两个协议地址的功能。

隧道技术：通过使用互联网的基础设施在网络之间传递数据的方式。使用隧道传递的数据（或负载）可以是不同协议的数据帧或包。隧道协议将其它协议的数据帧或包重新封装然后通过隧道发送。



4.5 路由协议

背景

- 因特网规模很大
- 许多单位不想让外界知道自己的路由选择协议，但还想连入因特网

4.5.1 自治系统

概述

自治系统 (Autonomous system, AS)：在单一的技术管理下的一组路由器，而这些

路由器使用一种AS内部的路由选择协议和共同的度量以确定分组在该AS内的路由，同时还使用一种AS之间的路由选择协议用以确定分组在AS之间的路由。

一个自治系统内的所有网络都属于一个行政单位（如 一个公司、一所大学、一个政府部门等）来管辖，一个自治系统的所有路由器在本自治系统内都必须是连通的。

4.5.2 域内路由与域间路由

概述

自治系统内部的路由选择称域内路由选择，自治系统之间的路由选择称域间路由选择。

两大类路由选择协议

- **内部网关协议（Interior Gateway Protocol, IGP）**

即在一个自治系统内部使用的路由选择协议，它与互联网中其他自治系统选用什么路由选择协议无关。目前这类路由选择协议使用得最多，如RIP协议和OSPF协议。

- **外部网关协议（External Gateway Protocol, EGP）**

若源站和目的站处在不同的自治系统中，当数据报传到一个自治系统的边界时（这两个自治系统可能使用不同的IGP），就需要使用一种协议将路由选择信息传递到另一个自治系统中。这样的协议就是外部网关协议EGP。目前使用最多的外部网关协议是BGP-4。



4.5.3 RIP协议

概述

- RIP是一种分布式的基于距离向量的路由选择协议，是因特网的协议标准，最大优点是**简单**。
- RIP协议规定：
 - 网络中每一个路由器都要维护从它自己到其他每一个目的网络的唯一最佳距离记录（即一组距离）。
 - 距离也称为“跳数”，即从源端口到目的端口所经过的路由器个数，经过一个路由器跳数+1。规定从一路由器到直接连接的网络距离为1。
 - RIP认为好的路由就是它通过的路由器的数目少，即优先选择跳数少的路径。
 - RIP允许一条路由最多只能包含15个路由器（即最多允许15跳），因此距离

等于16时，表示网络不可到达。可见RIP协议只适用于小型互联网。

注意：距离向量路由可能会出现环路的情况，规定路径上的最高跳数的目的是为了以防数据报不断循环在环路上，减少网络拥塞的可能性。

- RIP默认在任意两个使用RIP协议的路由器之间每30秒广播一次RIP路由更新信息。以便自动建立并维护路由表（动态维护）。
- 在RIP中不支持子网掩码的RIP广播，所以RIP中每个网络的子网掩码必须相同。但在新的RIP2中，支持变长子网掩码和CIDR。



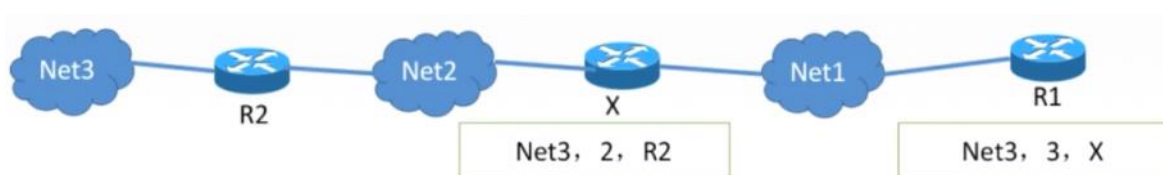
RIP协议特点

1. 仅和相邻路由器交换信息。
 2. 路由器交换的信息是当前本路由器所知道的全部信息，即自己的路由表。
 3. 按固定的时间间隔交换路由信息，如每30秒交换一次路由信息，然后路由器根据新信息更新路由表。若超过180s没收到邻居路由器的通告，则判定邻居没了，并更新自己路由表。
- 路由器刚开始工作时，只知道直接连接的网络的距离（距离为1），接着每一个路由器也只和数目非常有限的相邻路由器交换并更新路由信息。
 - 经过若干次更新后，所有路由器最终都会知道到达本自治系统任何一个网络的最短距离和下一跳路由器的地址，即“收敛”。

距离向量算法

每一个路由表项目都有3个关键数据：① 目的网络N，②距离d，③下一跳路由器X，对于每一个相邻路由器发送过来的RIP报文，进行以下步骤：

- 1、修改相邻路由器发来的RIP报文中**所有表项**：对地址为X的相邻路由器发来的RIP报文，先修改此报文中的所有项目：把“下一跳”字段中的地址都改为X，并把所有的“距离”字段的值加1。

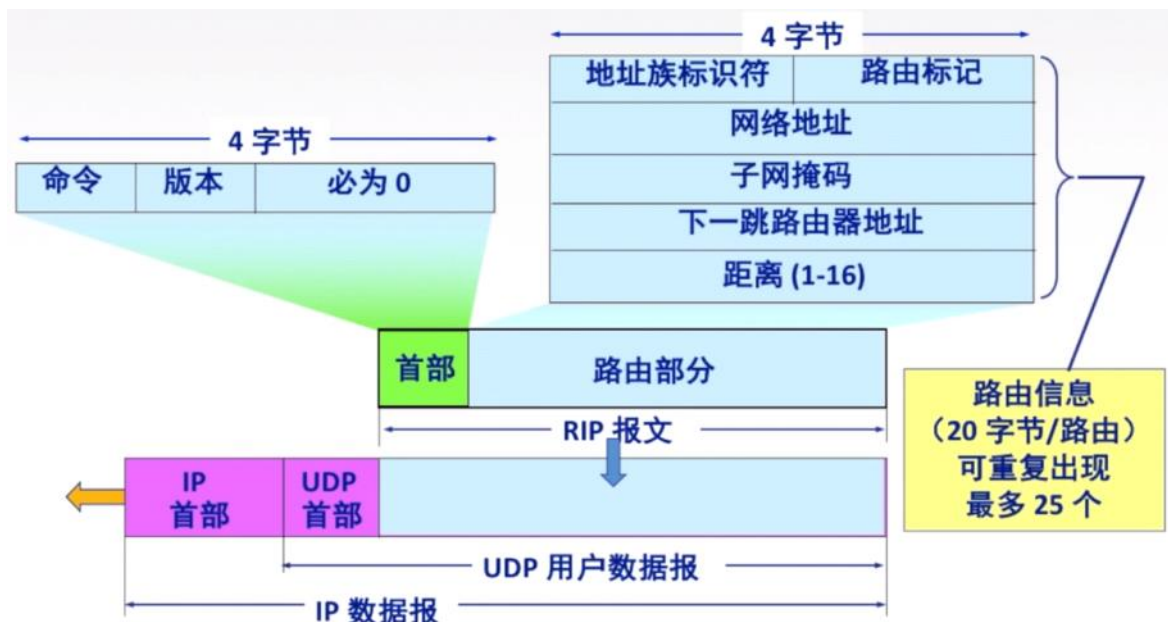


- 2、对修改后的RIP报文中的每一个项目，进行以下步骤：

- ① 原来的路由表中没有目的网络N时，则把该项目添加到路由表中。
 - ② 当原来的路由表中有目的网络N，若下一跳路由器地址是X，则用收到的项目替换原路由表中的项目；若下一跳不是X，原来距离比从X走的距离远则更新，否则不作处理。
- 3、若180s（RIP默认超时时间为180秒）还没收到相邻路由器X的更新路由表，则把此相邻路由器记为不可达的路由器，即把距离置为16（距离为16表示不可达）。

- 4、返回

RIP协议报文格式



RIP协议是应用层协议，使用UDP传送数据。

一个RIP报文最多个包括25个路由。如超过，必须再用一个RIP报文传送。

RIP优点 & 缺点

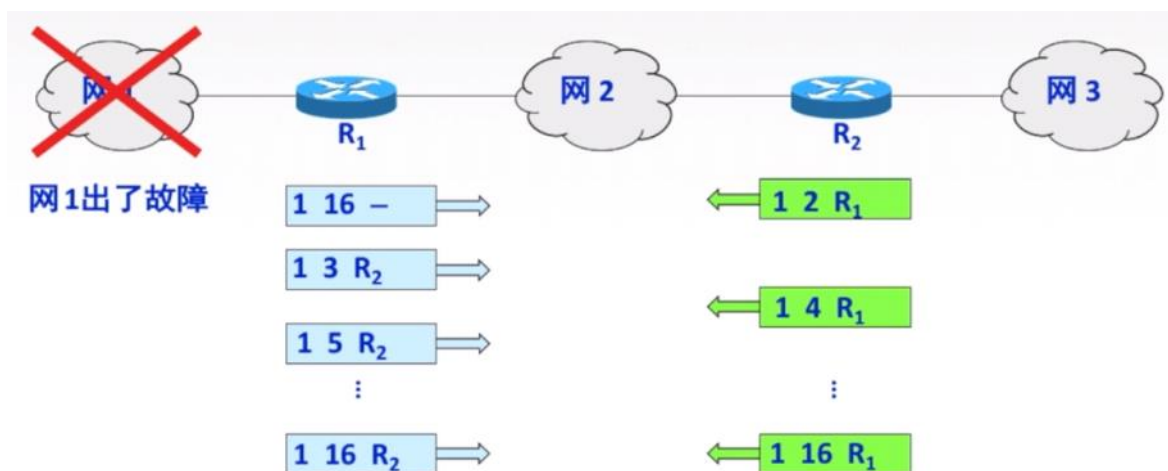
优点：实现简单，开销小，收敛过程较快。

缺点：

1. RIP限制了网络的规模，它能使用的最大距离为15（16表示不可达）。
2. 路由器之间交换的是路由器中的完整路由表，因此网络规模越大，开销也越大。
3. 当网络出现故障时，会出现慢收敛现象（即需要较长时间才能将此信息传送到所有路由器），俗称“坏消息传得慢”，使更新过程的收敛时间长。



正常情况



异常情况

RIP是应用层协议，它使用UDP传送数据（端口520）。RIP协议选择的路径不一定是时间最短的，但一定是具有最少路由器的路径。因为它是根据最少的跳数进行路径选择。

4.5.4 OSPF协议

概述

开放最短路径优先OSPF（Open Shortest Path First, OSPF）协议：“开放”表明OSPF协议不是受某一家厂商控制，而是公开发表的；“最短路径优先”是因为使用了Dijkstra提出的最短路径算法SPF。

OSPF最主要的特征就是使用分布式的链路状态协议，是使用分布式链路状态路由算法的典型代表，它也是内部网关协议IGP的一种。

基本特点

1. OSPF向本自治系统中所有路由器发送信息，这里使用的方法是洪泛法（即路由器通过广播方式，向通过输出端口向所有相邻的路由器发送信息，而每一个相邻路由器又再次将此信息发往其所有的相邻路由器）。
对比：RIP协议仅向自己相邻的几个路由器发送信息。
2. 发送的信息就是与本路由器**相邻的所有路由器的链路状态**（“链路状态”就是说明本路由器和哪些路由器相邻，以及该链路的度量/代价如费用、距离、时延、带宽等）。
对比：在RIP协议中，发送的信息是本路由器所知道的全部信息，即整个路由表。
3. 只有当**链路状态发生变化时**，路由器才用洪泛法向所有路由器发送此信息，并且更新过程收敛得快，不会出现RIP协议“坏消息传得慢”的问题。
对比：在RIP协议中，不管网络拓扑是否发生变化，路由器之间都会定期交换路由表的信息。
4. OSPF是网络层协议，它不使用UDP或TCP，而是直接IP数据报传送（其IP数据报首部的协议字段为89）。
对比：RIP是应用层协议，它在传输层使用UDP协议。
5. OSPF对不同的链路可根据IP分组的不同服务类型（TOS）而设置成不同的代价。因此，OSPF对于不同类型的业务可计算出不同的路由，因此十分灵活。
6. 如果到同一个目的网络有多条相同代价的路径，那么可以将通信量分配给这几条路径。这叫做多路径间的**负载均衡**。
7. 所有在OSPF路由器之间交换的分组都具有鉴别的功能，因而保证了仅在可信赖的路由器之间交换链路状态信息。
8. 支持可变长度的子网划分和无分类编址CIDR。
9. 每一个链路状态都带上一个32位的序号，序号越大状态就越新。

基本工作原理

1. 由于各路由器之间频繁地交换链路状态信息，因此所有的路由器最终都能建立一个链路状态数据库。这个数据库实际上就是全网的拓扑结构图，它在全网范围内是一致的（这称为链路状态数据库的同步）。
2. 然后，每个路由器就根据这个全网拓扑结构图，使用Dijkstra最短路径算法计算从自己到各目的网络最优路径，以此构造自己的路由表。

3. 在此之后，当链路状态发生变化时，每个路由器重新计算到各目的网络的最优路径，构造新的路由表。

注意：虽然使用Dijkstra算法能计算出完整的最优路径，但是路由表中不会存储完整路径，而只存储“下一跳”（只有到了下一跳路由器，才能知道再下一跳应当怎样走）。

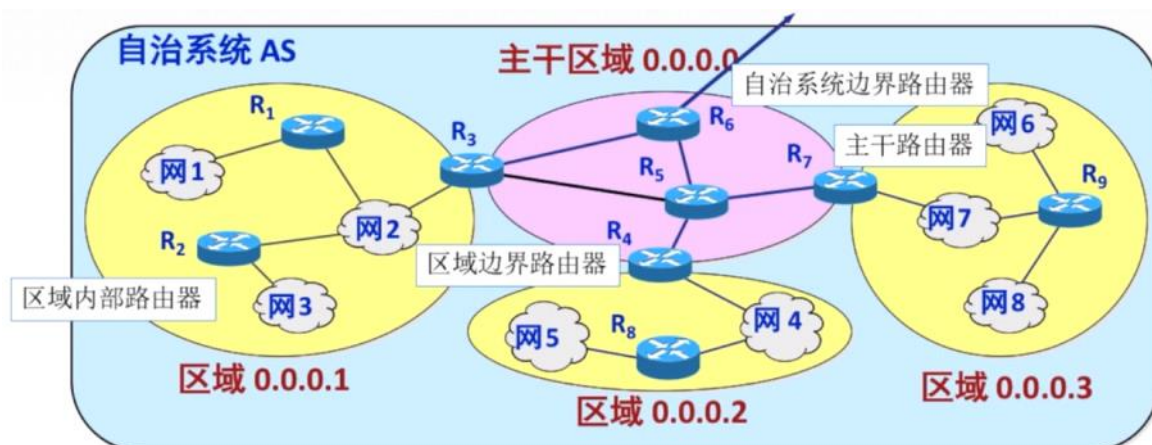
链路状态路由算法

1. 每个路由器发现它的邻居结点【hello问候组】，并了解邻居节点的网络地址。
2. 设置到它的每个邻居的成本度量metric。
3. 构造【DD数据库描述分组】，向邻站给出自己的链路状态数据库中的所有链路状态项目的摘要信息。
4. 如果DD分组中的摘要自己都有，则邻站不做处理；如果有没有的或者是更新的，则发送【LSR链路状态请求分组】请求自己没有的和比自己更新的信息。
5. 收到邻站的LSR分组后，发送【LSU链路状态更新分组】进行更新。
6. 更新完毕后，邻站返回一个【LSAck链路状态确认分组】进行确认。

只要一个路由器的链路状态发生变化：

- 泛洪发送【LSU链路状态更新分组】进行更新。
 - 更新完毕后，其他站返回一个【LSAck链路状态确认分组】进行确认。
7. 使用Dijkstra根据自己的链路状态数据库构造到其他节点间的最短路径。

OSPF区域

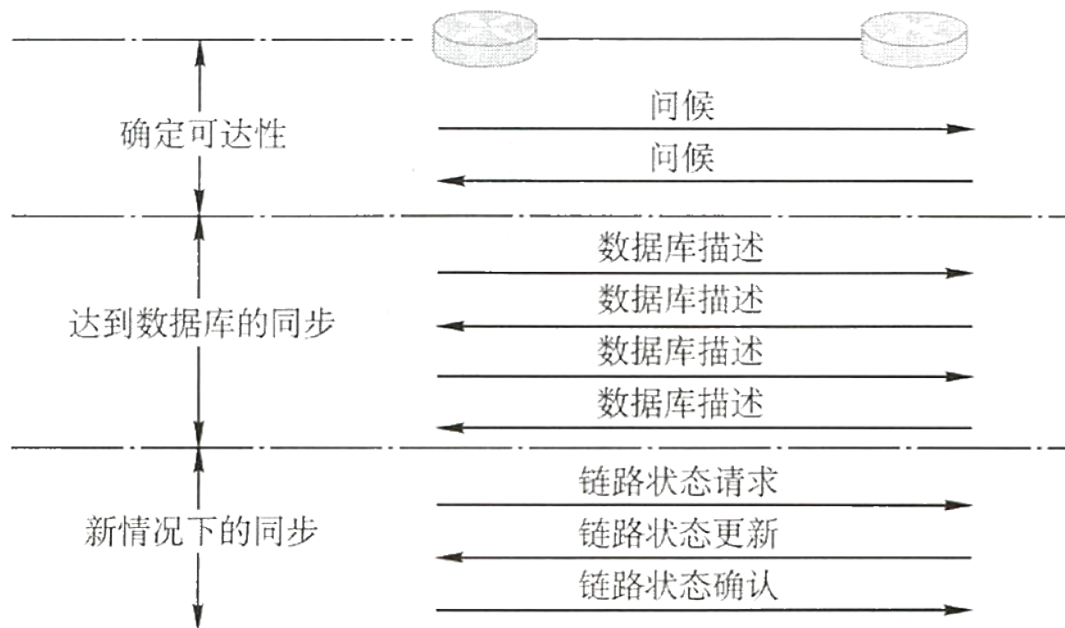


- 为了使OSPF能够用于规模很大的网络，OSPF将一个自治系统再划分为若干个更小的范围，叫做区域。
- **优点：**将利用洪泛法交换链路状态信息的范围局限于每一个区域而不是整个的自治系统，这就**减少了整个网络上的通信量**。在一个区域内部的路由器只知道本区域的完整网络拓扑，而不知道其他区域的网络拓扑的情况。
- 这些区域也有层次之分。处在上层的域叫主干区域，负责连通其他下层的区域，并且还连接其他自治域。
- 每一个区域都有一个32位的区域标识符（用点分十进制表示）。
- 区域也不能太大，在一个区域内的路由器最好不超过200个。

OSPF分组

OSPF五种分组类型：

1. 问候分组，用来发现和维持邻站的可达性。通常每隔10秒，每两个相邻路由器要交换一次问候分组，以便知道哪些站可达。
2. 数据库描述分组，向邻站给出自己的链路状态数据库中的所有链路状态项目的摘要信息。
3. 链路状态请求分组，向对方请求发送某些链路状态项目的详细信息。
4. 链路状态更新分组，用洪泛法对全网更新链路状态。
5. 链路状态确认分组，对链路更新分组的确认。



OSPF的基本操作

OSPF基本操作：

1. 在路由器刚开始工作时，OSPF让每一个路由器使用数据库描述分组和相邻路由器交换本数据库中已有的链路状态摘要信息。
2. 然后，路由器就使用链路状态请求分组，向对方请求发送自己所缺少的某些链路状态项目的详细信息。
3. 经过一系列的这种分组交换，全网同步的链路数据库就建立了。



注意：OSPF协议不使用UDP数据报传送，而是**直接使用IP数据报传送**。用UDP传送是指将该信息作为UDP报文的数据部分，而直接使用 IP数据报传送是指将该信息直接作为IP数据报的数据部分。RIP报文是作为UDP数据报的数据部分。

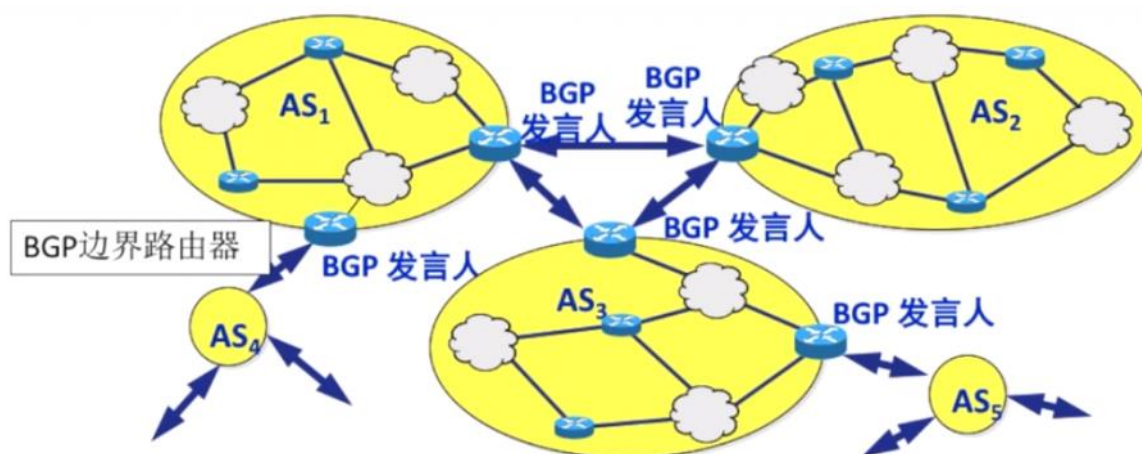
其他特点

1. 为了确保链路状态数据库与全网的状态保持一致，OSPF还规定每隔一段时间，如**30分钟要刷新一次数据库中的链路状态**。
2. 由于一个路由器的链路状态只涉及到与相邻路由器的连通状态，因而与整个互联网的规模并无直接关系。因此当互联网规模很大时，OSPF协议要比距离向量协议RIP好得多。
3. OSPF不存在坏消息传的慢的问题，它的收敛速度很快。

4.5.5 BGP协议

概述

边界网关协议 (Border Gateway Protocol, BGP) 是不同自治系统的路由器之间交换路由信息的协议。它是一种外部网关协议，边界网关协议常常应用于互联网的网关之间。路由表包含已知路由器的列表、路由器能够达到的地址以及到达每个路由器的路径的跳数。



内部网关协议主要是设法使数据报在一个AS中尽可能有效地从源站选送到目的站。在一个AS内部也不需要考虑其他方面的策略。然而 BGP使用的环境却不同，主要因为：

1. 因特网的规模太大，使得自治系统之间路由选择非常困难。
2. 对于自治系统之间的路由选择，要寻找最佳路由是很不现实的。
3. 自治系统之间的路由选择必须考虑有关策略。

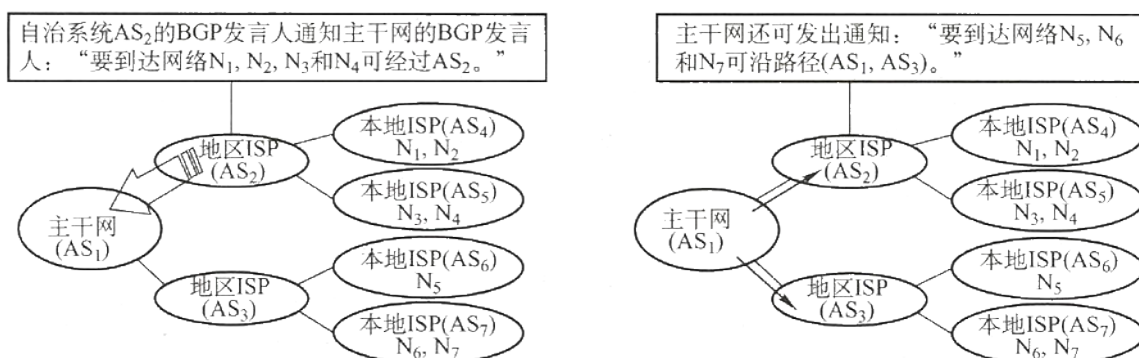
边界网关协议BGP只能是力求寻找一条能够到达目的网络且比较好的路由（不能兜圈子），而并非要寻找一条最佳路由。BGP采用的是路径向量路由选择协议，它与距离向量协议和链路状态协议有很大的区别。BGP协议是应用层协议，它是基于TCP的。

BGP协议工作原理

1. 每一个自治系统的管理员要选择至少一个路由器（可以有多个）作为该自治系统的“BGP发言人”。
2. 一个BGP发言人与其他自治系统中的BGP发言人要交换路由信息，就要先建立TCP连接（可见BGP报文是通过TCP传送，也就是说BGP报文是TCP报文的数据部分）。
3. 然后在此连接上交换BGP报文以建立BGP会话，再利用BGP会话交换路由信息。

4. 当所有BGP发言人都相互交换网络可达性的信息后，各BGP发言人就可找出到达各个自治系统的比较好的路由。

每一个BGP发言人除了必须运行BGP协议外，还必须运行该AS所使用的内部网关协议，如OSPF或RIP。BGP所交换的网络可达性信息就是要到达某个网络（用网络前缀表示）所要经过的一系列AS。

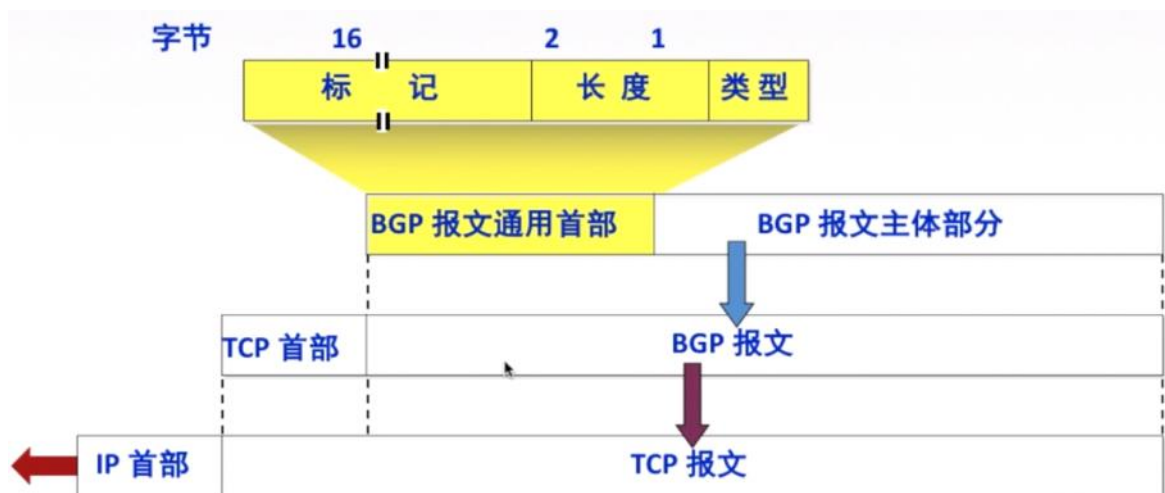


主干网与自治系统间路径向量的交换

报文格式

一个BGP发言人与其他自治系统中的BGP发言人要交换路由信息，就要先建立TCP连接，即通过TCP传送，然后在此连接上交换BGP报文以建立BGP会话（session），利用BGP会话交换路由信息。

BGP是应用层协议，借助TCP传送。



BGP协议特点

1. BGP协议交换路由信息的结点数量级是自治系统的数量级，这要比这些自治系统中的网络数少很多。
2. 每一个自治系统中BGP发言人（或边界路由器）的数目是很少的。这样就使得自治系统之间的路由选择不致过分复杂。
3. BGP支持CIDR，因此BGP的路由表也就应当包括目的网络前缀、下一跳路由器，以及到达该目的网络所要经过的各个自治系统序列。
4. 在BGP刚运行时，BGP的邻站是交换整个的BGP路由表。但以后只需要在发生变化时更新有变化的部分。这样做对节省网络带宽和减少路由器的处理开销方面都有好处。

BGP-4使用四种报文

1. 打开报文 (Open): 用来与相邻的另一个BGP发言人建立关系, 并认证发送方。
2. 更新报文 (Update): 通告新路径或撤销原路径。
3. 保活报文 (Keepalive): 在无update时, 周期性证实邻站的连通性; 也作为open的确认。
4. 通知报文 (Notification): 报告先前报文的差错; 也被用于关闭连接。

三种路由协议的比较

RIP是一种分布式的基于距离向量的内部网关路由选择协议, 通过广播UDP报文来交换路由信息。

OSPF是一个内部网关协议, 要交换的信息量较大, 应使报文的长度尽量短, 所以不使用传输层协议 (如UDP或TCP), 而是直接采用IP。

BGP是一个外部网关协议, 在不同的自治系统之间交换路由信息, 由于网络环境复杂, 需要保证可靠传输, 所以采用TCP。

协议	RIP	OSPF	BGP	
类型	内部	内部	外部	
路由算法	距离-向量	链路状态	路径-向量	
传递协议	UDP	IP	TCP	
路径选择	跳数最少	代价最低	较好, 非最佳	
交换结点	和本结点相邻的路由器	网络中的所有路由器	和本结点相邻的路由器	
交换内容	当前本路由器知道的全部信息, 即自己的路由表	与本路由器相邻的所有路由器的链路状态	首次	整个路由表
			非首次	有变化的部分

4.6 IP组播

4.6.1 组播的概念

概述

当网络中的某些用户需要特定数据, 像视频点播和视频会议这样的多媒体应用时, 网络必须实施某种有效的组播机制, 即让源计算机一次发送的单个分组可以抵达用一个组地址标识的若干台目标主机, 并被它们正确接收。

组播一定是仅应用于UDP, 数据发送者仅发送一次数据, 借助组播路由协议为组播数据包建立组播分发树, 被传递的数据到达距离用户端尽可能近的节点后才开始复制和分发, 用户主机可以选择加入或离开一个组, 从而一个主机可以同时属于多个组, 是一种点对多点传输方式。

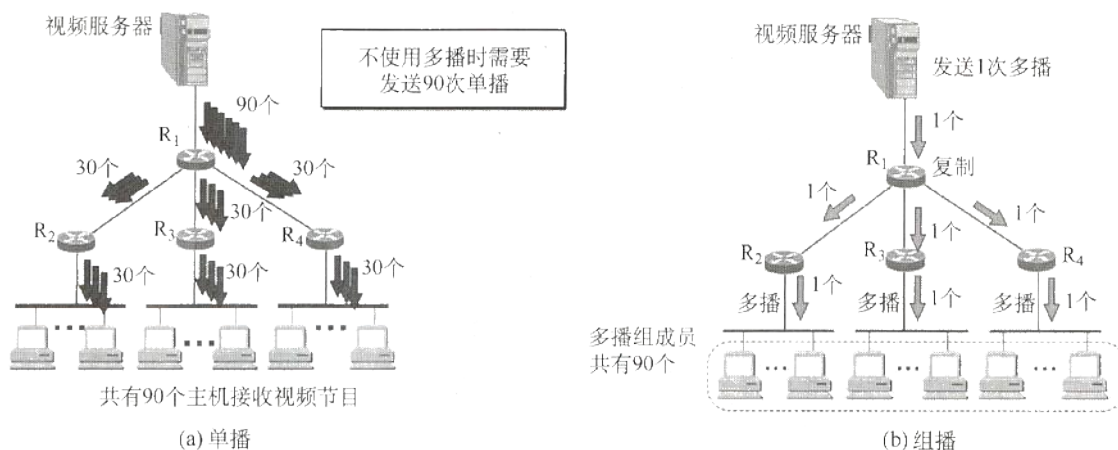
用户主机使用一个称作 IGMP (因特网组管理协议) 的协议加入组播组。它们使用该协议通知在本地网络上的路由器关于要接收发送给某个组播组的分组的愿望。通过扩展路由器的路由选择和转发功能, 可以在许多路由器互联的支持硬件组播的网络上面实现因特网组播。

优点

- 组播提高了数据传送效率。减少了主干网出现拥塞的可能性。组播组中的主机可以是在同一个物理网络, 也可以来自不同的物理网络 (如果有组播路由器的

支持)。

- 组播需要路由器的支持才能实现，能够运行组播协议的路由器称为组播路由器。



单播与组播的比较

4.6.2 IP组播地址

概述

IP组播使用D类地址格式。D类地址的前四位是1110，因此D类地址范围是 224.0.0.0 ~ 239.255.255.255。每一个D类IP地址标志一个组播组。

组播数据报和一般的IP数据报的区别在于它使用D类IP地址仅作为目的地址，并且首部中的协议字段值是2，表明使用IGMP协议。

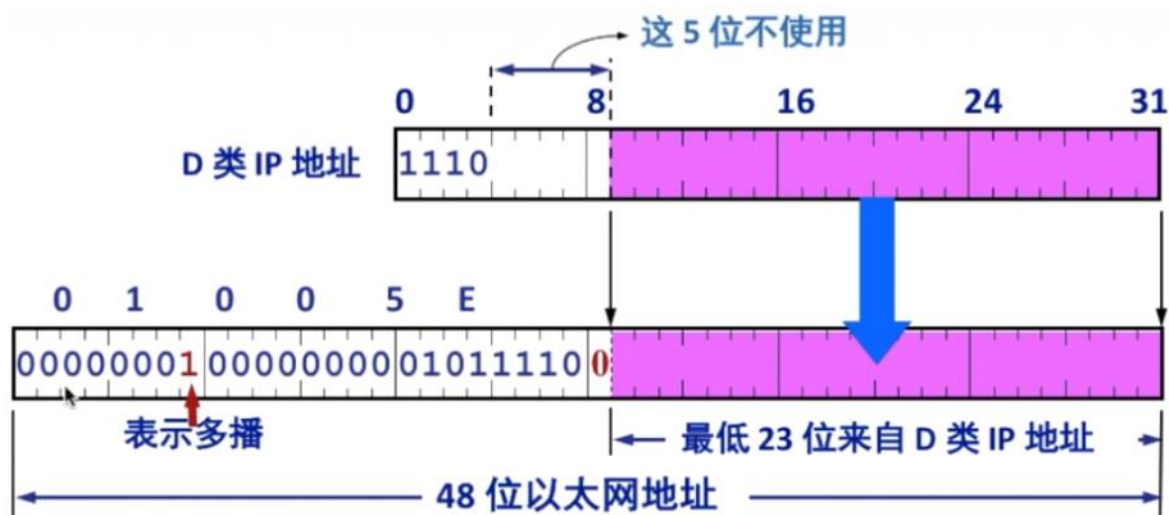
IP组播地址特点

- 组播数据报也是“尽最大努力交付”，不提供可靠交付。
- 组播地址只能用于目的地址，而不能用于源地址。
- 对组播数据报不产生ICMP差错报文。因此，若在PING命令后面键入组播地址，将永远不会收到响应。
- 并非所有的D类地址都可作为组播地址。

硬件组播

IP组播可以分为两种：① 只在本局域网上进行硬件组播；② 在因特网的范围内进行组播。在因特网上进行组播的最后阶段，还是要把组播数据报在局域网上用硬件组播交付给组播组的所有成员，即使用相应的组播MAC地址在本地网络中实际传送帧。

TCP/IP协议使用的以太网多播地址的范围是从01-00-5E-00-00-00 到 01-00-5E-7F-FF-FF。由于组播MAC地址以十六进制值01-00-5E打头，余下的6个十六进制位是根据IP组播组地址的最后23位转换得到的，所以在每一个地址中，只有23位可用作组播，和D类IP地址中的23位是一一对应关系。



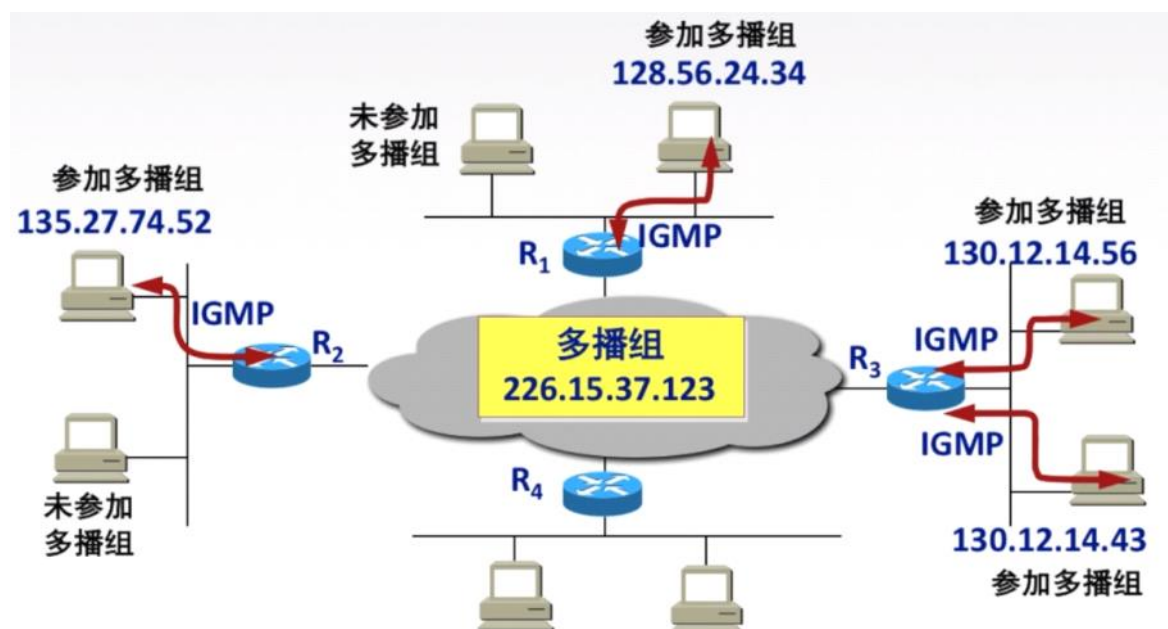
由于组播IP地址与以太网硬件地址的映射关系不是唯一的，因此收到组播数据报的主机，还要在IP层利用软件进行过滤，把不是本主机要接收的数据报丢弃。

4.6.3 IGMP协议与组播路由算法

概述

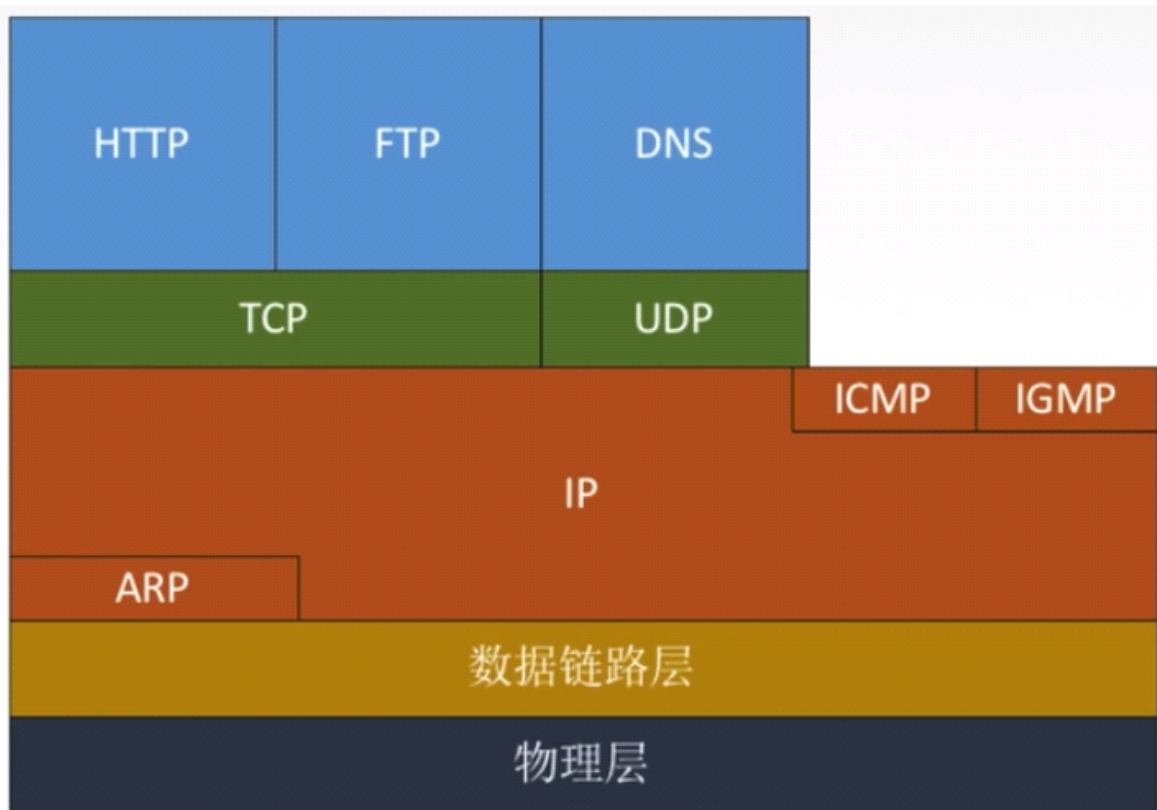
为了使路由器知道组播组成员的信息，需要利用到IGMP协议 (Internet Group Management Protocol)，IGMP协议让路由器知道本局域网上是否有主机 (的进程) 参加或退出了某个组播组。

连接在局域网上的组播路由器还必须和因特网上的其他组播路由器协同工作，以便把组播数据报用最小代价传送给所有的组成员，这就需要使用组播路由选择协议。



协议位置

ICMP和IGMP都使用IP数据报传递报文。



IGMP工作的两个阶段

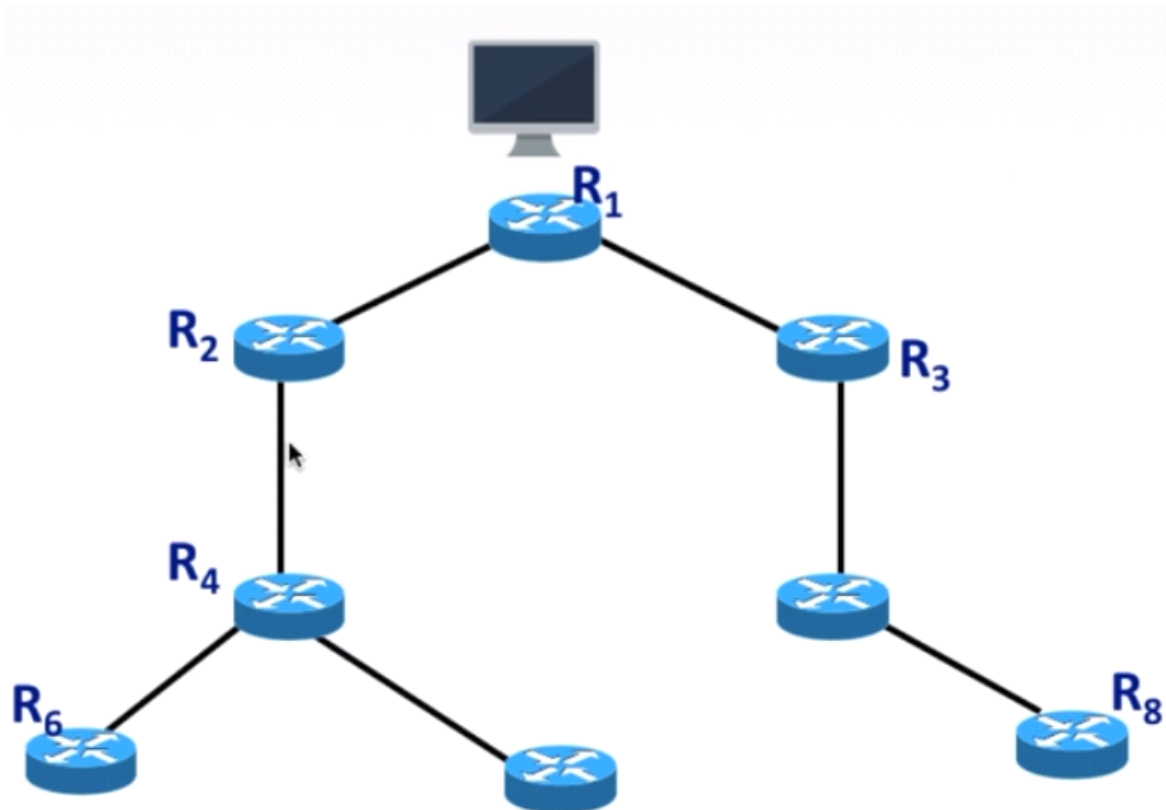
阶段1：某主机要加入组播组时，该主机向组播组的组播地址发送一个IGMP报文，声明自己要称为该组的成员。本地组播路由器收到IGMP报文后，要利用组播路由选择协议把这组成员关系发给因特网上的其他组播路由器。

阶段2：本地组播路由器周期性询问本地局域网上的主机，以便知道这些主机是否还是组播组的成员。只要有一个主机对某个组响应，那么组播路由器就认为这个组是活跃的；如果经过几次询问后没有一个主机响应，组播路由器就认为本网络上的没有此组播组的主机，因此就不再把这组的成员关系发给其他的组播路由器。

组播路由器知道的成员关系只是所连接的局域网中是否有组播组的成员。

组播路由选择协议

- 组播路由协议目的是找出以源主机为根节点的组播转发树。其中每一个分组在每条链路上只传送一次（即在组播转发树上的路由器不会收到重复的组播数据报）。
- 构造树可以避免在路由器之间兜圈子。
- 对不同的多播组对应于不同的多播转发树；同一个多播组，对不同的源点也会有不同的多播转发树。



主要路由算法

1. 基于链路状态的路由选择。
2. 基于距离-向量的路由选择。
3. 建立在任何路由器协议之上，因此称作协议无关的组播（PIM）。

4.7 移动IP

4.7.1 移动IP的概念

概述

支持移动性的因特网体系结构与协议被共同称为移动IP。**移动IP**是移动结点（计算机/服务器等）以固定的网络IP地址，实现跨越不同网段的漫游功能，并保证了基于网络IP的网络权限在漫游过程中不发生任何改变。移动IP的目标是把分组自动地投递给移动结点。

基于IPv4的移动IP定义三种功能实体：**移动结点**、**归属代理（也叫本地代理）**和**外埠代理（也叫外部代理）**。归属代理和外埠代理又统称为移动代理。

- **移动结点**：是把其连接点从一个网络或子网改变到另一个网络或子网的主机，即具有永久IP地址的移动设备。使用移动IP，一个移动结点可以在不改变其IP地址的情况下改变其驻留位置。
- **移动代理**
 - **归属代理（本地代理）**：一个移动结点的永久“居所”称为归属网络，在归属网络中代表移动节点执行移动管理功能的实体叫做归属代理（本地代理），它根据移动用户的转变地址，采用隧道技术转交移动结点的数据包。

- **外部代理（外地代理）**：在外部网络中帮助移动节点完成移动管理功能的实体称为外部代理。

移动IP和移动自组网络区别

- **移动IP**技术使漫游的主机可以用多种方式连接到因特网，移动IP的核心网络功能仍然是基于在固定互联网中一直在使用的各种路由选择协议。
- **移动自组网络**是将移动性扩展到无线领域中的自治系统，它具有自己独特的路由选择协议，并且可以不和因特网相连。

移动IP与动态IP区别

动态IP指的是局域网中的计算机可以通过网络中的DHCP服务器动态地获得一个IP地址，而不需要用户在计算机的网络设置中指定IP 地址。

4.7.2 移动IP通信过程

概述

在移动IP协议中，每一个移动结点都有一个唯一的本地地址，当移动结点移动时它的本地地址是不变的，在本地网络链路上每一个本地结点还必须有一个本地代理来为它维护当前的位置信息，这就需要引入**转交地址**。

当移动结点连接到外地网络链路上时，转交地址就用来标识移动结点现在所处的位置，以便进行路由选择。**移动结点的本地地址与当前转交地址的联合称作移动绑定或简称绑定**。当移动结点得到一个新的转交地址时，通过绑定向本地代理进行注册，以便让本地代理即时了解移动结点的当前位置。

永久地址（归属地址/主地址）：移动站点在归属网络中的原始地址。

转交地址（辅地址）：移动站点在外部网络使用的临时地址。

基本通信流程

1. 移动结点在本地网时，按传统的TCP/IP方式进行通信（在本地网固有的地址）。
2. 移动结点漫游到一个外地网络时，仍然使用固定的IP地址进行通信。为了能够收到通信对端发给它的IP分组，移动结点需要向本地代理注册当前的位置地址，这个位置地址就是转交地址（它可以是外部代理的地址或动态配置的一个地址）。
3. 本地代理接收来自转交地址的注册后，会构建一条通向转交地址的隧道，将截获的发给移动结点的IP分组通过隧道送到转交地址处。
4. 在转交地址处解除隧道封装，恢复出原始的IP分组，最后送到移动结点，这样移动结点在外网就能够收到这些发送给它的IP分组。
5. 移动结点在外网通过外网的路由器或者外代理向通信对端发送IP数据包。
6. 当移动结点来到另一个外网时，只需要向本地代理更新注册的转交地址，就可以继续通信。
7. 当移动结点回到本地网时，移动结点向本地代理注销转交地址，这时移动结点又将使用传统的 TCP/IP方式进行通信。

移动IP为移动主机设置了两个IP地址，即主地址和辅地址（转交地址）。移动主机

在本地网时，使用的是主地址。当移动到另一个网络时，需要获得一个临时的辅地址，但此时主地址仍然不变。当从外网移回本地网时，辅地址改变或撤销，而主地址仍然保持不变。

示例



1. A进入外部网络

1. 在外部代理登记获得一个转交地址，离开时注销；
2. 外地代理向本地代理登记转交地址。

2. A与B发生数据交互

- B给A发送数据报

1. 本地代理截获数据报；
2. 本地代理再封装数据报，新的数据报目的地址是转交地址，发给外部代理（隧道）；
3. 外部代理拆封数据报并发给A。

- A给B发送数据报

- A用自己的主地址作为数据报源地址，用B的IP地址作为数据报的目的地址。

- A移动到下一个网络

1. 在新外部代理登记注册一个转交地址；
2. 新外部代理给本地代理发送新的转交地址（覆盖旧的）；
3. 通信

3. A回到了归属网络

1. A向本地代理注销转交地址；
2. 按原始方式通信。

4.8 网络层设备

4.8.1 路由器的组成和功能

概念

路由器是一种具有多个输入端口和多个输出端口的专用计算机，其任务是连接不同的网络（连接异构网络）并完成路由转发。在多个逻辑网络（即多个广播域）互联时必须使用路由器。路由器也可以作为最基础的包过滤防火墙应用。

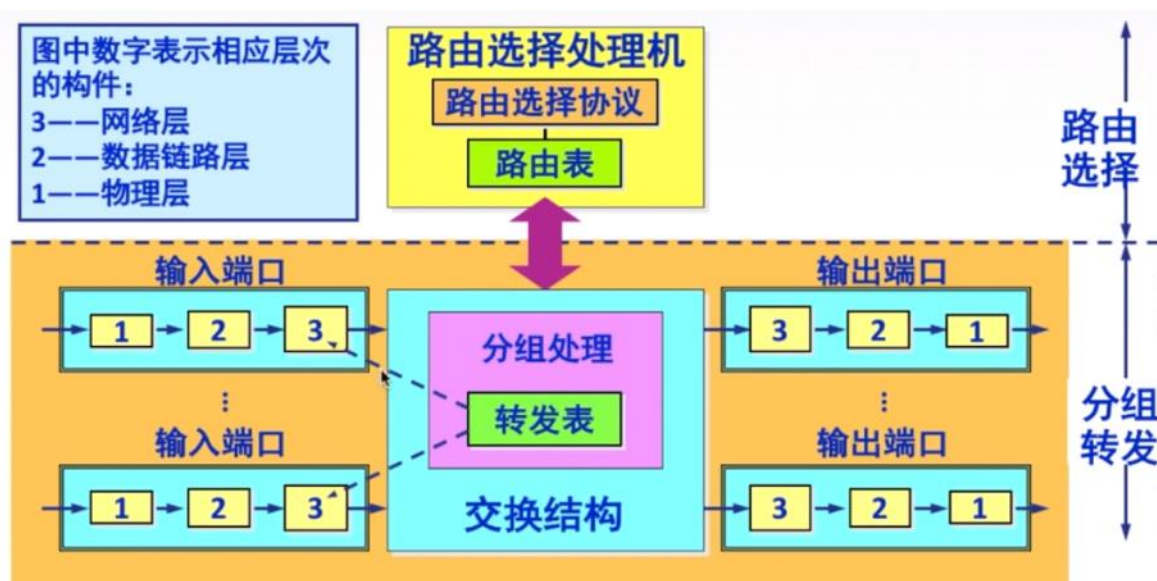
在同一个网络中传递数据无需路由器的参与，而跨网络通信必须通过路由器进行转

发：

1. 当源主机要向目标主机发送数据报时，路由器先检查源主机与目标主机是否连接在同一个网络上。
2. 如果源主机和目标主机在同一个网络上，则**直接交付**而无需通过路由器。
3. 如果源主机和目标主机不在同一个网络上，则路由器按照转发表(路由表)指出的路由将数据报转发给下一个路由器，称为**间接交付**。

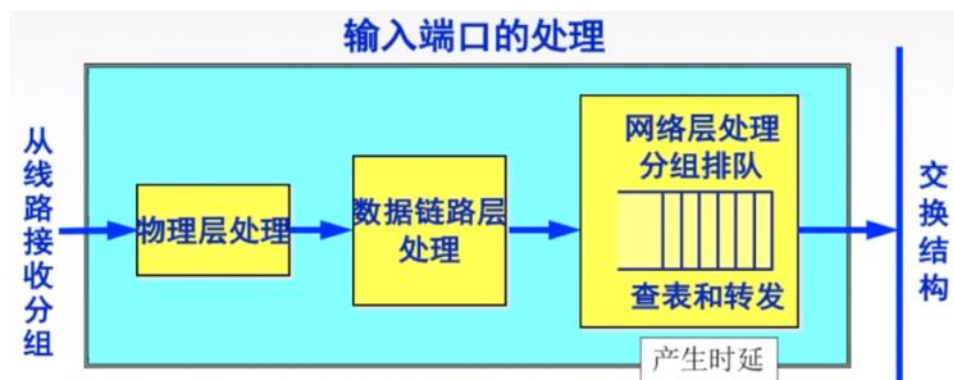
路由器构成与功能

从结构上看，路由器由路由选择和分组转发两部分构成，从模型的角度看，路由器是网络层设备，它实现了网络模型的下三层，即物理层、数据链路层和网络层。

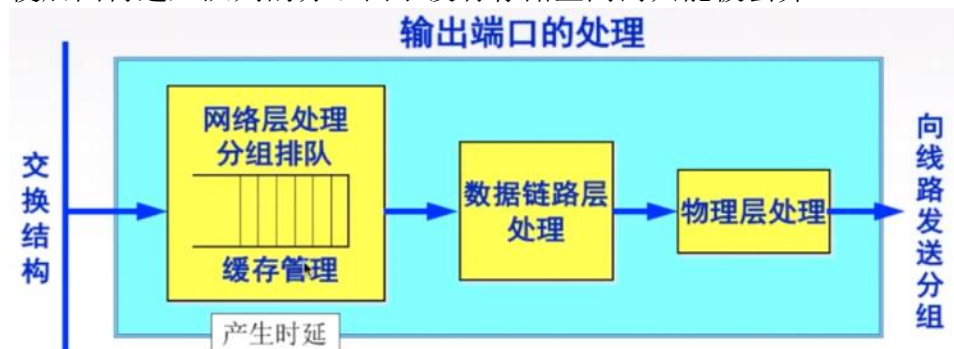


路由器体系结构

- **路由选择**
 - **结构**：也叫控制部分，其核心构件是路由选择处理机。
 - **功能**：根据所选定的路由选择协议构造出路由表，同时经常或定期地和相邻路由器交换路由信息而不断地更新和维护路由表。
- **分组转发**
 - **结构**：由 ① 交换结构、② 输入端口、③ 输出端口 三部分组成。
 - **交换结构**
 - **定义**：是路由器的关键部件，它根据转发表对分组进行处理，将某个输入端口进入的分组从一个合适的输出端口转发出去。交换结构本身就是一个网络。
 - **常用交换方法**：①通过存储器进行交换、②通过总线进行交换和③通过互联网络进行交换。
 - **输入端口**：把从物理层接收到的比特流中提取出链路层帧，进而从帧中提取出网络层数据报。输入端口中的查找和转发功能在路由器的交换功能中是最重要的。



- **输出端口**：与输入端口执行相反。若路由器处理分组的速率赶不上分组进入队列的速率，则队列的存储空间最终必定减少到零，这就使后面再进入队列的分组由于没有存储空间而只能被丢弃。



- 路由器中的输入或输出队列产生溢出是造成分组丢失的重要原因。
- **功能**：通过路由器的数据流，关键操作是转发表查询、转发以及相关的队列管理和任务调度等。

三层设备的区别

	能否隔离冲突域	能否隔离广播域
物理层设备【傻瓜】 (中继器、集线器)	×	×
链路层设备【路人】 (网桥、交换机)	√	×
网络层设备【大佬】 (路由器)	√	√

路由器可以互联两个不同网络层协议的网段。

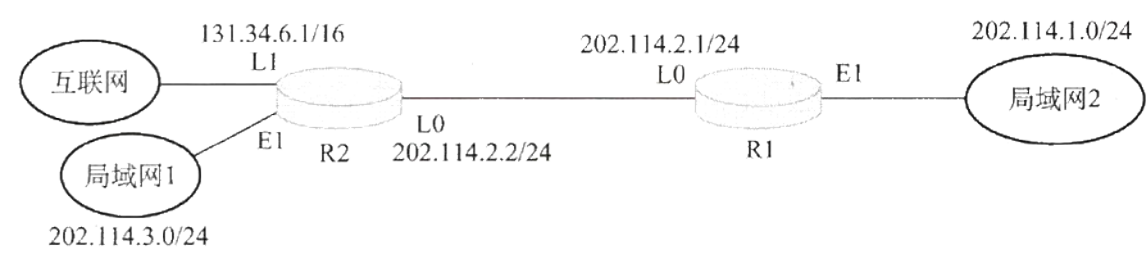
网桥可以互联两个物理层和链路层不同的网段。

集线器不能互联两个物理层不同的网段。

4.8.2 路由表和路由转发

路由表

根据路由选择算法得出的，主要用途是路由选择，总是用软件来实现。 标准的路由表具有四个项目：目的网络IP地址、子网掩码、下一跳IP地址、接口。



一个简单的网络拓扑

目的网络 IP 地址	子 网 掩 码	下一跳 IP 地址	接 口
202.114.1.0	255.255.255.0	Direct	E1
202.114.2.0	255.255.255.0	Direct	L0
202.114.3.0	255.255.255.0	202.114.2.2	L0
0.0.0.0	0.0.0.0	202.114.2.2	L0

R1中的路由表

转发表

由路由表得来，其表项和路由表项有直接对应关系。转发表必须包含完成转发功能所必需的信息，在转发表的每一行必须包含从要到达的目的网络到输出端口和某些MAC地址信息的映射。

与路由表区别

- **格式不同：**转发表结构应使查找过程最优化，而路由表则需对网络拓扑变化的计算最优化。
- **实现不同：**路由表总是用软件来实现；转发表可以用软件实现，甚至也可以用特殊的硬件来实现。

转发表中含有一个分组将要发往的目的地址，以及分组的下一跳（即下一步接收者的目的地址，实际为MAC地址）。为了减少转发表的重复项目，可以使用一个默认路由代替所有具有相同的“下一跳”的项目，并设置默认路由比其他项目的优先级低

目的站	下一跳
1	直接
2	3
3	2
4	3

a) 未使用默认路由

目的站	下一跳
1	直接
3	2
默认	3

b) 使用了默认路由

未使用默认路由的转发表和使用了默认路由的转发表的对比

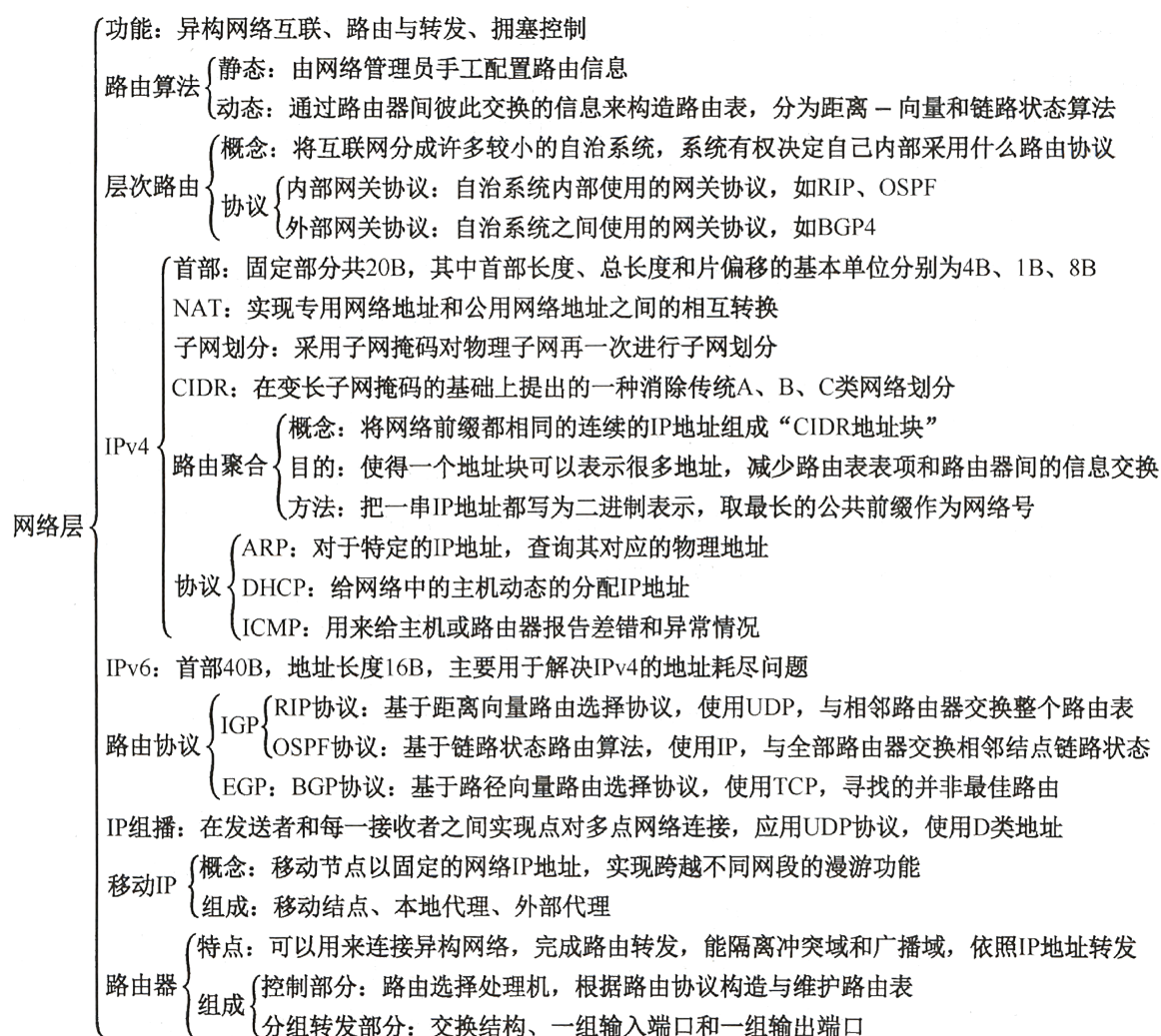
注意：分组的实际转发是靠直接查找转发表，而不是直接查找路由表。

转发和路由选择的区别

- **转发**是路由器根据转发表把收到的IP数据报从合适的端口转发出去，它仅涉及一个路由器。
- **路由选择**则涉及很多路由器，路由表是许多路由器协同工作的结果。这些路由器按照复杂的路由算法，根据从各相邻路由器得到的关于网络拓扑的变化情况，动态地改变所选择的路由，并由此构造出整个路由表。

4.9 小结

知识框架



第一章 计算机网络

星期日，三月 13，2022 5:21 下午

1.1 计算机网络概述

1.1.1 计算机网络的概念

概述

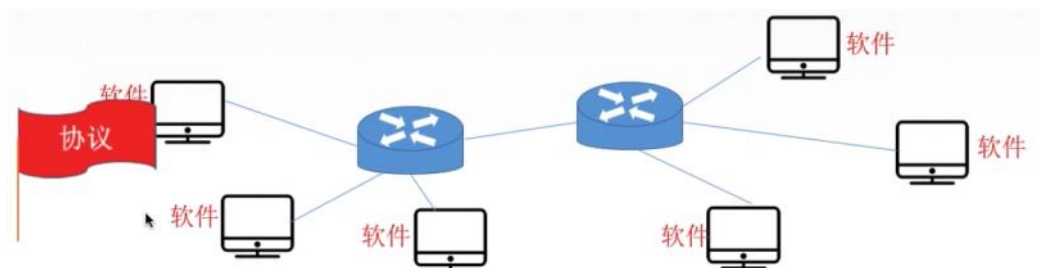
计算机网络是一个将分散的、具有独立功能的**计算机系统**，通过**通信设备与线路**连接起来，由功能完善的软件实现**资源共享**和**信息传递**的系统。简言之，12



关键词：① 计算机网络是互联的、自治的计算机集合。② 互联：互联互通，通信链路。③ 自治：无主从关系。

1.1.2 计算机网络的组成

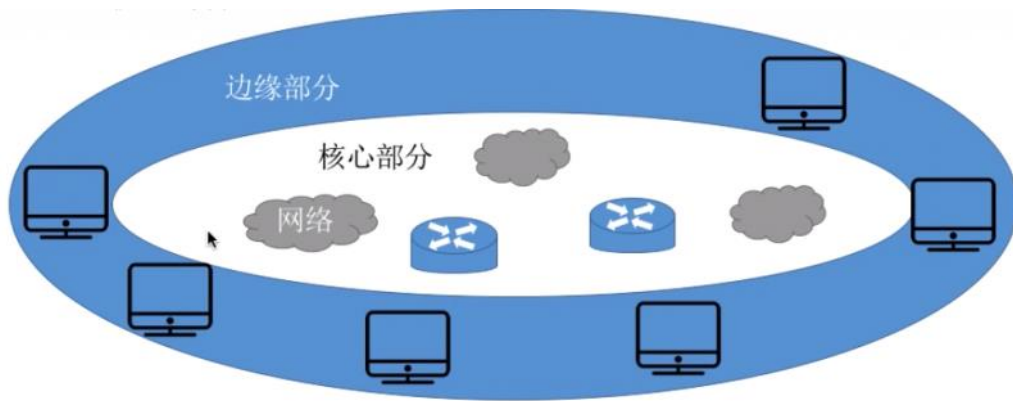
组成部分



一个完整的**计算机网络**主要由**硬件**、**软件**、**协议**三大组成部分，缺一不可。

- **硬件** 主要由主机（也叫端系统）、通信链路（如双绞线、光纤）、交换设备（如路由器、交换机等）和通信处理机（如网卡）等组成。
- **软件** 主要包括各种实现资源共享的软件、方便用户使用的各种工具软件，如网络操作系统、邮件收发程序、FTP程序、聊天程序等。软件部分多属于应用层。
- **协议** 是计算机网络的核心，如同交通规则约汽车驾驶一样，协议规定了网络传输数据时所遵循的规范。

作方式



计算机网络（这里主要指Internet）可分为边缘部分和核心部分。

- **边缘部分** 由所有连接在因特网上、供用户直接使用的主机组成，**用来进行通信**（如传输数据、音频或视频）和资源共享。
- **核心部分** 由大量的网络和连接这些网络的路由器组成，它为边缘部分提供连通性和交换服务。

功能组成



计算机网络由通信子网和资源子网组成。

- **通信子网** 由各种传输介质、通信设备和相应的网络协议组成，它使网络具有数据传输、交换、控制和存储的能力，实现联网计算机之间的数据通信。
- **资源子网** 是实现资源共享功能的设备及其软件的集合，向网络用户提供共享其他计算机上的硬件资源、软件资源和数据资源的服务。

1.1.3 计算机网络的功能

数据通信

它是计算机网络最基本和最重要的功能，用来实现联网计算机之间的各种信息的传输，并实现将分散在不同地理位置的计算机联系起来，进行统一的调配、控制和管理。比如，文件传输、电子邮件等应用，离开了计算机网络将无法实现。

资源共享

可以是软件共享、数据共享，也可以是硬件共享。使计算机网络中的资源互通有无、分工协作，从而极大地提高硬件资源、软件资源和数据资源的利用率。

分布式处理

当计算机网络中的某个计算机系统负荷过重时，可以将其处理的某个复杂任务分配给网络中的其他计算机系统，从而利用空闲计算机资源以提高整个系统的利用率。

提高可靠性

计算机网络中的各台计算机可以通过网络互为替代机。

负载均衡

将任务分配给多台计算机处理，提高系统的可靠性和性能。

将工作任务均衡地分配给计算机网络中的各台计算机。

除了以上几大主要功能，计算机网络还可以实现电子化办公与服务、远程教育、娱乐等功能，满足了社会的需求，方便了人们学习、工作和生活，具有巨大的经济效益。

1.1.4 计算机网络的分类

按分布范围分类

- **广域网(WAN)** - 广域网的任务是提供长距离通信，运送主机所发送的数据，其覆盖范围通常为几十到几千千米的区域，因而有时也称为远程网。广域网是因特网的核心部分。连接广域网的各结点交换机的链路一般都是高速链路，具有较大的通信容量。
- **城域网(MAN)** - 城域网的覆盖范围可以跨越几个街区甚至整个城市，覆盖范围约为5~50km。城域网大多采用以太网技术，因此有时也常并入局域网的范围进行讨论。
- **局域网(LAN)** - 局域网一般用微机或工作站通过高速线路相连，覆盖范围较小，一般是指几十米到几千米的区域。局域网在计算机配置的数量上没有太多的限制，少的可以只有两台，多的可达几百台。传统上，局域网使用广播技术，而广域网则使用交换技术。
- **个人区域网(PAN)** - 个人区域网是在个人工作的地方将消费电子设备（如平板电脑、智能手机等）用无线技术连接起来的网络，也常称为无线个人区域网WPAN，其范围大约在10m。

注意：若中央处理器之间的距离非常近（如仅1m的数量级或甚至更小），则一般就称之为多处理器系统，而不称它为计算机网络。

按传输技术分类

- **广播式网络** - 所有联网计算机都共享一个公共通信信道。当一台计算机利用共享通信信道发送报文分组时，所有其他的计算机都会“收听”到这个分组。接收到该分组的计算机将通过检查目的地址来决定是否接收该分组。
局域网基本上都采用广播式通信技术，广域网中的无线、卫星通信网络也采用广播式通信技术。
- **点对点网络** - 每条物理线路连接一对计算机。如果通信的两台主机之间没有直接连接的线路，那它们之间的分组传输就要通过中间结点的接收、存储和转发，直至目的结点。

是否采用分组存储转发与路由选择机制是点对点式网络与广播式网络的重要区别，广域网基本都属于点对点网络。

按拓扑结构分类

网络拓扑结构是指通过网中结点（路由器、主机等）与通信线路（网线）之间的几何关系（如总线形、环形）表示的网络结构，主要是指通信子网的拓扑结构。

按网络的拓扑结构，主要分为星形、总线形、环形和网状形网络等。星形、总线形和环形网络多用于局域网，网状形网络多用于广域网：

- **星形网络** - 每个终端或计算机都以单独的线路与中央设备相连。中央设备早期是计算机，现在一般是交换机或路由器。星形网络便于集中控制和管理，因为端用户之间的通信必须经过中央设备。缺点是成本高、中心结点对故障敏感。
- **总线形网络** - 用单根传输线把计算机连接起来。总线形网络的优点是建网容易、增减结点方便、节省线路。缺点是重负载时通信效率不高、总线任一处对故障敏感。
- **环形网络** - 所有计算机接口设备连接成一个环。环形网络最典型的例子便是令牌环局域网。环可以是单环，也可以是双环，环中信号是单向传输的。
- **网状形网络** - 一般情况下，每个结点至少要有两条路径与其他结点相连，多用在广域网中。有规则型和非规则型两种。其优点是可靠性高，缺点是控制复杂、线路成本高。

以上四种基本的网络拓扑结构可以互连组织成为更复杂的网络。

按使用者分类

- **公用网(Public Network)** - 它是指电信公司出资建造的大型网络。“公用”的意思就是所有愿意按电信公司的规定交纳费用的人都可以使用这种网络，因此也可称为公众网。
- **专用网(Private Network)** - 它是某个部门为本单位的特殊业务的需要而建造的网络。这种网络不向本单位以外的人提供服务。例如铁路、电力、军队等部门的专用网。

按交换技术分类

交换技术是指主机之间、通信设备之间或主机与通信设备之间为交换信息所采用的数据格式和交换装置的方式。按交换技术可将网络分为：

- **电路交换** - 在源结点和目的结点之间建立一条专用的通路用于传送数据，包括建立连接、传输数据和断开连接三个阶段。最典型的电路交换网是传统电话网络。
特点：整个报文的比特流连续地从源点直达终点，好像在一个管道中传送。优点是数据直接传送、延迟小。缺点是线路利用率低、不能充分利用线路容量、不便于进行差错控制。
- **报文交换** - 将用户数据加上源地址、目的地址、校验码等辅助信息，然后封装成报文。整个报文传送到相邻结点，全部存储下来后，再转发给下一个结点，重复这一过程直到到达目的结点。每个报文可以单独选择到达目的结点的路径。报文交换网络也称为存储-转发网络。
特点：整个报文先传送到相邻结点，全部存储下来后查找转发表，转发到下一个结点。
优点是可以较为充分利用线路容量；可以实现不同链路之间不同数据率的转换；可以实现格式转换；可以实现一对多、多对一的访问；可以实现差错控制。
缺点是增加了资源开销（如辅助信息导致处理时间和存储资源的开销）；增加缓冲延迟；额外的控制机制来保证多个报文的顺序不会乱序；缓冲区难以管理，因为报文的大小不确定，接收方在接收到报文之前不能预知报文的大小。
- **分组交换** - 也称为包交换网络。其原理是将数据分成较短的固定长度的数据块，在每个数据块中加上目的地址、源地址等辅助信息组成分组（包），以存储-转发方式传输。
特点：单个分组（这只是整个报文的一部分）传送到相邻结点，存储下来后查找转发表，转发到下一个结点。除了具备报文交换网络的优点外，分组交换网络还具有自身的优点：缓冲易于管理；包的平均延迟更小，网络中占用的平均缓冲区更少；更易于标准化；更适合应用。现在的主流网络基本上都可以看成是分组交换网络。

按传输介质分类

传输介质可以分为有线和无线两大类，故可以分为有线网络和无线网络：

- **有线网络** - 又分为双绞线网络、同轴电缆网络等。
- **无线网络** - 又可分为蓝牙、微波、无线电等类型。

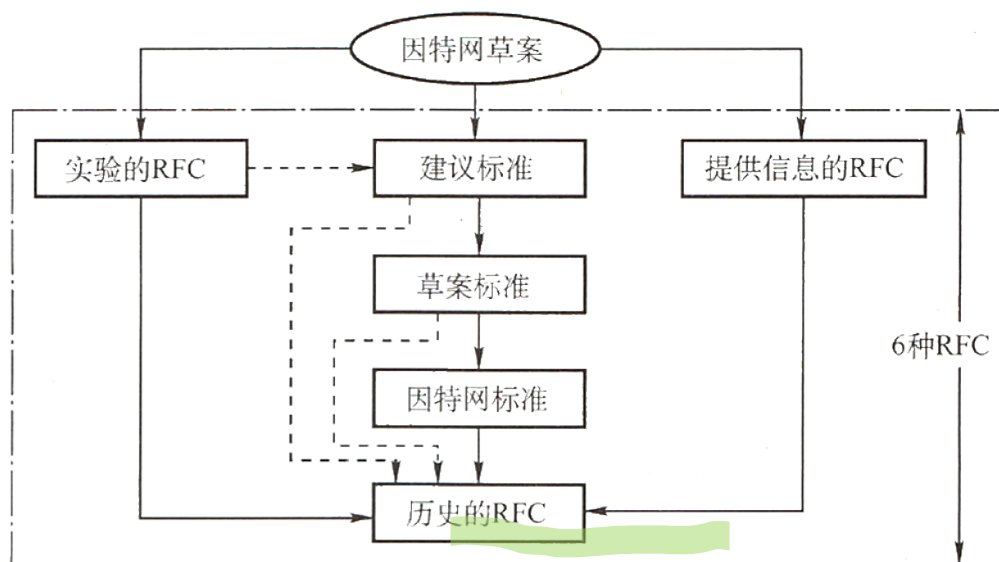
1.1.5 计算机网络的标准化工作及相关组织

标准化分类

计算机网络的标准化对计算机网络的发展和推广起到了极为重要的作用。

- **法定标准** - 由权威机构制定的正式的、合法的标准：OSI
- **事实标准** - 某些公司的产品在竞争中占据了主流，时间长了，这些产品的协议和技术就成了标准：TCP/IP

RFC



各种RFC之间的关系

因特网的所有标准都以RFC（Requests For Comments 因特网标准的形式）的形式在因特网上发布。但并非每个RFC都是因特网标准，RFC要上升为因特网正式标准需经过以下四个阶段：

1. **因特网草案 (Internet Draft)** - 这个阶段还不是RFC文档。
2. **建议标准 (Proposed Standard)** - 从这个阶段开始就成为RFC文档。
3. **草案标准 (Draft Standard)**
4. **因特网标准 (Internet Standard)**

此外，还有实验的RFC和提供信息的RFC。

相关组织

在国际上，有众多的标准化组织负责制定、实施相关网络标准。主要有以下几种：

- 国际标准化组织（ISO）- 制定的主要网络标准或规范有OSI参考模型、HDLC等。
- 国际电信联盟（ITU）- 其前身为国际电话电报咨询委员会(CCITT)，其下属机构ITU-T制定了大量有关远程通信的标准。
- 国际电气电子工程师协会（IEEE）- 世界上最大的专业技术团体，由计算机和工程专业人士组成。IEEE在通信领域最著名的研究成果是802标准。
- Internet工程任务组（IETF）- 负责英特网相关标准的制定RFC。

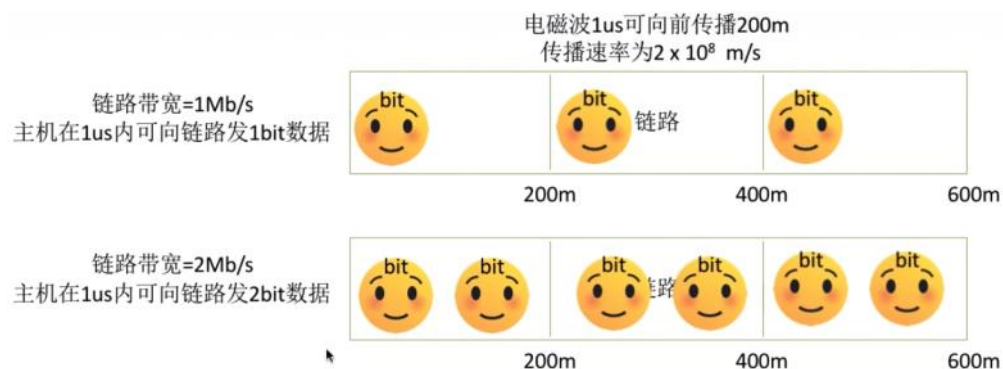
1.1.6 计算机网络的性能指标

概述

性能指标从不同的方面度量计算机网络的性能。下面介绍几种常用的性能指标。

带宽

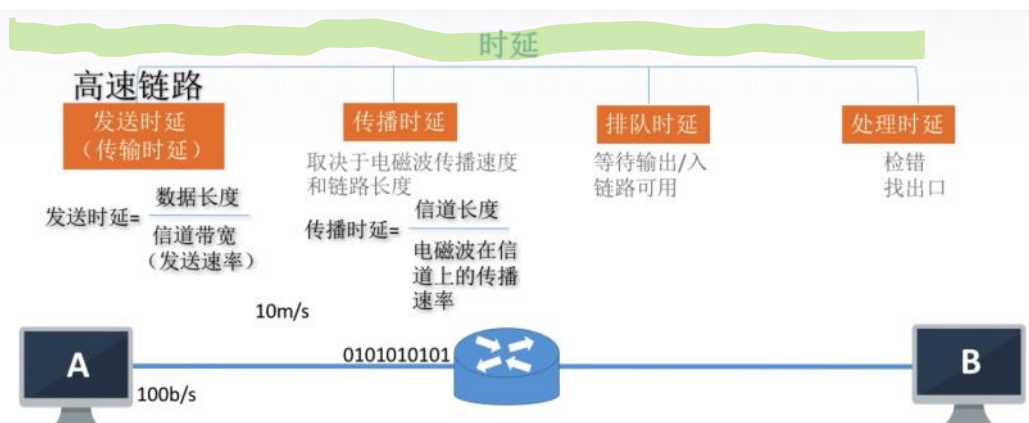
带宽 (Bandwidth) 原本指某个信号具有的频带宽度（即最高频率与最低频率之差），单位是赫兹(Hz)。在计算机网络中，带宽用来表示网络的通信线路传送数据的能力，通常是指单位时间内从网络中的某一点到另一点所能通过的“最高数据率”。单位是“比特每秒”(b/s)。



网络设备所支持的最高速度

时延

时延 (Delay) 是指数据（一个报文或分组）从网络（或链路）的一端传送到另一端所需要的总的时间，它由四个部分构成：发送时延、传播时延、处理时延和排队时延。

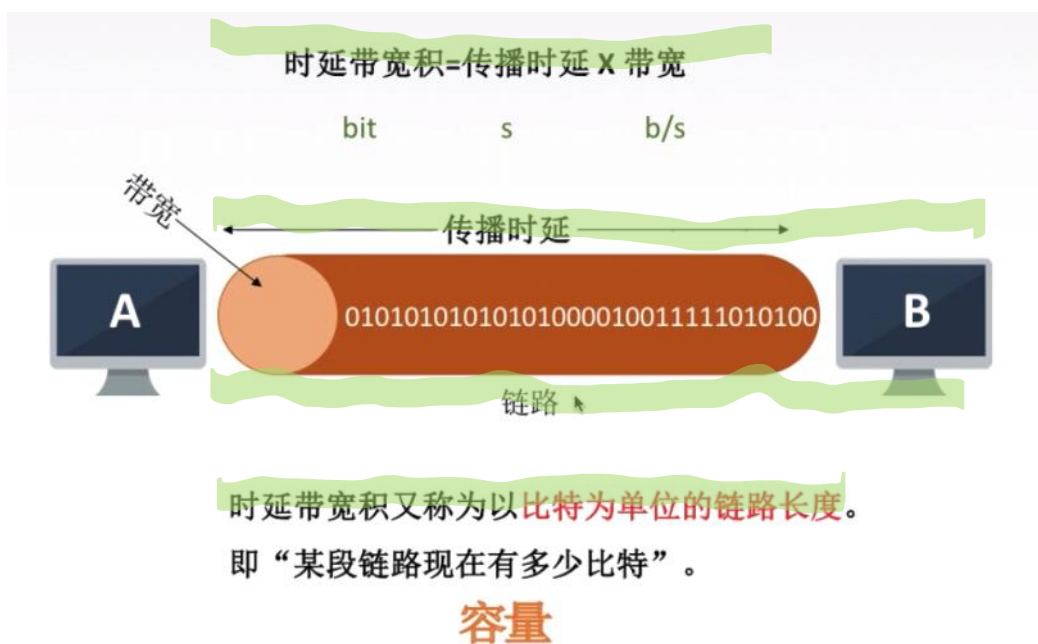


- **发送时延** - 结点将分组的所有比特推向(传输)链路所需的时间,也就是从发送分组的第一个比特算起,到该分组的最后一个比特发送完毕所需的时间,因此也称为传输时延。
发送时延计算公式: 发送时延 = 分组长度 / 信道宽度
 发送时延=分组长度/信道宽度
- **传播时延** - 电磁波在信道中传播一定的距离需要花费的时间,也就是一个比特从链路一端到另一端传播所需的时间。
传播时延计算公式: 传播时延 = 信道长度 / 电磁波在信道上的传播速率
 传播时延=信道长度/电磁波在信道上的传播速率
- **处理时延** - 数据在交换结点为存储转发而进行的一些必要的处理所花费的时间。例如,分析分组的首部、从分组中提取数据部分、进行差错检验或查找适当的路由等。
- **排队时延** - 分组在进入路由器后要先在输入队列中排队等待处理。在路由器确定了转发端口后,还要在输出队列中排队等待转发。这就产生了排队时延。

数据在网络中经历的**总时延就是以上四种时延之和**: 总时延 = 发送时延 + 传播时延 + 处理时延 + 排队时延
 总时延=发送时延+传播时延+处理时延+排队时延 总时延=发送时延+传播时延+处理时延+排队时延

时延带宽积

若发送端连续发送数据,在发送的第一个比特即将到达终点时,表示发送端已经发出的比特数。即 时延带宽积 = 传播时延 × 信道带宽
 时延带宽积=传播时延×信道带宽 时延带宽积=传播时延×信道带宽



如图所示,考虑一个代表链路的圆柱形管道,其长度表示链路的传播时延,横截面积表示链路带宽,则时延带宽积表示该管道可以容纳的比特数量。

往返时延RTT

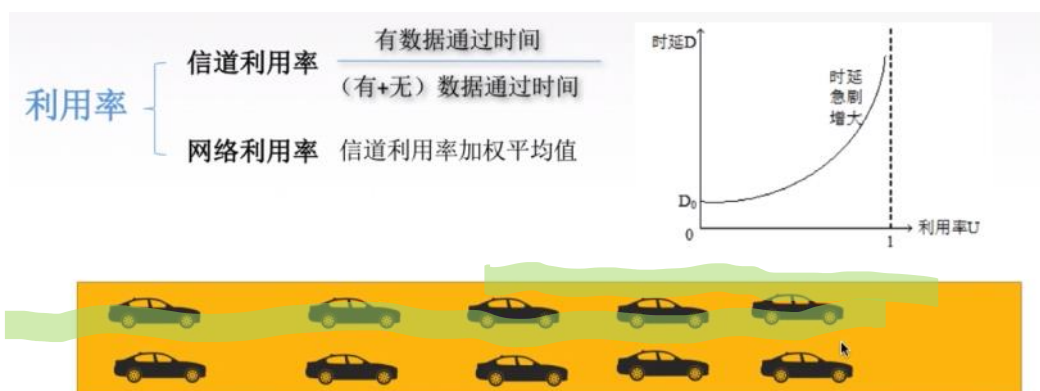
往返时延(Round-Trip Time, RTT) 表示从发送端发送数据开始,到发送端收到来自接收端的确认(接收端

收到数据后立即发送确认)，总共经历的时延。在互联网中，往返时延还包括各中间结点的处理时延、排队时延以及转发数据时的发送时延。

RTT越大，在收到确认之前，可以发送的数据越多。

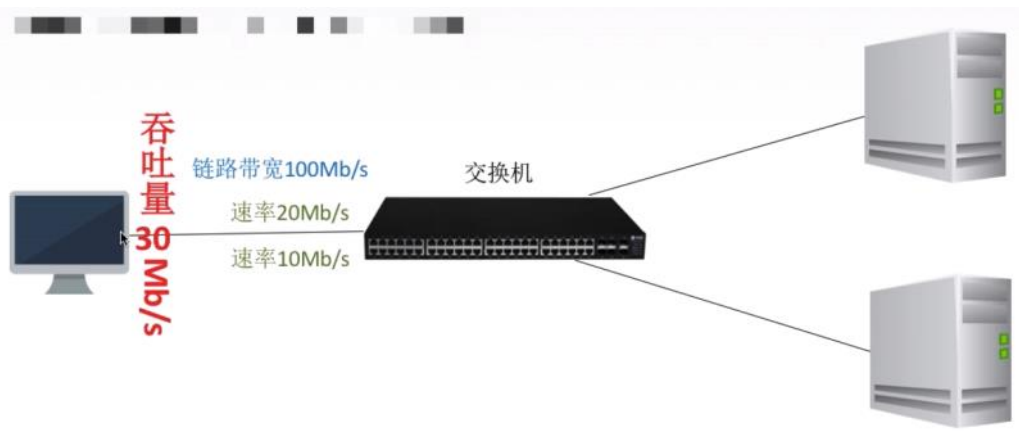
RTT包括 { 往返传播时延=传播时延 * 2
末端处理时间

利用率



吞吐量

吞吐量 (Throughput) 表示在单位时间内通过某个网络（或信道、接口）的数据量，单位 b/s、kb/s、Mb/s 等。吞吐量受网络的带宽或网络的额定速率的限制。



速率

网络中的**速率 (Speed)** 是指连接在计算机网络上的主机在数字信道上传送数据的速率，也称为**数据率或比特率**，单位是 b/s (比特每秒，或 bit/s，有时也写为 bps)。当数据率较高时，就可以用 Kb/s (K=10³)、Mb/s (M=10⁶) 或 Gb/s (G=10⁹)。在计算机网络中，通常把最高数据率称为“带宽”。

发送端	接收端	速率是10b/s
	0101010101	
速率		存储容量 1Byte (字节) = 8bit (比特)
千 1kb/s = 10 ³ b/s		1KB = 2 ¹⁰ B = 1024B = 1024 * 8b
兆 1Mb/s = 10 ³ kb/s = 10 ⁶ b/s		1MB = 2 ¹⁰ KB = 1024KB
吉 1Gb/s = 10 ³ Mb/s = 10 ⁶ kb/s = 10 ⁹ b/s		1GB = 2 ¹⁰ MB = 1024MB
太 1Tb/s = 10 ³ Gb/s = 10 ⁶ Mb/s = 10 ⁹ kb/s = 10 ¹² b/s		1TB = 2 ¹⁰ GB = 1024GB

1.2 计算机网络体系结构与参考模型

1.2.1 计算机网络分层结构

概述

两个系统中实体间的通信是一个很复杂的过程，为了降低协议设计和调试过程的复杂性，也为了便于对网络进行研究、实现和维护，促进标准化工作，通常对计算机网络的体系结构以分层的方式进行建模。

我们把计算机网络的各层及其协议的集合称为网络的**体系结构 (Architecture)**。计算机网络的体系结构就是这个计算机网络及其所应完成的功能的精确定义，它是计算机网络中的 层次、各层 的协议以及层间接口的集合。

分层基本原则



计算机网络的体系结构通常都具有可分层的特性，将复杂的大系统分成若干较容易实现的层次。

1. 各层之间相互独立，每层只实现一种相对独立的功能。
2. 每层之间界面自然清晰，易于理解，相互交流尽可能少。
3. 各层功能的精确定义独立于具体的实现方法，每层都采用最合适的技术来实现。
4. 保持下层对上层的独立性，上层单向使用下层提供的服务。
5. 整个分层结构应该能促进标准化工作。

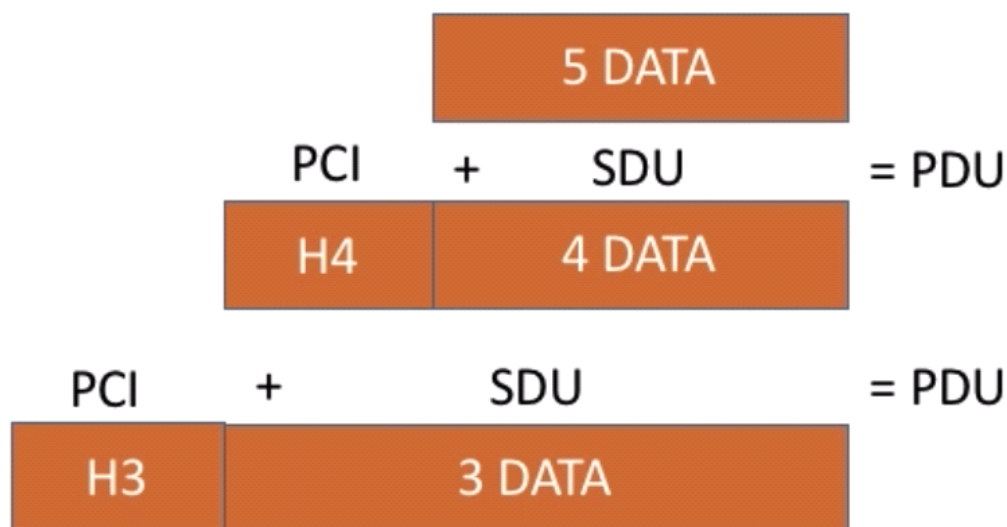
由于分层后各层之间相对独立，灵活性好，因而分层的体系结构易于更新（替换单个模块），易于调试，易于交流，易于抽象，易于标准化。但层次越多，有些功能在不同层中难免重复出现，产生了额外的开销，整体运行效率就越低。而层次越少，就会使每一层的协议太复杂。因此，在分层时应考虑层次的清晰程度与运行效率间的折中、层次数量的折中。

分层结构

在计算机网络的分层结构中，第n层中的活动元素通常称为**n层实体**。具体地，**实体**指任何可发送或接收信息的硬件或软件进程，通常是一个特定的软件模块。不同机器上同一层称为对等层，同一层的实体叫做**对等实体**。n层实体实现的服务为n+1层所利用。在这种情况下，n层被称为服务提供者，n+1层是服务用户。

每一层还有自己传送的数据单位，其名称、大小、含义也各有不同。

在计算机网络体系结构的各个层次中，每一个报文都分为两个部分，一个是数据部分，也就是SDU， 另外一部分是控制信息部分，也就是PCI， 它们共同组成PDU。

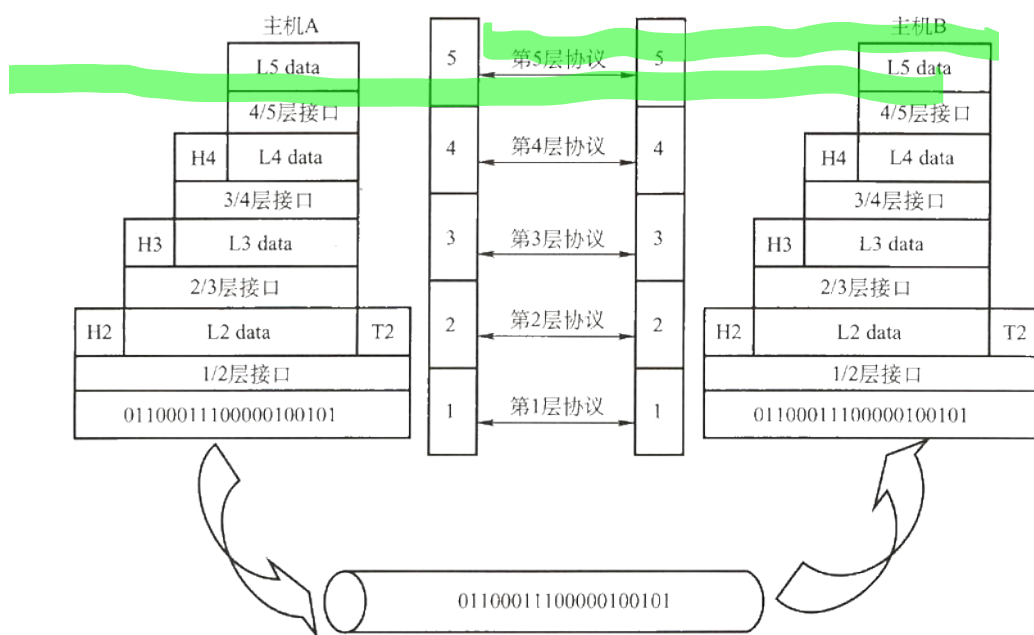


服务数据单元 (SDU)： 为完成用户所要求的功能而应传送的数据。第n层的服务数据单元记为n-SDU。

协议控制信息 (PCI)： 控制协议操作的信息。第n层的协议控制信息记为n-PCI。

协议数据单元 (PDU)： 对等层次之间传送的数据单位称为该层的PDU。第n层的协议数据单元记为n-PDU。

在实际的网络中，每层的协议数据单元都有一个通俗的名称，如物理层的PDU叫比特，链路层的PDU叫帧，网络层的PDU叫分组，传输层的PDU叫报文。



网络各层数据单元的联系

在各层间传输数据时，把从第n+1层收到的PDU作为第n层的SDU，加上第n层的PCI，就变成了第n层的PDU，交给第n-1层后作为SDU发送，接收方接收时做相反的处理，故可知三者的关系为 $n\text{-SDU} + n\text{-PCI} = n\text{-PDU} = (n-1)\text{-SDU}$ 。

层次结构的含义：

1. 第n层的实体不仅要使用第n-1层的服务，实现自身定义的功能，还要向第n+1层提供本层的服务，该服务是第n层及其下面各层提供的服务总和。
2. 最低层只提供服务，是整个层次结构的基础；中间各层既是下一层的服务使用者，又是上一层的服务提供者；最高层面向用户提供服务。

3. 最低层只提供服务，是整个层次结构的基础；中间各层既是下一层的服务使用者，又是上一层的服务提供者；最高层面向用户提供服务。
4. 两个主机通信时，对等层在逻辑上有一条直接信道，表现为不经过下层就把信息传送到对方。

1.2.2 计算机网络协议、接口、服务的概念

协议

协议，就是规则的集合。在网络中要做到有条不紊地交换数据，就必须遵循一些事先约定好的规则。这些规则明确规定了所交换的数据的格式以及有关的同步问题。这些为进行网络中的数据交换而建立的规则、标准或约定称为**网络协议 (Network Protocol)**，它是控制两个(或多个)对等实体进行通信的规则集合，也就是水平的。

不对等实体之间是没有协议的，比如用TCP/IP协议栈通信的两个结点，结点A的传输层和结点B的传输层之间存在协议，但结点A的传输层和结点B的网络层之间不存在协议。**网络协议也简称为协议。**

协议由语法、语义和同步三部分组成：

- **语法** - 规定了传输数据的格式。
- **语义** - 规定了所要完成的功能，即需要发出何种控制信息、完成何种动作以及做出何种应答。
- **同步** - 规定了执行各种操作的条件、时序关系等，即事件实现顺序的详细说明。

一个完整的协议通常应具有线路管理（建立、释放连接）、差错控制、数据转换等功能。

接口

接口 是同一结点内相邻两层间交换信息的连接点，是一个系统内部的规定。每一层只能为紧邻的层次之间定义接口，不能跨层定义接口。

在典型的接口上，同一结点相邻两层的实体通过 **服务访问点 (Service Access Point, SAP)** 进行交互。服务是通过服务访问点SAP提供给上层使用的，第n层的SAP就是第n+1层可以访问第n层服务的地方。每个SAP都有一个能够标识它的地址。服务访问点SAP是一个抽象的概念，它实际上就是一个逻辑接口(类似于邮政信箱)，但和通常所说的两个设备之间的硬件接口是很不一样的。

服务

服务是指下层为紧相邻的上层提供的功能调用，也就是垂直的。对等实体在协议的控制下，使得本层能为上一层提供服务，但要实现本层协议还需要使用下一层所提供的服务。

上层使用下层所提供的服务必须通过与下层交换一些命令，这些命令在OSI中称为**服务原语**。OSI将原语划分为四类：

1. **请求(Request)** - 由服务用户发往服务提供者，请求完成某项工作。
2. **指示(Indication)** - 由服务提供者发往**服务用户**，**指示用户做某件事**。
3. **响应(Response)** - 由服务用户发往服务提供者，作为对指示的响应。
4. **证实(Confirmation)** - 由服务提供者发往服务用户，作为对请求的证实。

这四类原语用于不同的功能，如建立连接、传输数据和断开连接等。有应答服务包括全部四类原语，而无应答服务则只有请求和指示两个原语。



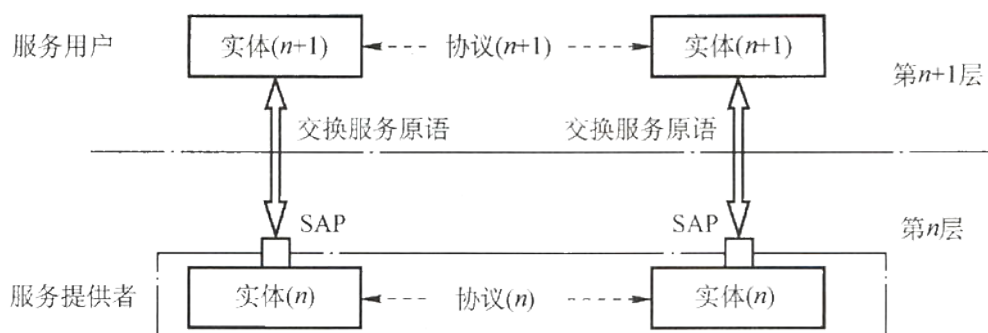
四类原语的关系

注意：协议和服务在概念上是不一样的。

- 首先，本层协议的实现才能保证向上一层提供服务。**本层的服务用户只能看见服务而无法看见下面的协议**，即下面的协议对上层的服务用户是透明的。
- 其次，协议是“水平的”，即协议是控制对等实体之间通信的规则。但服务是“垂直的”，即服务是由下层通过层间接口向上层提供的。

- 另外，并非在一个层内完成的全部功能都称为服务，只有那些能够被高一层实体“看得见”的功能才称为服务。

协议/接口/服务三者之间关系



协议、接口、服务三者之间的关系

计算机网络提供服务分类

• 面向连接服务与无连接服务

- **面向连接服务** - 通信前双方必须先建立连接，分配相应的资源(如缓冲区)，以保证通信能正常进行，传输结束后释放连接和所占用的资源。因此这种服务可以分为连接建立、数据传输和连接释放这三个阶段。例如 TCP 就是一种面向连接服务的协议。
- **无连接服务** - 通信前双方不需要先建立连接，需要发送数据时就直接发送，把每个带有目的地址的包(报文分组)传送到线路上，由系统选定路线进行传输，是一种不可靠的服务。这种服务常被描述为“尽最大努力交付”(Best-Effort-Delivery)，并不保证通信的可靠性。例如 IP、UDP 就是一种无连接服务的协议。

• 可靠服务和不可靠服务

- **可靠服务** - 是指网络具有纠错、检错、应答机制，能保证数据正确、可靠地传送到目的地。
- **不可靠服务** - 是指网络只是尽量正确、可靠地传送，但不能保证数据正确、可靠地传送到目的地，是一种尽力而为的服务。

对于提供不可靠服务的网络，其网络的正确性、可靠性就要由应用或用户来保障。

• 有应答服务和无应答服务

- **有应答服务** - 是指接收方在收到数据后向发送方给出相应的应答，该应答由传输系统内部自动实现，而不是由用户实现。所发送的应答可以是肯定应答，也可以是否定应答，通常在接收到的数据有错误时发送否定应答。例如，文件传输服务就是一种有应答服务。
- **无应答服务** - 是指接收方收到数据后不自动给出应答。若需要应答，则由高层实现。例如WWW服务，客户端收到服务器发送的页面文件后不给出应答。

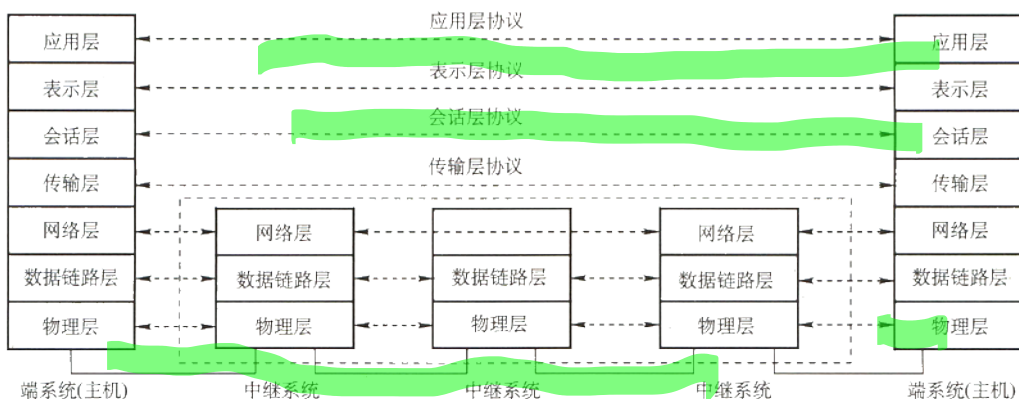
1.2.3 ISO/OSI参考模型和TCP/IP模型

OSI参考模型



国际标准化组织（ISO）于1984年提出的网络体系结构模型，称为**开放系统互连参考模型（OSI/RM）**，通常简称为**OSI参考模型**（理论成功，市场失败）。OSI有七层，自下而上依次为物理层、数据链路层、网络层、运输层、会话层、表示层、应用层。**低三层统称为通信子网**，它是为了联网而附加上去的通信设备，完成数据的传输功能；**高三层统称为资源子网**，它相当于计算机系统，完成数据的处理等功能。运输层承上启下。

OSI层次结构



• 物理层（Physical Layer）

- 物理层的传输单位是比特，任务是透明的传输比特流，功能是在物理媒体上为数据端设备透明的传输原始比特流。
- 物理层主要定义 数据终端设备(DTE) 和 数据通信设备(DCE) 的物理和逻辑连接方法，所以物理层协议也称为物理层接口标准。由于在通信技术的早期阶段，通信规则称为规程（Procedure），故物理层协议也称为物理层规程。

• 数据链路层（Data Link Layer）

- 数据链路层的传输单位是帧，任务是将网络层传下来的IP数据报组装成帧。数据链路层的功能可以概括为：成帧、差错控制、流量控制和传输管理等。
- 典型的数据链路层协议有SDLC、HDLC、PPP、STP和帧中继等。

• 网络层（Network Layer）

- 网络层的传输单位是数据报，它关心的是通信子网的运行控制，主要任务是把网络层的协议数据单元（分组）从源端传到目的端，为分组交换网上的不同主机提供通信服务。关键问题是对分组进行路由选择，并实现流量控制、拥塞控制、差错控制和网际互联等功能。
- 网络层的协议有：IP、IPX、ICMP、IGMP、ARP、RARP和OSPF等。

• 运输层（Transport Layer）

- 运输层也叫运输层，传输单位是报文段(TCP)或用户数据报(UDP)，运输层的任务是负责主机中两个进程之间的通信。功能是为端到端连接提供可靠的传输服务；为端到端连接提供流量控制、差错控制、服务质量、数据传输管理等服务。
- 运输层的协议有：TCP，UDP。



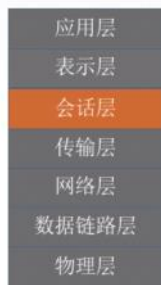
• 会话层（Session Layer）

- 会话层允许不同主机上各进程之间的会话。会话层利用运输层提供的端到端的服务，向表示层提供它的增值服务。这种服务主要是向表示层实体或用户进程提供建立连接并在连接上有序地传输数据，这就是会话，也称为建立同步(SYN)。
- 会话层负责管理主机间的会话进程，包括建立、管理以及终止进程间的会话。会话层使用校验点

可使通信会话在通信失效时从校验点继续恢复通信，实现数据同步。

向表示层实体/用户进程提供**建立连接**并在连接上**有序地传输**数据。

这是会话，也是**建立同步**（SYN）



功能一：建立、管理、终止会话

功能二：使用校验点可使会话在通信失效时从**校验点/同步点**继续恢复通信，实现数据同步。

适用于传输大文件。

主要协议：

ADSI、ASP

• 表示层（Presentation Layer）

- 表示层主要用于处理在两个通信系统中交换信息的表示方式。不同机器采用的编码和表示方法不同，使用的数据结构也不同。为了使不同表示方法的数据和信息之间能互相交换，表示层采用抽象的标准方法定义数据结构，并采用标准的编码形式。**数据压缩、加密和解密**也是表示层可提供的数据表示变换功能。

用于处理在两个通信系统中交换信息的表示方式（语法和语义）

功能一：数据格式变换 **翻译官**

....0111011000....



功能二：数据加密解密

“我的微信支付密码是XXXX”

功能三：数据压缩和恢复

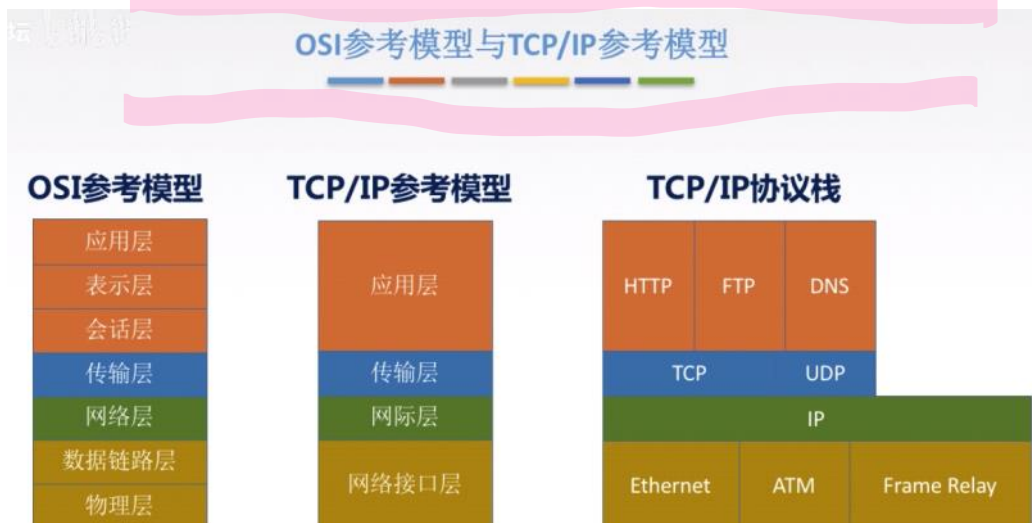


• 应用层（Application Layer）

- 应用层是OSI模型的最高层，是用户与网络的界面。应用层为特定类型的网络应用提供访问OSI环境的手段。因为用户的实际应用多种多样，这就要求应用层采用不同的应用协议来解决不同类型的应用要求，因此应用层是最复杂的一层，使用的协议也最多。
- 典型的有文件传送的FTP、电子邮件的SMTP、万维网应用的HTTP等。



TCP/IP模型，从低到高依次为：网络接口层(对应OSI参考模型中的物理层和数据链路层)、网际层、传输层和应用层(对应OSI参考模型中的会话层、表示层和应用层)。TCP/IP由于得到广泛应用而成为事实上的国际标准。



• 网络接口层

- 功能类似于OSI的物理层和数据链路层。它表示与物理网络的接口，但实际上TCP/IP本身并没有真正描述这一部分，只是指出主机必须使用某种协议与网络连接，以便能向其上传递IP分组。具体的物理网络可以是各种类型的局域网，如以太网、令牌环网、令牌总线网等，也可以是诸如电话网、SDH、X.25、帧中继和ATM等公共数据网络。
- 网络接口层的责任是从主机或结点接收IP分组，并把它们发送到指定的物理网络上。

• 网际层（主机-主机）

- 是TCP/IP体系结构的关键部分。它和OSI网络层在功能上非常相似。网际层将分组发往任何网络，并为之独立地选择合适的路由，但它不保证各个分组有序的到达，各个分组的有序交付由高层负责。网际层定义了标准的分组格式和协议，即IP协议。当前采用的IP协议是第4版，即IPv4，它的下一个版本是IPv6。

• 传输层（应用-应用 或 进程-进程）

- 功能同样和OSI中的传输层类似，是使发送端和目的端主机上的对等实体可以进行会话。传输层主要使用以下两种协议：
 - 传输控制协议（Transmission Control Protocol, TCP）。面向连接的，数据传输的单位是报文段，能够提供可靠的交付。
 - 用户数据报协议（User Datagram Protocol, UDP）。无连接的，数据传输的单位是用户数据报，不保证提供可靠的交付，只能提供“尽最大努力交付”。

• 应用层（用户-用户）

- 包含所有的高层协议。如虚拟终端协议（Telnet）、文件传输协议（FTP）、域名解析服务（DNS）、电子邮件协议（SMTP）和超文本传输协议（HTTP）。

TCP/IP模型与OSI参考模型的比较

相同点

1. 二者都采取分层的体系结构，将庞大且复杂的问题划分为若干个较容易处理的、范围较小的问题，而且分层的功能也大体相似。
2. 二者都是基于独立的协议的概念。
3. 二者都可以解决异构网络的互联，实现世界上不同厂家生产的计算机之间的通信。

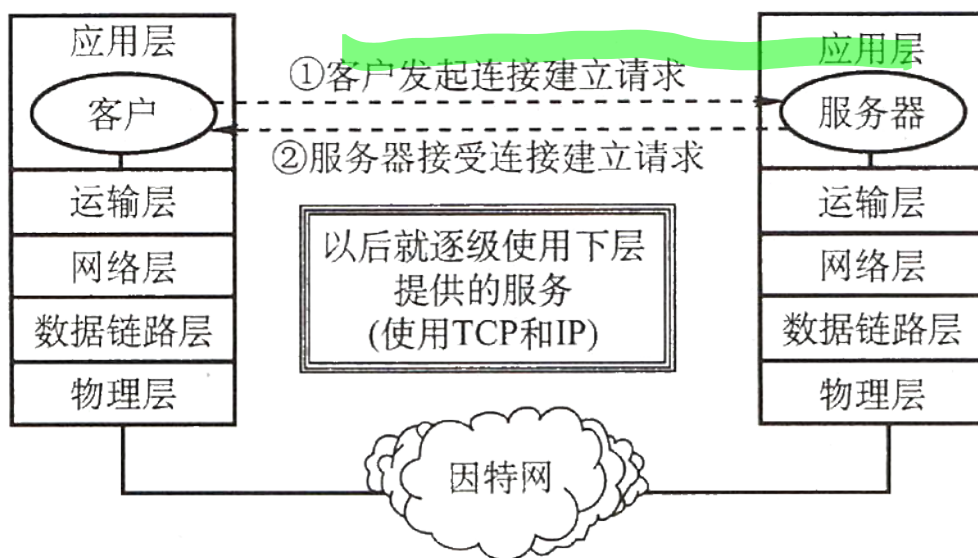
不同点

1. OSI参考模型的最大贡献就是精确地定义了三个主要概念：服务、协议和接口，这与现代的面向对象程序设计思想非常吻合。而TCP/IP模型在这三个概念上却没有明确区分，不符合软件工程的思想。
2. OSI参考模型产生在协议发明之前，没有偏向于任何特定的协议，通用性良好。TCP/IP模型正好相反。首先出现的是协议，模型实际上是对已有协议的描述，因此不会出现协议不能匹配模型的情况，但该模型不适合于任何其他非TCP/IP的协议栈。

2. OSI参考模型产生在协议发明之前，没有偏向于任何特定的协议，通用性良好。TCP/IP模型正好相反。首先出现的是协议，模型实际上是对已有协议的描述，因此不会出现协议不能匹配模型的情况，但该模型不适合于任何其他非TCP/IP的协议栈。
3. TCP/IP模型在设计之初就考虑到多种异构网的互联问题，并将网际协议IP作为一个单独的重要层次。OSI参考模型最初只考虑到用一种标准的公用数据网将各种不同的系统互联。后来，OSI参考模型认识到网际协议IP的重要性，因此只好在网络层中划分出一个子层来完成类似于TCP/IP模型中IP的功能。
4. OSI参考模型在网络层支持无连接和面向连接的通信，但在传输层仅有面向连接的通信。而TCP/IP模型认为可靠性是端到端的问题，因此它在网际层仅有一种无连接的通信模式，但在传输层支持无连接和面向连接两种模式。

5层参考模型

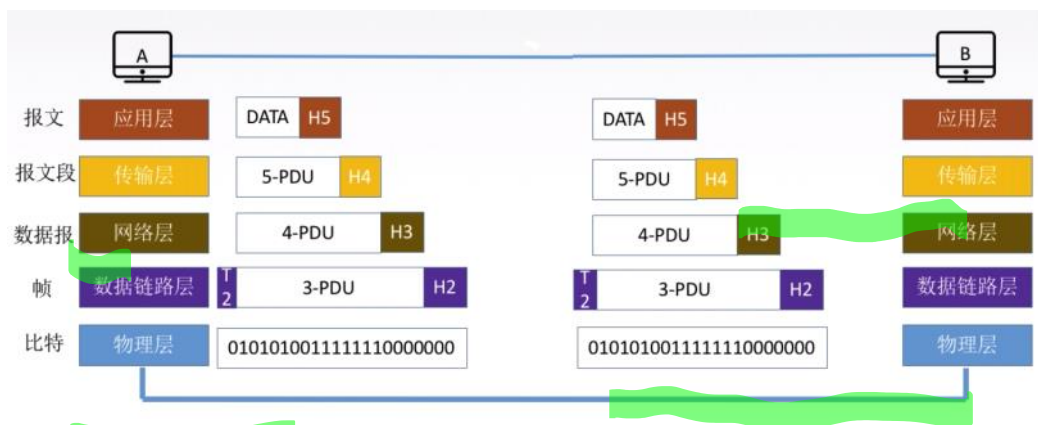
无论是OSI参考模型，还是TCP/IP模型都不是完美的，对二者的讨论和批评都很多。OSI 参考模型的设计者从工作的开始，就试图建立一个全世界的计算机网络都要遵循的统一标准。从技术角度来看，他们希望追求一种完美的理想状态，这也导致基于OSI参考模型的软件效率极低。OSI参考模型缺乏市场与商业动力，结构复杂，实现周期长，运行效率低，这是它没有能够达到预想目标的重要原因。



网络的五层体系结构模型

所以在学习计算机网络时往往采取折中的办法，即综合OSI和TCP/IP的优点，采用一种只有五层协议的体系结构，即我们所熟知的物理层、数据链路层、网络层、传输层和应用层。

通信协议栈通信过程



通信协议栈的通信过程示例

每个协议栈的最顶端都是一个面向用户的接口，下面各层是为通信服务的协议。

1. 当用户要传输一个数据报时，通常是给出用户能够理解的自然语言，然后通过应用层，自然语言会转化为用于通信的通信数据。
2. 通信数据到达运输层，作为运输层的数据部分(运输层SDU)，加上运输层的控制信息(运输层PCI)，组成运输层的PDU，然后交到网络层。

化为用于通信的通信数据。

2. 通信数据到达运输层，作为运输层的数据部分(运输层SDU)，加上运输层的控制信息(运输层PCI)，组成运输层的PDU，然后交到网络层。
3. 运输层的PDU下放到网络层后，就成为网络层的SDU，然后加上网络层的PCI，又组成了网络层的PDU，下放到数据链路层。
4. 就这样层层下放，层层包裹，最后形成的数据报通过通信线路传输，到达接收方结点协议栈，接收方再逆向地一层一层把“包裹”拆开，然后把收到的数据提交给用户。

第二章 物理层

星期日，三月 13，2022 5:26 下午

已剪辑自: <https://blog.csdn.net/donkey010/article/details/114698200>

基本概念

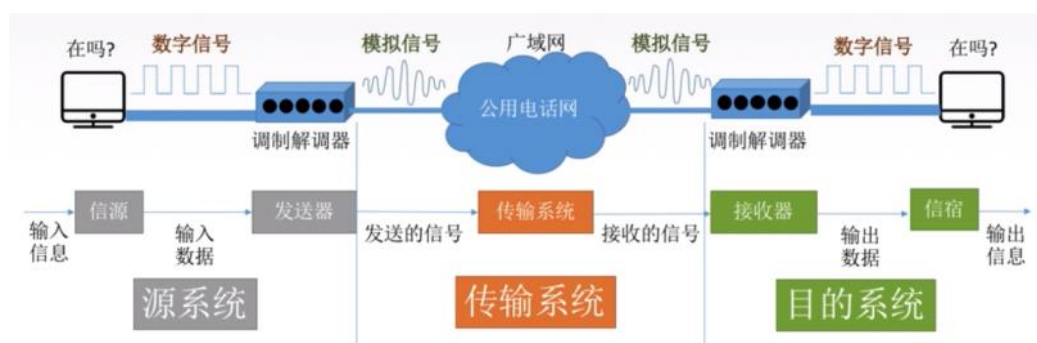
物理层解决如何在连接各种计算机的网络媒体上传输数据比特流，而不是指具体的传输媒体。

物理层主要任务：确定与传输媒体接口有关的一些特性 → 定义标准

1. 机械特性：定义物理连接的特性，规定物理连接时所采用的规格、接口形状、引线数目、引脚数量和排列情况。
2. 电气特性：规定传输二进制位时，线路上信号的电压范围、阻抗匹配、传输速率和距离限制等。
例如：某网络在物理层规定，信号的电平用+10V ~ +15V表示二进制0，用-10V ~ -15V表示二进制1，电线长度限于15m内。
3. 功能特性：指明某条线上出现的某一电平表示何种含义，接口部件的信号线的用途。
例如：描述一个物理层接口引脚处于高电平时的含义。
4. 规程特性：（过程特性）定义各条物理线路的工作规程和时序的关系。

数据通信基础知识

典型数据通讯模型



术语

通讯的目的的传送消息

数据：传送消息的实体，通常是有意义的符号序列。

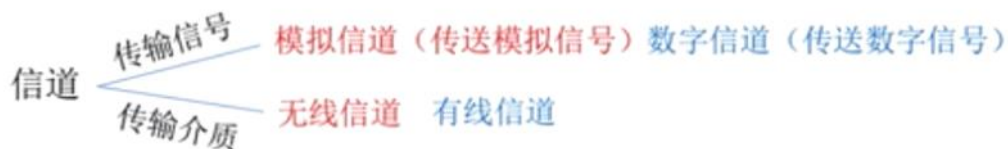
信号：数据的电气/电磁表示，是数据在传输过程中的存在形式。

- 数字信号：代表消息的参数取值是离散的。
- 模拟信号：代表消息的参数取值是连续的。

信源：产生和发送数据的源头。

信宿：接收数据的终点。

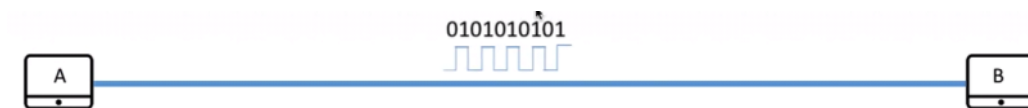
信道：信号的传输媒介。一般用来表示向某一个方向传送信息的介质，因此一条通讯线路往往包含一条发送信道和一条接收信道。



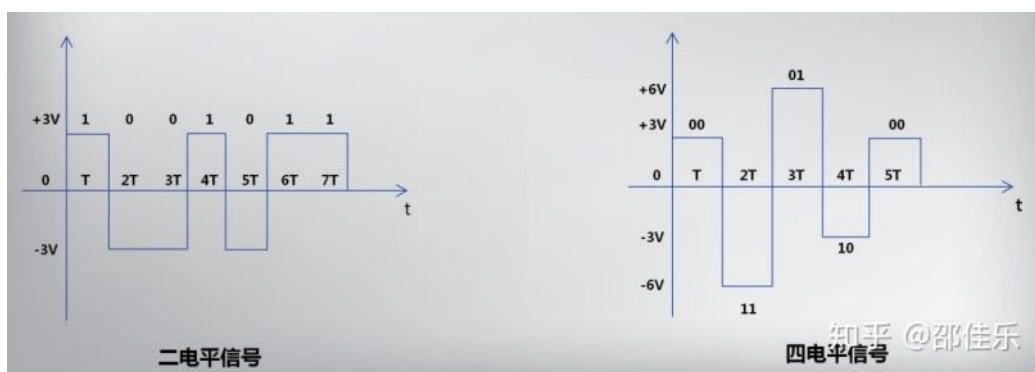
码元

码元是指用一个固定时长的信号波形（数字脉冲），代表不同离散数值的基本波形，是数字通信中数字信号的计量单位，这个时长内的信号称为k进制码元，而该时长称为码元宽度。当码元的离散状态有M个时（M大于2），此时码元为M进制码元。

1码元可以携带多个比特的信息量。例如，在使用二进制编码时，只有两种不同的码元，一种代表0状态，另一种代表1状态。



k进制码元 —— 4进制码元 → 码元的离散状态有4个 → 4种高低不同的信号波形
：00 01 10 11



速率、波特

速率也叫数据率，是指数据的传输效率，表示单位时间内传输的数据量。可以用码元传输效率和信息传输效率表示。

码元传输速率：别名码元速率、波形速率、调制速率、符号速率等，它表示单位时间内数字通信系统所传输的码元个数（也可称为脉冲个数或信号变化的次数），单位是**波特（Baud）**。1波特表示数字通信系统每秒传输一个码元。这里的码元可以是多进制的，也可以是二进制的，但码元速率与进制数无关。

信息测试速率：别名信息速率、比特率等，表示单位时间内数字通信系统传输的二进制码元个数（即比特数），单位是比特/秒（b/s）。

关系：若一个码元携带n bit的信息量，则M Baud的码元传输速率所对应的信息传输速率为 $M \times n$ bit/s。

带宽

带宽：表示在单位时间内从网络中的某一点到另一点所能通过的”最高数据率“，常用来表示网络的通信线路所能传输数据的能力。单位是b/s。

习题

练习题

某一数字通信系统传输的是四进制码元,4s传输了8000个码元,求系统的码元传输速率是多少?信息传输速率是多少?若另一通信系统传输的是十六进制码元,6s传输了7200个码元,求他的码元传输速率是多少?信息传输速率是多少?并指出哪个系统传输速率快?

2000Baud, 4000b/s; 1200Baud, 4800b/s; 十六进制更快

四进制码元系统

码元传输速率就是 $8000/4=2000\text{Baud}$, 信息传输速率就是 $2000*\log_2 4=4000\text{b/s}$

十六进制码元系统

码元传输速率就是 $7200/6=1200\text{Baud}$, 信息传输速率就是 $1200*\log_2 16=4800\text{bit/s}$

系统传输的是**比特流**, 通常比较的是信息传输速率, 所以传输十六进制码元的通信系统传输速率较快, 如果用该系统去传输四进制码元会有更高的码元传输速率。

三种通讯方式

从通信双方信息的交互方式看, 可以有三种基本方式

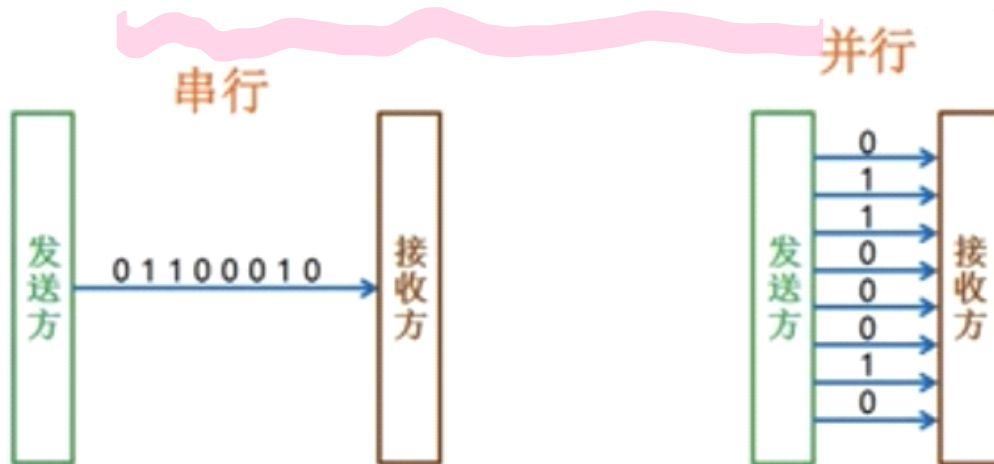
1. **单工通信**: 只有一个方向的通信, 而没有反方向的交互, 仅需要一条信道。
2. **半双工通信**: 通信的双方都可以发送和接收信息, 但任何一方都不能同时发送和接收, 需要两条信道。
3. **全双工通信**: 通信双方可以同时发送和接收信息, 也需要两条信道。

两种数据传输方式

传输方式

串行传输: 速度慢, 费用低, 适合远距离

并行传输: 速度快, 费用高, 适合近距离, 常用于计算机内部数据传输



编码与调制

信号

概念

信道: 信号的传输媒介。一般用来表示向某一个方向传送信息的介质, 因此一条通信线路往往包含一条发送信道和一条接收信道。



基带信号

将数字信号1和0直接用两种不同的电压表示，再送到数字信道上去传输（基带传输）。**来自信源**的信号，像计算机输出的代表各种文字或图像文件的数据信号都属于基带信号。基带信号就是发出的**直接表达了要传输的信息的信号**，比如我们是说话的声波就是基带信号。

宽带信号

将基带信号进行调整后形成的频分复用模拟信号，再传送到模拟信道上去传输（宽带传输）。把基带信号经过**载波调制**后，把信号的**频率范围搬移到较高的频段**以便在信道中传输（即仅在一段频率范围内能够通过信道）。

区别

在传输距离较远时，计算机网络采用宽带传输方式（远距离衰减大，即使信号变化大也能最后过滤出来基带信号）

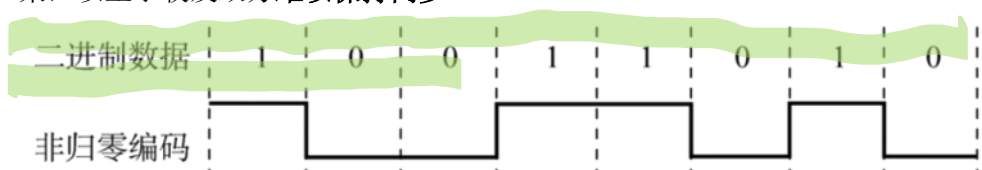


编码

数字数据编码为数字信号

1. 非归零编码【NRZ】

高1低0，编码容易实现，但没有检错功能，且无法判断一个码元的开始和结束，以至于收发双方**难以保持同步**



2. 曼彻斯特编码

将一个码元分成两个相等的间隔，前一个间隔为低电平后一个间隔为高电平表示码元1，码元0则正好相反。也可以采用相反的规定。该编码的特点是在每一个码元的中间出现电平跳变，位中间的跳变既做时钟信号（可用于同步），又做数据信号，但它所占的频带宽度是原始的基带宽度的两倍。每一个码元都被调成两个电平，所以数据传输速率只有调制速率的1/2。



3. 差分曼彻斯特编码

同1异0，常用于局域网传输，其规则是：若码元为1，则前半个码元的电平与上一个码元的后半半个码元的电平相同，若为0，则相反。该编码的特点是，在每个码元的中间，都有一次电平的跳转，可以实现自同步，且抗干扰性强于曼彻斯特编码。

以太网使用曼彻斯特编码

上一个码元的高电平与下一个码元的电平相同，则为0，则相反。该编码的特点是，在每个码元的中间，都有一次电平的跳转，可以实现自同步，且抗干扰性强于曼彻斯特编码。



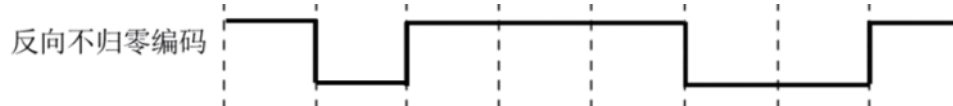
4. 归零编码【RZ】

信号电平在一个码元之内都要恢复到0的这种编码成编码方式。



5. 反向不归零编码【NRZI】

信号电平翻转表示0，信号电平不变表示1。



6. 4B/5B编码

比特流中插入额外的比特以打破一连串的0或1，就是用5个比特来编码4个比特的数据，之后再传给接收方，因此称为4B/5B，编码效率为80%。

只采用16种对应16种不同的4位码，其他的16种作为控制码（帧的开始和结束，路线的状态信息等）或保留。

4 比特数据符号	5 比特编码
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011

模拟数据编码为数字信号

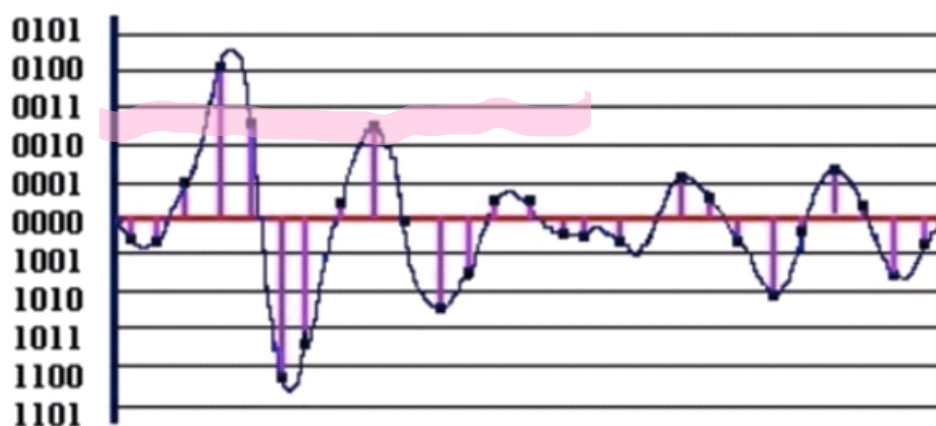
计算机内部处理的是二进制数据，处理的都是**数字音频**，所以需要将模拟音频通过采样、量化转换成有限个数字表示的离散序列（即实现音频数字化）。

最典型的例子就是对音频信号进行编码的脉冲调制（PCM），在计算机应用中，能够达到**最高保真水平**的就是PCM编码，被广泛用于素材保存及音乐欣赏，CD、DVD以及我们常见的WAV文件中均有应用。它主要包括三步：抽样、量化、编码。

1. **抽样**：对模拟信号周期性扫描，把时间上连续的信号变成时间上离散的信号。为了使所得的离散信号能无失真的代表被抽样的模拟数据，要使用采样定理进

行采样： $f_{\text{采样频率}} \geq 2f_{\text{信号最高频率}}$

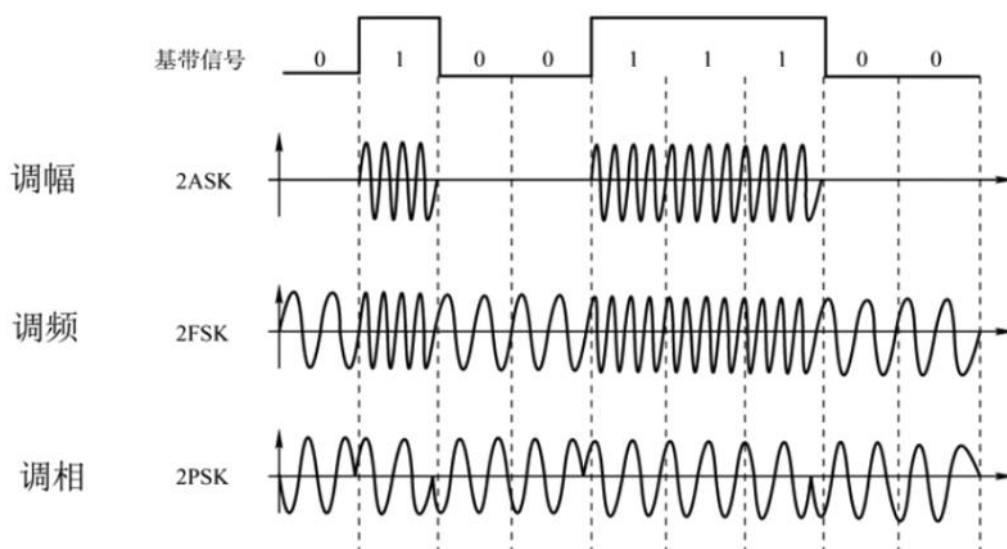
2. **量化**：把抽样取得的电平幅值按照一定的分级标度转化为对应的数字值，并取整数，这就把连续的电平幅值转换为离散的数字量。
3. **编码**：把量化的结果转换为与之对应的二进制编码。



调制

数字数据调制为模拟信号

数字数据调制技术在发送端将数字信号转换为模拟信号，而在接收端将模拟信号还原为数字信号，分别对应于调制解调器的调制和解调过程。



模拟数据调制为模拟信号

为了实现传输的有效性，可能需要较高的频率。这种调制方式还可以使用频分复用技术，充分利用带宽资源。在电话机和本机交换机所传输的信号是采用模拟信号传输模拟数据的方式；模拟的声音数据是加载到模拟的载波信号中传输的。



四种特性

两个定理

奈氏准则

失真

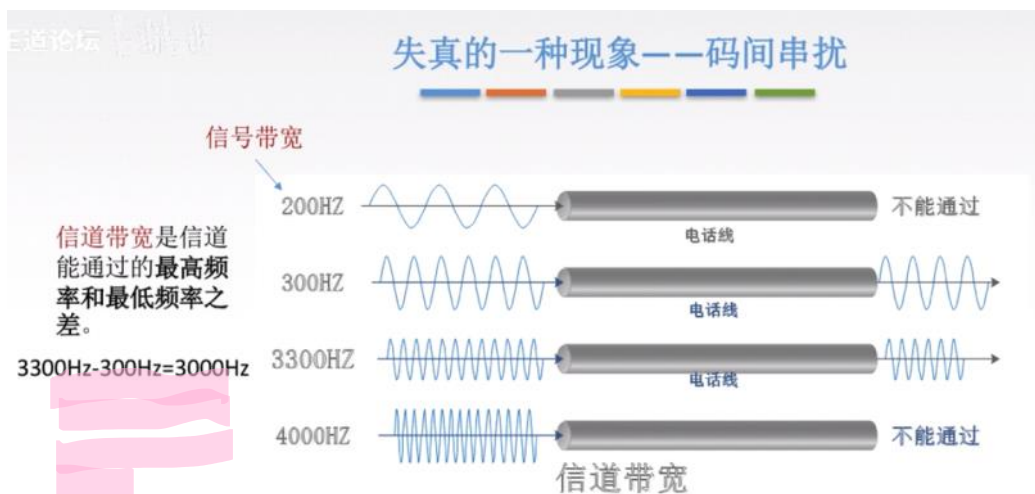
影响失真程度的因素：

1. 码元传输速率
2. 信号传输距离
3. 噪声干扰
4. 传输媒体质量



码间串扰

码间串扰：接收端接到的信号波形失去了码元之间清晰界限的现象。



奈氏准则（奈奎斯特定理）

奈氏准则：在理想低通（无噪声、宽带受限）条件下，为了避免码间串扰，极限码元传输速率为 $2W$ Baud， W 是通道带宽，单位是 Hz。

$$\text{理想低通信道下的极限数据传输率} = 2W \log_2 V \quad (\text{b/s})$$

波特

带宽(Hz)

几种码元/码元的离散电平数目

1. 在任何信道中，码元传输的速率是有上限的，若传输速率超过此上限，就会出现严重的码间串扰问题，使接收端对码元的完全正确识别成为不可能。
2. 信道的**频带越宽**（即能通过的信号高频分量越多），就可以用更高的速率进行码元的有效传输。
3. 奈氏准则给出了码元传输速率的限制，但并没有对信息传输速率给出限制。
4. 由于码元的传输速率受奈氏准则的制约，所以要提高数据的传输速率，就必须

设法使每个码元能携带更多个比特的信息量，这就需要采用多元制的调制方法。

例. 在无噪声的情况下，若某通信链路的带宽为3kHz，采用4个相位，每个相位具有4种振幅的QAM调制技术，则该通信链路的最大数据传输率是多少？

信号有 $4 \times 4 = 16$ 种变化

最大数据传输率 $= 2 \times 3 \text{ k} \times 4 = 24 \text{ kb/s}$

香农定理

概念

在宽带受限且有噪声的信道中，为了不产生误差，信息的数据传输速率有上限值。



1. 信道的**带宽**或信道中的**信噪比**越大，则信息的极限传输速率就**越高**。
2. 对一定的传输带宽和一定的信噪比，信息传输速率的上限就确定了。
3. 只要信息的传输速率低于信道的极限传输速率，就一定能找到某种方法来试实现无差错的传输。
4. 香农定理得出的为极限信息传输速率，实际信道的传输速率要比它低不少。

例. 电话系统的典型参数是信道带宽为3000Hz，信噪比为30dB，则该系统最大数据传输率是多少？

$30\text{dB} = 10 \log_{10}(S/N)$
则 $S/N = 1000$

信道的极限数据传输速率 $= W \log_2(1+S/N) = 3000 \times \log_2(1+1000) \approx 30 \text{ kb/s}$

区别

区别

奈氏准则 内忧	香农定理 外患
带宽受限无噪声条件下，为了避免码间串扰，码元传输速率的上限 $2W$ Baud。	带宽受限有噪声条件下的信息传输速率。
理想低通信道下的极限数据传输率 $= 2W \log_2 V$	信道的极限数据传输速率 $= W \log_2(1+S/N)$
要想提高数据率，就要提高带宽/采用更好的编码技术。	要想提高数据率，就要提高带宽/信噪比。

案例

题目：二进制信号在信噪比为127：1的4kHz信道上传输，最大的数据速率可达到多少？

Nice: $2 \times 4000 \times \log_2 2 = 8000 \text{ b/s}$

香农: $4000 \times \log_2(1+127) \approx 28000 \text{ b/s}$



物理层传输介质

传输媒介及分类

传输媒介：也称传输媒体/传输媒介，它就是数据传输系统中在发送设备和接收设备之间的**物理通路**。

传输媒体并不是物理层。传输媒体在物理层的下面，因为物理层是体系结构的第一层，因此有时称传输媒体为0层。在传输媒体中传输的是信号，但传输媒体并不知道所传输的信号代表什么意思。但物理层规定了**电气特性**，因此能够识别所传送的比

传输媒体并不是物理层。传输媒体在物理层的下面，因为物理层是体系结构的第一层，因此有时称传输媒体为0层。在传输媒体中传输的是信号，但传输媒体并不知道所传输的信号代表什么意思。但物理层规定了**电气特性**，因此能够识别所传送的比特流。

分类：

- **导向性传输介质**
电磁波被导向沿着固体媒介（铜线/光纤）传播。
- **非导向性传输介质**
自由空间，介质可以是空气、真空、海水等。

导向

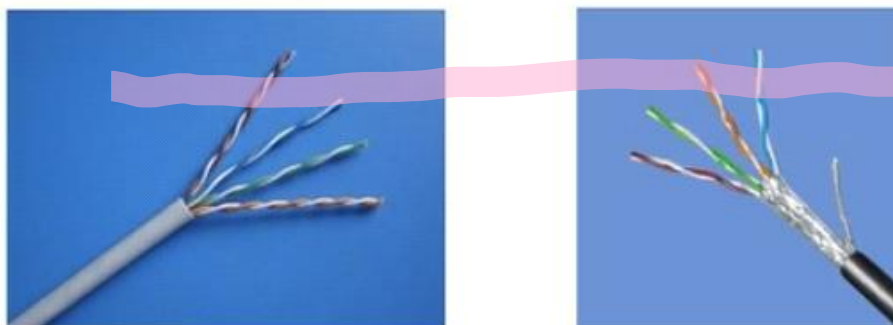
双绞线

导向传输介质1：双绞线

双绞线是古老、又最常用的传输介质，它由两根采用一定规则并排绞合的、相互绝缘的**铜导线**组成。



为了进一步提高抗电磁干扰能力，可在双绞线的外面再加上一个由金属丝编织成的屏蔽层，这就是**屏蔽双绞线**（STP），无屏蔽层的双绞线就称为**非屏蔽双绞线**（UTP）。



非屏蔽双绞线

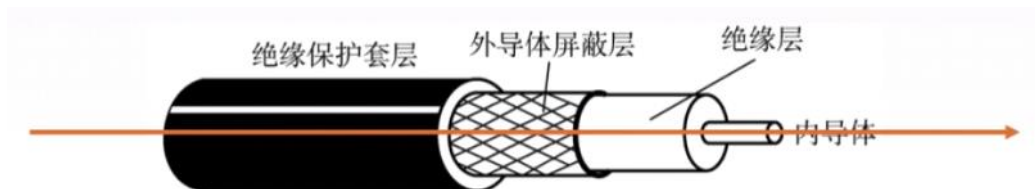
屏蔽双绞线

双绞线价格便宜，是最常用的传输介质之一，在局域网和传统电话网中普遍使用。模拟传输和数字传输都可以使用双绞线，其通信距离一般为几公里到数十公里。距离太远时，对于模拟传输要用放大器放大衰减的信号；对于数字传输，要用中继器将失真的信号整形。

同轴电缆

导向传输介质2：同轴电缆

同轴电缆由导体铜质芯线、绝缘层、网状编织屏蔽层和塑料层构成。按特性阻抗数值的不同，通常将同轴电缆分为两类：50 Ω 同轴电缆和75 Ω 同轴电缆。



其中，50 Ω 同轴电缆主要用于传送基带数字信号，又称为**基带同轴电缆**，它在局域网中得到广泛应用；75 Ω 同轴电缆主要用于传送宽带信号，又称为**宽带同轴电缆**，它主要用于有线电视系统。



同轴电缆VS双绞线

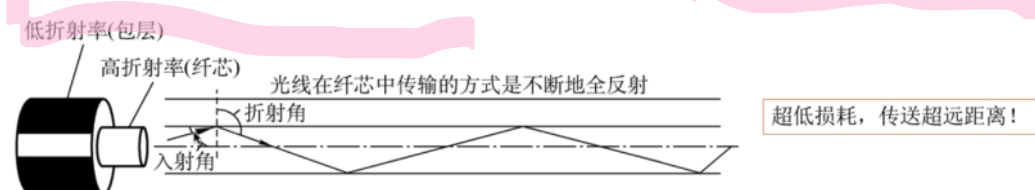
由于外导体屏蔽层的作用，同轴电缆抗干扰特性比双绞线好，被广泛用于传输较高速率的数据，其传输距离更远，但价格较双绞线贵。

光纤

导向传输介质3：光纤

光纤通信就是利用光导纤维（简称光纤）传递光脉冲来进行通信。有光脉冲表示1，无光脉冲表示0。而可见光的频率大约是 10^{14} Hz，因此光纤通信系统的**带宽远远大于**目前其他各种传输媒体的带宽。

光纤在发送端有光源，可以采用发光二极管或半导体激光器，它们在电脉冲作用下能产生出光脉冲；在接收端用光电二极管做成光检测器，在检测到光脉冲时可还原出电脉冲。



光纤主要由纤芯（实心的！）和包层构成，光波通过纤芯进行传导，包层较纤芯有较低的折射率。当光线从高折射率的介质射向低折射率的介质时，其折射角将大于入射角。因此，如果入射角足够大，就会出现全反射，即光线碰到包层时候就会折射回纤芯、这个过程不断重复，光也就沿着光纤传输下去。

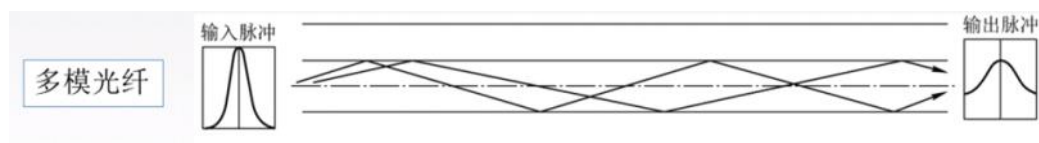


分类

单模光纤：是一种在横向模式直接传输光信号的光纤，光源为定向性很好的激光二极管，特点是衰耗小，适合远距离传输。



多模光纤：有多种传输光信号模式的光纤，光源为发光二极管，特点是易失真，适合近距离传输。



一根光缆少则只有一根光纤，多则包括十至数百根光纤。

特点

- 1 . 传输损耗小，中继距离长，对远距离传输特别经济。
- 2 . 抗雷电和电磁干扰性能好。
- 3 . 无串音干扰，保密性好，也不易被窃听或截取数据。
- 4 . 体积小，重量轻。

非导向

无线电波

无线电波

较强**穿透能力**，可传远距离，广泛用于通信领域（如手机通信）。信号向所有方向传播。

微波

概念

微波通信频率较高、频道范围宽，因此数据率很高。信号固定方向传播。

应用分类

- 地面微波接力通信
- 卫星通信
 - 优点
 - 通信容量大
 - 距离远
 - 覆盖广
 - 广播通信和多址通信
 - 缺点
 - 传播时延长（250-270ms）
 - 受气候影响大（强风、太阳黑子爆发、日凌）
 - 误码率较高
 - 成本高

红外线、激光

概念

要把传输的信号分别**转换为各自的信号格式**，即红外光信号和激光信号，再在空间中传播。信号固定方向传播。

物理层设备

中继器

诞生原因

由于存在损耗，在线路上传输的信号功率会逐渐衰减，衰减到一定程度时将造成信号失真，因此会导致接收错误。

中继器的功能

对信号进行再生和还原，对衰减的信号进行放大，保持与原数据相同，以增加信号传输的距离，延长网络的长度。

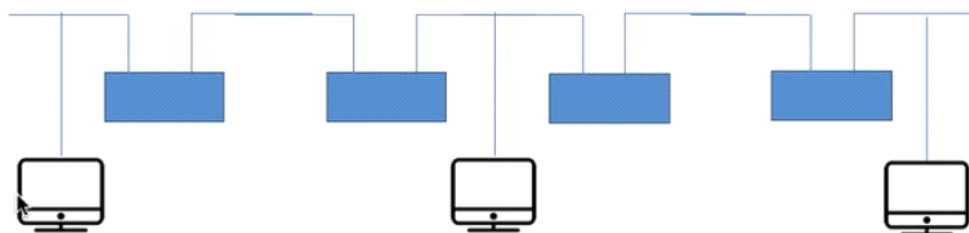


中继器的两端

1. 两端的网络部分是网段，而不是子网，适用于完全相同的两类网络的互连，且两个网段速率要相同。
2. 中继器只将任何电缆段上的数据发送到另一段电缆上，它仅作用于信号的电气部分，并不管数据中是否有错误数据或不适于网段的数据。
3. 两端可连相同媒体，也可连不同媒体。
4. 中继器两端的网段一定要是同一个协议（中继器不会存储转发）。

5-4-3规则

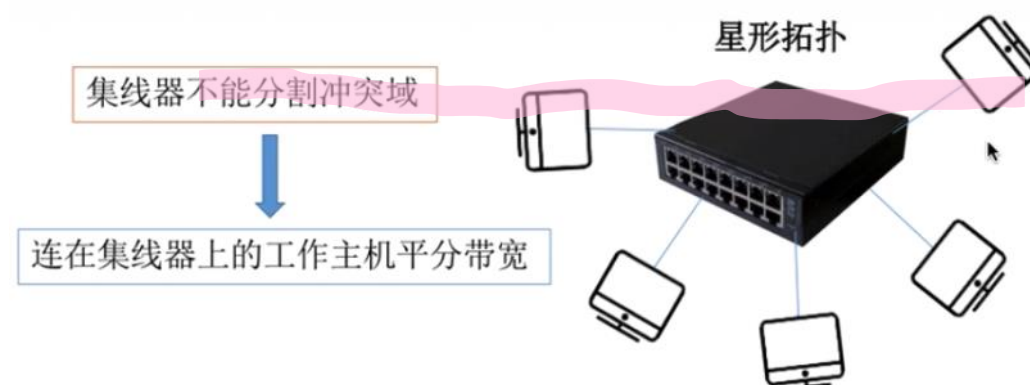
网络标准中都对信号的延迟范围作了具体的规定，因而中继器只能在规定的范围内进行，否则会网络故障。



集线器

集线器（多口中继器）

集线器的功能：对信号进行再生放大转发，对衰减的信号进行放大，接着转发到其他所有（除输入端口外），处于工作状态的端口上，以增加信号传输的距离，延长网络的长度。不具备信号的定向传送能力，是一个共享式设备。



第五章 传输层

星期日，三月 13，2022 5:28 下午

已剪辑自: <https://blog.csdn.net/donkey010/article/details/114691294>

第五章：传输层

5.1 传输层提供的服务

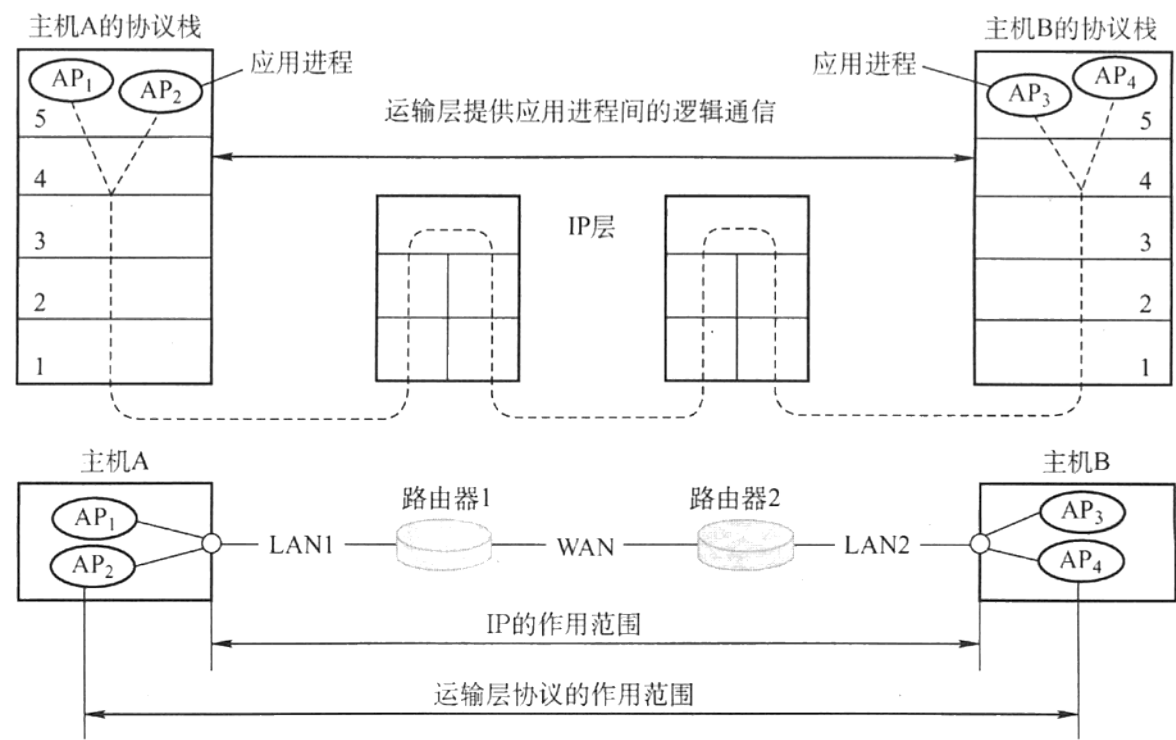
5.1.1 传输层的功能

概述

从通信和信息处理的角度看，传输层向它上面的应用层提供通信服务，它属于面向通信部分的最高层，同时也是用户功能中的最低层。

传输层位于网络层之上，它为运行在不同主机上的进程之间提供了逻辑通信，而网络层提供了主机之间的逻辑通信。显然，即使当网络层协议是不可靠的，也就是即使网络层协议会使分组丢失、混乱和重复，传输层同样也能为应用程序提供可靠的服务。

传输层为相互通信的进程提供逻辑通信



传输层的功能

1、**传输层**提供应用进程之间的逻辑通信（即端到端的通信）。与网络层的区别是，网络层提供的是主机之间的逻辑通信。

2、复用和分用。

- 复用：是指发送方不同的应用进程都可以使用同一个传输层协议传送数据；
- 分用：是指接收方的传输层在剥去报文的首部后能够把这些数据正确交付到目的应用进程。

3、传输层对收到的报文进行差错检测（首部和数据部分）。而网络层只检查IP数据报的首部，不检验数据部分是否出错。

4、提供两种不同的传输协议，即面向连接的TCP和无连接的UDP。而网络层无法同时实现两种协议。

当传输层采用面向连接的TCP时，尽管下面的网络是不可靠的，但这种逻辑通信信道就相当于是一条全双工的可靠信道。但当传输层采用无连接的UDP时，这种逻辑通信信道仍然是一条不可靠信道。

5.1.2 传输层的寻址与端口

端口的作用

- 端口能够让应用层的各种应用进程将其数据通过端口向下交付给传输层，以及让传输层知道应当将其报文段中的数据向上通过端口交付给应用层相应的进程。
- 端口就是传输层服务访问点TSAP，标识的是主机中的应用进程。
- 数据链路层的SAP是MAC地址，网络层的SAP是IP地址，传输层的SAP是端口。
- 硬件端口是不同硬件设备进行交互的接口，而软件端口是应用层的各种协议进程与传输实体进行层间交互的一种地址。传输层使用的是软件端口。
- 端口号只具有本地意义，即端口号只是为了标志本计算机应用层中的各进程。在因特网中不同计算机的相同端口号是没有联系的。

端口号

应用进程通过端口号进行标识，端口号长度为16bit，能够表示65536 (2^{16}) 个不同的端口号。端口号只具有本地意义，即端口号只是为了标志本计算机应用层中的各进程，不同计算机的相同 端口号是没有联系的。

根据端口号范围可将端口分为两类

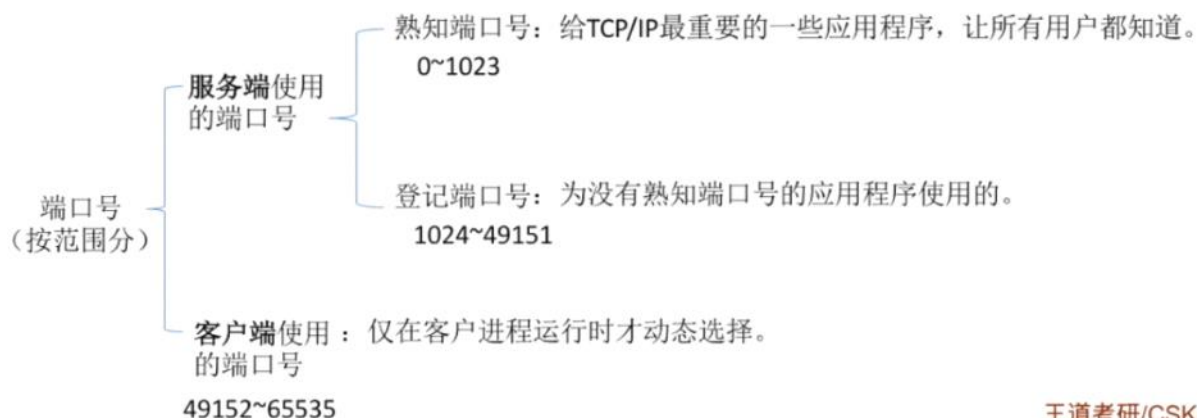
• 服务端使用的端口号

- 熟知端口号：数值为0~1023，IANA（互联网地址指派机构）把这些端口号指派给了TCP/IP最重要的一些应用程序，让所有的用户都知道。
- 登记端口号：数值为1024~49151。它是为没有熟知端口号的应用程序使用的，使用这类端口号必须在IANA 登记，以防止重复。

应用程序	FTP	TELNET	SMTP	DNS	TFTP	HTTP	SNMP
熟知端口号	21	23	25	53	69	80	161

• 客户端使用的端口号

- 数值为49152~65535，由于这类端口号仅在客户进程运行时才动态选择，因此又叫短暂端口号(也称临时端口)。通信结束后，刚才使用过的客户端端口号就不复存在，这个端口号就可以供其他客户进程使用。



王道考研/CSKA

套接字

网络中通过IP地址来标识和区别不同的主机，通过端口号来标识和区分一台主机中的不同应用进程。在网络中采用发送方和接收方的套接字(Socket) 组合来识别端点。

套接字实际上是一个通信端点，即 套 接 字 = (主 机 I P 地 址 ， 端 口 号)
套接字 = (主机IP地址, 端口号) 套接字=(主机IP地址, 端口号) ，它唯一地标识了网络中的一个主机和其上的一个应用(进程)。

5.1.3 无连接服务与面向连接服务

概述

面向连接服务：是在通信双方进行通信之前，必须先建立连接，在通信过程中，整个连接的情况一直被实时地监控和管理。当通信结束后，则应该释放这个连接。

无连接服务：两个实体之间的通信不需要先建立好连接，需要通信的时候，直接将信息发送到“网络”中，让该信息的传递在网上尽力而为地往目的地传送。

TCP/IP 协议族在IP层之上使用了两个传输协议：

- 面向连接的传输控制协议TCP，当采用TCP时，传输层向上提供的是一条全双工的可靠逻辑信道。
- 无连接的用户数据报协议UDP，当采用UDP 时，传输层向上提供的是一条不可靠的逻辑信道。

IP数据报和UDP数据报的区别

1. IP数据报在网络层要经过路由的存储转发；
2. UDP数据报是在传输层的端到端的逻辑信道中传输，而封装成IP数据报在网络层传输时，UDP数据报的信息对路由是不可见的。

TCP和网络层虚电路的区别

1. TCP报文段是在传输层抽象的逻辑信道中传输，对路由器不可见；
2. 虚电路所经过的交换结点都必须保存虚电路状态信息。在网络层若采用虚电方式，则无法提供无连接服务；
3. 而传输层采用TCP协议不影响网络层提供无连接服务。

5.2 UDP协议

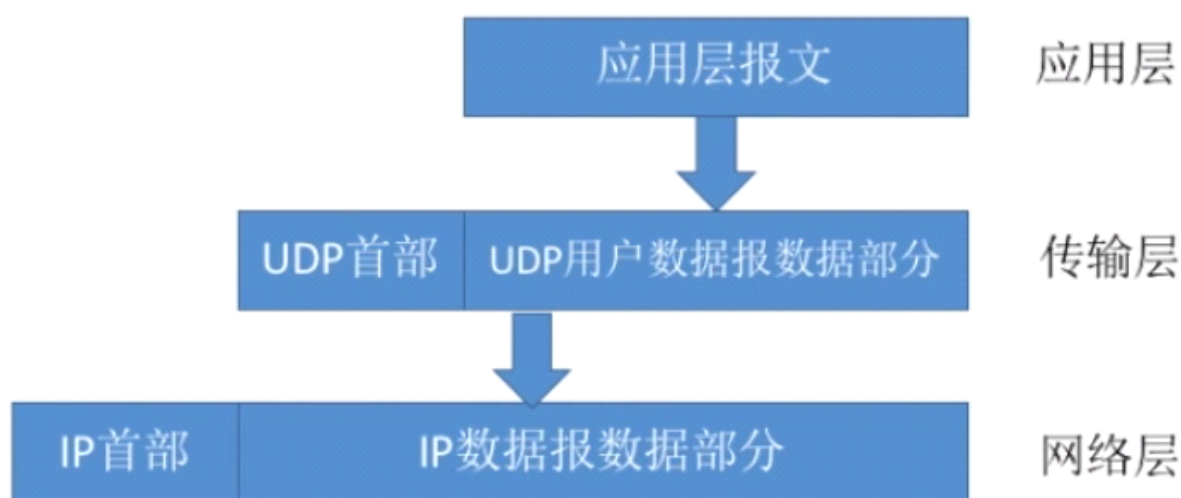
5.2.1 UDP数据报

概述

[RFC 768]定义的UDP只是做了传输协议能够做的最少工作，只在IP的数据报服务之上增加了两个最基本的服务：复用和分用以及差错检测。

特点

- UDP是无连接的，减少开销和发送数据之前的时延。
- UDP使用最大努力交付，即不保证可靠交付。
- UDP是面向报文的，适合一次性传输少量数据的网络应用。
- UDP无拥塞控制，适合很多实时应用。
- UDP首部开销小，8B，TCP20B。



UDP优点:

1. UDP无需建立连接。因此UDP不会引入建立连接的时延。
2. 无连接状态。TCP需要在端系统中维护连接状态。此连接状态包括接收和发送缓存、拥塞控制参数和序号与确认号的参数。而UDP不维护连接状态，也不跟踪这些参数。
3. 分组首部开销小。TCP有20字节的首部开销，而UDP仅有8字节的开销。
4. 应用层能更好地控制要发送的数据和发送时间。UDP没有拥塞控制，因此网络中的拥塞也不会影响主机的发送效率。

UDP常用于一次性传输比较少量数据的网络应用，如DNS、SNMP等，也常用于多媒体应用如IP电话、实时视频会议、流媒体等。

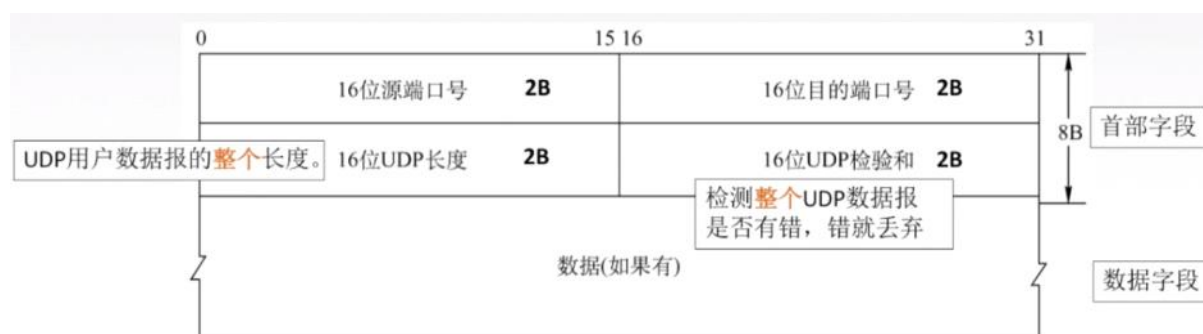
UDP提供尽最大努力的交付，即不保证可靠交付，但这并不意味着应用对数据的要求是不可靠的，因此所有维护传输可靠性的工作需要用户在设计自己的可靠性机制。

UDP是面向报文的。报文不可分割，是UDP数据报处理的最小单位。

UDP的首部格式

UDP数据报包含两个部分：UDP首部和用户数据，整个UDP数据报作为IP数据报的数据部分封装在IP数据报中。UDP首部有8个字节，由4个字段组成，每个字段的长度都是两个

字节。



各字段意义：

1. **源端口**：源端口号。在需要对方回信时选用。不需要时可用全0。
 2. **目的端口**：目的端口号。这在终点交付报文时必须使用到。
 3. **长度**：UDP数据报的长度(包括首部和数据)，其最小值是8(仅有首部)。
 4. **校验和**：检测UDP数据报在传输中是否有错。有错就丢弃。该字段是可选的，当源主机不想计算校验和，则直接令该字段为全0。
- 当传输层从IP层收到UDP数据报时，就根据首部中的目的端口，把UDP数据报通过相应的端口，上交给应用进程。
 - 如果接收方UDP发现收到的报文中的目的端口号不正确(即不存在对应于端口号的应用进程)，就丢弃该报文，并由ICMP发送“端口不可达”差错报文给发送方。

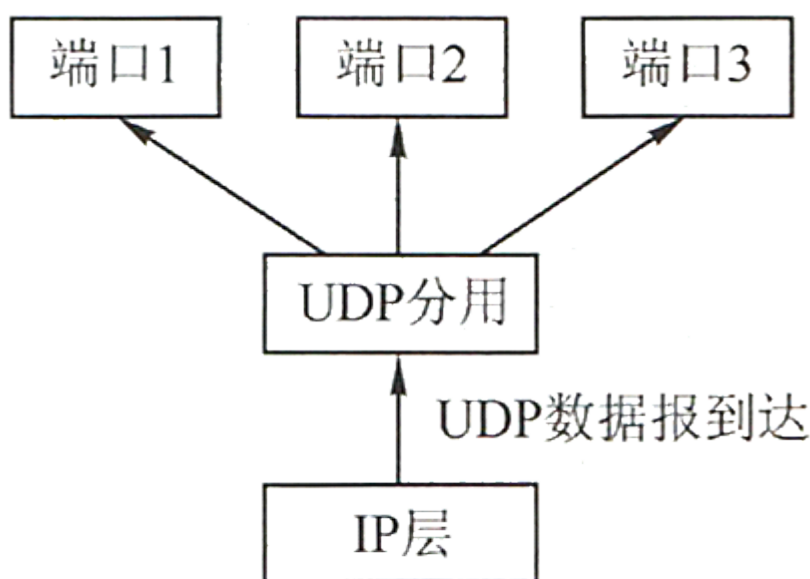


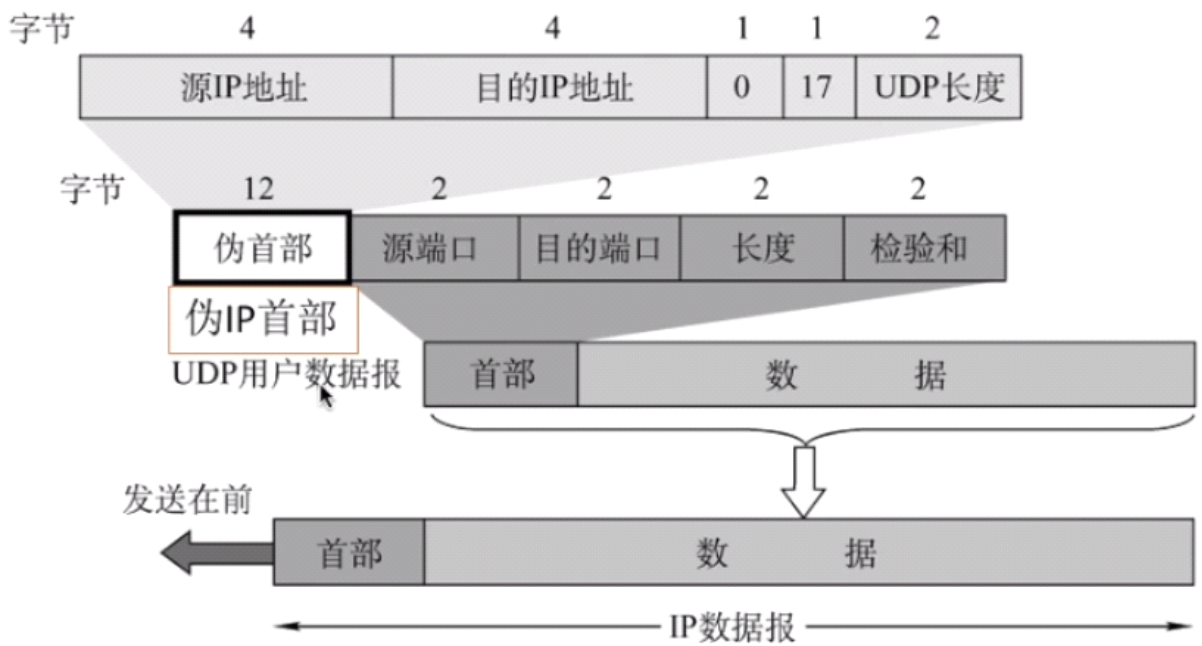
图 5-3 UDP 基于端口的分用

5.2.2 UDP校验

概述

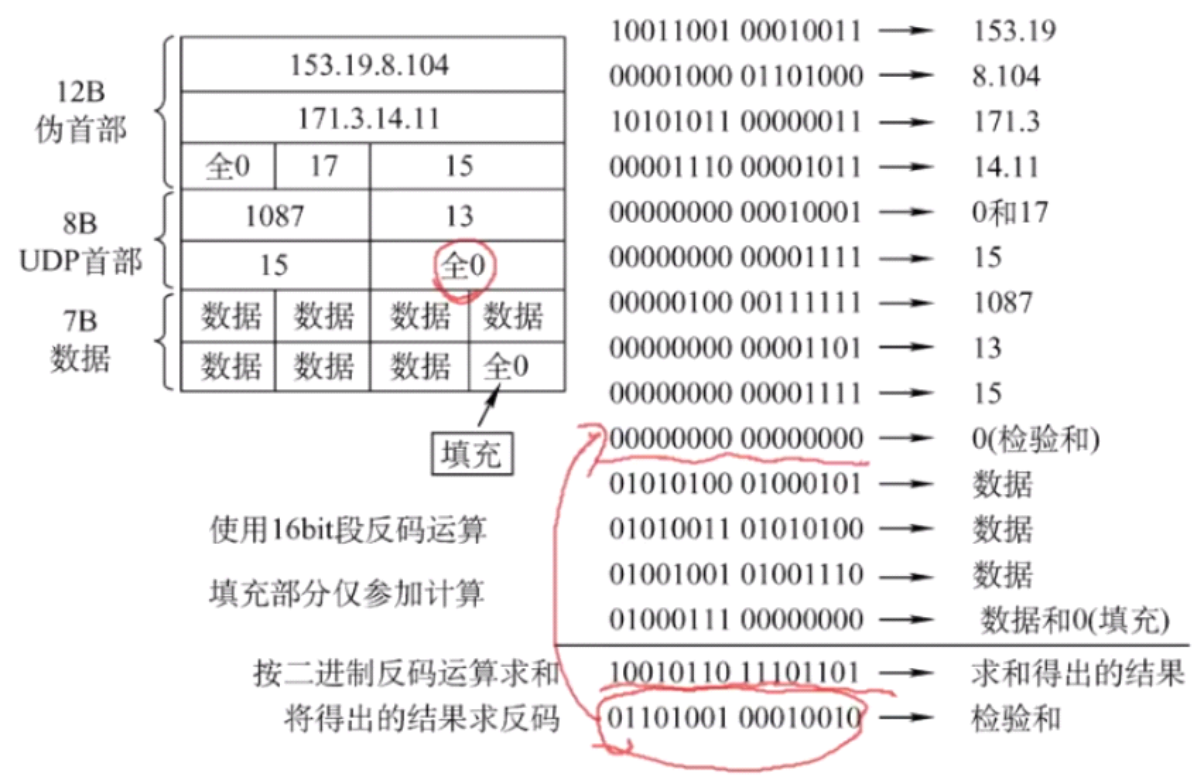
在计算校验和时，要在UDP数据报之前增加12个字节的伪首部，伪首部并不是UDP真正的首部。只是在计算校验和时，临时添加在UDP数据报的前面，得到一个临时的UDP数

据报。**校验和就是按照这个临时的UDP数据报计算的。**伪首部既不向下传送也不向上递交，而仅仅是为了计算校验和。这样的校验和，既检查了UDP数据报，又对IP数据报的源IP地址和目的IP地址进行了检验。



UDP校验和计算过程

UDP校验和的计算方法和IP数据报首部校验和的计算方法相似，都使用二进制反码运算求和再取反。但不同的是：IP数据报的校验和只检验IP数据报的首部，但UDP的校验和是把首部和数据部分一起都检验。



在发送端：

1. 填上伪首部
2. 全0填充校验和字段

3. 全0填充数据部分（UDP数据报要看成许多4B的字串接起来）
4. 伪首部+首部+数据部分采用二进制反码求和
5. 把和求反码填入检验和字段
6. 去掉伪首部，发送

在接收端：

1. 填上伪首部
2. 伪首部+首部+数据部分采用二进制反码求和
3. **结果全为1则无差错**，否则丢弃数据/交给应用层附上出差错的警告。

注意

1. 校验时，若UDP数据报部分的长度不是偶数个字节，则需要填入一个全0字节，但是此字节和伪首部一样，是不发送的。
2. 如果UDP校验和校验出UDP数据报是错误的，可以丢弃，也可以交付给上层，但是需要附上错误报告，即告诉上层这是错误的的数据报。
3. 通过伪首部，不仅可以检查源端口号、目的端口号和UDP用户数据报的数据部分，还可以检查IP数据报的源IP地址和目的地址。

这种简单的差错检验方法的检错能力并不强，但它的好处是**简单，处理速度快**。

5.3 TCP协议

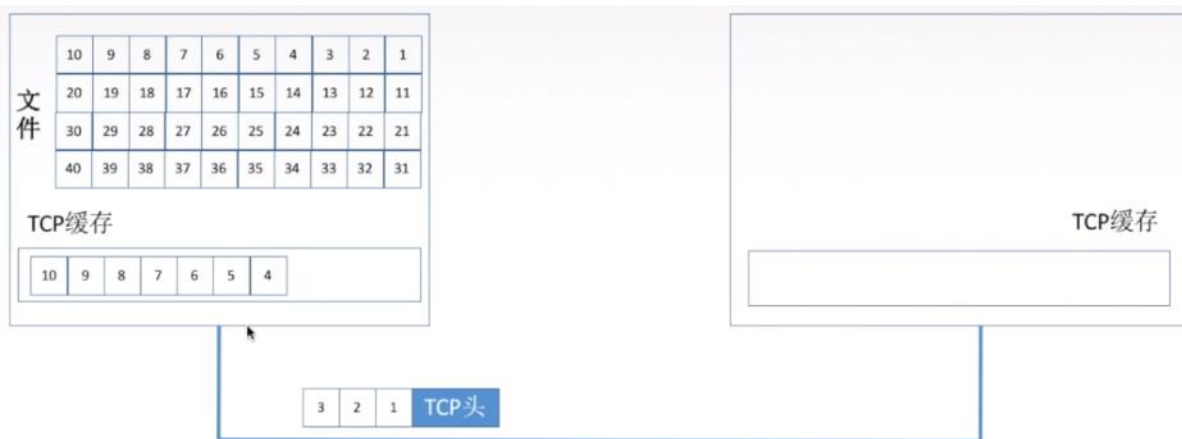
5.3.1 TCP协议的特点

概述

TCP是在不可靠的IP层之上实现的可靠的数据传输协议，它主要解决传输的可靠、有序、无丢失和不重复的问题。

特点

1. TCP是面向连接（虚连接）的传输层协议。
2. 每一条TCP连接只能有两个端点，每一条TCP连接只能是点对点的（一对一）。
3. TCP提供可靠交付的服务，保证传送的数据无差错、不丢失、不重复、按序到达。可靠有序，不丢不重。
4. TCP提供全双工通信，TCP 允许通信双方的应用进程在任何时候都能发送数据，为此TCP连接的两端都设有发送缓存和接收缓存，用来临时存放双向通信的数据。
 - 发送缓存：① 准备发送的数据；② 已发送但尚未收到确认的数据。
 - 接收缓存：① 按序到达但尚未被接受应用程序读取的数据；② 不按序到达的数据。
5. TCP是面向字节流的，虽然应用程序和TCP的交互是一次一个数据块（大小不等），但TCP把应用程序交下来的数据看成仅仅是一连串的无结构的字节流。（**流**：流入到进程或从进程流出的字节序列）。



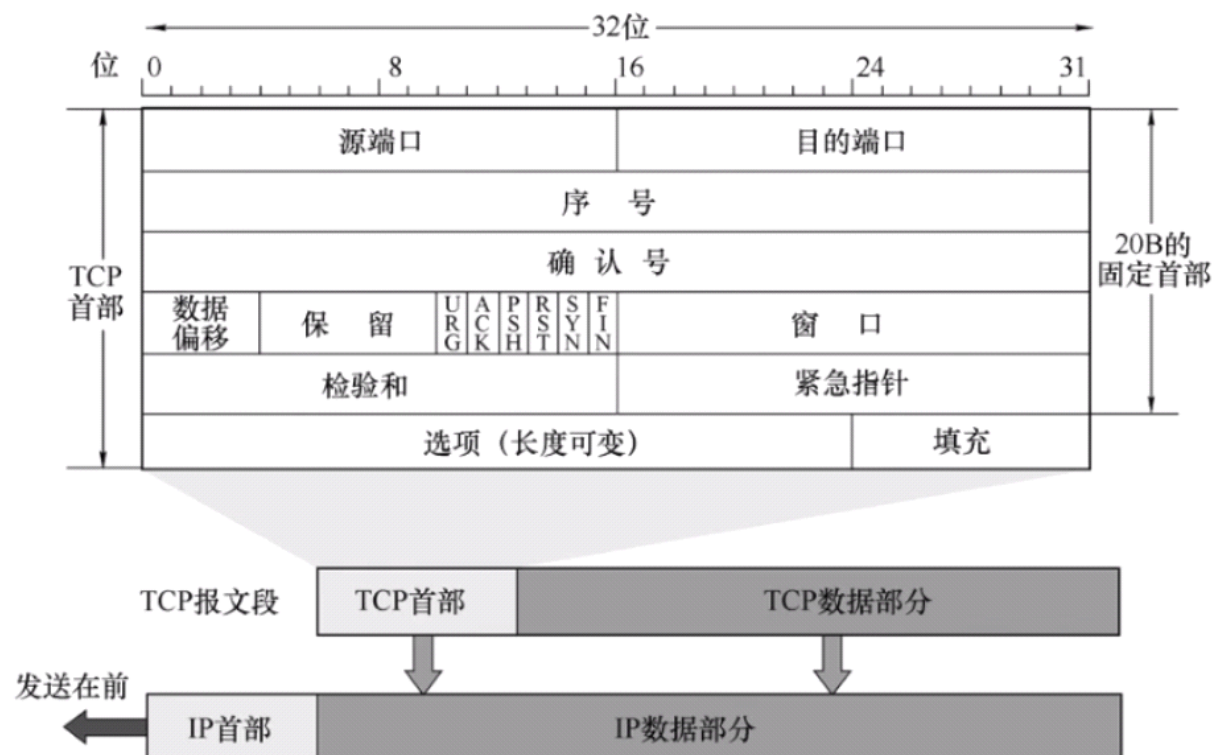
5.3.2 TCP报文段

概述

TCP传送的数据单元称为报文段。一个TCP报文段分为TCP首部和TCP数据两部分，整个TCP段作为IP数据报的数据部分封装在IP数据报中。其首部的前20字节是固定的。TCP报文段的首部最短为20字节，后面有4N字节是根据需要而增加的选项，通常长度为4字节的整数倍。

TCP报文段既可以用来运载数据，也可以用来建立连接、释放连接和应答。

TCP报文段首部



各字段意义：

1、**源端口和目的端口字段：**各占2字节。端口是运输层与应用层的服务接口。运输层的复用和分用功能都要通过端口才能实现。

2、**序号字段：**占4字节，TCP是面向字节流的，所以TCP连接中传送的数据流中的每一

个字节都编上一个序号。序号字段的值则指的是本报文段所发送的数据的第一个字节的序号。

3、**确认号字段**：占 4 字节，是期望收到对方的下一个报文段的数据的第一个字节的序号。若确认号=N，则表明到序号N-1为止的所有数据都已正确收到。

4、**数据偏移字段**：（表示首部长度）占4位，指出TCP报文段的数据起始处距离TCP报文段的起始处有多远，单位是32位（以4字节为计算单位）。因此当此字段的值为15时，达到TCP首部的最大长度60字节。

5、**保留字段**：占6位，保留为今后使用，但目前应置为0，该字段可以忽略不计。

6、6个控制位

- **紧急位URG**：URG=1时，表明此报文段中有紧急数据，是高优先级的数据，应尽快传送，不用在缓存里排队，但是URG需要和紧急指针配套使用，也就是说数据从第一个字节到紧急指针所指字节就是紧急数据。
- **确认位ACK**：ACK=1时确认号有效，当ACK=0时，确认号无效。TCP规定，在连接建立后所有传送的报文段都必须把ACK置为1。
- **推送位PSH**：PSH=1时，接收方尽快交付接收应用进程，不再等到缓存填满再向上交付。
- **复位RST**：RST=1时，表明TCP连接中出现严重差错，必须释放连接，然后再重新建立传输链接。
- **同步位SYN**：SYN=1时，表示这是一个连接请求或连接接收报文。
 - 当SYN=1，ACK=0时，表明这是一个连接请求报文，对方若同意建立连接，则在响应报文中使用 SYN=1，ACK=1。即，SYN=1就表示这是一个连接请求或连接接收报文。
- **终止位FIN**：FIN=1时，表明此报文段发送方数据已发完，要求释放连接。

7、**窗口字段**：占2字节。它指出了现在允许对方发送的数据量，接收方的数据缓存空间是有限的，故用窗口值作为接收方让发送方设置其发送窗口的依据，单位为字节。

8、**检验和字段**：占2字节。检验和字段检验的范围包括首部和数据这两部分。在计算检验和时，和UDP一样，要在TCP报文段的前面加上 12字节的伪首部。

9、**紧急指针字段**：占 16 位，指出在本报文段中紧急数据共有多少个字节（紧急数据放在本报文段数据的最前面）。

10、**选项字段**：长度可变。TCP最初只规定了一种选项，即最大报文段长度。MSS是TCP报文段中的数据字段的最大长度。

11、**填充字段**：为了使整个首部长度是4字节的整数倍。

5.3.3 TCP连接管理

概述

TCP是面向连接的协议。因此每一个TCP连接都有三个阶段：连接建立、数据传送和连接释放。TCP连接的管理就是使运输连接的建立和释放都能正常进行。

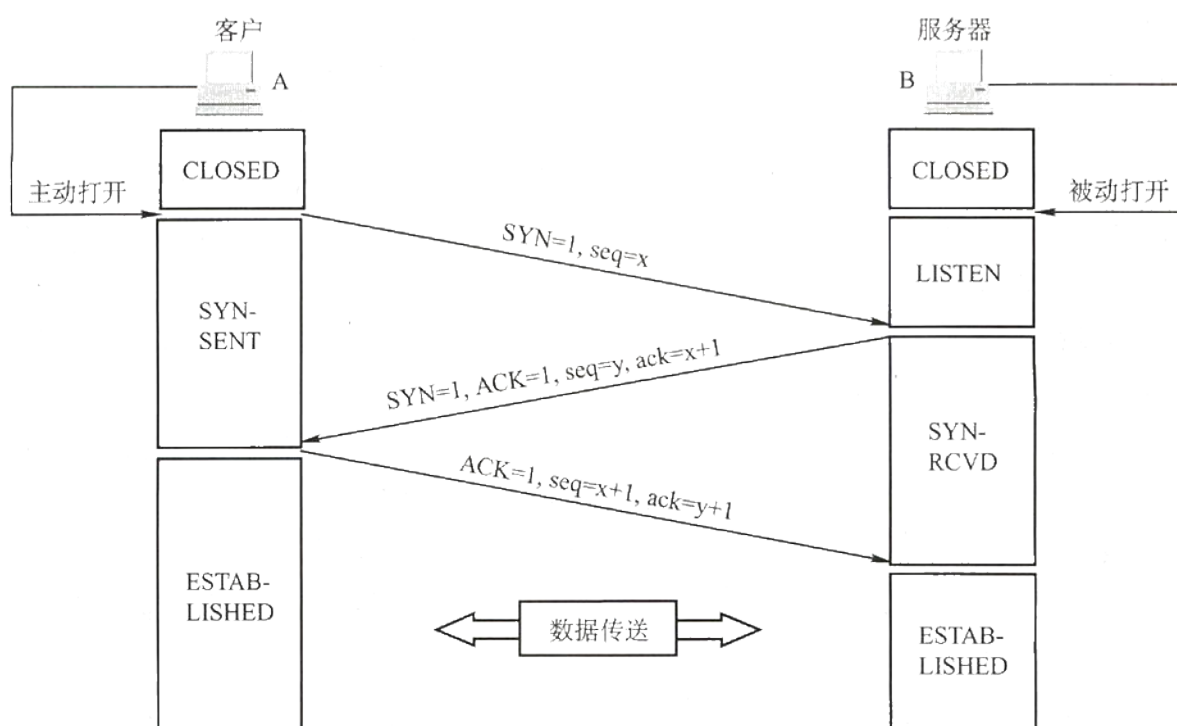
TCP把连接作为最基本的抽象，每一条TCP连接有两个端点，TCP连接的端口叫做套接字(socket)或插口。端口拼接到IP地址即构成了套接字。每一条TCP连接唯一地被通信两端的两个端点(即两个套接字)所确定。

TCP连接的建立采用客户/服务器方式。主动发起连接建立的应用进程叫做客户机(Client)，而被动等待连接建立的应用进程叫做服务器(Server)。



TCP的连接的建立

连接的建立经历以下 3 个步骤，通常称为“三次握手”：



阶段1：客户机的TCP首先向服务器的TCP发送一个连接请求报文段。这个特殊的报文段中**不含应用层数据**，其首部中的SYN标志位被置为1。另外，客户机会随机选择一个起始序号 $seq=x$ （连接请求报文不携带数据，但要消耗掉一个序号）。

- $SYN=1, seq=x$ （随机）

阶段2：服务器的TCP收到连接请求报文段后，如同意建立连接，就向客户机发回确认，并为该TCP连接分配TCP缓存和变量。在确认报文段中，SYN和ACK位都被置为1，确认号字段的值为 $x+1$ ，并且服务器随机产生起始序号 $seq=y$ （确认报文不携带数据，但也要消耗掉一个序号）。确认报文段同样不包含应用层数据。

- $SYN=1, ACK=1, seq=y$ （随机）， $ack=x+1$

阶段3：当客户机收到确认报文段后，还要向服务器给出确认，并且也要给该连接分配缓存和变量。这个报文段的ACK标志位被置1，序号字段为 $x+1$ ，确认号字段 $ack=y+1$ 。该报文段可以携带数据，如果不携带数据则不消耗序号。

- SYN=0, ACK=1, seq=x+1, ack=y+1

注意：

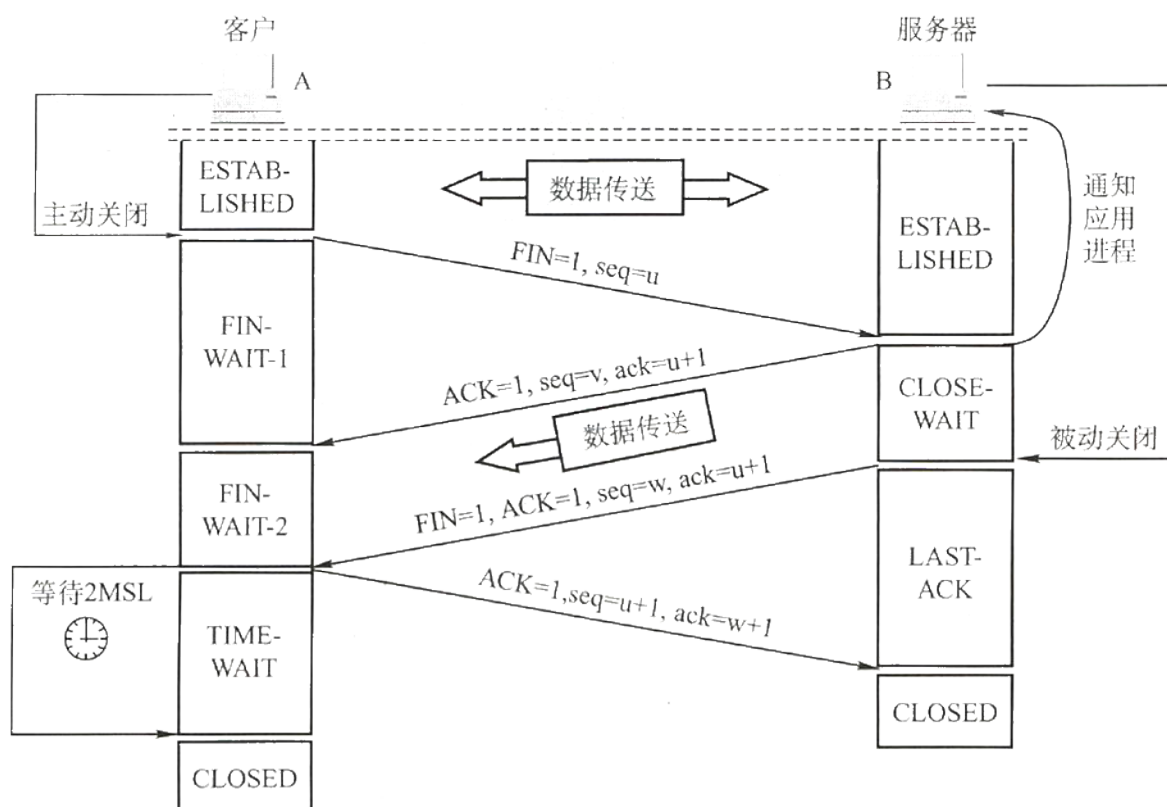
- TCP提供的是全双工通信，因此通信双方的应用进程在任何时候都能发送数据。
- 服务器端的资源是在完成第二次握手时分配的，而客户端的资源是在完成第三次握手时分配的。这就使得服务器易于受到SYN洪泛攻击。

SYN洪泛攻击

SYN洪泛攻击发生在OSI第四层，这种方式利用TCP协议的特性，就是三次握手。攻击者发送 TCP SYN，SYN是TCP三次握手中的**第一个数据包**，而当服务器返回ACK后，该攻击者就不对其进行再确认，那这个TCP连接就处于挂起状态，也就是所谓的半连接状态，服务器收不到再确认的话，还会重复发送ACK给攻击者。这样更加会浪费服务器的资源。攻击者就对服务器发送非常大量的这种TCP连接，由于每一个都没法完成三次握手，所以在服务器上，这些TCP连接会因为挂起状态而消耗CPU和内存，最后服务器可能死机，就无法为正常用户提供服了。

TCP连接的释放

参与一条TCP连接的两个进程中的任何一个都能终止该连接，连接结束后，主机中的“资源”（缓存和变量）将被释放。TCP连接释放的过程通常称为“四次挥手”。



阶段1：客户机打算关闭连接，就向其TCP发送一个连接释放报文段，并停止再发送数据，主动关闭TCP连接，该报文段FIN标志位被置1，seq=u，它等于前面已传送过的数据的最后一个字节的序号加1（FIN报文段即使不携带数据，也要消耗掉一个序号）。

- FIN=1, seq=u

阶段2：服务器收到连接释放报文段后即发出确认，确认号是 ack=u+1，而这个报文段自己的序号是v，等于它前面已传送过的数据的最后一个字节的序号加1。此时，从客户机到服务器这个方向的连接就释放了，TCP连接处于半关闭状态。但服务器若发送数

据，客户机仍要接收，即从服务器到客户机这个方向的连接并未关闭。

- ACK=1, seq=v, ack=u+1

阶段3: 若服务器已经没有要向客户机发送的数据，就通知TCP释放连接，此时其发出FIN=1的连接释放报文段。

- FIN=1, ACK=1, seq=w, ack=u+1

阶段4: 客户机收到连接释放报文段后，必须发出确认。在确认报文段中，ACK字段被置为1，确认号ack=w+1，序号 seq=u+1。此时 TCP连接还没有释放掉，必须经过时间等待计时器设置的时间2MSL后，A才进入到连接关闭状态。

- ACK=1, seq=u+1, ack=w+1

5.3.4 TCP可靠传输

概述

TCP的任务是在IP层的不可靠的、尽力而为服务的基础上建立一种可靠数据传输服务。**TCP提供的可靠数据传输服务就是要保证接收方进程从缓存区读出的字节流与发送方发出的字节流是完全一样的。**TCP使用了校验、序号、确认和重传等机制来达到这个目的。

序号

TCP首部的序号字段用来保证数据能有序提交给应用层，TCP把数据看成一个无结构但是有序的字节流，而序号是建立在传送的字节流之上，而不是建立在报文段之上。

TCP连接中传送的数据流中的每一个字节都编上一个序号。序号字段的值则指的是本报文段所发送的数据的第一个字节的序号。

假设A和B之间建立了一条TCP连接，A的发送缓存区中总共有10个字节，序号从0开始标号，第一个报文包含第0~2个字节，则该TCP报文段的序号是0；第二个报文段的序号是3。

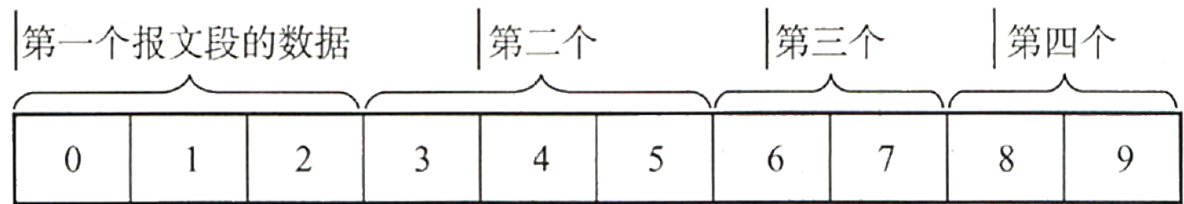


图 5-9 A 的发送缓存区里的数据划分成 TCP 段

确认

TCP首部的**确认号**是期望收到对方的下一个报文段的数据的第一个字节的序号。发送方缓存区会继续存储那些已经发送但未收到确认的报文段，以便在需要的时候重传。

TCP默认使用**累计确认**，即TCP只确认数据流中至第一个丢失字节为止的字节。



重传

有两种事件会导致TCP对报文段进行重传：**超时**和**冗余ACK**。

1、超时

TCP每发送一个报文段，就对这个报文段设置一次计时器。只要计时器设置的重传时间到期但还没有收到确认，就要重传这一报文段。

由于TCP的下层是一个互联网环境，IP数据报所选择的路由变化很大。因而传输层的往返时延的方差也很大。为了计算超时计时器的重传时间，TCP采用一种自适应算法，它记录一个报文段发出的时间，以及收到相应确认的时间，这两个时间之差叫做报文段的**往返时间(Round-Trip Time, RTT)**。

TCP保留了RTT的一个加权平均往返时间RTTs，当第一次测量RTT样本时，RTTs值就为所测量到的RTT样本的值，但以后每测量一个新的 RTT样本，就重新计算一次RTTs。

公式： 新的 RTTs = (1 - α) * (旧的 RTTs) + α * (新的 RTT 样本)

$$\text{新的RTT_S} = (1 - \alpha) * (\text{旧的RTT_S}) + \alpha * (\text{新的RTT样本})$$

$$\text{新的RTTS} = (1 - \alpha) * (\text{旧的RTTS}) + \alpha * (\text{新的RTT样本})$$

参数： $0 \leq \alpha \leq 1$ $0 \leq \alpha \leq 1$ ，若 α 很接近于零，表示新的RTTs值和旧的RTTs值相比变化不大，而受新的RTT样本影响不大（RTT 值更新较慢）。若选择 α 接近于1，则表示新的RTTs值受新的RTT样本的影响较大（RTT值更新较快）。

超时计时器设置的**超时重传时间**（Retransmission Time-Out, RTO）应略大于加权平均往返时间RTTs：

公式： $RTO = RTTs + 4 * RTTD$ $RTO = RTTS + 4 * RTTD$

参数： RTTD RTT_D RTTD 是RTT的偏差的加权平均值，它与 RTTs RTT_S RTTS 和新的RTT样本之差有关。当第一次测量时， RTTD RTT_D RTTD 取为测量到的RTT样本值的一半。

公式： 新的 RTTD = (1 - β) * (旧的 RTTD) + β * |RTTs - 新的 RTT 样本|

$$\text{新的RTT_D} = (1 - \beta) * (\text{旧的RTT_D}) + \beta * |\text{RTT_S} - \text{新的RTT样本}|$$

$$\text{新的RTTD} = (1 - \beta) * (\text{旧的RTTD}) + \beta * |\text{RTTS} - \text{新的RTT样本}|$$

参数： β \beta β 是个小于1的系数，它的推荐值是0.25。

2、冗余ACK（冗余确认）

每当比期望序号大的失序报文段到达时，发送一个**冗余ACK**，指明下一个期待字节的序号。冗余ACK就是再次确认某个报文段的ACK，而发送方先前已经收到过该报文段的确认。

TCP规定每当比期望序号大的失序报文段到达时，发送一个冗余ACK，指明下一个期待字节的序号。

TCP规定当发送方收到对同一个报文段的3个冗余ACK时，就可以认为跟在这个被确认报文段之后的报文段已经丢失。

发送方已发送1, 2, 3, 4, 5报文段

接收方收到1，返回给1的确认（确认号为2的第一个字节）

接收方收到3，仍返回给1的确认（确认号为2的第一个字节）

接收方收到4，仍返回给1的确认（确认号为2的第一个字节）

接收方收到5，仍返回给1的确认（确认号为2的第一个字节）

发送方收到**3个对于报文段1的冗余ACK** - 认为2报文段丢失，重传2号报文段（快速重传）

5.3.5 TCP的流量控制

概述

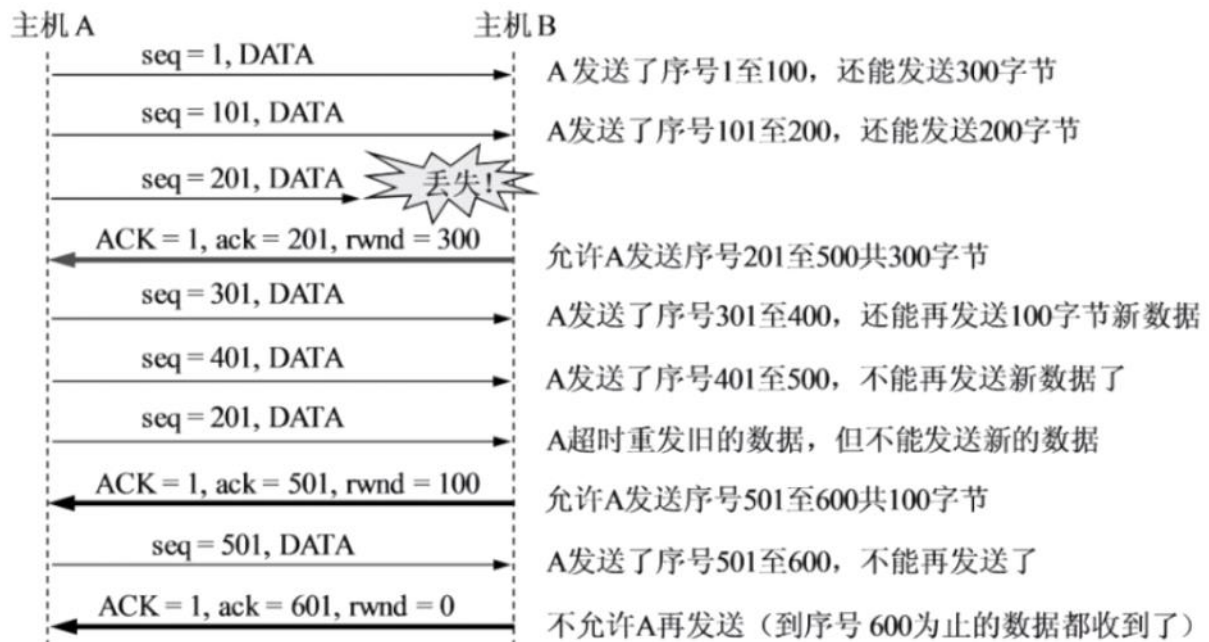
TCP提供了**流量控制**服务以消除发送方使接收方缓存区溢出的可能性，因此可以说流量控制是一个速度匹配服务（匹配发送方的发送速率与接收方的读取速率）。TCP利用**滑动窗口**机制实现流量控制。

在通信过程中，接收方根据自己**接收缓存的大小**，动态地调整发送方的发送窗口大小，这就是接收窗口rwnd，即调整TCP报文段首部中的“窗口”字段值，来限制发送方向网络注入报文的速率。

同时，发送方根据其对当前网络拥塞程序的估计而确定的窗口值，称为拥塞窗口cwnd，其大小与网络的带宽和时延密切相关。发送方的**发送窗口取接收窗口rwnd和拥塞窗口cwnd的最小值**。

示例

A向B发送数据，连接建立时，B告诉A：“我的rwnd=400(字节)”，假设每一个报文段100B，报文段序号初始值为1。



问题：当响应主机B的滑动窗口值为0时，主机A会一直等待B的确认，进而形成一个死锁局面。

解决：

1. TCP为每一个连接设有一个持续计时器，只要TCP连接的一方收到对方的零窗口通知，就启动持续计时器。
2. 若持续计时器设置的时间到期，就发送一个零窗口探测报文段。接收方收到探测报文段时给出现在的窗口值。
3. 若窗口仍然是0，那么发送方就重新设置持续计时器。

注意

传输层 & 数据链路层流量控制的区别：传输层定义了端到端用户之间的流量控制，数据链路层定义了两个中间的相邻结点的流量控制。另外，数据链路层的滑动窗口协议的窗口大小不能动态变化，传输层则可以动态变化。

5.3.6 TCP拥塞控制

概述

拥塞控制就是防止过多的数据注入网络中，这样可以使网络中的路由器或链路不致过载。当出现拥塞时，端点并不能了解到拥塞发生的细节，对通信连接的端点来说，拥塞往往表现为通信时延的增加。当然拥塞控制和流量控制也有相似的地方，都是通过控制发送方发送数据的速率来达到效果的。

出现拥塞的条件

本质：对资源需求的总和 > 可用资源

网络中有许多资源同时呈现供应不足 → 网络性能变坏 → 网络吞吐量将随输入负荷增大而下降

拥塞控制 & 流量控制区别

拥塞控制：是让网络能够承受现有的网络负荷，它是一个全局性的过程，涉及所有的主机、所有的路由器，以及与降低网络传输性能有关的所有因素。

流量控制：往往是指点对点的通信量的控制，即接收端控制发送端，它所做的就是抑制发送端发送数据的速率，以便使接收端来得及接收。

拥塞控制算法

为了更好地对传输层进行拥塞控制，因特网建议标准定义了以下**四种算法**：**慢开始、拥塞避免、快重传、快恢复**。

发送方在确定发送报文段的速率时，既要根据接收方的接收能力，又要从全局考虑不要使网络发生拥塞。因此，TCP协议要求发送方维护以下两个窗口：

接收窗口rwnd：接收方根据目前接收缓存大小所许诺的最新的窗口值，反映了接收方的容量。由接收方根据其放在TCP报文的首部的窗口字段通知发送方。

拥塞窗口cwnd：发送方根据自己估算的网络拥塞程度而设置的窗口值，反映了网络的当前容量。只要网络没有出现拥塞，拥塞窗口就再增大一些，以便把更多的分组发送出去。但只要网络出现拥塞，拥塞窗口就减小一些，以减少注入网络中的分组数。

发送窗口的上限值取接收窗口rwnd和拥塞窗口cwnd中较小的一个：**发送窗口上限值 = Min[接收窗口rwnd, 拥塞窗口cwnd]** 发送窗口上限值=Min[接收窗口rwnd, 拥塞窗口cwnd]

慢开始和拥塞避免

1. **慢开始算法：**在 TCP刚刚连接好，开始发送TCP报文段时，先令拥塞窗口cwnd=1，即一个最大报文段长度MSS。而在每收到一个对新的报文段的确认后，将cwnd加1，即增大一个MSS。用这样的方法逐步增大发送方的拥塞窗口cwnd，可以使分组注入到网络的速率更加合理。
2. **拥塞避免算法：**发送端的拥塞窗口cwnd每经过一个往返时延RTT就增加一个MSS的大小，而不是加倍，使cwnd按线性规律缓慢增长（即**加法增大**），而当出现一次超时（网络拥塞）时，则令慢开始门限 ssthresh等于当前cwnd的一半（即**乘法减小**）。

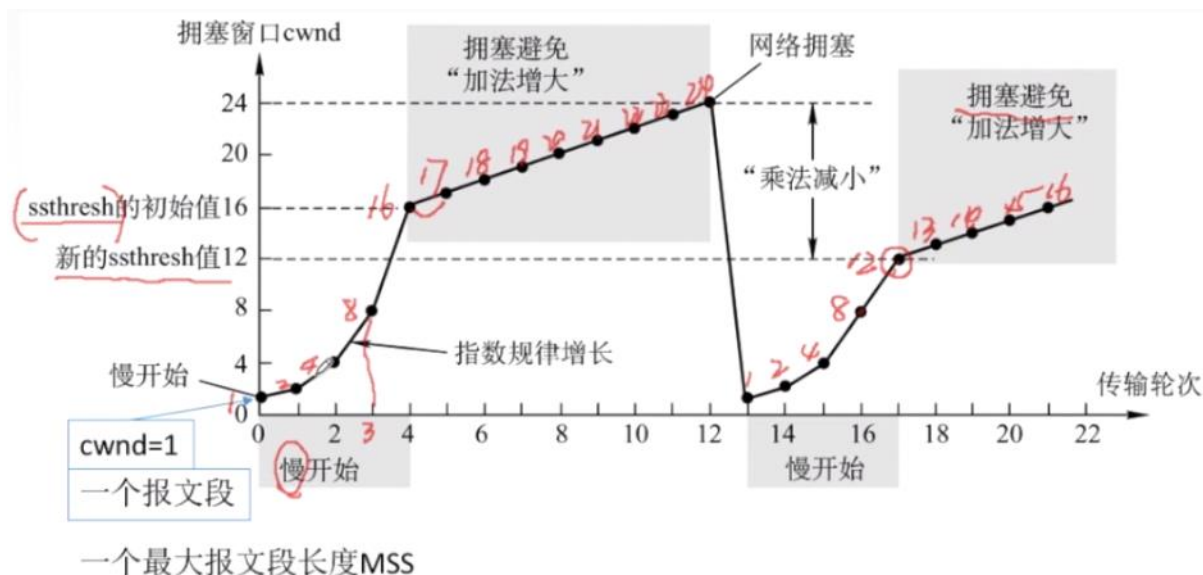
根据 cwnd 的大小执行不同的算法，可归纳如下：

- 当 $cwnd < ssthresh$ 时，使用慢开始算法。
- 当 $cwnd > ssthresh$ 时，停止使用慢开始算法而改用拥塞避免算法。
- 当 $cwnd = ssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法（通常做法）。

3. **网络拥塞的处理：**当网络出现拥塞时，无论在慢开始阶段还是在拥塞避免阶段，只要发送方检测到超时事件的发生（没有按时收到确认，重传计时器超时），就要把慢开始门限ssthresh设置为出现拥塞时的发送方cwnd值的一半（但不能小于2）。然后把拥塞窗口cwnd重新设置为1，执行慢开始算法。

这样做的目的就是要迅速减少主机发送到网络中的分组数，使得发生拥塞的路由器有足够时间把队列中积压的分组处理完毕。

拥塞避免是指在拥塞避免阶段把拥塞窗口控制为按线性规律增长，使网络比较不容易出现拥塞，并非完全能避免拥塞。



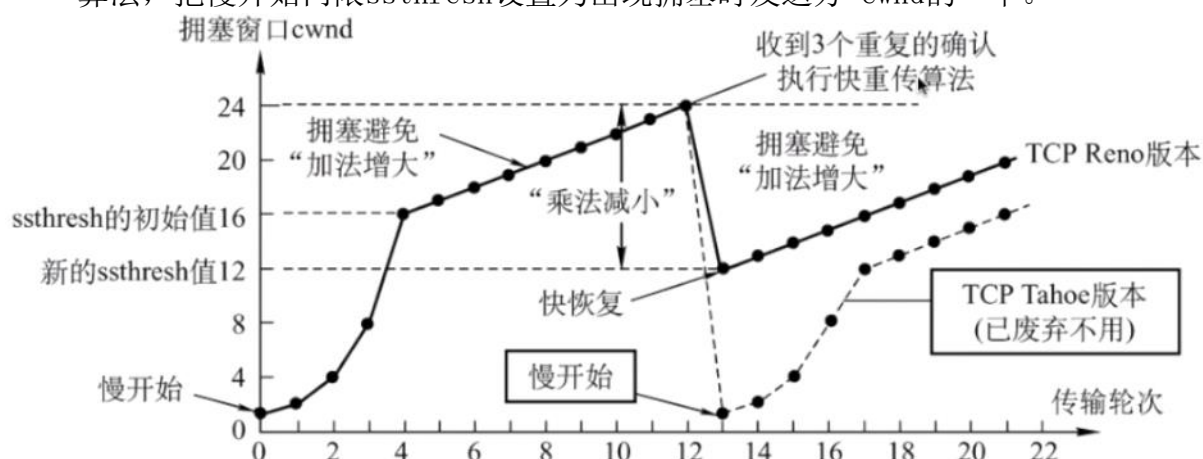
慢开始和拥塞避免算法的实现举例

1. 初始时，拥塞窗口置为1，即 $cwnd=1$ ，慢开始门限置为16，即 $ssthresh=16$ 。慢开始阶段， $cwnd$ 初值为1，以后发送方每收到一个确认ACK， $cwnd$ 值加1，也即经过每个传输轮次(RTT)， $cwnd$ 呈指数规律增长。
2. 当拥塞窗口 $cwnd$ 增长到慢开始门限 $ssthresh$ 时(即当 $cwnd=16$ 时)，就改用拥塞避免算法， $cwnd$ 按线性规律加性增长。
3. 假定 $cwnd=24$ 时，网络发生拥塞，更新 $ssthresh$ 值为12(即变为超时 $cwnd$ 值24的一半)， $cwnd$ 重置1，并执行慢开始算法，当 $cwnd=12$ 时，改为拥塞避免算法。

快重传和快恢复

快重传和快恢复算法是对慢开始和拥塞避免算法的改进。

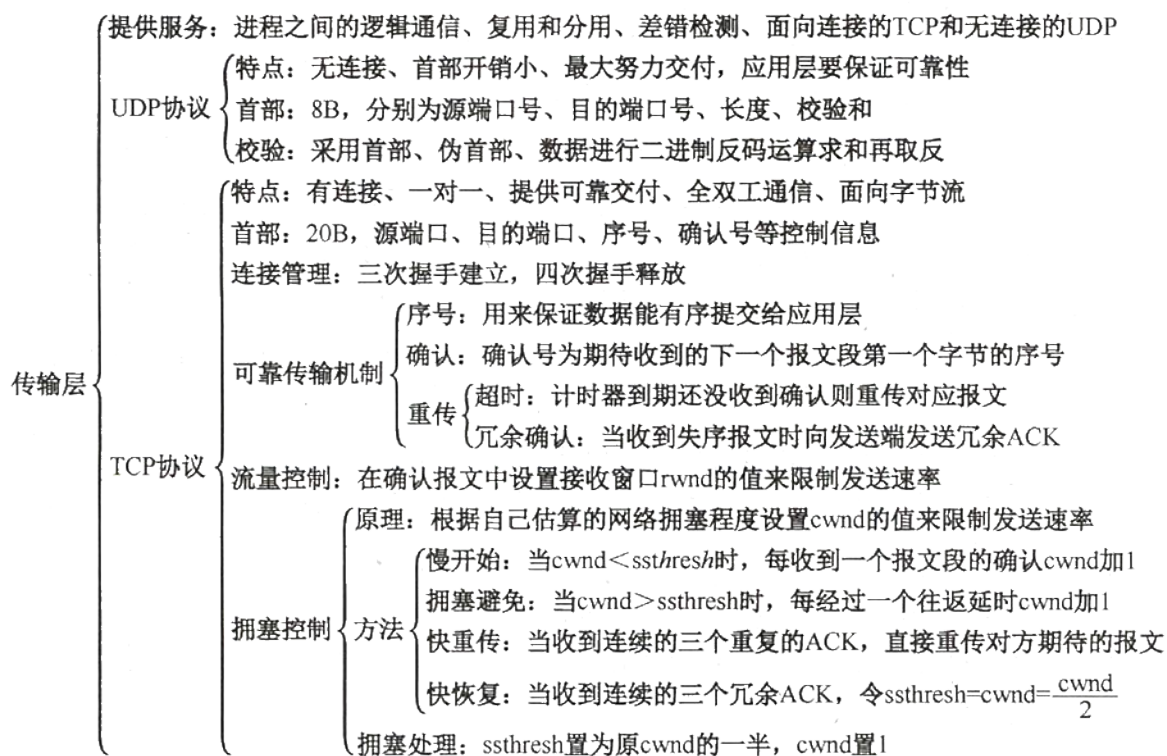
1. **快重传**：当发送方连续收到三个重复的ACK报文时，直接重传对方尚未收到的报文段，而不必等待那个报文段设置的重传计时器超时。
2. **快恢复**：当发送端收到连续三个冗余ACK(即重复确认)时，就执行“乘法减小”算法，把慢开始门限 $ssthresh$ 设置为出现拥塞时发送方 $cwnd$ 的一半。



实际上，慢开始、拥塞避免算法、快重传和快恢复几种算法应该是同时应用在拥塞控制机制之中的，当发送方检测到超时的时候就采用慢开始和拥塞避免，当发送方接收到冗余ACK的时候就采用快重传和快恢复。

5.4 小结

知识架构



1-1 MQTT是什么

星期一, 三月 7, 2022 12:51 下午

已剪辑自: https://www.bilibili.com/read/cv8227034?spm_id_from=333.999.0.0



首页 番剧 直播 游戏中心 会员购 漫画 赛事

咱就说, 咱中国车也可以



专栏 / 科技 / 学习 / 【零基础学物联网】MQTT篇 1-1 MQTT是什么

【零基础学物联网】MQTT篇 1-1 MQTT是什么

学习 2020-11-05 11:32 1454阅读 · 53喜欢 · 4评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注



为了帮助您学习, 我们专门为本教程制作了视频。



【太极创客】零基础入门学用物联网 - MQTT篇 1-1 MQTT是什么

3.5万 71

视频 太极创客团队

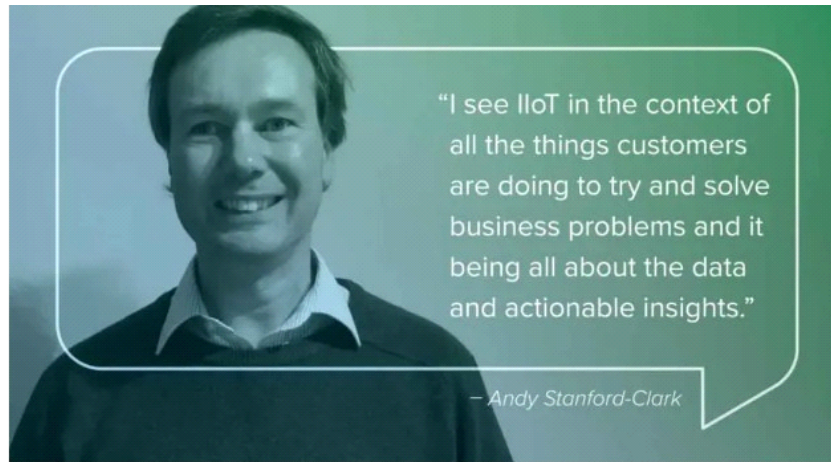
MQTT是一个客户端服务端架构的发布/订阅模式的消息传输协议。它的设计思想是轻巧、开放、简单、规范, 易于实现。这些特点使得它对很多场景来说都是很好的选择, 特别是对于受限的环境如机器与机器的通信 (M2M) 以及物联网环境 (IoT)。

关于MQTT协议是什么这个问题, 该协议的官方规范文档很好地描述了MQTT的全部含义。MQTT协议是一种很轻的通讯协议。与HTTP之类的协议相比, MQTT在通过网络传输数据时表现出众。该协议的另一个重要特点是易于在客户端实现。因此, MQTT成为了当今世界上最受欢迎的物联网协议。它已广泛应用于车联网、智能家居、即时聊天应用和工业互联网等领域。目前通过MQTT协议连接的设备已经过亿, 这些都得益于MQTT协议为设备提供了稳定、可靠、易用的通信基础。

MQTT历史

MQTT协议最初版本是在1999年建立的。该协议的发明人是的Andy Stanford-Clark和Arlen Nipper。





MQTT协议的发明人之一Andy Stanford-Clark



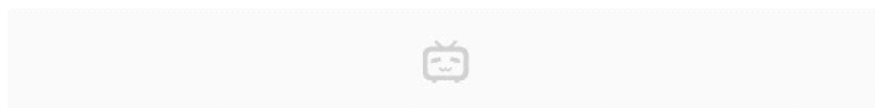
MQTT协议发明人之一Arlen Nipper

他们当时正在开发一个利用卫星通讯监控输油管道的的项目。为了实现这个项目要求，他们需要开发一种用于嵌入式设备的通讯协议，这种通讯协议必须满足以下条件：

- 易于实现
- 数据传输的服务质量可控
- 占用带宽小
- 传输数据内容不可预知
- 设备连接状态可知

从以上几点不难看出，MQTT 从诞生之初就是专为低带宽、高延迟或不可靠的网络而设计的。虽然历经几十年的更新和变化，以上这些特点仍然是MQTT协议的核心特点。但是与最初不同的是，MQTT协议已经从嵌入式系统应用拓展到开放的物联网（IoT）领域。

OASIS标准



OASIS(结构化信息标准促进组织)

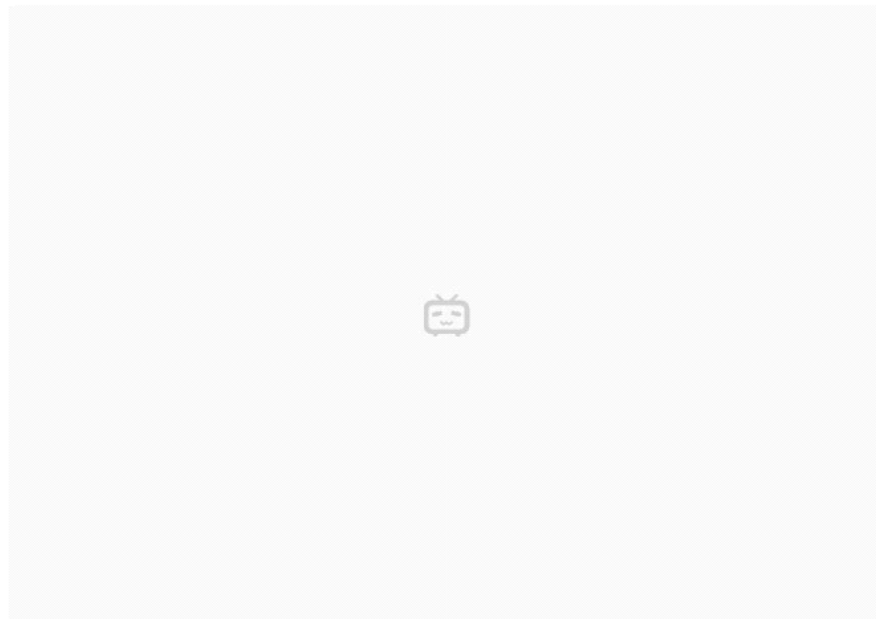
2014年10月29日，MQTT成为OASIS（结构化信息标准促进组织）正式批准的通讯标准。OASIS是一个推进电子商务标准的发展、融合与采纳的非盈利性国际化组织。相比其他组织，OASIS形成了Web服务标准的同时也提出了安全的电子商务标准，同时在针对公众领域和特定应用市场的标准化方面也付出很多的努力。自1993年成立开始，OASIS已经发展成为了由来自100多个国家的600多家组织、企业。简言之，由众多业内专家组成的OASIS愿意为MQTT背书，组件该协议在物联网领域的重要性。

为公共领域内容在世界上许多国家也得到很多的支持。自1999年建立开始，OASIS已经发展成为了由来自100多个国家的600多家组织、企业。简言之，由众多业内专家组成的OASIS愿意为MQTT背书，组件该协议在物联网领域的重要性。

MQTT版本

目前MQTT主流版本有两个，分别是MQTT3.1.1和MQTT5。MQTT3.1.1是在2014年10月发布的，而MQTT5是在2019年3月发布的。由于MQTT3.1.1与MQTT5的时间相差了将近五年，且MQTT5的发布时间距今不久，因此在本文书写时（2020年10月），MQTT3.1.1仍然主流版本。

MQTT5是在MQTT3.1.1的基础上进行了升级。**因此MQTT5是完全兼容MQTT3.1.1的。**而MQTT5是在MQTT3.1.1的基础上添加了更多的功能补充完善MQTT协议。



MQTT5与MQTT3.1.1的关系

本教程将会重点针对**MQTT3.1.1**进行介绍，这么做有几点原因：

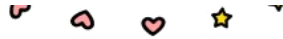
1. MQTT3.1.1是MQTT5的基础，因此掌握了MQTT3.1.1后可以通过进一步学习掌握MQTT5的使用。
2. 当前多种流行编程语言的MQTT客户端库仍然只支持MQTT3.1.1，而不支持MQTT5。
3. 目前物联网环境中的大量设备仍然使用MQTT3.1.1协议而不支持MQTT5协议。

基于以上几点原因，**本教程将以MQTT3.1.1作为重点讲解。**毕竟掌握了MQTT3.1.1，对于学习MQTT5是没有任何浪费的。学会MQTT3.1.1后，如果想要使用MQTT5。那么无需重新学习MQTT5，而**只要利用MQTT3.1.1的基础再学习MQTT5的新功能就可以了。**



本教程为 **太极创客团队** 制作的《零基础入门学用物联网》系列教程第二篇- MQTT篇。假如您学习本教程感到有些吃力，那么您可能还没有观看本系列教程 **第一篇 - 基础知识篇**。您可以通过以下链接前往本教程的第一篇 - 基础知识篇 教程页面。





假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的每一份**点赞与投币**，都是对我们**最温暖的鼓励与支持**。

本文编辑制作：CYNO朔（<https://space.bilibili.com/415252426/>）

本文为我原创 本文禁止转载或摘编

🔗 人工智能 物联网 创客 ESP8266 IoT 太极创客 MQTT 物联网教程 MQTT协议

分享到：

投诉或建议

推荐文章

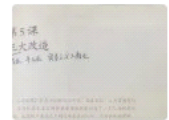
更多精彩内容 >

教师备课教材：八下历史 第5课 三大改造 | 课堂笔记

第5课 三大改造一、农业合作化背景：（1）农产品满足不了国家工业化建设的需要。（2）农民也有进行互助合作的要求。主要...



苏子不胜酒 学习 6 0 1



薪火艺考攻略 | 2022年艺考为今后的艺考生发出了什么讯号

[点击进入查看全文>](#)



薪火艺考 学习 1 0 0



供应链OKR大声吼道:你跑错赛道了!KPI邪恶地看着,笑而不语.

全文5010个字符, 阅读需要8分钟关于OKR和KPI的区别,理论上我就不说了从彼得德鲁克的MBO到OKR变迁,网上一大把,你玩命...



供应链数据治理 学习 0 0 0



评论

全部评论

按时间排序

目录

53

2

24

4



1-2 MQTT基本原理

星期一, 三月 7, 2022 12:55 下午

已剪辑自:

https://www.bilibili.com/read/cv8232098?spm_id_from=333.999.0.0



首页 番剧 直播 游戏中心 会员购 漫画 赛事

梵高也玩原神?



专栏 / 生活 / 日常 / 【零基础学物联网】MQTT篇 1-2 MQTT基本原理

【零基础学物联网】MQTT篇 1-2 MQTT基本原理

日常 2020-11-05 17:15 521阅读 · 44喜欢 · 4评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注

为了帮助您学习，我们专门为本教程制作了视频。

在上节课“MQTT篇 1-1 MQTT是什么”里我们学习了MQTT协议的背景知识和基本特点，这节课我们一起了解MQTT的基本工作原理。

在MQTT协议通讯中，有两个最为重要的角色。它们分别是**服务端**和**客户端**。首先我们来初步了解一下它们。

MQTT服务端：

MQTT服务端通常是一台服务器。它是MQTT信息传输的枢纽，负责将MQTT客户端发送来的信息传递给MQTT客户端。MQTT服务端还负责管理MQTT客户端。确保客户端之间的通讯顺畅，保证MQTT消息得以正确接收和准确投递。

MQTT客户端：有发布订阅功能

MQTT客户端可以向服务端发布信息，也可以从服务端收取信息。我们把客户端发送信息的行为成为“发布”信息。而客户端要想从服务端收取信息，则首先要向服务端“订阅”信息。“订阅”信息这一操作很像我们在视频网站订阅某一部电视剧。当这部电视剧上新后，视频网站会向订阅了该剧的用户发送信息，告诉他们有新剧上线了。

MQTT主题

刚刚我们在讲解MQTT客户端订阅信息时，使用了用户在视频网站订阅电视剧这个例子。在MQTT通讯中，客户端所订阅的肯定不是一部电视剧，而是一个个“主题”。MQTT服务端在管理MQTT信息通讯时，就是使用“主题”来控制的。

为了便于您更好理解服务端是如何通过主题来控制客户端之间的信息通讯，我们来看看下图实例：





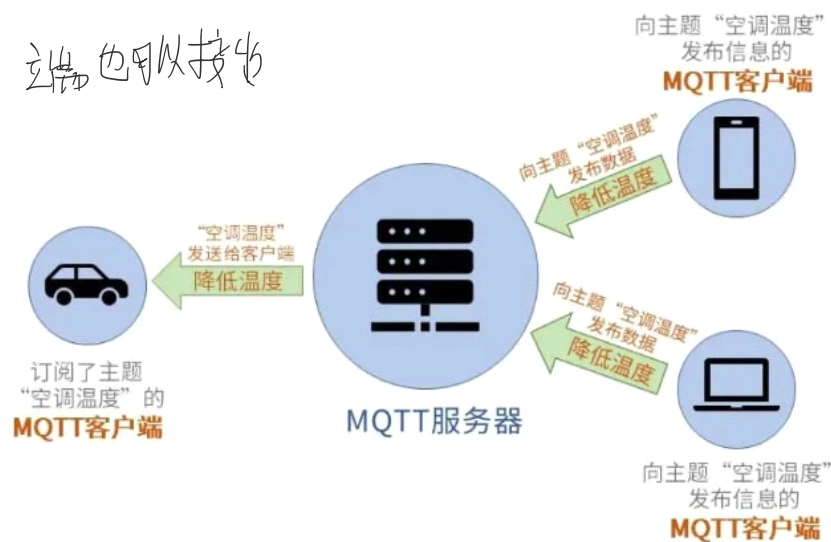
MQTT通讯实例-1

在以上图示中一共有三个MQTT客户端。它们分别是汽车，手机和电脑。MQTT服务端在管理MQTT通讯时使用了“主题”来对信息进行管理的。比如上图所示，假设我们需要利用手机和电脑获取汽车的速度，那么我们先要利用电脑和手机向MQTT服务器订阅主题“汽车速度”。接下来，当汽车客户端向服务端的“汽车速度”主题发布信息后，服务端就会首先检查以下都有哪些客户端订阅了“汽车速度”这一主题的信息。当它发现订阅了该主题的客户端有一个手机和一个电脑，于是服务端就会将刚刚收到的“汽车速度”信息转发给订阅了该主题的手机和电脑客户端。

在以上实例中，汽车是“汽车速度”主题的发布者，而手机和电脑则是该主题的订阅者。

值得注意的是，MQTT客户端在通讯时，往往角色不是单一的。它既可以作为信息发布者也可以同时作为信息订阅者。如下图所示：

发布一端也可以接收



MQTT通讯实例-2

上图中的所有客户端都是围绕“空调温度”这一主题进行通讯的。对于“空调温度”这一主题，手机和电脑客户端成为了MQTT信息的发布者而汽车则成为了MQTT信息的订阅者（接收者）。

可以看到，针对不同的主题，MQTT客户端可以切换自己的角色。它们可能对主题A来说是信息发布者，但是对于主题B就成了信息订阅者。

MQTT 发布/订阅 特性

从以上实例我们可以看到，MQTT通讯的核心枢纽是MQTT服务端。有了服务端对MQTT信息的接收、储存、处理和发送，客户端在发布和订阅信息时，可以相互独立，且在空间上可以分离，时间上可以异步。这里所说的相互独立、空间和时间分离具体指的是什么呢？

- **相互可独立**：MQTT客户端是一个个独立的个体。它们无需了解彼此的存在，依然可以实现信息交流。比如以上实例中汽车客户端在发布“汽车速度”信息时，汽车客户端本身可以完全不知道有多少个MQTT客户端订阅了“汽车速度”这一主题。而订阅了“汽车速度”主题的手机和电脑客户端也完全不知道彼此的存在。大家只要订阅了“汽车速度”主题，MQTT服务端就会在每次收到新信息时，将信息发送给订阅了“汽车速度”主题的客户端。
- **空间可分离**：空间分离相对容易理解，MQTT客户端在通讯比要条件是连接到了同一

度”主题，MQTT服务端就会在每次收到新信息时，将信息发送给订阅了“汽车速度”主题的客户。

- **空间可分离**：空间分离相对容易理解，MQTT客户端在通讯必要条件是连接到了同一个MQTT通讯网络。这个网络可以是互联网或者局域网。只要客户端联网，无论他们远在天边还是近在眼前，都可以实现彼此间的通讯交流。
- **时间可异步**：MQTT客户端在发送和接收信息时无需同步。这一特点对物联网设备尤为重要。有时物联网设备会发生意外离线情况。我们使用以上实例二的场景来作为示例。当我们的汽车在行驶过程中，可能会突然进入隧道，这时汽车可能会断开与MQTT服务端的连接。假设在此时我们的手机客户端向汽车客户端所订阅的“空调温度”主题发布了信息，而汽车恰恰不在线。这时，MQTT服务端可以将“空调温度”主题的新信息保存，待汽车再次上线后，服务端再将“空调温度”信息推送给汽车。

以上几点概括了MQTT通讯时客户端的相互关系以及服务端在其中所起的作用。讲到这里请您注意：以上总结的几个特点中都有一个“可”字。这个“可”字意味着客户端彼此之间可以独立，空间可以分离，时间可以异步。在我们实际应用中，客户端之间的关系既可以独立也可以相互依存。在空间上，既可以相距甚远，也可以彼此相邻。在时间上，既可以异步也可以同步。这个“可”字所体现的是MQTT通讯的灵活性。

可能有些朋友看过以上文字后感觉有些抽象。毕竟这些都是纯理论知识，在后面的教程里，我们将通过实例向您讲解MQTT的应用，届时相信您会对MQTT有深切的认识。到这里，请您务必留意MQTT通讯的三个特点，彼此可独立，空间可分离、时间可异步。



本教程为 太极创客团队 制作的《零基础入门学用物联网》系列教程第二篇- MQTT篇。假如您学习本教程感到有些吃力，那么您可能还没有观看本系列教程 第一篇 - 基础知识篇。您可以通过以下链接前往本教程的第一篇 - 基础知识篇 教程页面。



假如您也喜欢太极创客，希望您将本文“分享”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYNO朔 (<https://space.bilibili.com/415252426/>)

本文禁止转载或摘编

分享到：



投诉或建议

推荐文章

更多精彩内容 >

她宣布退出娱乐圈

▲参加毕业演出的星野南乃木坂46成员星野南（24岁）于2月12日在东京国际会展中心举办毕业演出，并宣布于同日退出娱乐圈...



猪肉熟了

日常

9.1万

448

65



评论

评论

全部评论

按时间排序



44



3



9



4



1-3 连接MQTT服务端 - 哔哩哔哩

星期一, 三月 7, 2022 2:02 下午

已剪辑自: https://www.bilibili.com/read/cv8373124?spm_id_from=333.999.0.0



首页 ▾ 番剧 直播 游戏中心 会员购 漫画 赛事

双雄对决, 谁是胜者?



专栏 / 生活 / 日常 / 【零基础学物联网】MQTT篇 1-3 连接MQTT服务端

【零基础学物联网】MQTT篇 1-3 连接MQTT服务端

日常 2020-11-15 14:24 640阅读 · 50喜欢 · 4评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注



为了帮助您学习, 我们专门为本教程制作了视频。



如需阅读本系列教程完整内容, 欢迎前往本教程配套专栏文章文集



<https://member.bilibili.com/platform/text-read-list?id=337632>



MQTT客户端之间要想实现通讯, 必须要通过MQTT服务端。因此MQTT客户端无论是发布消息还是订阅消息, 首先都要连接MQTT服务端。下面我们看一下MQTT客户端连接服务端的详细过程。

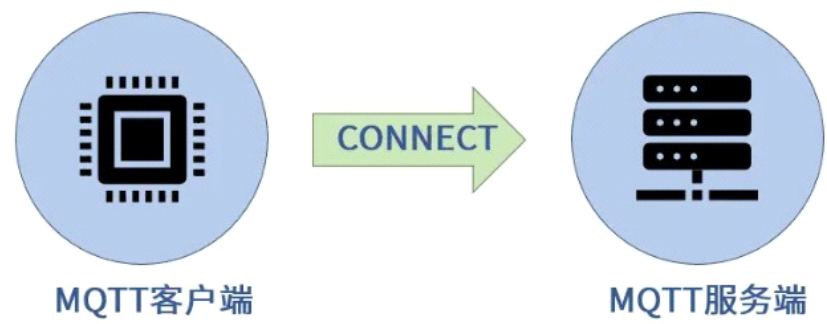
MQTT客户端连接服务端一共有两步。

1 首先MQTT客户端将会向服务端发送连接请求。该请求实际上是一个包含有连接请求信息的数据包。这个数据包的官方名称为CONNECT。

CONNECT

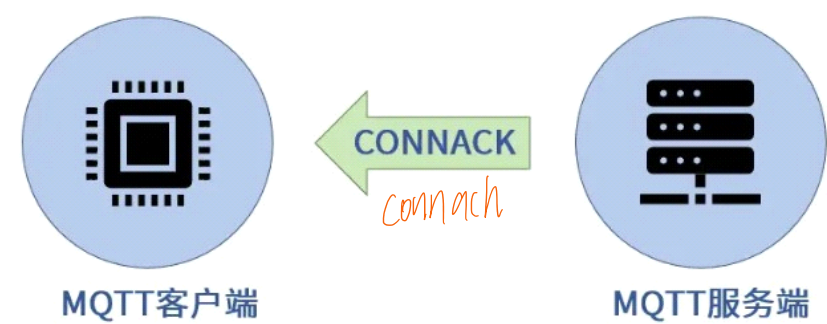
1 客户端向服务端发送连接请求

1 客户端向服务端发送连接请求



2 MQTT服务端收到客户端连接请求后，会向客户端发送连接确认。同样的，该确认也是一个数据包。这个数据包官方名称为CONNACK。

2 服务端向客户端发送连接确认



以上就是MQTT客户端在连接服务端的两步操作。接下来，我们一起来了解一下客户端在连接服务端时所发送的CONNECT报文内容。

CONNECT – 连接服务端

在上面的描述中我们看到。MQTT客户端要想连接服务端，首先要向服务端发送CONNECT报文。如果此CONNECT报文的格式或内容不符合MQTT规范，则服务器会拒绝客户端的连接请求。

下图是CONNECT报文所包含的信息内容。

MQTT-Packet: CONNECT	
名称	内容
clientId	"client-1"
cleanSession	true
username (可选)	"hans"
password (可选)	"letmein"
lastWillTopic (可选)	"/hans/will"
lastWillQos (可选)	2
lastWillMessage (可选)	"unexpected exit"
lastWillRetain (可选)	false
keepAlive	60

MQTT CONNECT 报文内容

在接下来的课程讲解中我们将会频繁接触到两个概念。第一个概念是**报文**，第二个概念是**信息**。

所谓**报文**就是一个MQTT数据包。这个数据包中可能包含有多个信息。比如以上图片就是描绘了一个CONNECT报文（数据包）的详细内容。

在这个CONNECT报文（数据包）中包含有多个信息。上图左侧栏中的内容是CONNECT报文所包含的信息名称。右侧是信息的具体内容。如上图示例中，此CONNECT报文包含有名称为clientId的信息，该信息的内容是"client-1"。当然，上图只是举个例子，不是所有的CONNECT报文中的信息内容都是这样的。

CONNECT报文所包含的信息名称。右侧是信息的具体内容。如上图示例中，此CONNECT报文包含有名称为clientId的信息，该信息的内容是“client-1”。当然，上图只是一个示例，不是所有的CONNECT报文中的clientId信息内容都是“client-1”。

另外也请注意，上图中有些信息名称旁边标注了“可选”字样，而有些则没有。那些没有标注“可选”字样的信息是必须包含在CONNECT报文中的。而对于标注了“可选”字样的信息，CONNECT报文既可以包含它们也可以没有它们。

考虑到我们刚刚开始接触MQTT协议，目前我们先从最基础的内容开始学起。那么在本节课里，**我们只讲解未标注“可选”字样的信息以及它们的功能**。在后续的课程里，我们会详细讲解标有“可选”字样的信息。

clientId – 客户端ID

ClientId是MQTT客户端的标识。MQTT服务端用该标识来识别客户端。因此ClientId必须是独立的。**如果两个MQTT客户端使用相同ClientId标识，服务端会把它们当成同一个客户端来处理**。通常ClientId是由一串字符所构成的，如上图所示，此示例中的clientId是“client-1”。

cleanSession – 清除会话

所谓“清除会话”这一翻译源自MQTT官方文档中文版。要说明cleanSession的具体含义，首先要从MQTT网络环境讲起。

MQTT客户端与服务端的连接可能不是非常稳定，在不稳定的网络环境下，要想保证所有信息传输都能够做到准确无误，这是非常困难的。因此，我们就要根据客户端对系统运行的重要性来区别对待。

有些MQTT客户端对整个系统运行起着关键作用，这些客户端一定要准确无误的收到服务端发来的报文。比如一辆自动驾驶汽车的导航系统。假如这个导航系统错过了服务端发来的报文，可能会导致交通事故甚至人员伤亡。因此，即使网络不是非常稳定，我们仍然要求汽车导航系统一定要准确无误的收到服务端所发来的报文。

但是有些MQTT客户端对整个系统运行并不是很重要。比如同样是这辆自动驾驶汽车。它的音乐播放系统如果没有及时收到服务端发来的音乐播放报文，这对驾驶系统来说影响不大。

以上所举的两个例子说明，MQTT通讯中有些客户端必须准确无误的收到报文，有些则不需要。

为了保证重要的MQTT报文可以被客户端准确无误的收到。在服务端向客户端发送报文后，客户端会向服务端返回一个确认报文。如果服务端没有收到客户端返回的确认报文，那么服务端就会认为刚刚发送给客户端的报文没有被准确无误的送达。在这种情况下，服务端将会执行以下两个操作：

操作1：将尚未被客户端确认的报文保存起来

操作2：再次尝试向客户端发送报文，并且再次等待客户端发来确认信息。

讲到这里就要看看cleanSession的作用了。

✎ 如果cleanSession被设置为“true”。那么服务端不需要客户端确认收到报文，也不会保存任何报文。在这种情况下，即使客户端错过了服务端发来的报文，也没办法让服务端再次发送报文。其实我们从字面上也很容易理解。cleanSession的第一个词是clean。这个词的意思是clean（干净）的。服务端一旦发送完报文，就会把报文忘得“干干净净”了。





反过来，如果我们将**cleanSession** 设置为“false”。那么服务端就知道，后续通讯中，客户端可能会要求我保存没有收到的报文。

从以上的描述不难看出，如果某个客户端用于收发非常重要的信息（比如前文示例中汽车自动驾驶系统），那么该客户端在连接服务端时，应该将**cleanSession**设置为“false”。这样才能让服务端保存那些没有得到客户端接收确认的信息。以便服务端再次尝试将这些重要信息再次发送给客户端。

相反的，如果某个客户端用于收发不重要的信息（比如前文示例中车载音乐系统）那么该客户端在连接服务端时，应该将**cleanSession**设置为“true”。

请注意，如果需要服务端保存重要报文，光设置**cleanSession** 为false是不够的，还需要传递的MQTT信息QoS级别大于0。

关于QoS的概念，我们会在本教程后续课程中详细讲解。到目前请您务必牢记，**如果想让服务器记住重要报文，那么客户端在连接服务端时，需要把**cleanSession**中设置为false。**这一点非常关键，请务必牢记。

KeepAlive – 心跳时间间隔

MQTT服务端运行过程中，当有客户端因为某种原因断开了与服务端的连接，服务端需要实时了解这一情况。**KeepAlive**（心跳时间间隔）正是用于服务端了解客户端连接情况的。不过关于**KeepAlive**（心跳时间间隔）目前讲解还为时过早，我们会在后续的课程中给您做详细介绍。目前您只需要记住，**KeepAlive**用于服务端实时了解客户端是否与其保持连接的情况。 *定时向服务器发送心跳信息*

以上就是CONNECT报文的主要内容。关于CONNECT报文中的其它内容，我们会在接下来的课程里给大家讲解。下面我们再看看MQTT服务端接收到客户端发来的连接请求后所回复的CONNACK报文详细内容。

CONNACK – 确认连接请求

下图是CONNACK报文所包含的信息内容。

MQTT-Packet:	
CONNACK	
名称	内容
sessionPresent	true
<u>returnCode</u>	0

MQTT CONNACK 报文内容

CONNACK报文包括两个信息。一个是returnCode(连接返回码)，另一个是sessionPresent (当前会话)。以下是这两个信息的说明：

returnCode – 连接返回码

当服务端收到了客户端的连接请求后，会向客户端发送returnCode(连接返回码)，用以说明连接情况。如果客户端与服务端成功连接，则返回数字“0”。如果未能成功连接，连接返回码将会是一个非零的数值，具体这个数值的含义，请见下列说明：

明连接情况。如果客户端与服务端成功连接，则返回数字“0”。如果未能成功连接，连接返回码将会是一个非零的数值，具体这个数值的含义，请见下列说明：

0 - 成功连接

1 - 连接被服务端拒绝，原因是不支持客户端的MQTT协议版本

2 - 连接被服务端拒绝，原因是不支持客户端标识符的编码。

3 - 连接被服务端拒绝，原因是服务端不可用。

4 - 连接被服务端拒绝，原因是用户名或密码无效。

5 - 连接被服务端拒绝，原因是客户端未被授权连接到此服务端

sessionPresent – 当前会话

要说明sessionPresent，首先我们要回顾一下CONNECT报文中的cleanSession – 清除会话。

我们还用自动驾驶汽车为例。对于自动驾驶汽车来说，自动导航系统属于非常重要的MQTT客户端。服务端发送给导航系统的报文必须要准确无误的送达。相反，音乐播放系统就不那么重要了。即使音乐播放系统错过服务端发送的报文也没有关系。

对于不重要的MQTT客户端，它们在向服务器发送连接请求时，CONNECT报文中的cleanSession通常设置为true。原因是这类不重要的MQTT客户端即使丢失信息也不会影响整体系统运行。因此服务端在看到客户端的cleanSession为true时，就不会保存发送给它们的信息。

然而对于汽车导航系统这类重要的MQTT客户端来说。当它在连接服务端时，cleanSession肯定时设置为false。原因是重要客户端需要服务端确保信息发送准确无误。如果服务端发现发送给重要客户端的信息没有得到确认，会将报文进行保存。

当重要客户端连接服务端时，服务端可能保存着没有得到确认的报文。如果是这样的话，那么客户端在连接服务端时，就会通过sessionPresent来了解服务端是否有之前未能确认的信息。

下面我们分几种情况来讲述sessionPresent的作用。

首先，当客户端发送的CONNECT报文中的cleanSession设置为true。在这种情况下，客户端是不需要服务端保存任何报文的。那么服务端发送的确认连接CONNACK报文中，sessionPresent肯定是false，也就是说，服务端没有保存任何报文。

当客户端发送的CONNECT报文中的cleanSession设置为false时，客户端是要求服务端保存报文的。在这种情况下，如果服务端的确保存了没有收到客户端接收确认的报文信息，那么cleanSession为true，否则为false。

简言之，CONNACK报文的sessionPresent与CONNECT报文的cleanSession相互配合。其作用是客户端发送连接请求时，服务端告知客户端有没有保存报文信息。这个被服务端保存的报文信息是来自于上一次客户端连接时，服务端曾经发送此报文给客户端，但是发送后没有收到客户端接收确认。



本教程为 **太极创客团队** 制作的《零基础入门学物联网》系列教程第二篇- MQTT篇。假如您学习本教程感到有些吃力，那么您可能还没有观看本系列教程 第一篇 - 基础知识篇。您可以通过以下链接前往本教程的第一篇 - 基础知识篇 教程页面。



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYNO朔（<https://space.bilibili.com/415252426/>）

每一件**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYNO朔（<https://space.bilibili.com/415252426/>）

本文为我原创 本文禁止转载或摘编

🔗 DIY 物联网 创客 ESP8266 Arduino 网络知识 IoT 太极创客 MQTT

分享到：    

投诉或建议

推荐文章

更多精彩内容 >

【每日观优】桃乃木香奈（桃乃木かな）



枫花恋-fg

日常

👁 1352

👍 67

💬 5



评论

全部评论

按时间排序

📖
目录

👍
50

🚫
0

★
9

💬
4

^

1-4 MQTT服务端连接操作 - 哔哩哔哩

星期一, 三月 7, 2022 2:02 下午

已剪辑自: https://www.bilibili.com/read/cv8399263?spm_id_from=333.999.0.0



首页 ▾ 番剧 直播 游戏中心 会员购 漫画 赛事

咱就说, 咱中国车也可以



专栏 / 生活 / 日常 / 【零基础学物联网】MQTT篇 1-4 MQTT服务端连接操作

【零基础学物联网】MQTT篇 1-4 MQTT服务端连接操作

日常 2020-11-17 11:42 514阅读 · 42喜欢 · 2评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注



为了帮助您学习, 我们专门为本教程制作了视频。



零基础入门学用物联网
MQTT篇
1-4 服务端连接操作
太极创客 32:12

【太极创客】零基础入门学用物联网 - MQTT篇 1-4 服务端连接操作

1.6万 46

视频 太极创客团队

如需阅读本系列教程完整内容, 欢迎前往本教程配套专栏文章文集



<https://member.bilibili.com/platform/text-read-list?id=337632>



到目前为止, 我们已经学习了MQTT客户端连接服务端的基本原理。光有理论知识还不够, 在本节课程里我们将一起学习如何使用电脑客户端和ESP8266客户端来连接MQTT服务端。

电脑客户端连接MQTT服务端

首先我们一起来学习如何使用电脑软件来实现客户端与服务端的连接。我们将为电脑安装一款免费的MQTT客户端软件—MQTTfx。MQTT客户端软件有很多款, 不过经过我们的使用和比较, 发现免费的MQTTfx软件在易用性和功能性都非常出色。

通过MQTTfx软件, 我们的电脑将会成为MQTT客户端, 你可以使用以下命令验证

装一款免费的MQTT客户端软件—MQTTfx。MQTT客户端软件有很多款，不过经过我们的使用和比较，发现免费的MQTTfx软件在易用性和功能性都非常出色。

通过MQTTfx软件，我们的电脑将会成为一台MQTT客户端。您可以通过以下两个途径下载到MQTTfx软件。

1183

首先，您可以通过MQTT官网下载到MQTTfx软件。MQTTfx的官网地址是：
<http://mqttfx.org>

由于网络连接问题，有些朋友可能无法打开此网址。如果是这样的话，您也可以通过我们太极创客网站的下载页面获取此软件。

特别提示一点，建议您尽量使用以上的MQTTfx官网下载此软件，因为官网上的软件版本肯定是最新的。

接下来我们就来实际使用MQTTfx客户端来尝试连接服务端。具体操作方法，请见本文首部嵌入的视频教程。

ESP8266连接MQTT服务端

ESP8266的Arduino开发环境里有多款MQTT库，我们将使用最为流行的PubSubClient库来作为本教程的主要介绍对象，我们将通过一系列课程教您如何使用ESP8266通过PubSubClient库来实现MQTT物联网应用。关于PubSubClient库，以下是其基本信息：

作者名：Nick O'Leary

官网地址：<https://pubsubclient.knolleary.net/>

GitHub：<https://github.com/knolleary/pubsubclient/>

百度网盘下载：<https://pan.baidu.com/s/12MHGbdFiOdwOGip5RMSSEQ> 提取码: sizz

当您把PubSubClient库安装后，我们就可以使用该库进行开发了。

关于ESP8266连接MQTT服务端的示例程序。您可以通过太极创客官网免费获取。以下是该程序所在的页面链接。

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-tutorial/mqtt-tutorial/client-connect-mqtt-server/>



本教程为太极创客团队制作的《零基础入门学物联网》系列教程第二篇- MQTT篇。假如您学习本教程感到有些吃力，那么您可能还没有观看本系列教程 第一篇 - 基础知识篇。您可以通过以下链接前往本教程的第一篇 - 基础知识篇 教程页面。



假如您也喜欢太极创客，希望您将本文“分享”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYNO朔 (<https://space.bilibili.com/415252426/>)

本文为我原创 本文禁止转载或摘编

🔗 编程 物联网 创客 ESP8266 太极创客 MQTT 物联网应用 物联网教程 MQTT协议 物联网协议

分享到：

投诉或建议



推荐文章

更多精彩内容 >

支持远程管理的智能灌溉系统解决方案

点击进入查看全文>

聚英电子 日常 2 0 0



小型气象站对精准农业有什么用

点击进入查看全文>

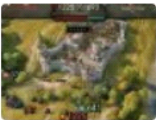
聚英电子 日常 1 0 0



骑马与砍杀2、帝国兵种用法与实用指挥战术、最详细，第一篇

在骑马与砍杀2当中，各个国家有不一样的特色兵种，他们的外表风格，武器，护甲都展现不同文化的魅力。在打仗能力方面，...

江诗贤Sailing22k 日常 1052 33 19



评论

全部评论 按时间排序



1-5 发布、订阅和取消订阅 - 哔哩哔哩

星期一, 三月 7, 2022 2:04 下午

已剪辑自: https://www.bilibili.com/read/cv8465978?spm_id_from=333.999.0.0



首页 番剧 直播 游戏中心 会员购 漫画 赛事

咱就说, 咱中国车也可以



专栏 / 科技 / 学习 / 【零基础学物联网】MQTT篇 1-5 发布、订阅和取消订阅

【零基础学物联网】MQTT篇 1-5 发布、订阅和取消订阅

学习 2020-11-22 10:00 911阅读 · 49喜欢 · 0评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注



为了帮助您学习, 我们专门为本教程制作了视频。



零基础入门学用物联网 MQTT篇 1-5 发布、订阅和取消订阅 太极创客 20:50	【太极创客】零基础入门学用物联网 - MQTT篇 1-5 发布订阅和取消订阅 1万 15 太极创客团队
---	--

在之前的课程里, 我们学习了如何使用MQTT客户端连接MQTT服务端。在这节课里, 我们学习客户端如何实现发布消息、订阅主题以及取消订阅主题。在本节课里我们将重点讲解以下MQTT报文:

- * PUBLISH – 发布信息
- * SUBSCRIBE – 订阅主题
- * SUBACK – 订阅确认
- * UNSUBSCRIBE – 取消订阅

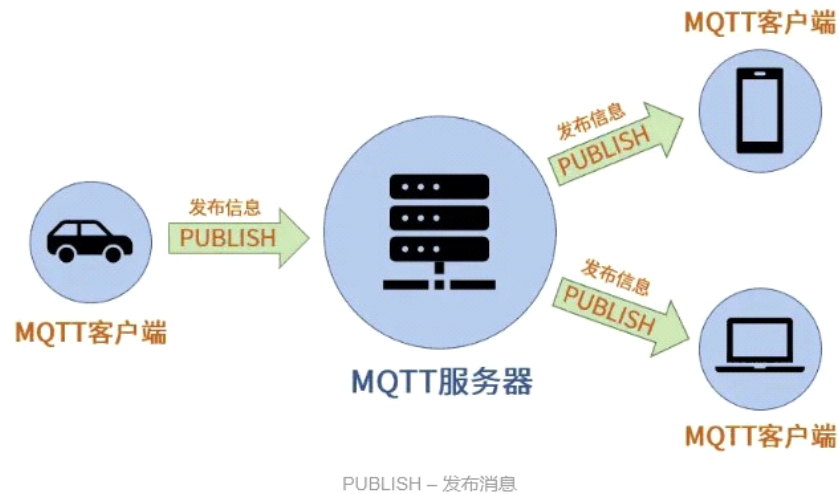
PUBLISH – 发布消息

(持久)

MQTT客户端一旦连接到服务端, 便可以发布消息。每条发布的MQTT消息必须包含一个主题。MQTT服务器可以通过主题确定将消息转发给哪些客户端。(注: 这里的消息指的是MQTT报文。)

MQTT客户端

指的是MQTT报文。)



MQTT客户端发布消息时，会向服务端发送PUBLISH报文。以下是PUBLISH报文的详细信息。

packetId	474	
topicName	"topic1"	
qos	1	
retainFlag	false	
payload	"temperature (22)"	具体信息内容
dupFlag	false	

MQTT PUBLISH 报文

上图左侧栏中的内容是PUBLISH报文所包含的信息名称。右侧是信息的具体内容。

topicName – 主题名

主题名用于识别此信息应发布到哪一个主题。关于MQTT主题的应用，我们在之前的课程中已经做了详细介绍，在后续课程中我们还会对主题的高级应用进行更加详细的讲解。

QoS – 服务质量等级

QoS (Quality of Service) 表示MQTT消息的服务质量等级。QoS有三个级别：0、1和2。QoS决定MQTT通讯有什么样的服务保证。有关QoS的详细信息我们会在后续课程中详细讲解。

packetId – 报文标识符

报文标识符可用于对MQTT报文进行标识。不同的MQTT报文所拥有的标识符不同。MQTT设备可以通过该标识符对MQTT报文进行甄别和管理。**请注意：报文标识符的内容与QoS级别有密不可分的关系。只有QoS级别大于0时，报文标识符才是非零数值。如果QoS等于0，报文标识符为0。**

retainFlag – 保留标志

在默认情况下，当客户端订阅了某一主题后，并不会马上接收到该主题的信息。只有在客户端订阅该主题后，服务端接收到该主题的新信息时，服务端才会将最新接收到的该主题信息推送给客户端。

但是在有些情况下，我们需要客户端在订阅了某一主题后马上接收到一条该主题的信息。这时候就需要用到保留标志这一信息。关于保留标志的具体使用方法，我们将在本教程的后续部分进行详细讲解。

但在任何情况下，我们需要客户端订阅主题。一旦主题订阅成功，客户端将接收该主题的消息。这时候就需要用到**保留标志**这一信息。关于**保留标志**的具体使用方法，我们将在本教程的后续部分进行详细讲解。

true 服务端会把它保存起来有人订阅立即发送

Payload – 有效载荷

有效载荷是我们希望通过MQTT所发送的实际内容。我们可以使用MQTT协议发送文本，图像等格式的内容。这些内容都是通过有效载荷所发送的。

dupFlag – 重发标志

当MQTT报文的接收方没有及时确认收到报文时，发送方会重复发送MQTT报文。在重复发送MQTT报文时，发送方会将此“重发标志”设置为true。请注意，重发标志只在QoS级别大于0时使用。有关QoS的详细信息，我们将会在后续教程中为您做详细介绍。

SUBSCRIBE – 订阅主题

当客户端连接到服务端后，除了可以发布消息，也可以接收消息。我们在之前的课程讲过，所有MQTT消息都有主题。客户端要想接收消息，首先要订阅该主题的消息。这样，当有客户端向该主题发布消息后，订阅了该主题的客户端就能接收到消息了。

客户端要想订阅主题，首先要向服务端发送主题订阅请求。客户端是通过向服务端发送SUBSCRIBE报文来实现这一请求的。该报文包含有一系列“**订阅主题名**”。**请注意，一个SUBSCRIBE报文可以包含有单个或者多个订阅主题名。**也就是说，一个SUBSCRIBE报文可以用于订阅一个或者多个主题。

在以上PUBLISH报文讲解中，我们曾经提到过QoS（服务质量等级）这一概念。同样的，客户端在订阅主题时也可以明确QoS。服务端会根据SUBSCRIBE中的QoS来提供相应的**服务保证**。

另外每一个SUBSCRIBE报文还包含有“**报文标识符**”。报文标识符可用于对MQTT报文进行标识。不同的MQTT报文所拥有的标识符不同。MQTT设备可以通过该标识符对MQTT报文进行甄别和管理。

SUBACK – 订阅确认

服务端接收到客户端的订阅报文后，会向客户端发送SUBACK报文确认订阅。

SUBACK报文包含有“**订阅返回码**”和“**报文标识符**”这两个信息。

returnCode – 订阅返回码

客户端向服务端发送订阅请求后，服务端会给客户端返回一个订阅返回码。

在之前的讲解中我们说过，客户端可通过一个SUBSCRIBE报文发送多个主题的订阅请求。服务端会针对SUBSCRIBE报文中的所有订阅主题来逐一回复给客户端一个返回码。

这个返回码的作用是告知客户端是否成功订阅了主题。以下是返回码的详细说明。

0 - 订阅成功 - QoS 0

1 - 订阅成功 - QoS 1

2 - 订阅成功 - QoS 2

128 - 订阅失败

看这个客户端订阅质量是

请注意，如上表所示，针对不同的主题订阅QoS，服务端的返回码会有所不同。

另外每一个SUBACK报文也包含有“**报文标识符**”。MQTT设备可以通过该标识符对报文进行管理。

UNSUBSCRIBE – 取消订阅

顾名思义，当客户端要取消订阅某主题时，可通过向服务端发送UNSUBSCRIBE – 取消订阅报文来实现。

以上三章内容，UNSUBSCRIBE报文包含两个重要信息：一个是取消订阅的主题名

顾名思义，当客户端要取消订阅某主题时，可通过向服务端发送UNSUBSCRIBE – 取消订阅报文来实现。

以上示意图显示，UNSUBSCRIBE报文包含两个重要信息，第一个是取消订阅的主题名称。同一个UNSUBSCRIBE报文可以同时包含多个取消订阅的主题名称。另外，UNSUBSCRIBE报文也包含“报文标识符”，MQTT设备可以通过该标识符对报文进行管理。

当服务端接收到UNSUBSCRIBE报文后，会向客户端发送取消订阅确认报文 – UNSUBACK报文。该报文含有客户端所发送的“取消订阅报文标识符”。

客户端接收到UNSUBACK报文后就可以确认取消主题订阅已经成功完成了。



本教程为 太极创客团队 制作的《零基础入门学物联网》系列教程第二篇- MQTT篇。假如您学习本教程感到有些吃力，那么您可能还没有观看本系列教程 第一篇 - 基础知识篇。您可以通过以下链接前往本教程的第一篇 - 基础知识篇 教程页面。



假如您也喜欢**太极创客**，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的**鼓励与支持**。

本文编辑制作：CYNO朔 (<https://space.bilibili.com/415252426/>)

本文为我原创 本文禁止转载或摘编

🔗 嵌入式 物联网 创客 ESP8266 网络知识 IoT 太极创客 MQTT 物联网教程 MQTT协议

分享到：

[投诉或建议](#)

推荐文章

[更多精彩内容 >](#)

Windows环境下NCL软件的安装方法

感谢感谢知乎用户 @一米粒 同学帮助我找出了关于WSL的问题，我开始忽略了图形化界面对WSL和Windows版本的新要求...



刘芦苇杆子

学习

👁 49

👍 1

💬 1



机械工程师怎么才能赶上工业 4.0、工业自动化等浪潮？

关于制造业的未来发展方向，现在有几个提法，我们可以大致的了解一下：中国的定义：中国制造2025德国的定义：工业4.0美...



华维单片机速学

学习

👁 59

👍 1

💬 0



数学中“神出鬼没”的策略问题探究（1）

作者：zc（第14届中学生数理化竞赛一等奖）我相信大家从小到大肯定玩过策略游戏吧，策略游戏中涉及到逻辑推理，计算等方...



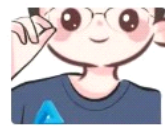
子楚工作室

学习

👁 14

👍 1

💬 0



评论



目录

评论

全部评论

按时间排序



目录



49



3



16



0



1-6 MQTT服务端连接操作 - 哔哩哔哩

星期一, 三月 7, 2022 11:21 下午

已剪辑自: https://www.bilibili.com/read/cv8487184?spm_id_from=333.999.0.0



首页 番剧 直播 游戏中心 会员购 漫画 赛事

真正的打脸现场



专栏 / 科技 / 学习 / 【零基础学物联网】MQTT篇 1-6 MQTT服务端连接操作

【零基础学物联网】MQTT篇 1-6 MQTT服务端连接操作

学习 2020-11-23 10:44 363阅读 · 46喜欢 · 1评论



太极创客团队

粉丝: 8.6万 文章: 29

已关注



为了帮助您学习，我们专门为本教程制作了视频。



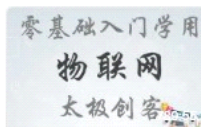
【太极创客】零基础入门学物联网 - MQTT篇 1-6 ESP8266发布MQTT消息

1万 24

视频 太极创客团队

到目前为止，我们已经学习了发布和订阅MQTT消息的基本知识，这节课我们来将目前所学的知识应用到实际操作中。我们将要利用ESP8266来实现最基本的MQTT消息发布。

本节课的学习将需要您掌握ESP8266的基本操作，假如您对ESP8266的基本操作不太了解，您可以先完成《零基础入门学物联网》系列教程的第一部分“基础知识篇”。



【ESP8266教程】零基础入门学物联网-基础知识篇（太极创客团队）

41.2万 979

视频 太极创客团队

假如您已经完成以上教程，那么接下来您就可以继续这节课的学习了。

ESP8266的Arduino开发环境里有多多个MQTT库，我们将使用最为流行的PubSubClient库来作为本教程的主要介绍对象，我们将通过一系列课程教您如何使用ESP8266通过PubSubClient库来实现MQTT物联网应用。关于PubSubClient库,以下是其基本信息：

分区 新分区 1 的第 177 页

来作为本教程的主要介绍对象，我们将通过一系列课程教您如何使用ESP8266通过PubSubClient库来实现MQTT物联网应用。关于PubSubClient库,以下是其基本信息：

作者名：Nick O’Leary

官网地址：<https://pubsubclient.knolleary.net/>

GitHub：<https://github.com/knolleary/pubsubclient/>

百度网盘下载：<https://pan.baidu.com/s/12MHGbdfiOdwOGip5RMSSEQ> 提取码: sizz

当您把PubSubClient库安装后，我们就可以使用该库进行开发了。

发布MQTT消息示例程序

通过以下示例程序，我们将可以让ESP8266向公用MQTT服务器发布消息。如果您需要了解此程序的详细说明，请点击本文首部的视频。

/******

项目名称/Project：零基础入门学物联网

程序名称/Program name：publish_ranye_url

团队/Team：太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author：CYNO朔

日期/Date (YYYYMMDD)：20200813

程序目的/Purpose：

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器发布信息。

本示例程序为太极创客团队制作的《零基础入门学物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

#include <ESP8266WiFi.h>

#include <PubSubClient.h>

#include <Ticker.h>

// 设置wifi接入信息(请根据您的WiFi信息进行修改)

const char* ssid = "taichi-maker";

const char* password = "12345678";

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案

// <http://www.taichi-maker.com/public-mqtt-broker/>

Ticker ticker;

WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);

int count; // Ticker计数由变量

```

int count; // Ticker计数用变量

void setup() {
    Serial.begin(9600);

    //设置ESP8266工作模式为无线终端模式
    WiFi.mode(WIFI_STA);

    // 连接WiFi
    connectWifi();

    // 设置MQTT服务器和端口号
    mqttClient.setServer(mqttServer, 1883);

    // 连接MQTT服务器
    connectMQTTServer();

    // Ticker定时对象
    ticker.attach(1, tickerCount);
}

void loop() {
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
        // 每隔3秒钟发布一次信息
        if (count >= 3){
            pubMQTTmsg();
            count = 0;
        }

        // 保持心跳
        mqttClient.loop();
    } else { // 如果开发板未能成功连接服务器
        connectMQTTServer(); // 则尝试连接服务器
    }
}

void tickerCount(){
    count++;
}

```



```

    count++;
}

void connectMQTTServer(){
    // 根据ESP8266的MAC地址生成客户端ID (避免与其它ESP8266的客户端ID重名)
    String clientId = "esp8266-" + WiFi.macAddress();
    // mac地址

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address: ");
        Serial.println(mqttServer);
        Serial.println("ClientId:");
        Serial.println(clientId);
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(3000);
    }
}

// 发布信息
void pubMQTTmsg(){
    static int value; // 客户端发布信息用数字

    // 建立发布主题。主题名称以Taichi-Maker-为前缀，后面添加设备的MAC地址。
    // 这么做是为确保不同用户进行MQTT信息发布时，ESP8266客户端名称各不相同，
    String topicString = "Taichi-Maker-Pub-" + WiFi.macAddress();
    char publishTopic[topicString.length() + 1];
    strcpy(publishTopic, topicString.c_str());

    // 建立发布信息。信息内容以Hello World为起始，后面添加发布次数。
    String messageString = "Hello World " + String(value++);
    char publishMsg[messageString.length() + 1];
    strcpy(publishMsg, messageString.c_str());

    // 实现ESP8266向主题发布信息
    if(mqttClient.publish(publishTopic, publishMsg)){
        Serial.println("Publish Topic:");Serial.println(publishTopic);
        Serial.println("Publish message:");Serial.println(publishMsg);
    }
}

```

```
Serial.println("Publish topic:");Serial.println(publishTopic);

Serial.println("Publish message:");Serial.println(publishMsg);

} else {

Serial.println("Message Publish Failed.");

}

}
```

```
// ESP8266连接wifi
```

```
void connectWifi(){
```

```
WiFi.begin(ssid, password);
```

```
//等待WiFi连接,成功连接后输出成功信息
```

```
while (WiFi.status() != WL_CONNECTED) {
```

```
delay(1000);
```

```
Serial.print(".");
```

```
}
```

```
Serial.println("");
```

```
Serial.println("WiFi Connected!");
```

```
Serial.println("");
```

```
}
```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的**鼓励与支持**。

本文编辑制作：CYNO朔（<https://space.bilibili.com/415252426/>）

本文为我原创 本文禁止转载或摘编

📁 科技 物联网 创客 ESP8266 网络知识 IoT 太极创客 MQTT 物联网教程 MQTT协议

分享到：👍👤👥🌟👤

投诉或建议

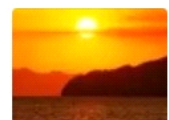
推荐文章

更多精彩内容 >

今日教员语录—赤辰社【3月份系列】

—

👤 北-不羁之民 学习 👁 66 👍 18 💬 0



圆周率 π^2 是無理數？



圓周率 π^2 是無理數？

20世紀上半葉，「一萬零一元」(Ivan Niven) 給出一個大一微積分能懂的方式證明：假使，令。我們要利用 來製造矛盾。當時...



數學老師張旭

学习

132

18

3



趣味数学：比大小 $\text{Log}[5, 6]$ vs $\text{Log}[6, 7]$

—



Mathhouse

学习

95

11

4



评论

全部评论

按时间排序

目录

46

2

5

1



1-7 MQTT主题进阶 - 哔哩哔哩

星期一, 三月 7, 2022 11:22 下午

已剪辑自: https://www.bilibili.com/read/cv8518346?spm_id_from=333.999.0.0



首页 番剧 直播 游戏中心 会员购 漫画 赛事

登高也玩原神?



专栏 / 生活 / 日常 / 【零基础学物联网】MQTT篇 1-7 MQTT主题进阶

【零基础学物联网】MQTT篇 1-7 MQTT主题进阶

日常 2020-11-25 10:03 279阅读 · 38喜欢 · 0评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注



为了帮助您学习，我们专门为本教程制作了视频。



【太极创客】零基础入门学用物联网 - MQTT篇 1-7 MQTT主题进阶

6433 14

视频 太极创客团队

通过以前的学习，我们了解到**主题**是MQTT的核心概念之一。在本节课里，我们来一起进一步了解MQTT主题这一概念。我们将从以下几个方面来学习MQTT主题的高级知识：

1. 主题基本形式
2. 主题分级
3. 主题通配符
4. 主题应用注意事项

1. 主题基本形式

主题的最基本形式就是一个字符串。以下是几个主题示例：

- myTopic
- motorSpeed
- MotorSpeed
- current time

- motorSpeed
- MotorSpeed
- current time

虽然主题看起来很简单，但是我们在使用主题时需要注意以下几点：

1. **主题是区分大小写的。**如上列表中的主题 motor_speed和Motor_speed是两个完全不同的主题。
2. **主题可以使用空格** 如以上列表中的current time，虽然有空格分隔current和time这两个词，但这实际是一个MQTT主题。不过，虽然我们可以使用空格，但是笔者强烈建议您**不要在主题中使用空格**。我们在开发时一不小心，可能就会漏掉空格，这将造成不必要的麻烦。
3. 大部分MQTT服务端是不支持中文主题的，所以我们应使用英文字符或ASCII字符来作为MQTT主题。

2. 主题分级

MQTT主题可以是一个简单的字符串，比如motor_speed，myTopic。这些都是单一级别的主题。

为了更好的对主题进行管理和分类，我们可以对主题进行分级处理。MQTT主题各个级别之间可以使用“/”来分隔。如下例所示：

Tyler-1/motor/1/speed

在以上示例中一共有四级主题，分别是第1级 Tyler-1、第2级motor、第三级1、第4级speed。主题的每一级至少需要一个字符，比如以上示例中，数字1既是一级主题。

我们再来看几个分级主题的示例：

```
home/sensor/kitchen/temperature
home/sensor/kitchen/brightness
home/sensor/bedroom/temperature
home/sensor/bedroom/brightness
```

3 主题通配符

当客户端订阅主题时，可以使用通配符同时订阅多个主题。通配符只能在订阅主题时使用，下面我们将介绍两种通配符：单级通配符和多级通配符。

单级通配符: +

顾名思义，单级通配符可以代替一个主题级别。以下为含有单级通配符的主题示例。

home/sensor+/temperature

当客户端订阅了以上主题后，它将会收到以下主题的信息内容：

```
home/sensor/kitchen/temperature
home/sensor/bedroom/temperature
```

我们可以看到，在home后面的级别中，由于客户端订阅的主题使用了+ 单级通配符，因此无论home级别后面的内容是什么，客户端都能收到这些主题的信息。

相反，客户端将无法收到以下主题的信息。

```
home/sensor/bedroom/brightness
office/sensor/bedroom//temperature
home/screen/livingroom/temperature
```

以上主题的红色部分都是客户端无法收到信息的原因。这些红色的部分都是与客户端订阅的主题“home/sensor+/temperature”不相符的部分。

多级通配符

单级通配符仅可代替一个主题级别，而多级通配符“#”可以涵盖任意数量的主题级别。如下示例所示，多级通配符必须是主题中的最后一个字符。

home/sensor/#

多级通配符可以代替主题级别，而多级通配符 # 可以匹配任意数量的主题级别。如下示例所示，多级通配符必须是主题中的最后一个字符。

`home/sensor/#`

当客户端订阅了以上含有“#”的主题后，可以收到以下主题的信息。

```
home/sensor/kitchen/temperature
home/sensor/bedroom/brightness
home/sensor/data
```

多级通配符可以代替多级主题信息，因此无论“home/sensor”后面有一级还是多级主题，都可以被订阅了“home/sensor/#”的客户端接收到。

4. 主题应用注意事项

- 以\$开始的主题

以\$开始的主题是MQTT服务端系统保留的特殊主题，我们不能随意订阅或者向其发布信息。以下是此类主题的示例：

```
$SYS/broker/clients/connected
$SYS/broker/clients/disconnected
$SYS/broker/clients/total
$SYS/broker/messages/sent
$SYS/broker/uptime
```

类似的主题还有很多。不过请记住一点，以\$符号开头的主题是系统保留的特殊主题，我们不能随意订阅或者向其发布信息。

- 不要用“/”作为主题开头

MQTT允许使用“/”作为主题的开头，例如/home/sensor/data。但是这将这么做毫无意义，而且会额外产生一个没有用处的主题级别。所以我们应避免使用/作为主题的开头。

- 主题中不要使用空格

MQTT协议允许我们在主题中使用空格，但是阅读和调试含有空格的主题会显得异常困难。所以我们尽量不要在主题中使用空格或者特殊字符。

- 保持主题简洁明了

MQTT是一种轻量级的通讯协议，它常用于网络带宽受限的环境，因此我们应尽量让主题简洁明了，从而让设备间交互的内容更加简洁，以更好的适应网络带宽受限的环境。

- 主题中尽量使用ASCII字符

虽然很多MQTT设备支持UTF-8字符作为MQTT主题，但是笔者建议您在主题中尽量使用ASCII字符。

- 在主题中嵌入客户端ID

在主题中嵌入发布消息的客户端ID，这一操作可以为开发和管理MQTT信息提供便利。通过主题中的客户端ID内容，我们可以很容易的了解该主题信息是由哪一台设备所发布的。



本教程为 太极创客团队 制作的《零基础入门学用物联网》系列教程第二篇- MQTT篇。假如您学习本教程感到有些吃力，那么您可能还没有观看本系列教程 第一篇 - 基础知识篇。您可以通过以下链接前往本教程的第一篇 - 基础知识篇 教程页面。

零基础入门学用
物联网
太极创客

【ESP8266教程】零基础入门学用物联网-基础知识篇（太极创客团队）

41.2万 979

视频 太极创客团队



假如您也喜欢太极创客，希望您将本文“分享”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

假如您也喜欢太极创客，希望您将本文“分享”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYNO朔（<https://space.bilibili.com/415252426/>）

本文为我原创 本文禁止转载或摘编

物联网 创客 ESP8266 前沿科技 Arduino IoT 太极创客 MQTT MQTT协议 物联网通讯

分享到：



投诉或建议

推荐文章

更多精彩内容>

（莎头）蓄谋已久（十七）

他怎么也没想到这丫头，买东西都不看清楚的，看了看手里的东西，有些觉得好笑，最后还是选择直接扔进垃圾桶里而那一盒子...



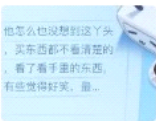
栗桔桔

日常

6152

61

14



评论

全部评论

按时间排序

目录

38

0

7

0

1-8 ESP8266订阅MQTT主题 - 哔哩哔哩

星期一, 三月 7, 2022 11:23 下午

已剪辑自: https://www.bilibili.com/read/cv8565906?spm_id_from=333.999.0.0



首页 番剧 直播 游戏中心 会员购 漫画 赛事

双雄对决, 谁是胜者?



专栏 / 科技 / 学习 / 【零基础学物联网】MQTT篇 1-8 ESP8266订阅MQTT主题

【零基础学物联网】MQTT篇 1-8 ESP8266订阅MQTT主题

学习 2020-11-28 11:03 446阅读 · 49喜欢 · 1评论



太极创客团队
粉丝: 8.6万 文章: 29

已关注



为了帮助您学习, 我们专门为本教程制作了视频。



零基础入门学用物联网
MQTT篇
1-8 ESP8266订阅主题
太极创客 27:15

【太极创客】零基础入门学用物联网 - MQTT篇 1-8 ESP8266订阅MQTT主题

7108 21

太极创客团队

在上一节课程里, 我们学习了如何使用ESP8266发布MQTT消息, 在这节课里, 我们将学习如何使用ESP8266来订阅MQTT消息。我们将使用ESP8266实现以下订阅MQTT消息的操作:

- 订阅单个主题
- 订阅多个主题
- 使用单级通配符订阅主题
- 使用多级通配符订阅主题

我们将针对以上列表中的每一项内容提供一个示例程序。这些示例程序均配有视频讲解以帮助您更好的了解我们是如何实现每一项MQTT操作的。您可以点击本文首部的视频链接, 打开讲解视频。

订阅单个主题

项目名称/Project : 零基础入门学用物联网

项目名称/Project : 零基础入门学用物联网
程序名称/Program name : subscribe_ranye_url
团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)
作者/Author : CYN0朔
日期/Date (YYYYMMDD) : 20200708
程序目的/Purpose :
本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器订阅信息。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichimaker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
void setup() {
```

```
    pinMode(LED_BUILTIN, OUTPUT); // 设置板上LED引脚为输出模式
```

```
    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED
```

```
    Serial.begin(9600); // 启动串口通讯
```

```
//设置ESP8266工作模式为无线终端模式
```

```
WiFi.mode(WIFI_STA);
```

```
// 连接WiFi
```

```
connectWifi();
```

```
// 设置MQTT服务器和端口号
```

```
mqttClient.setServer(mqttServer, 1883);
```

```

// 设置MQTT服务器和端口号
mqttClient.setServer(mqttServer, 1883);

// 设置MQTT订阅回调函数
mqttClient.setCallback(receiveCallback);


// 连接MQTT服务器
connectMQTTserver();
}

void loop() {
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
        mqttClient.loop();        // 处理信息以及心跳
    } else {                      // 如果开发板未能成功连接服务器
        connectMQTTserver();      // 则尝试连接服务器
    }
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver(){
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address:");
        Serial.println(mqttServer);
        Serial.println("ClientId: ");
        Serial.println(clientId);
        subscribeTopic(); // 订阅指定主题
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(5000);
    }
}

// 收到信息后的回调函数
void receiveCallback(char* topic, byte* payload, unsigned int length) {
    Serial.print("Message Received l'");

```



```

void receiveCallback(char* topic, byte* payload, unsigned int length) {

    Serial.print("Message Received [");

    Serial.print(topic);

    Serial.print("] ");

    for (int i = 0; i < length; i++) {

        Serial.print((char)payload[i]);

    }

    Serial.println("");

    Serial.print("Message Length(Bytes) ");

    Serial.println(length);

    if ((char)payload[0] == '1') { // 如果收到的信息以“1”为开始

        digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。

        Serial.println("LED ON");

    } else {

        digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。

        Serial.println("LED OFF");

    }

}

// 订阅指定主题

void subscribeTopic(){

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

    if(mqttClient.subscribe(subTopic)){

        Serial.println("Subscrib Topic:");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }

}

// ESP8266连接wifi

```

```
// ESP8266连接wifi

void connectWifi(){

    WiFi.begin(ssid, password);

    //等待WiFi连接,成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}

}
```

订阅多个主题

/******

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_multitopic_ranye_url

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYNO朔

日期/Date (YYYYMMDD) : 20201125

程序目的/Purpose :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器订阅多个主题

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

***** /

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichimaker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```

// http://www.taichi-maker.com/public-mqtt-broker/

WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);

void setup() {
    pinMode(LED_BUILTIN, OUTPUT); // 设置板上LED引脚为输出模式
    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED
    Serial.begin(9600);           // 启动串口通讯

    //设置ESP8266工作模式为无线终端模式
    WiFi.mode(WIFI_STA);

    // 连接WiFi
    connectWifi();

    // 设置MQTT服务器和端口号
    mqttClient.setServer(mqttServer, 1883);
    mqttClient.setCallback(receiveCallback);

    // 连接MQTT服务器
    connectMQTTserver();
}

void loop() {
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
        mqttClient.loop();       // 处理信息以及心跳
    } else {                     // 如果开发板未能成功连接服务器
        connectMQTTserver();     // 则尝试连接服务器
    }
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver(){
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
    }
}

```

```

if (mqttClient.connect(clientId.c_str())) {
    Serial.println("MQTT Server Connected.");
    Serial.println("Server Address:");
    Serial.println(mqttServer);
    Serial.println("ClientId: ");
    Serial.println(clientId);
    subscribeTopic(); // 订阅指定主题
} else {
    Serial.print("MQTT Server Connect Failed. Client State:");
    Serial.println(mqttClient.state());
    delay(5000);
}
}

// 收到信息后的回调函数
void receiveCallback(char* topic, byte* payload, unsigned int length) {
    Serial.print("Message Received [");
    Serial.print(topic);
    Serial.print("] ");
    for (int i = 0; i < length; i++) {
        Serial.print((char)payload[i]);
    }
    Serial.println("");
    Serial.print("Message Length(Bytes) ");
    Serial.println(length);

    if ((char)payload[0] == '1') { // 如果收到的信息以“1”为开始
        digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
    } else {
        digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。
    }
}

// 订阅指定主题
void subscribeTopic(){

    // 建立订阅主题1。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。
    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同
    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

```

称不同

```
String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

char subTopic[topicString.length() + 1];

strcpy(subTopic, topicString.c_str());


// 建立订阅主题2

String topicString2 = "Taichi-Maker-Sub2-" + WiFi.macAddress();

char subTopic2[topicString2.length() + 1];

strcpy(subTopic2, topicString2.c_str());


// 通过串口监视器输出是否成功订阅主题1以及订阅的主题1名称

if(mqttClient.subscribe(subTopic)){

    Serial.println("Subscrib Topic:");

    Serial.println(subTopic);

} else {

    Serial.print("Subscribe Fail...");

}


// 通过串口监视器输出是否成功订阅主题2以及订阅的主题2名称

if(mqttClient.subscribe(subTopic2)){

    Serial.println("Subscrib Topic:");

    Serial.println(subTopic2);

} else {

    Serial.print("Subscribe Fail...");

}

}


// ESP8266连接wifi

void connectWifi(){

    WiFi.begin(ssid, password);

    //等待WiFi连接,成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}
```



```
Serial.println("");  
}
```

使用单级通配符订阅主题

```
/******
```

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_single_level_wildcard_ranye

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20200813

程序目的/Purpose :

本程序旨在演示ESP8266订阅信息时的单级通配符+应用。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

```
*****/
```

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichi-maker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
void setup() {
```

```
    pinMode(LED_BUILTIN, OUTPUT); // 设置板上LED引脚为输出模式
```

```
    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED
```

```
    Serial.begin(9600); // 启动串口通讯
```

```
    //设置ESP8266工作模式为无线终端模式
```

```
    WiFi.mode(WIFI_STA);
```

```

WiFi.mode(WIFI_STA);

// 连接WiFi
connectWifi();

// 设置MQTT服务器和端口号
mqttClient.setServer(mqttServer, 1883);
mqttClient.setCallback(receiveCallback);

// 连接MQTT服务器
connectMQTTserver();
}

void loop() {
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
        mqttClient.loop();      // 处理信息以及心跳
    } else {                    // 如果开发板未能成功连接服务器
        connectMQTTserver();    // 则尝试连接服务器
    }
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver(){
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address:");
        Serial.println(mqttServer);
        Serial.println("ClientId: ");
        Serial.println(clientId);
        subscribeTopic(); // 订阅指定主题
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(5000);
    }
}
}

```

```

    }
}

// 收到信息后的回调函数
void receiveCallback(char* topic, byte* payload, unsigned int length) {

    Serial.print("Message Received [");

    Serial.print(topic);

    Serial.print("] ");

    for (int i = 0; i < length; i++) {

        Serial.print((char)payload[i]);

    }

    Serial.println("");

    Serial.print("Message Length(Bytes) ");

    Serial.println(length);

    if ((char)payload[0] == '1') { // 如果收到的信息以“1”为开始
        digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
    } else {
        digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。
    }
}

// 订阅指定主题
void subscribeTopic(){

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。
    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress()+"/+/data";
    char subTopic[topicString.length() + 1];
    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称
    if(mqttClient.subscribe(subTopic)){
        Serial.println("Subscrib Topic:");
        Serial.println(subTopic);
    } else {
        Serial.print("Subscribe Fail...");
    }
}
}

```

```

    }
}

// ESP8266连接wifi
void connectWifi(){

    WiFi.begin(ssid, password);

    //等待WiFi连接,成功连接后输出成功信息
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi Connected!");
    Serial.println("");
}

```

使用多级通配符订阅主题

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_multi_level_wildcard_ranye

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYNO朔

日期/Date (YYYYMMDD) : 20200813

程序目的/Purpose :

本程序旨在演示ESP8266订阅信息时的多级通配符#应用。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

#include <ESP8266WiFi.h>

#include <PubSubClient.h>

// 设置wifi接入信息(请根据您的WiFi信息进行修改)

const char* ssid = "taichi-maker";

const char* password = "12345678";

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案

```

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
// http://www.taichi-maker.com/public-mqtt-broker/

WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);

void setup() {
    pinMode(LED_BUILTIN, OUTPUT); // 设置板上LED引脚为输出模式
    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED
    Serial.begin(9600);           // 启动串口通讯

    //设置ESP8266工作模式为无线终端模式
    WiFi.mode(WIFI_STA);

    // 连接WiFi
    connectWifi();

    // 设置MQTT服务器和端口号
    mqttClient.setServer(mqttServer, 1883);
    mqttClient.setCallback(receiveCallback);

    // 连接MQTT服务器
    connectMQTTserver();
}

void loop() {
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
        mqttClient.loop();       // 处理信息以及心跳
    } else {                     // 如果开发板未能成功连接服务器
        connectMQTTserver();     // 则尝试连接服务器
    }
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver(){
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器

```



```

// 连接MQTT服务器

if (mqttClient.connect(clientId.c_str())) {

    Serial.println("MQTT Server Connected.");

    Serial.println("Server Address:");

    Serial.println(mqttServer);

    Serial.println("ClientId: ");

    Serial.println(clientId);

    subscribeTopic(); // 订阅指定主题

} else {

    Serial.print("MQTT Server Connect Failed. Client State:");

    Serial.println(mqttClient.state());

    delay(5000);

}

}

// 收到信息后的回调函数

void receiveCallback(char* topic, byte* payload, unsigned int length) {

    Serial.print("Message Received [");

    Serial.print(topic);

    Serial.print("] ");

    for (int i = 0; i < length; i++) {

        Serial.print((char)payload[i]);

    }

    Serial.println("");

    Serial.print("Message Length(Bytes) ");

    Serial.println(length);

    if ((char)payload[0] == '1') { // 如果收到的信息以“1”为开始

        digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。

    } else {

        digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。

    }

}

// 订阅指定主题

void subscribeTopic(){

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为了确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名

```

```

// 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

// 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress()+"/sensor/#";

char subTopic[topicString.length() + 1];

strcpy(subTopic, topicString.c_str());


// 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

if(mqttClient.subscribe(subTopic)){

    Serial.println("Subscrib Topic:");

    Serial.println(subTopic);

} else {

    Serial.print("Subscribe Fail...");

}

}

// ESP8266连接wifi

void connectWifi(){

    WiFi.begin(ssid, password);

    //等待WiFi连接,成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}

```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的**每一份点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYNO朔（<https://space.bilibili.com/415252426/>）

本文为我原创 本文禁止转载或摘编

📁 嵌入式 物联网 创客 ESP8266 Arduino 太极创客 MQTT 物联网技术 IoT MQTT协议

分享到：

投诉或建议

推荐文章

更多精彩内容 >

15.我在B站学云原生之Docker镜像安全最佳实践

帅哥（靓仔）、美女，点个关注后续不迷路！本章目录0x00 前言简述1.容器安全概述2.容器安全机制内核能力机制服务端防护机...

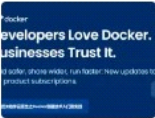
 WeiYiGeek

学习

 8

 0

 0



人大代表李君：建议关闭直播打赏功能，禁止未成年人玩网络游戏

点击进入查看全文>

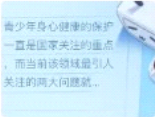
 电科技

学习

 21

 0

 0



erp软件完结对企业有什么好处？

企业施行erp软件需要的投入是非常巨大的，进步利润，降低运作本钱，加强供应链办理。在这个意义上的投入是“软技能”，关于...

 数加科技李工

学习

 1

 0

 0



评论

全部评论 按时间排序

目录

 49

 0

 12

 1



1-8 ESP8266订阅MQTT主题

星期日, 三月 13, 2022 5:32 下午

已剪辑自: https://www.bilibili.com/read/cv8565906?spm_id_from=333.999.0.0

为了帮助您学习, 我们专门为本教程制作了视频。

↓

在上一节课程里, 我们学习了如何使用ESP8266发布MQTT消息, 在这节课里, 我们将学习如何使用ESP8266来订阅MQTT消息。我们将使用ESP8266实现以下订阅MQTT消息的操作:

- 订阅单个主题
- 订阅多个主题
- 使用单级通配符订阅主题
- 使用多级通配符订阅主题

我们将针对以上列表中的每一项内容提供一个示例程序。这些示例程序均配有视频讲解以帮助您更好的了解我们是如何实现每一项MQTT操作的。您可以点击本文首部的视频链接, 打开讲解视频。

订阅单个主题

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_ranye_url

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20200708

程序目的/Purpose :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器订阅信息。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息, 请参考以下网页:

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

```

*****/

#include <ESP8266WiFi.h>

#include <PubSubClient.h>


// 设置wifi接入信息(请根据您的WiFi信息进行修改)

const char* ssid = "taichimaker";

const char* password = "12345678";

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接, 请前往以下页面寻找解决方案

// http://www.taichi-maker.com/public-mqtt-broker/


WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);


void setup() {

    pinMode(LED_BUILTIN, OUTPUT);    // 设置板上LED引脚为输出模式

    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED

    Serial.begin(9600);               // 启动串口通讯


    //设置ESP8266工作模式为无线终端模式

    WiFi.mode(WIFI_STA);


    // 连接WiFi

    connectWifi();


    // 设置MQTT服务器和端口号

    mqttClient.setServer(mqttServer, 1883);

    // 设置MQTT订阅回调函数

```



```

mqttClient.setCallback(receiveCallback);

// 连接MQTT服务器
connectMQTTserver();
}

void loop() {
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
        mqttClient.loop();          // 处理信息以及心跳
    } else {                        // 如果开发板未能成功连接服务器
        connectMQTTserver();        // 则尝试连接服务器
    }
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver() {
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address:");
        Serial.println(mqttServer);
        Serial.println("ClientId: ");
        Serial.println(clientId);
        subscribeTopic(); // 订阅指定主题
    } else {

```

```

    Serial.print("MQTT Server Connect Failed. Client State:");

    Serial.println(mqttClient.state());

    delay(5000);
}
}

// 收到信息后的回调函数
void receiveCallback(char* topic, byte* payload, unsigned int length) {

    Serial.print("Message Received [");

    Serial.print(topic);

    Serial.print("] ");

    for (int i = 0; i < length; i++) {

        Serial.print((char)payload[i]);

    }

    Serial.println("");

    Serial.print("Message Length(Bytes) ");

    Serial.println(length);

    if ((char)payload[0] == '1') { // 如果收到的信息以“1”为开始

        digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。

        Serial.println("LED ON");

    } else {

        digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。

        Serial.println("LED OFF");

    }

}

// 订阅指定主题

```

```

void subscribeTopic() {

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());


    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

    if(mqttClient.subscribe(subTopic)) {

        Serial.println("Subscrib Topic:");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }

}


// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }
}

```

```

Serial.println("");

Serial.println("WiFi Connected!");

Serial.println("");
}

```

订阅多个主题

/*****

```

项目名称/Project      : 零基础入门学用物联网

程序名称/Program name : subscribe_multitopic_ranye_url

团队/Team             : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author           : CYN0朔

日期/Date (YYYYMMDD)  : 20201125

程序目的/Purpose        :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器订阅多个主题

```

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```

#include <ESP8266WiFi.h>

#include <PubSubClient.h>

```

```

// 设置wifi接入信息(请根据您的WiFi信息进行修改)

```

```

const char* ssid = "taichimaker";

```

```

const char* password = "12345678";

```

```

const char* mqttServer = "test.ranye-iot.net";

```

```

// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案

```

// <http://www.taichi-maker.com/public-mqtt-broker/>

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
void setup() {
```

```
    pinMode(LED_BUILTIN, OUTPUT);    // 设置板上LED引脚为输出模式
```

```
    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED
```

```
    Serial.begin(9600);                // 启动串口通讯
```

```
    //设置ESP8266工作模式为无线终端模式
```

```
    WiFi.mode(WIFI_STA);
```

```
    // 连接WiFi
```

```
    connectWifi();
```

```
    // 设置MQTT服务器和端口号
```

```
    mqttClient.setServer(mqttServer, 1883);
```

```
    mqttClient.setCallback(receiveCallback);
```

```
    // 连接MQTT服务器
```

```
    connectMQTTserver();
```

```
}
```

```
void loop() {
```

```
    if (mqttClient.connected()) { // 如果开发板成功连接服务器
```

```
        mqttClient.loop();        // 处理信息以及心跳
```

```
    } else {                        // 如果开发板未能成功连接服务器
```



```

        connectMQTTserver();          // 则尝试连接服务器
    }
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver() {
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address:");
        Serial.println(mqttServer);
        Serial.println("ClientId: ");
        Serial.println(clientId);
        subscribeTopic(); // 订阅指定主题
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(5000);
    }
}

// 收到信息后的回调函数
void receiveCallback(char* topic, byte* payload, unsigned int length) {
    Serial.print("Message Received [");
    Serial.print(topic);

```

```

Serial.print("] ");

for (int i = 0; i < length; i++) {

    Serial.print((char)payload[i]);

}

Serial.println("");

Serial.print("Message Length(Bytes) ");

Serial.println(length);


if ((char)payload[0] == '1') {    // 如果收到的信息以“1”为开始

    digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。

} else {

    digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。

}

}

// 订阅指定主题

void subscribeTopic() {

    // 建立订阅主题1。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());


    // 建立订阅主题2

    String topicString2 = "Taichi-Maker-Sub2-" + WiFi.macAddress();

    char subTopic2[topicString2.length() + 1];

```

```

strcpy(subTopic2, topicString2.c_str());

// 通过串口监视器输出是否成功订阅主题1以及订阅的主题1名称
if(mqttClient.subscribe(subTopic)) {
    Serial.println("Subscrib Topic:");
    Serial.println(subTopic);
} else {
    Serial.print("Subscribe Fail...");
}

// 通过串口监视器输出是否成功订阅主题2以及订阅的主题2名称
if(mqttClient.subscribe(subTopic2)) {
    Serial.println("Subscrib Topic:");
    Serial.println(subTopic2);
} else {
    Serial.print("Subscribe Fail...");
}
}

// ESP8266连接wifi
void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
}

```

```

}

Serial.println("");

Serial.println("WiFi Connected!");

Serial.println("");

}

```

使用单级通配符订阅主题

/*****

```

项目名称/Project      : 零基础入门学用物联网

程序名称/Program name  : subscribe_single_level_wildcard_ranye

团队/Team             : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author           : CYN0朔

日期/Date (YYYYMMDD)  : 20200813

程序目的/Purpose        :

```

本程序旨在演示ESP8266订阅信息时的单级通配符+应用。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichi-maker";
```

```
const char* password = "12345678";
```

```

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
// http://www.taichi-maker.com/public-mqtt-broker/

WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);

void setup() {

    pinMode(LED_BUILTIN, OUTPUT);    // 设置板上LED引脚为输出模式

    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED

    Serial.begin(9600);               // 启动串口通讯

    //设置ESP8266工作模式为无线终端模式

    WiFi.mode(WIFI_STA);

    // 连接WiFi

    connectWifi();

    // 设置MQTT服务器和端口号

    mqttClient.setServer(mqttServer, 1883);

    mqttClient.setCallback(receiveCallback);

    // 连接MQTT服务器

    connectMQTTserver();

}

void loop() {

    if (mqttClient.connected()) { // 如果开发板成功连接服务器

```



```

    mqttClient.loop();          // 处理信息以及心跳
} else {                      // 如果开发板未能成功连接服务器
    connectMQTTserver();      // 则尝试连接服务器
}
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver() {
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address:");
        Serial.println(mqttServer);
        Serial.println("ClientId: ");
        Serial.println(clientId);
        subscribeTopic(); // 订阅指定主题
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(5000);
    }
}

// 收到信息后的回调函数
void receiveCallback(char* topic, byte* payload, unsigned int length) {

```

```

Serial.print("Message Received [");
Serial.print(topic);
Serial.print("] ");
for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
}
Serial.println("");
Serial.print("Message Length(Bytes) ");
Serial.println(length);

if ((char)payload[0] == '1') {    // 如果收到的信息以“1”为开始
    digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
} else {
    digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。
}
}

// 订阅指定主题

void subscribeTopic() {

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress()+"/+/data";
    char subTopic[topicString.length() + 1];
    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

```

```

    if(mqttClient.subscribe(subTopic)) {

        Serial.println("Subscrib Topic:");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }

}

// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}

```

使用多级通配符订阅主题

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_multi_level_wildcard_ranye

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20200813

程序目的/Purpose :

本程序旨在演示ESP8266订阅信息时的多极通配符#应用。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichi-maker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
void setup() {
```

```
    pinMode(LED_BUILTIN, OUTPUT);    // 设置板上LED引脚为输出模式
```

```
    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED
```

```
    Serial.begin(9600);                // 启动串口通讯
```

```

//设置ESP8266工作模式为无线终端模式

WiFi.mode(WIFI_STA);


// 连接WiFi

connectWifi();


// 设置MQTT服务器和端口号

mqttClient.setServer(mqttServer, 1883);

mqttClient.setCallback(receiveCallback);


// 连接MQTT服务器

connectMQTTserver();

}


void loop() {

    if (mqttClient.connected()) { // 如果开发板成功连接服务器

        mqttClient.loop();          // 处理信息以及心跳

    } else {                        // 如果开发板未能成功连接服务器

        connectMQTTserver();        // 则尝试连接服务器

    }

}


// 连接MQTT服务器并订阅信息

void connectMQTTserver() {

    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）

    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器

```



```

if (mqttClient.connect(clientId.c_str())) {

    Serial.println("MQTT Server Connected.");

    Serial.println("Server Address:");

    Serial.println(mqttServer);

    Serial.println("ClientId: ");

    Serial.println(clientId);

    subscribeTopic(); // 订阅指定主题

} else {

    Serial.print("MQTT Server Connect Failed. Client State:");

    Serial.println(mqttClient.state());

    delay(5000);

}

}

// 收到信息后的回调函数

void receiveCallback(char* topic, byte* payload, unsigned int length) {

    Serial.print("Message Received [");

    Serial.print(topic);

    Serial.print("] ");

    for (int i = 0; i < length; i++) {

        Serial.print((char)payload[i]);

    }

    Serial.println("");

    Serial.print("Message Length(Bytes) ");

    Serial.println(length);

    if ((char)payload[0] == '1') { // 如果收到的信息以“1”为开始

        digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
    }
}

```

```

    } else {

        digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。

    }
}

// 订阅指定主题

void subscribeTopic() {

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress()+"/sensor/#";

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

    if(mqttClient.subscribe(subTopic)) {

        Serial.println("Subscrib Topic:");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }

}

// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

```

```
//等待WiFi连接,成功连接后输出成功信息

while (WiFi.status() != WL_CONNECTED) {

    delay(1000);

    Serial.print(".");

}

Serial.println("");

Serial.println("WiFi Connected!");

Serial.println("");

}
```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。
您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

1-9 自我测试 – 哔哩哔哩

星期日, 三月 13, 2022 5:32 下午

已剪辑自: https://www.bilibili.com/read/cv8641342?spm_id_from=333.999.0.0

为了帮助您学习, 我们专门为本教程制作了视频。

↓

到目前为止, 我们已经掌握了MQTT通讯的基本流程以及如何使用ESP8266来发布和订阅MQTT消息。这节课我们来进行自我测试。

以下示例程序可以让ESP8266同时订阅和发布MQTT消息。请将本程序上传到ESP8266开发板并且仔细阅读程序内容。关于此程序的具体使用方法, 您可以参考本文首部的视频说明。

假如您对程序的所有内容全部理解, 那么恭喜您, 目前我们所教授的知识您已经完全掌握了。

假如您对程序有任何不理解的地方, 那么请务必将整个程序搞清楚, 然后再继续后面的学习。

目前我们所学的都是基础知识, 因此一定要牢固掌握所有程序内容, 这样才能让我们后续的学习更加顺利。

假如您需要收看本教程的全部内容, [请点击此处进入教程合辑页面](#)。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : pub_and_sub

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20200719

程序目的/Purpose :

利用PubSubClient同时订阅和发布信息。

定时向主题发布D3按键的状态

通过订阅主题收到信息是数字1则打开LED，否则关闭LED。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
#include <Ticker.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichi-maker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```
Ticker ticker;
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
int count;    // Ticker计数用变量
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    pinMode(D3, INPUT_PULLUP);
```



```

// Ticker定时对象

ticker.attach(1, tickerCount);

//设置ESP8266工作模式为无线终端模式

WiFi.mode(WIFI_STA);

// 连接WiFi

connectWifi();

// 设置MQTT服务器和端口号

mqttClient.setServer(mqttServer, 1883);

mqttClient.setCallback(receiveCallback);

// 连接MQTT服务器

connectMQTTServer();

}

void loop() {

    if (mqttClient.connected()) { // 如果开发板成功连接服务器

        // 每隔3秒钟发布一次信息

        if (count >= 3) {

            pubMQTTmsg();

            count = 0;

        }

        // 保持心跳

        mqttClient.loop();

    } else { // 如果开发板未能成功连接服务器

```

```

        connectMQTTServer();    // 则尝试连接服务器
    }
}

void tickerCount() {
    count++;
}

void connectMQTTServer() {
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address: ");
        Serial.println(mqttServer);
        Serial.println("ClientId:");
        Serial.println(clientId);
        subscribeTopic(); // 订阅指定主题
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(3000);
    }
}
}

```

```

// 发布信息

void pubMQTTmsg() {

    // 建立发布主题。主题名称以Taichi-Maker-为前缀，后面添加设备的MAC地址。
    // 这么做是为确保不同用户进行MQTT信息发布时，ESP8266客户端名称各不相同，
    String topicString = "Taichi-Maker-Pub-" + WiFi.macAddress();

    char publishTopic[topicString.length() + 1];
    strcpy(publishTopic, topicString.c_str());

    // 定时向服务器主题发布当前D3引脚状态

    String messageString;

    if(digitalRead(D3)) {

        messageString = "on";

    } else {

        messageString = "off";

    }

    char publishMsg[messageString.length() + 1];
    strcpy(publishMsg, messageString.c_str());

    // 实现ESP8266向主题发布信息

    if(mqttClient.publish(publishTopic, publishMsg)) {

        Serial.println("Publish Topic:");Serial.println(publishTopic);

        Serial.println("Publish message:");Serial.println(publishMsg);

    } else {

        Serial.println("Message Publish Failed.");

    }

}

// 订阅指定主题

```

```

void subscribeTopic() {

    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

    if(mqttClient.subscribe(subTopic)) {

        Serial.println("Subscribe Topic:");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }

}

// 收到信息后的回调函数

void receiveCallback(char* topic, byte* payload, unsigned int length) {

    Serial.print("Message Received [");

    Serial.print(topic);

    Serial.print("] ");

    for (int i = 0; i < length; i++) {

        Serial.print((char)payload[i]);

    }

    Serial.println("");

    Serial.print("Message Length(Bytes) ");

```

```

Serial.println(length);

if ((char)payload[0] == '1') {    // 如果收到的信息以“1”为开始
    digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
} else {
    digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。
}
}

// ESP8266连接wifi
void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }

    Serial.println("");
    Serial.println("WiFi Connected!");
    Serial.println("");
}

```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。
您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-1 QoS 服务质量等级

星期日, 三月 13, 2022 5:34 下午

已剪辑自: https://www.bilibili.com/read/cv8681431?spm_id_from=333.999.0.0

为了帮助您学习, 我们专门为本教程制作了视频。



什么是MQTT服务质量?

在之前的课程里我们提到过, 一个物联网系统中有些信息非常重要, 我们需要确保这类重要信息可以准确无误的发送和接收, 而有些信息则相对不那么重要, 这类信息如果在传输中丢失不会影响系统的运行。

MQTT服务质量(Quality of Service 缩写 QoS)正是用于告知物联网系统, 哪些信息是重要信息需要准确无误的传输, 而哪些信息不那么重要, 即使丢失也没有问题。

MQTT协议有三种服务质量级别:

QoS = 0 - 最多发一次

QoS = 1 - 最少发一次

QoS = 2 - 保证收一次

以上三种不同的服务质量级别意味着不同的MQTT传输流程。对于较为重要的MQTT消息, 我们通常会选择QoS>0的服务级别(即QoS 为1或2)。

另外这里提到的“发”与“收”有两种可能。一种是客户端发布消息时, 将消息发送给服务端。一种是客户端订阅了某一主题消息后, 服务端将消息发送给客户端。因此发布消息和接收消息的可能是服务端也可能是客户端。

为了避免为您造成混淆, 我们在本节教程后面的描述中将使用“发送端”来描述发送MQTT消息的设备, 而使用“接收端”来描述接收MQTT消息的设备。

接下来我们仔细看一下这三种服务质量级别的具体含义。



QoS = 0 - 最多发一次

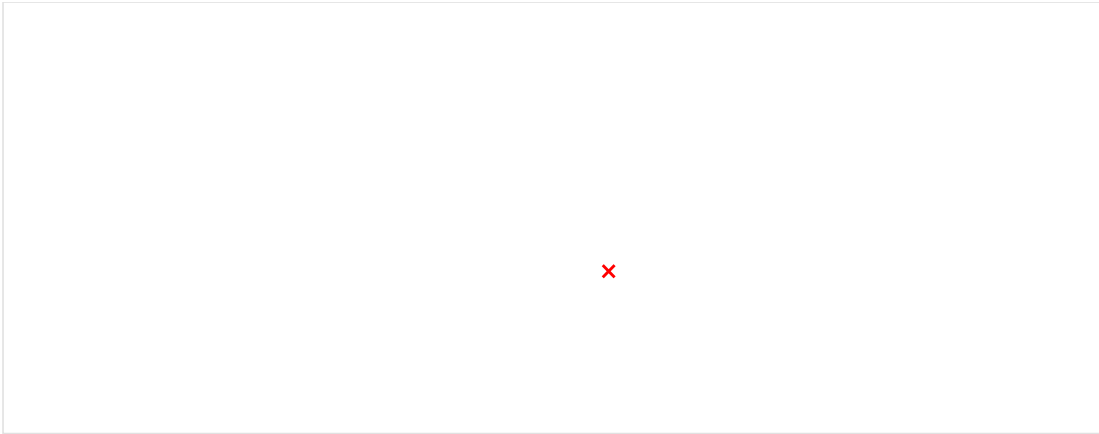
0是服务质量QoS的最低级别。当QoS为0级时, MQTT协议并不保证所有信息都能得以传输。也就是说, QoS=0的情况下, MQTT服务端和客户端不会对消息传输是否成功进行确认和检查。消息能否成功传输全看网络环境是否稳定。

也就是说, 在QoS为0时。发送端一旦发送完消息后, 就完成任务了。发送端不会检查发出的消息能否被正确接收到。

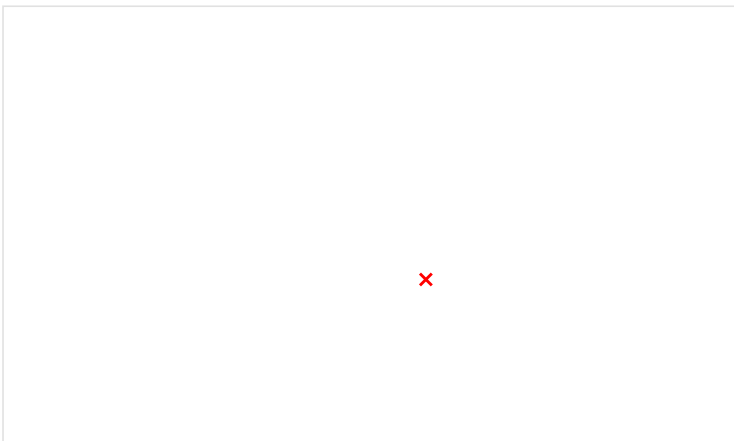
在网络环境稳定的情况下, 信息传输一般是不会出现问题的。但是在环境不稳定的情况下, 可能会在传输过程中出现MQTT消息丢失的情况。

QoS = 1 - 最少发一次

当QoS级别为1时, 发送端在消息发送完成后, 会检查接收端是否已经成功接收到了消息。但是发送端是如何实现这一检查的呢? 请看下图:



发送端将消息发送给接收端后，会等待接收端的确认。接收端成功接收消息后，会发送一条确认报文PUBACK给发送端。如果发送端收到了这条PUBACK确认报文，那么它就知道消息已经成功接收。

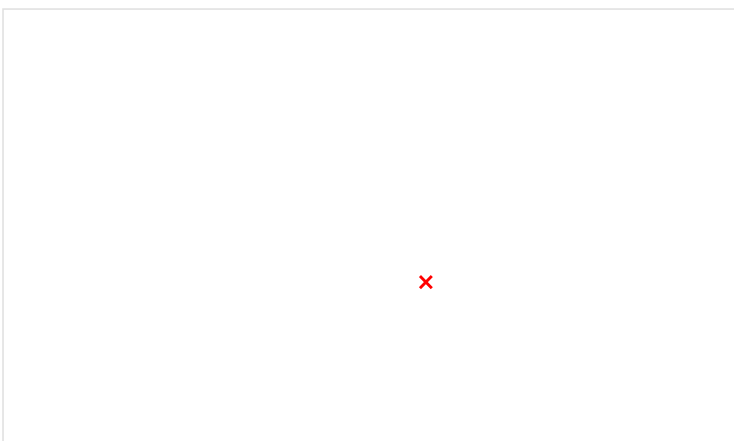


PUBACK 报文

假如过了一段时间后，发送端没有收到PUBACK报文，那么发送端会再次发送消息，然后再次等待接收端的PUBACK确认报文。因此，当QoS=1时，发送端在没有收到接收端的PUBACK确认报文以前，会重复发送同一条消息。

所以QoS = 1时，每一条消息都至少传输一次。

另外请您回忆一下PUBLISH报文的内容。



MQTT PUBLISH 报文 dupFlag 设置为 true

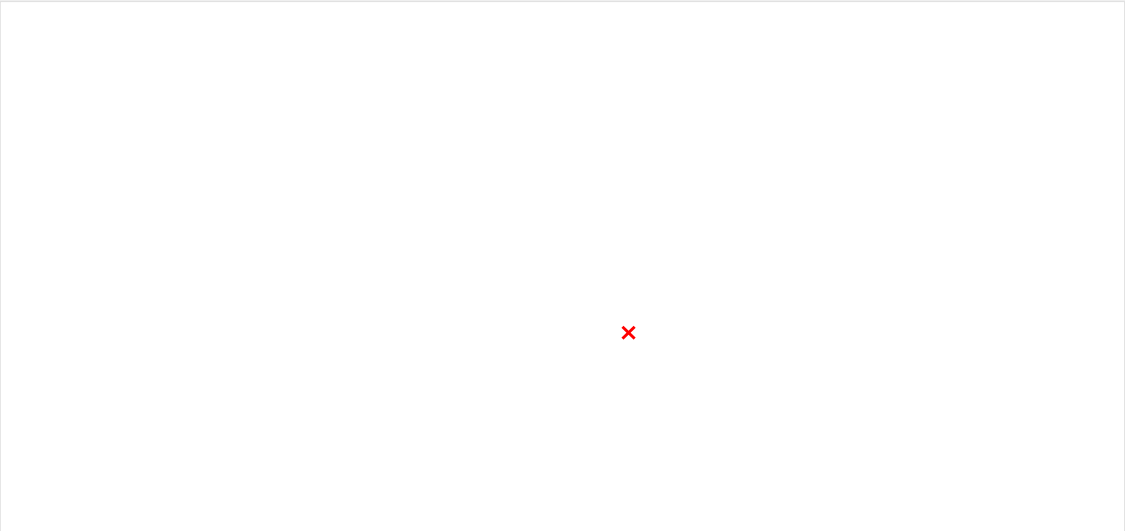
当发送端重复发送一条消息时，PUBLISH报文中的dupFlag会被设置为True（如上图黑色横线所标注的部分）。这是为了告诉接收端，此消息为重复发送的消息。

QoS = 2 - 保证收一次

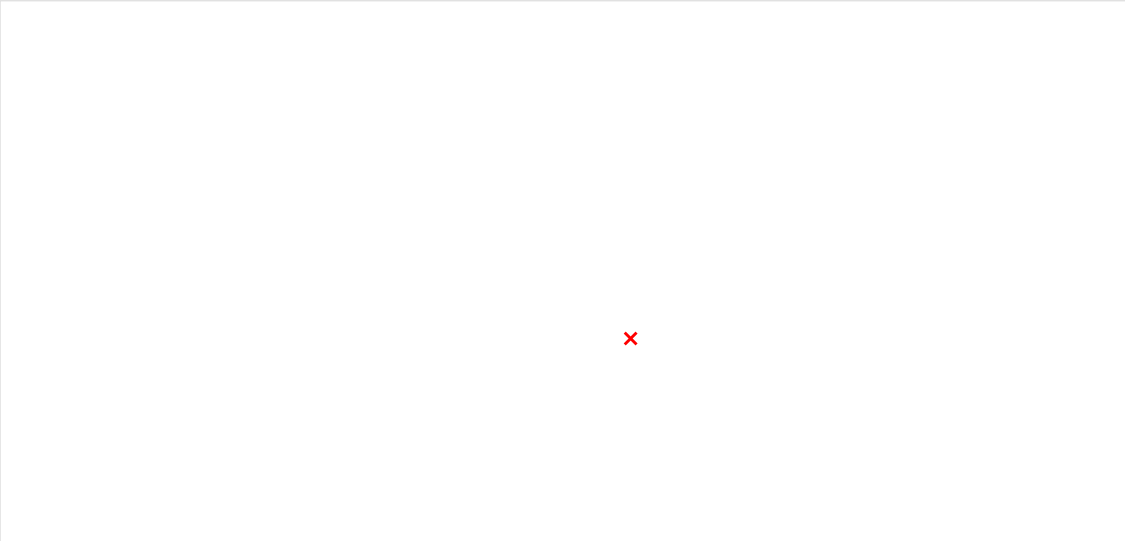
MQTT服务质量最高级是2级，即QoS = 2。当MQTT服务质量为2级时，MQTT协议可以确保接收端只接收一次消息。

如下图所示，QoS=2的收发相对更加复杂。发送端需要接收端进行两次消息确认。因此，2级MQTT服务质量是最安全的服务级别，也是最慢的服务级别。

下面我们来分步看一下QoS=2时的消息发送和接收基本流程。

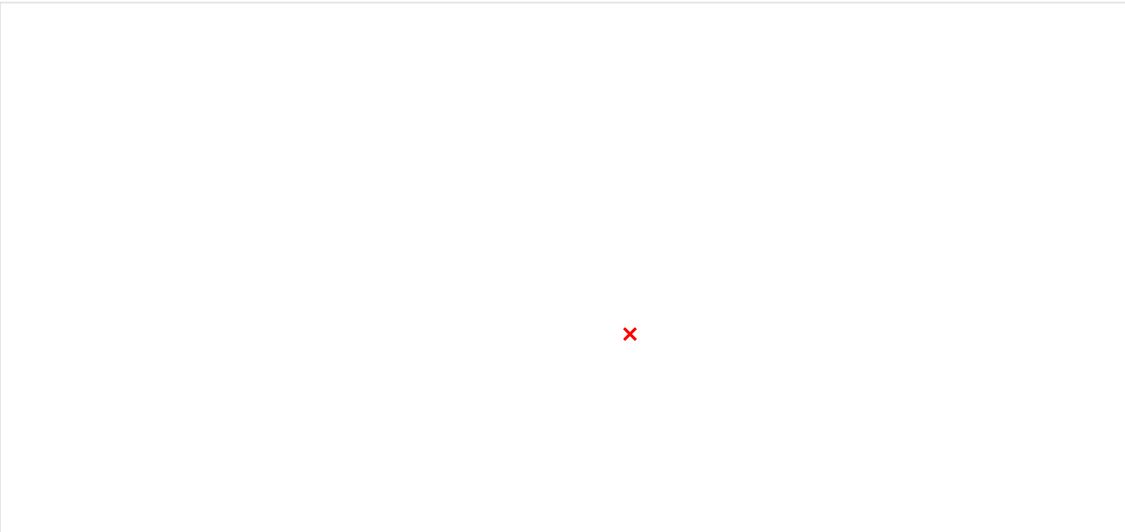


MQTT QoS2 PUBLISH



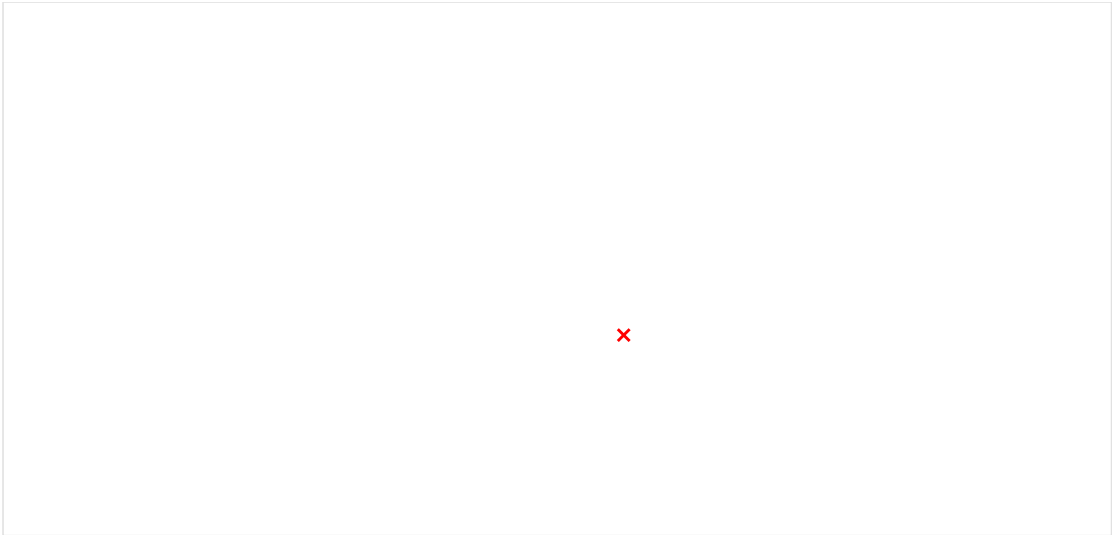
MQTT QoS2 PUBREC

接收端收到QoS为2的消息后，会返回PUBREC报文作为应答。



MQTT QoS2 PUBREL

发送端收到PUBREC报文后，会把此报文进行存储，并且返回PUBREL报文作为应答。



MQTT QoS2 PUBCOMP

当接收端收到PUBREL报文后，会应答发送端一条PUBCOMP报文。至此，一次QoS2的MQTT消息传输就结束了。

以上是QoS=2时的MQTT通讯基本过程。这里我们只给您列出了基本流程，而没有过多的讲解MQTT协议是如何控制接收端只接收一次消息。这么做是因为本教程的重点是MQTT应用。关于QoS=2的MQTT服务端内部控制机制，我们在实际开发MQTT物联网的过程中是不会涉及到的。

所以您只需要牢记一点，那就是**QoS=2可以保证接收端只收一次消息。**

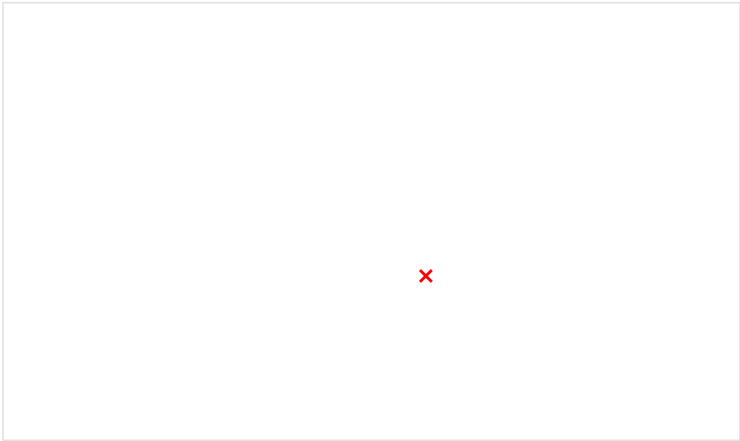


设置QoS

了解了QoS的含义后，我们该如何在MQTT通讯中设置QoS呢？下面我们来分别讲解客户端在发布消息和订阅消息时如何设置QoS。

发布消息

如下图所示，客户端发布信息时，PUBLISH数据包中专有一个信息为qos。该信息正是用于设置客户端发布MQTT消息的QoS等级。



MQTT Publish QoS

订阅消息

同样的，在客户端订阅MQTT主题时，SUBSCRIBE数据包中也同样有一个信息用于设置订阅主题的QoS级别。客户端正是通过该主题来设置订阅主题的QoS级别的。

换句话说，无论是发布(PUBLISH)还是订阅(SUBSCRIBE)，都可以使用数据包中的qos消息设置服务质量级别。

接收端连接服务端

另外，要想实现QoS>0的MQTT通讯，客户端在连接服务端时必须要把cleanSession设置为false。如果这一步没有实现，那么客户端是无法实现QoS>0的MQTT通讯。这一点非常关键，请您务必要留意。

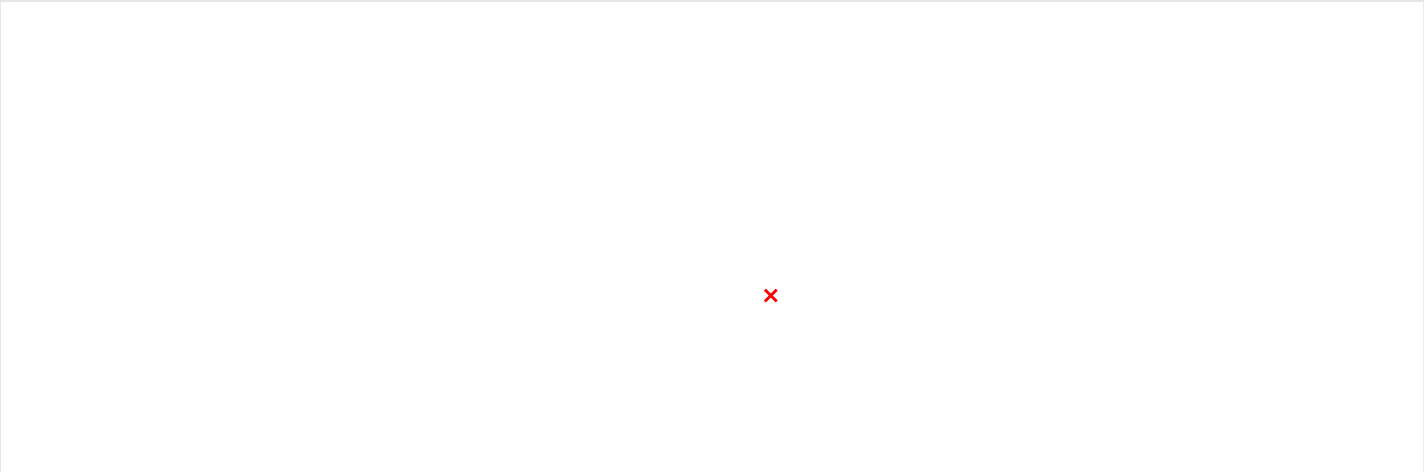


MQTT-QoS-cleanSession-false



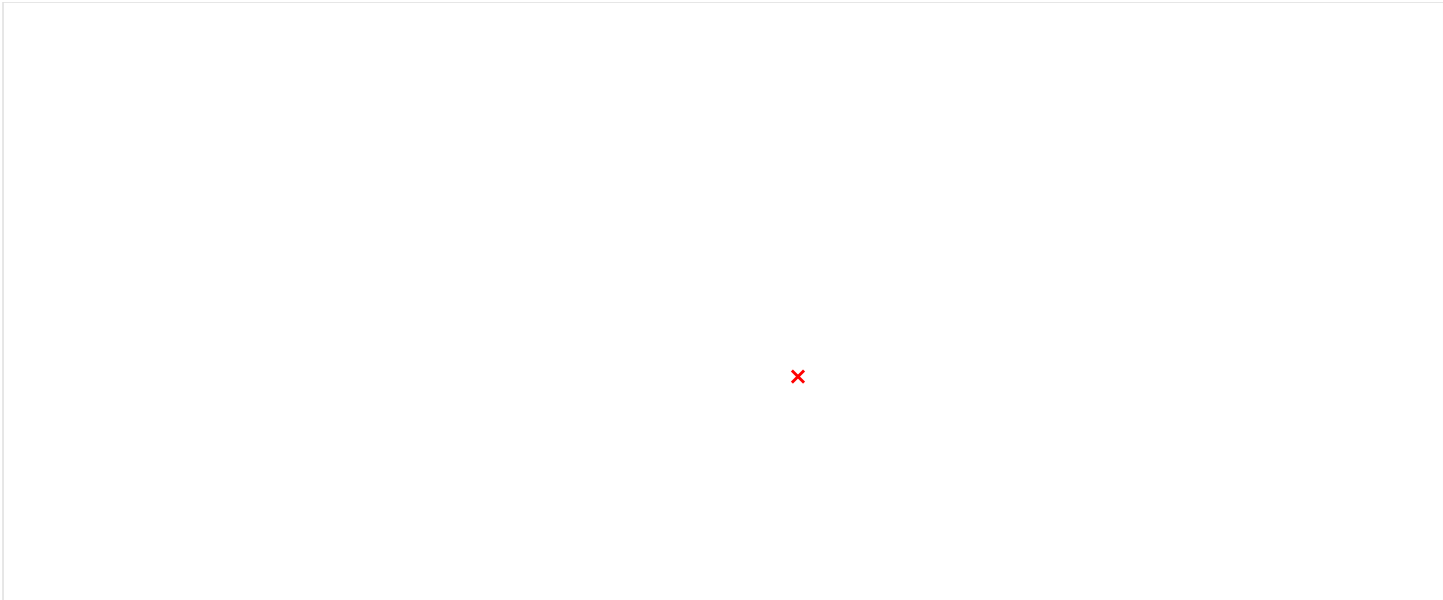
服务质量降级

讲到这里，不知道有没有朋友会感到好奇。假如客户端在发布和订阅信息时使用不同级别的QoS，将会发生什么情况呢。如下图所示，假如客户端A发布到主题1的消息是采用QoS = 2，然而客户端B订阅主题1采用QoS = 1。那么服务端该如何来应对这一情况呢？



MQTT-QoS-设置-1

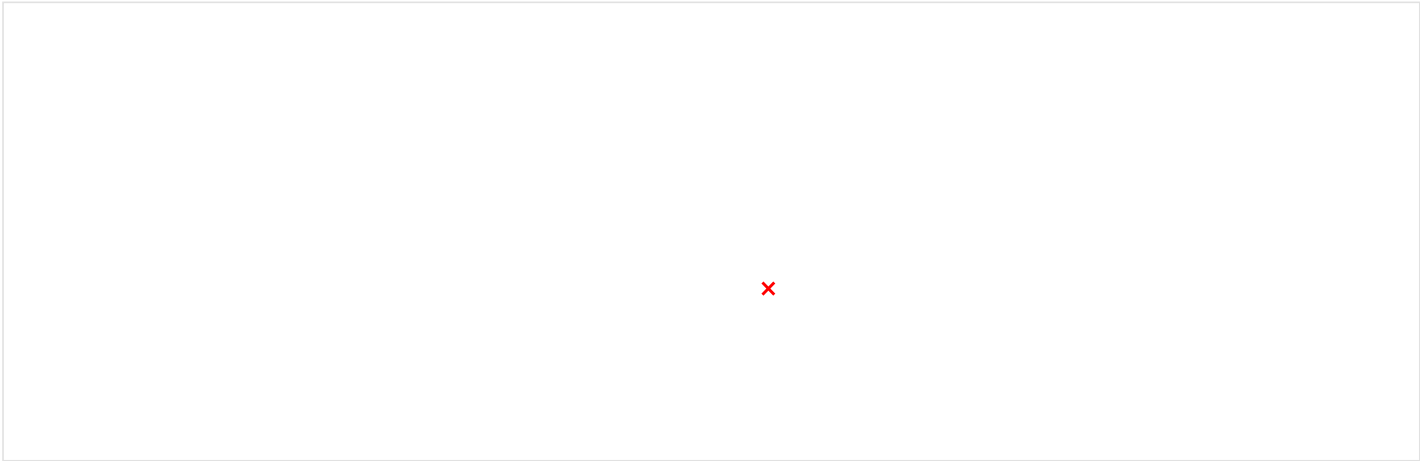
在这种情况下，服务端会使用较低级别来提供服务。如下图所示，虽然A发送到主题1的消息采用QoS为2，但是服务端发送主题1的消息给B时，采用的QoS为1。这是因为B在订阅主题1时采用的QoS为1。



MQTT-QoS-设置-2

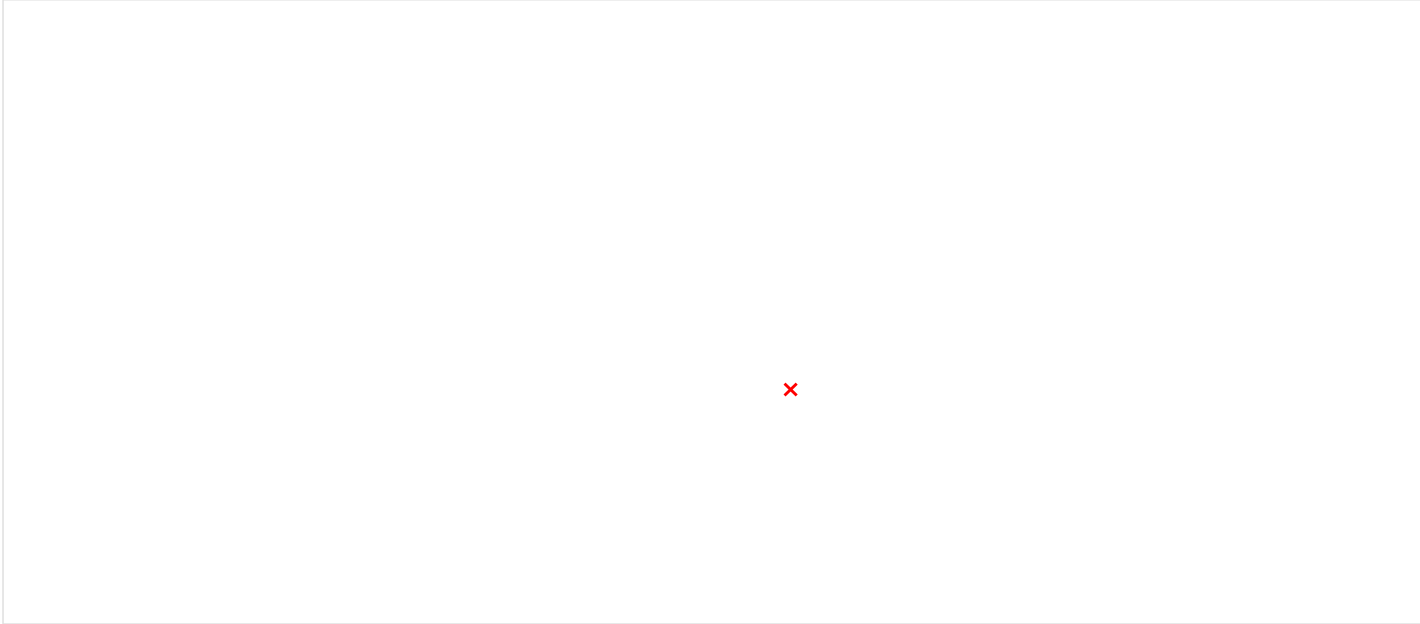
下面我们再看一种情况。

如下图所示，假如客户端A发布主题1消息时使用QoS为0，而客户端B订阅主题1消息时使用QoS为1。



MQTT-QoS-设置

在这种情况下，虽然客户端B订阅主题1消息时QoS为1，但是由于客户端A发送主题1消息时QoS为0，所以服务端发送消息给B的QoS为0。



通过以上两个示例我们可以看到。对于发布和订阅消息的客户端，服务端会主动采用较低级别的QoS来实现消息传输。



QoS=1通讯时的注意事项

如想在MQTT通讯中实现服务质量等级为1级（QoS=1），我们要分别对消息的发布端接收端进行相应的设置。以下列表中的内容是具体需要采取的措施。

- 接收端连接服务端时cleanSession设置为false
- 接收端订阅主题时QoS=1
- 发布端发布消息时QoS=1



QoS=2通讯时的注意事项

如想在MQTT通讯中实现服务质量等级为2级（QoS=2），我们要分别对消息的发布端和接收端进行相应的设置。以下列表中的内容是具体需要采取的措施。

- 接收端连接服务端时cleanSession设置为false
- 接收端订阅主题时QoS=2
- 发布端发布消息时QoS=2



小结

- 若想实现QoS>0，订阅端连接服务端时cleanSession需要设置为false，订阅端订阅主题时QoS>0，发布端发布消息时的QoS>0。
- 服务端会选择发布消息和订阅消息中较低的QoS来实现消息传输，这也被称作“服务降级”。
- QoS = 0，占用的网络资源最低，但是接收端可能会出现无法接收消息的情况，所以适用于传输重要性较低的信息。
- QoS = 1，MQTT会确保接收端能够接收到消息，但是有可能出现接收端反复接收同一消息的情况。
- QoS = 2，MQTT会确保接收端只接收到一次消息。但是QoS为2时消息传输最慢，另外消息传输需要多次确认，因此所占用的网络资源也是最多的。此类服务等级适用于重要消息传输。
- 由于QoS1和QoS2都能确保客户端接收到消息，但是QoS1所占用的资源较QoS2占用资源更小。因此建议使用QoS1来实现网络资源较为珍贵的环境下传输重要信息。



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-2 ESP8266 QoS 应用

星期日, 三月 13, 2022 5:34 下午

已剪辑自: https://www.bilibili.com/read/cv8737107?spm_id_from=333.999.0.0

为了帮助您学习, 我们专门为本教程制作了视频。



在本节课里, 我们将使用ESP8266配合MQTTfx客户端软件, 通过实际操作来看一下MQTT的QoS是如何发挥作用的。

在开始本节课以前请确保您的电脑已经成功安装了MQTTfx软件。另外我们还将使用PubSubClient库。您可以通过以下链接获取此库。

- 官网地址: <https://pubsubclient.knolleary.net/>
- GitHub: <https://github.com/knolleary/pubsubclient/>
- 百度网盘下载: <https://pan.baidu.com/s/12MHGbdfi0dwOGip5RMSSEQ> 提取码: sizzy

PubSubClient库目前只支持1级QoS订阅, 因此我们将仅介绍如何使用ESP8266通过QoS=1模式订阅和接收MQTT消息。



使用ESP8266接收QoS=1的MQTT消息

要使用QoS=1订阅MQTT消息, 需要满足以下要求:

- 1 接收端要有确定的clientID
- 2 接收端连接服务器时 cleanSession=False
- 3 发送端发布消息时 QoS=1或QoS=2
- 4 接收端订阅消息时 QoS=1

通过以下示例程序, 您将可以实现ESP8266 QoS1订阅主题。如需了解本程序的详细使用方法和内容, 请参考本文首部的视频。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_qos1

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20200813

程序目的/Purpose :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器订阅信息。

订阅QoS级别为1。

- 在此程序控制下，ESP8266启动后将会尝试连接MQTT服务端。

连接时cleanSession=false。

- 接下来ESP8266在订阅主题时，将QoS设置为1。

- 保持ESP8266在线，使用MQTTfx向ESP8266所订阅的主题发布QoS=1的信息。

- ESP8266将会收到信息（至此还没有体现QoS=1的优势）

- 将ESP8266断电，然后再次使用MQTTfx向ESP8266订阅主题发布QoS=1的信息。

此时由于ESP8266未通电，所以无法接收到MQTTfx发送的信息。因此MQTT服务端将会保存此信息。

- 将ESP8266再次通电，ESP8266连接到MQTT服务端后，将会马上收到了MQTTfx在ESP8266断电时所发送的信息。（这就是QoS=1的优势，即客户端断电再通电后依然可以收到QoS=1信息。）

要使用QoS=1订阅MQTT消息，需要满足以下要求：

- 1 接收端要有确定的clientID
- 2 接收端连接服务器时 cleanSession=False
- 3 发送端发布消息时 QoS=1或QoS=2
- 4 接收端订阅消息时 QoS=1

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```

#include <PubSubClient.h>

// 设置wifi接入信息(请根据您的WiFi信息进行修改)

const char* ssid = "taichi-maker";

const char* password = "12345678";

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接, 请前往以下页面寻找解决方案
// http://www.taichi-maker.com/public-mqtt-broker/

WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);

const int subQoS = 1;    // 客户端订阅主题时使用的QoS级别 (截止
2020-10-07, 仅支持QoS = 1, 不支持QoS = 2)

const bool cleanSession = false; // 清除会话 (如QoS>0必须要设为false)

const char* willTopic = "willTopic"; // 遗嘱主题名称

const char* willMsg = "willMsg";    // 遗嘱主题信息

const int willQos = 0;                // 遗嘱QoS

const int willRetain = false;         // 遗嘱保留

void setup() {

    pinMode(LED_BUILTIN, OUTPUT);    // 设置板上LED引脚为输出模式

    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED

    Serial.begin(9600);               // 启动串口通讯

    //设置ESP8266工作模式为无线终端模式

    WiFi.mode(WIFI_STA);

```

```

// 连接WiFi

connectWifi();


// 设置MQTT服务器和端口号

mqttClient.setServer(mqttServer, 1883);
mqttClient.setCallback(receiveCallback);


// 连接MQTT服务器

connectMQTTserver();
}


void loop() {

    // 如果开发板未能成功连接服务器，则尝试连接服务器

    if (!mqttClient.connected()) {

        connectMQTTserver();

    }


    // 处理信息以及心跳

    mqttClient.loop();

}


// 连接MQTT服务器并订阅信息

void connectMQTTserver() {

    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）

    String clientId = "client-" + WiFi.macAddress();

```

```
/* 连接MQTT服务器
```

```
boolean connect(const char* id, const char* user,  
  
               const char* pass, const char* willTopic,  
  
               uint8_t willQos, boolean willRetain,  
  
               const char* willMessage, boolean cleanSession);
```

若让设备在离线时仍然能够让qos1工作，则connect时的cleanSession需要设置为false

```
*/
```

```
if (mqttClient.connect(clientId.c_str(), NULL, NULL, willTopic, willQos,  
willRetain, willMsg, cleanSession)) {
```

```
    Serial.print("MQTT Server Connected. ClientId: ");
```

```
    Serial.println(clientId);
```

```
    subscribeTopic(); // 订阅指定主题
```

```
} else {
```

```
    Serial.print("MQTT Server Connect Failed. Client State:");
```

```
    Serial.println(mqttClient.state());
```

```
    delay(5000);
```

```
}
```

```
}
```

```
// 收到信息后的回调函数
```

```
void receiveCallback(char* topic, byte* payload, unsigned int length) {
```

```
    Serial.print("Message Received [");
```

```
    Serial.print(topic);
```

```
    Serial.print("] ");
```

```
    for (int i = 0; i < length; i++) {
```

```
        Serial.print((char)payload[i]);
```

```
    }
```

```
    Serial.println("");
```

```

Serial.print("Message Length(Bytes) ");

Serial.println(length);

if ((char)payload[0] == '1') {    // 如果收到的信息以“1”为开始
    digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
} else {
    digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。
}
}

// 订阅指定主题

void subscribeTopic() {
    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

    // 请注意subscribe函数第二个参数数字为QoS级别。这里为QoS = 1

    if(mqttClient.subscribe(subTopic, subQoS)) {

        Serial.print("Subscribed Topic: ");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }
}

```



```
}

// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}
```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。
您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-2 ESP8266 QoS 应用

星期日，三月 13，2022 5:34 下午

已剪辑自: https://www.bilibili.com/read/cv8737107?spm_id_from=333.999.0.0

为了帮助您学习，我们专门为本教程制作了视频。

↓

在本节课里，我们将使用ESP8266配合MQTTfx客户端软件，通过实际操作来看一下MQTT的QoS是如何发挥作用的。

在开始本节课以前请确保您的电脑已经成功安装了MQTTfx软件。另外我们还将使用PubSubClient库。您可以通过以下链接获取此库。

- 官网地址: <https://pubsubclient.knolleary.net/>
- GitHub: <https://github.com/knolleary/pubsubclient/>
- 百度网盘下载: <https://pan.baidu.com/s/12MHGbdfi0dwOGip5RMSSEQ> 提取码: sizzy

PubSubClient库目前只支持1级QoS订阅，因此我们将仅介绍如何使用ESP8266通过QoS=1模式订阅和接收MQTT消息。



使用ESP8266接收QoS=1的MQTT消息

要使用QoS=1订阅MQTT消息，需要满足以下要求：

- 1 接收端要有确定的clientID
- 2 接收端连接服务器时 cleanSession=False
- 3 发送端发布消息时 QoS=1或QoS=2
- 4 接收端订阅消息时 QoS=1

通过以下示例程序，您将可以实现ESP8266 QoS1订阅主题。如需了解本程序的详细使用方法和内容，请参考本文首部的视频。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : subscribe_qos1

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20200813

程序目的/Purpose :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器订阅信息。

订阅QoS级别为1。

- 在此程序控制下，ESP8266启动后将会尝试连接MQTT服务端。

连接时cleanSession=false。

- 接下来ESP8266在订阅主题时，将QoS设置为1。

- 保持ESP8266在线，使用MQTTfx向ESP8266所订阅的主题发布QoS=1的信息。

- ESP8266将会收到信息（至此还没有体现QoS=1的优势）

- 将ESP8266断电，然后再次使用MQTTfx向ESP8266订阅主题发布QoS=1的信息。

此时由于ESP8266未通电，所以无法接收到MQTTfx发送的信息。因此MQTT服务端将会保存此信息。

- 将ESP8266再次通电，ESP8266连接到MQTT服务端后，将会马上收到了MQTTfx在ESP8266断电时所发送的信息。（这就是QoS=1的优势，即客户端断电再通电后依然可以收到QoS=1信息。）

要使用QoS=1订阅MQTT消息，需要满足以下要求：

- 1 接收端要有确定的clientID
- 2 接收端连接服务器时 cleanSession=False
- 3 发送端发布消息时 QoS=1或QoS=2
- 4 接收端订阅消息时 QoS=1

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```

#include <PubSubClient.h>

// 设置wifi接入信息(请根据您的WiFi信息进行修改)

const char* ssid = "taichi-maker";

const char* password = "12345678";

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接, 请前往以下页面寻找解决方案
// http://www.taichi-maker.com/public-mqtt-broker/

WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);

const int subQoS = 1;    // 客户端订阅主题时使用的QoS级别 (截止
2020-10-07, 仅支持QoS = 1, 不支持QoS = 2)

const bool cleanSession = false; // 清除会话 (如QoS>0必须要设为false)

const char* willTopic = "willTopic"; // 遗嘱主题名称

const char* willMsg = "willMsg";    // 遗嘱主题信息

const int willQos = 0;                // 遗嘱QoS

const int willRetain = false;         // 遗嘱保留

void setup() {

    pinMode(LED_BUILTIN, OUTPUT);    // 设置板上LED引脚为输出模式

    digitalWrite(LED_BUILTIN, HIGH); // 启动后关闭板上LED

    Serial.begin(9600);               // 启动串口通讯

    //设置ESP8266工作模式为无线终端模式

    WiFi.mode(WIFI_STA);

```

```

// 连接WiFi

connectWifi();


// 设置MQTT服务器和端口号

mqttClient.setServer(mqttServer, 1883);
mqttClient.setCallback(receiveCallback);


// 连接MQTT服务器

connectMQTTserver();
}


void loop() {

    // 如果开发板未能成功连接服务器，则尝试连接服务器

    if (!mqttClient.connected()) {

        connectMQTTserver();

    }


    // 处理信息以及心跳

    mqttClient.loop();

}


// 连接MQTT服务器并订阅信息

void connectMQTTserver() {

    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）

    String clientId = "client-" + WiFi.macAddress();

```

```
/* 连接MQTT服务器
```

```
boolean connect(const char* id, const char* user,  
  
               const char* pass, const char* willTopic,  
  
               uint8_t willQos, boolean willRetain,  
  
               const char* willMessage, boolean cleanSession);
```

若让设备在离线时仍然能够让qos1工作，则connect时的cleanSession需要设置为false

```
*/
```

```
if (mqttClient.connect(clientId.c_str(), NULL, NULL, willTopic, willQos,  
willRetain, willMsg, cleanSession)) {
```

```
    Serial.print("MQTT Server Connected. ClientId: ");
```

```
    Serial.println(clientId);
```

```
    subscribeTopic(); // 订阅指定主题
```

```
} else {
```

```
    Serial.print("MQTT Server Connect Failed. Client State:");
```

```
    Serial.println(mqttClient.state());
```

```
    delay(5000);
```

```
}
```

```
}
```

```
// 收到信息后的回调函数
```

```
void receiveCallback(char* topic, byte* payload, unsigned int length) {
```

```
    Serial.print("Message Received [");
```

```
    Serial.print(topic);
```

```
    Serial.print("] ");
```

```
    for (int i = 0; i < length; i++) {
```

```
        Serial.print((char)payload[i]);
```

```
    }
```

```
    Serial.println("");
```



```

Serial.print("Message Length(Bytes) ");

Serial.println(length);

if ((char)payload[0] == '1') {    // 如果收到的信息以“1”为开始
    digitalWrite(BUILTIN_LED, LOW); // 则点亮LED。
} else {
    digitalWrite(BUILTIN_LED, HIGH); // 否则熄灭LED。
}
}

// 订阅指定主题

void subscribeTopic() {
    // 建立订阅主题。主题名称以Taichi-Maker-Sub为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同设备使用同一个MQTT服务器测试消息订阅时，所订阅的主题名称不同

    String topicString = "Taichi-Maker-Sub-" + WiFi.macAddress();

    char subTopic[topicString.length() + 1];

    strcpy(subTopic, topicString.c_str());

    // 通过串口监视器输出是否成功订阅主题以及订阅的主题名称

    // 请注意subscribe函数第二个参数数字为QoS级别。这里为QoS = 1

    if(mqttClient.subscribe(subTopic, subQoS)) {

        Serial.print("Subscribed Topic: ");

        Serial.println(subTopic);

    } else {

        Serial.print("Subscribe Fail...");

    }
}

```

```
}

// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}
```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。
您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-3 保留消息 - 哔哩哔哩

星期日, 三月 13, 2022 5:35 下午

已剪辑自: https://www.bilibili.com/read/cv8875311?spm_id_from=333.999.0.0

为了帮助您学习, 我们专门为本教程制作了视频。



“保留消息”是十分重要的MQTT概念。通过“保留消息”这一名称不难判断, “保留消息”是一种被保留下来的消息。但是这个“保留消息”为何要被保留? 而保留消息又是有什么特殊的用途? 这些答案我们将在本节课揭晓。

保留消息的作用

要讲明“保留消息”这一概念, 我们先看一个场景。假设我们正在利用MQTT协议开发一套智能家居物联网系统。在该系统中有一台专门用于检测和发布室温信息的MQTT客户端, 它每到整点时就会测量当前室温并且向MQTT服务端发布室温测量结果。

假设在该智能家居物联网系统中, 还有一台环境信息显示客户端。这台客户端的作用就是把当前的室温显示在屏幕上以便我们实时了解室内温度。换句话说, 这台环境信息显示客户端一启动就会订阅室温主题, 这样室温检测客户端一发布消息, 显示客户端就能获取到最新的温度消息并显示在屏幕上了。

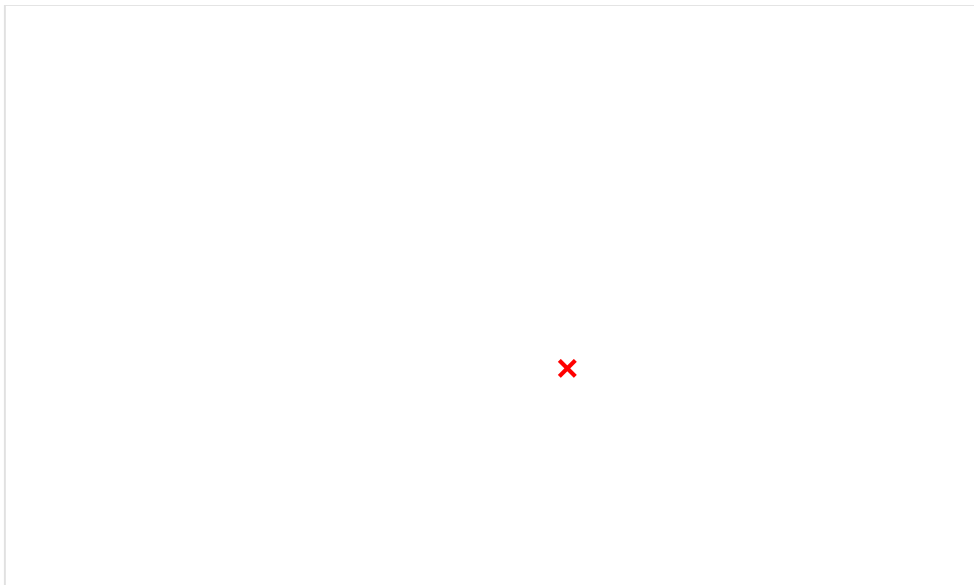
假设某天上午7:00, 我们的室温检测客户端将最新的室温消息发布到了服务端, 那么订阅了室温消息的显示客户端也就马上获取到室温消息并且显示在屏幕上。

然而在7:10的时候, 家里的小狗不小心把显示客户端的电源碰掉了, 显示客户端没有电也就自动关机了。我们发现这一问题后, 马上把显示客户端重新通电, 客户端通电启动后会立刻订阅室温主题。

但这时候问题出现了, 室温测量客户端每到整点才发布一次温度信息。上一次发布时间是7:00, 下一次发布时间是8:00。所以, 尽管显示客户端订阅了室温主题, 它还要等到8:00钟才能收到最新室温消息。在8:00前的几十分钟里, 显示客户端无法获知当前室温信息, 也就无法将室温信息显示在屏幕上供我们查阅。

为了避免以上情况出现, 我们可以让室温测量客户端在每次向室温主题发布消息时都使用“保留消息”这一模式将温度信息发布到服务端。这样无论显示客户端在任何时间订阅室温主题, 都会马上收到该主题中的“保留消息”, 也就是温度测量客户端发布的最新室温消息。

发布保留消息的方法



发布MQTT保留消息Publish报文

MQTT设备发布的保留消息的流程与发布普通消息的流程十分类似。唯一区别是，在发布保留消息时，MQTT设备需要将PUBLISH报文中retainFlag设置为true（如上图所示）。

当然，如果要发布非保留消息，那么PUBLISH报文中retainFlag设置为false。

修改保留消息的方法

每一个主题只能有一个“保留消息”，如果客户端想要更新“保留消息”，就需要向该主题发送一条新的“保留消息”，这样服务端会将新的“保留消息”覆盖旧的“保留消息”。当有客户端订阅该主题时，服务端就会将最新的“保留消息”发送给订阅客户端了。

删除保留消息的方法

如果要删除主题的“保留消息”，可以通过向该主题发布一条空的“保留消息”，也就是发送一条0字节payload的“保留消息”



假如您也喜欢太极创客，希望您将本文“分享”给更多的人，让更多朋友认识我们。您的每一份点赞与投币，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔（<https://space.bilibili.com/415252426/>）

2-4 ESP8266 保留消息应用 - 哔哩哔哩

星期日, 三月 13, 2022 5:35 下午

已剪辑自: https://www.bilibili.com/read/cv8901903?spm_id_from=333.999.0.0

在本节课里, 我们将向您讲解如何使用ESP8266发布保留消息。和以往课程一样, 我们还将使用PubSubClient库来实现这一功能。

在开始本节课以前请确保您的电脑已经成功安装了MQTTfx软件。另外我们还将使用PubSubClient库。您可以通过以下链接获取此库。

- 官网地址: <https://pubsubclient.knolleary.net/>
- GitHub: <https://github.com/knolleary/pubsubclient/>
- 百度网盘下载: <https://pan.baidu.com/s/12MHGbdfi0dwOGip5RMSSEQ> 提取码: sizzy

以下是使用ESP8266实现保留消息发布的示例程序, 关于本程序的使用方法以及说明信息, 请收看本文首部的视频教程。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : publish_retained_msg

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20201014

程序目的/Purpose :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器发布保留信息。

此程序在a_publish_ranye_url程序基础上修改。重点修改内容是publish函数添加了第三个参数, 用于设置发布信息是否是保留信息(retained msg)

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息, 请参考以下网页:

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichi-maker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    //设置ESP8266工作模式为无线终端模式
```

```
    WiFi.mode(WIFI_STA);
```

```
    // 连接WiFi
```

```
    connectWifi();
```

```
    // 设置MQTT服务器和端口号
```

```
    mqttClient.setServer(mqttServer, 1883);
```



```

// 连接MQTT服务器

connectMQTTServer();

if (mqttClient.connected()) { // 如果开发板成功连接服务器
    pubRetMQTTmsg();          // 发布信息
}
}

void loop() {
    mqttClient.loop();         // 保持心跳
}

void connectMQTTServer() {
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器
    if (mqttClient.connect(clientId.c_str())) {
        Serial.println("MQTT Server Connected.");
        Serial.println("Server Address: ");
        Serial.println(mqttServer);
        Serial.println("ClientId:");
        Serial.println(clientId);
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(3000);
    }
}

```

```

    }
}

// 发布信息

void pubRetMQTTmsg() {

    // 建立发布主题。主题名称以Taichi-Maker-为前缀，后面添加设备的MAC地址。
    // 这么做是为确保不同用户进行MQTT信息发布时，ESP8266客户端名称各不相同，
    String topicString = "Taichi-Maker-Ret-" + WiFi.macAddress();

    char publishTopic[topicString.length() + 1];
    strcpy(publishTopic, topicString.c_str());

    // 建立即将发布的保留消息。消息内容为"Retained Msg"。
    String messageString = "Retained Msg";

    char publishMsg[messageString.length() + 1];
    strcpy(publishMsg, messageString.c_str());

    // 实现ESP8266向主题发布retained信息
    // 以下publish函数第三个参数用于设置保留信息 (retained message)
    if(mqttClient.publish(publishTopic, publishMsg, true)){

        Serial.println("Publish Topic:");Serial.println(publishTopic);

        Serial.println("Publish Retained
message:");Serial.println(publishMsg);

    } else {

        Serial.println("Message Publish Failed.");

    }

}

// ESP8266连接wifi

```

```
void connectWifi() {  
  
    WiFi.begin(ssid, password);  
  
    //等待WiFi连接, 成功连接后输出成功信息  
  
    while (WiFi.status() != WL_CONNECTED) {  
  
        delay(1000);  
  
        Serial.print(".");  
  
    }  
  
    Serial.println("");  
  
    Serial.println("WiFi Connected!");  
  
    Serial.println("");  
  
}
```



假如您也喜欢**太极创客**，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。
您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-5 心跳机制

星期日，三月 13，2022 5:35 下午

已剪辑自: https://www.bilibili.com/read/cv8921110?spm_id_from=333.999.0.0

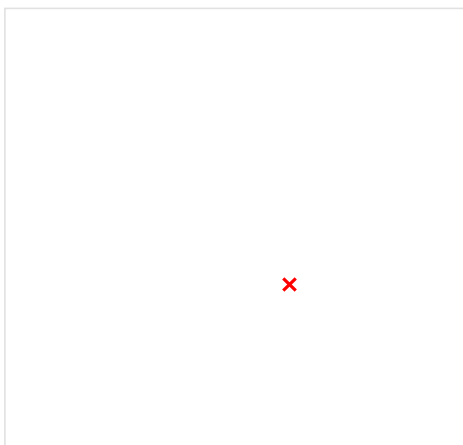
为了帮助您学习，我们专门为本教程制作了视频。

↓

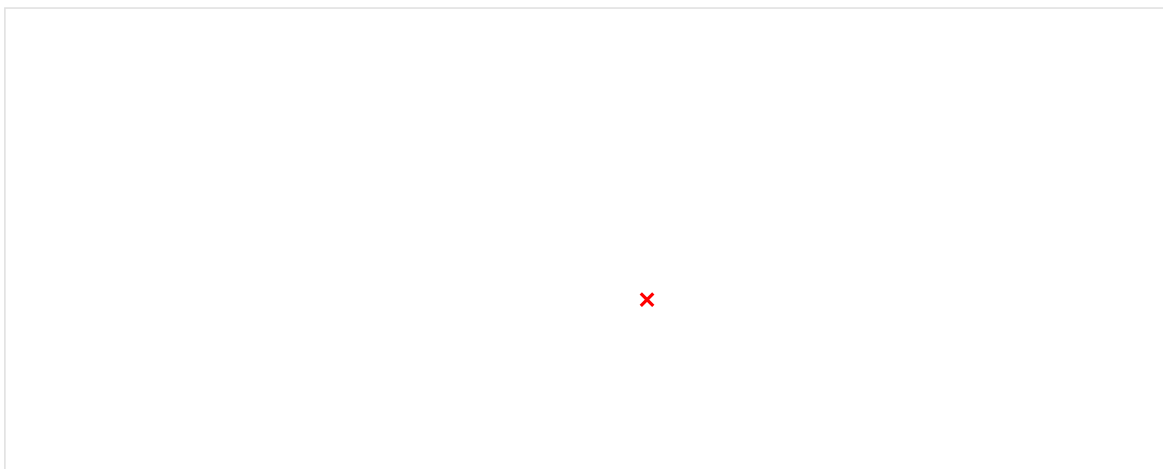
在医院里，医生利用心跳来判断患者是否还有生命体征。对于MQTT服务器来说，它要判断一台MQTT客户端是否依然保持连接可以检查这台客户端是不是经常发送消息给服务端。如果服务端经常收到客户端的消息，那么没问题，这个客户端肯定在线。

但是有些客户端并不经常发送消息给服务端。对于这种客户端，服务端可以使用类似心跳检测的方法，来判断客户端是否在线。

GIF



不过客户端设备没有心脏，自然不会跳动。所以MQTT协议为它们配上一个类似心脏的机制，这个心脏机制就是让客户端在没有向服务端发送信息时，可以定时向服务端发送一条消息。这条用于心跳机制的消息也被称作心跳请求（PINGREQ）。



心跳请求的作用正是用于告知服务端，当前客户端依然在线。服务端在收到客户端的心跳请求后，会回复一条消息。这条回复消息被称作心跳响应（PINGRESP）。

由于心跳请求是客户端定时发送的，一旦服务端发现客户端停止发送请求信息，那么服务端就会知道，这台客户端已经断开了连接。

这个心跳机制不仅可以用于服务端判断客户端是否保持连接，也可以用于客户端判断自己与服务端是否保持连接。如果客户端在发送心跳请求（PINGREQ）后，没有收到服务端的心跳响应（PINGRESP），那么客户端就会认为自己与服务端的连接已经被断开了。

以上是心跳机制(Keep Alive)的简单介绍，为了更深入的了解心跳机制，我们来看一下MQTT客户端在连接服务端的过程。请见下图，客户端连接服务端时会像服务端发送CONNECT报文。



我刚刚给大家讲过，在心跳机制中，客户端要定时向服务端发送心跳请求(PINGREQ)报文。那么客户端发送心跳请求的时间间隔是多少呢？

这个心跳时间间隔是我们在开发客户端时进行设置的。假如我们使用ESP8266开发板作为物联网客户端，那么我们在编写控制程序时，会在程序中对心跳时间间隔进行设置。

设置好心跳时间间隔后，客户端就知道多久要发送一条心跳请求给服务端。但是这里存在一个问题。光是客户端知道心跳时间间隔还不够，服务端也需要知道客户端的心跳时间间隔，这样服务端才能定时检查客户端的心跳请求消息。

因此，在客户端连接服务端时，会将心跳时间间隔信息放入CONNECT报文。也就是上图中最后一行的信息keepAlive。这个keepAlive正是用于告知服务端心跳时间间隔的。

以上示例图中我们看到keepAlive数值为60。这就意味着，**客户端的心跳间隔时间是60秒。**

接下来我要给您讲解的内容十分关键，请您务必留意。

在继续后面的学习以前我们先来问您个问题。**假如客户端的心跳间隔时间是60秒，那么服务端是不是每隔60秒就检查一次客户端是否发来心跳请求呢？**

我们本节课刚开始的时候曾给大家介绍过，如果客户端在心跳时间间隔内发布了消息给服务端，那么服务端不需要客户端发送心跳请求也可以确定该客户端肯定在线。



但是当客户端在心跳间隔内没有发布消息给服务端，这时客户端会主动发送一个心跳请求消息给服务端。以表明自己仍让在线



简而言之，客户端在心跳间隔时间内，如果有消息发布，那就不发布心跳请求。但是在心跳间隔时间内，客户端没有消息发布，那么它就会发布一条心跳请求给服务端，这个心跳请求的目的就是为了告诉服务端，“我还在线，你放心吧。”

另外，在实际运行中，如果服务端没有在1.5倍心跳时间间隔内收到客户端发布消息(PUBLISH)或发来心跳请求(PINGREQ)，那么服务端就会认为这个客户端已经掉线。

举例来说，如果心跳时间间隔是60秒。那么服务端在90秒内没有收到客户端发布的信息也没有收到PINGREQ请求，那么它就会认为客户端已经掉线。

另外，心跳机制不仅仅用于服务端判断客户端是否在线。客户端也可以利用这一机制来判断自己是否与服务端仍保持连接。如果客户端发送了心跳请求(PINGREQ)给服务端一段时间后，仍然没有收到服务端回复的心跳确认。那么客户端也会认为自己已经断开了与服务端的连接。

了解了MQTT心跳机制后，不知道您有没有想过，如果服务端知道了某一台客户端已经掉线，它会采取什么措施吗？关于这个问题答案，我们将在下一节课为您讲解。敬请关注~

2-6 MQTT遗嘱

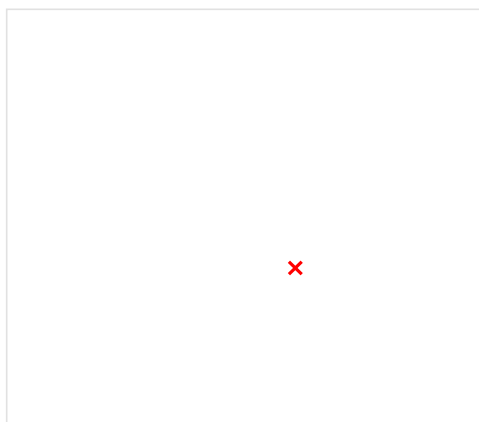
星期日，三月 13，2022 5:36 下午

已剪辑自: https://www.bilibili.com/read/cv8967394?spm_id_from=333.999.0.0

为了帮助您学习，我们专门为本教程制作了视频。

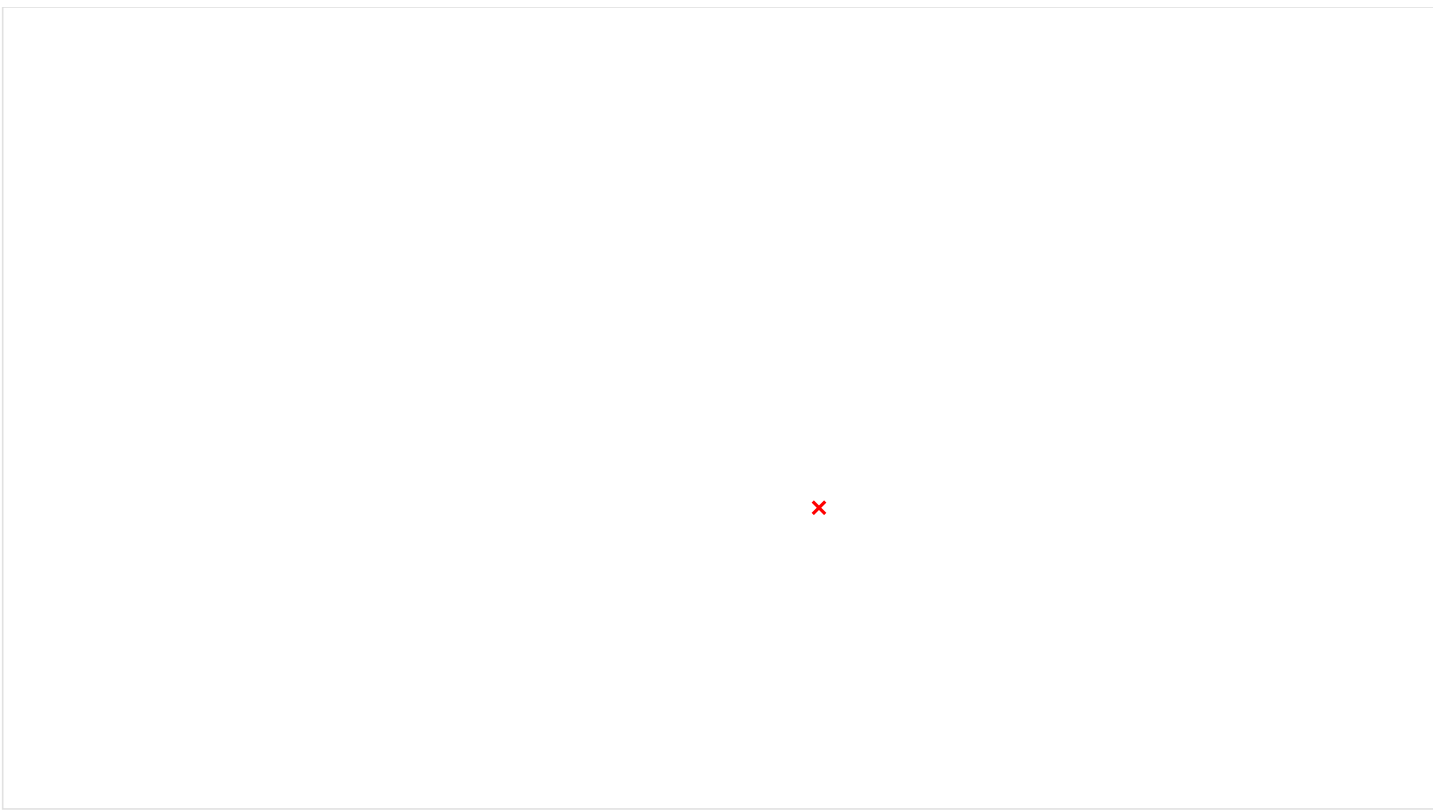
↓

在之前的课程里，我们学习了MQTT协议的心跳机制。该机制可以让服务端随时掌握客户端连接情况。当客户端“心跳”正常时，服务端即知道客户端仍然在线（活着）。当心跳一旦停止，服务端就会发现该客户端已经断线（死亡）。



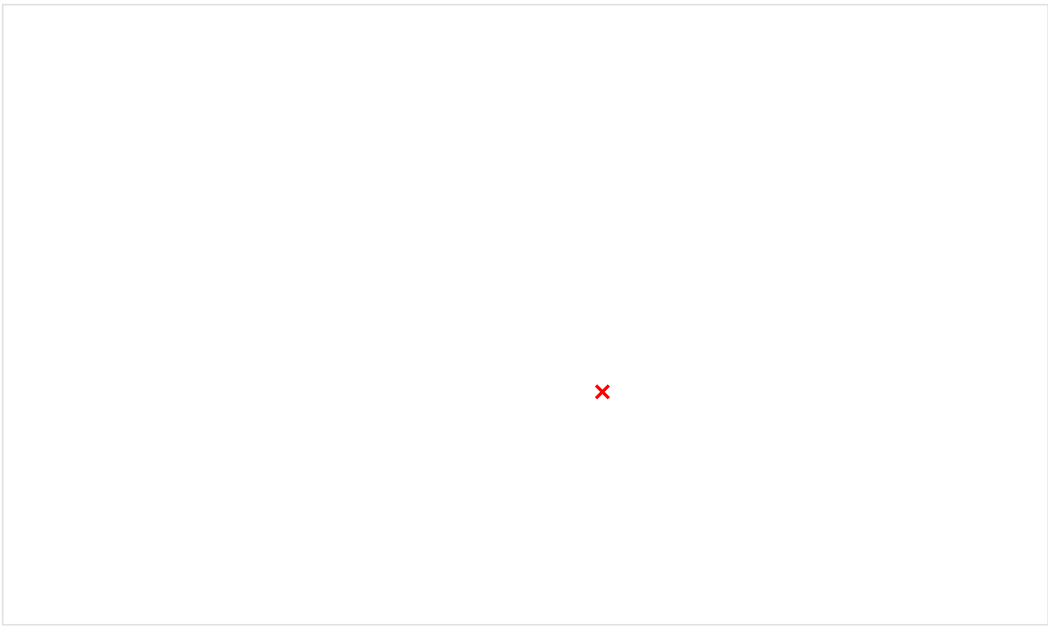
为了让客户端可以更好的发挥作用，便于服务端管理，MQTT协议允许客户端在“活着”的时候就写好遗嘱，这样一旦客户端意外断线，服务端就可以将客户端的遗嘱公之于众。

请注意，在上面的这段话中，我将意外断线这几个字特意做了标注，这是因为，客户端的遗嘱只在意外断线时才会发布，如果客户端正常的断开了与服务端的连接，这个遗嘱机制是不会启动的，服务端也不会将客户端的遗嘱公布。



那么什么是意外断线呢？

当客户端正常断开连接时，会向服务端发送DISCONNECT报文，服务端接收到该报文后，就知道，客户端是正常断开连接，而并非意外断开连接。

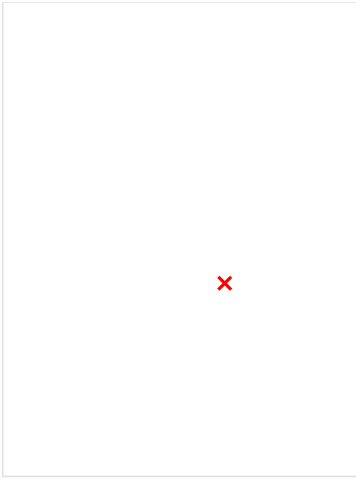


MQTT断开连接报文

然而，当服务端在**没有收到DISCONNECT报文**的情况下，发现客户端“心跳”停止了，这时服务端就知道客户端是意外断线了。

那究竟是什么原因会导致客户端意外断线呢？我们知道MQTT协议作为物联网协议可能用于不稳定的网络环境，假如客户端的网络信号突然出现问题，就会导致意外断线。

另外，有些客户端设备使用电池供电，当电池没电时，也会出现意外断网的情况。当然，造成意外断网的情况还有很多，以上是较为常见的情况。



通过以上讲解，我们了解了客户端遗嘱的作用，接下来我们仔细看一下关于客户端遗嘱的几个重要话题：

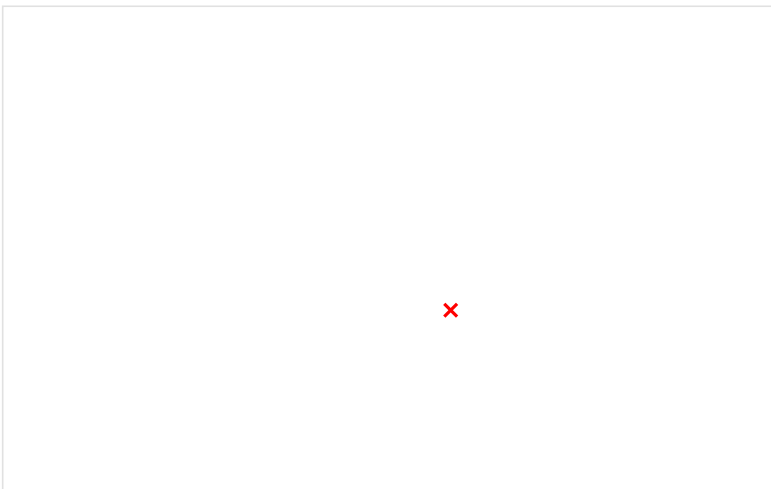
1. 客户端如何将遗嘱消息发送给服务端
2. MQTT遗嘱使用建议

1. 客户端如何将遗嘱消息发送给服务端

在之前的课程里我为大家讲解了MQTT客户端是如何连接服务端的。

MQTT客户端要想连接服务端，首先要向服务端发送CONNECT报文。下图是CONNECT报文所包含的信息内容。

GIF



MQTT CONNECT报文中的遗嘱信息

在上图的CONNECT报文中，我特意使用红色对一系列信息进行了标注。客户端正是在连接服务端时，利用这一系列信息将遗嘱消息发送给服务端。下面我们来分别看一下这些信息的作用。

lastWillTopic - 遗嘱主题

遗嘱消息和普通MQTT消息很相似，也有主题和正文内容。lastWillTopic的作用正是告知服务端，本客户端的遗嘱主题是什么。只有那些订阅了这一遗嘱主题的客户端才会收到本客户端的遗嘱消息。

以上图为例，此遗嘱主题为”hans/will”。也就是说，只有订阅了主题”hans/will”的客户端，才会收到这台客户端的遗嘱消息。

lastWillMessage - 遗嘱消息

遗嘱消息定义了遗嘱消息内容。在本示例中，那些订阅了主题”hans/will”的客户端会在客户端意外断线时，收到服务端发布的“unexpected exit”。

lastWillQoS - 遗嘱QoS

在之前的课程里，我们学习了服务质量的概念。对于遗嘱消息来说，同样可以使用服务质量来控制遗嘱消息的传递和接收。这里的服务质量与普通MQTT消息的服务质量是一样的概念。也可以设置为0、1、2。对于不同的服务质量级别，服务端会使用不同的服务质量来发布遗嘱消息。

lastWillRetain - 遗嘱保留

遗嘱消息也可以设置为保留消息，关于保留消息的具体内容，我们在之前的课程中曾经学习过。遗嘱保留用于设置遗嘱消息是否需要进行保留处理。服务端会根据此处内容，对遗嘱消息进行相应的保留与否处理。

2. MQTT遗嘱操作建议

在使用MQTT遗嘱时，我们建议您通过以下方法让设备的MQTT遗嘱机制可以更好的发挥作用。

假设我们现在有一台MQTT客户端。它的client id是 client-1。它的遗嘱主题是“client-1-will”

1. 当client-1连接服务端时，CONNECT报文中的遗嘱消息是“offline”。并且它的遗嘱保留设置为true。
2. 当client-1成功连接服务端后，立即向遗嘱主题“client-1-will”发布一条消息“online”。同时在发布此消息时，保留标志设置为true。这样，只要client-1在线，那么任何设备一订阅“client-1-will”就能收到设备在线的消息“online”。
3. 如果client-1发生意外离线。那么任何设备一订阅“client-1-will”就会收到设备离线的消息“offline”。
4. 如果client-1恢复连接，那么它会将遗嘱主题“client-1-will”的保留消息更改为“online”，这样任何设备一订阅“client-1-will”就能收到设备在线的消息“online”。



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-7 ESP8266 MQTT 遗嘱应用

星期日, 三月 13, 2022 5:37 下午

已剪辑自: https://www.bilibili.com/read/cv9023580?spm_id_from=333.999.0.0

为了帮助您学习, 我们专门为本教程制作了视频。



在这节课里, 我们一起来学习如何使用ESP8266来实现MQTT遗嘱应用。我们将向您介绍两段示例程序。以下是这两段实例的简要介绍:

示例1: MQTT遗嘱基本应用

示例2: 利用MQTT遗嘱实现设备在线状态发布

在开始本节课以前请确保您的电脑已经成功安装了MQTTfx软件。另外我们还将使用PubSubClient库。您可以通过以下链接获取此库。

官网地址: <https://pubsubclient.knolleary.net/>

GitHub: <https://github.com/knolleary/pubsubclient/>

百度网盘下载: <https://pan.baidu.com/s/12MHGbdfiOdwOGip5RMSSEQ> 提取码: sizz

当您将PubSubClient库安装后, 我们就可以使用该库进行开发了。



示例1: MQTT遗嘱基本应用

本示例程序将实现ESP8266的最基本MQTT遗嘱应用。程序使用connect函数对遗嘱消息实现设置。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : last_will_no_retain

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20201223

程序目的/Purpose :

本程序旨在演示如何设置客户端遗嘱机制。客户端在连接服务器时，设置遗嘱的主题和信息。

MQTT服务器会定时检查客户端是否仍然与服务器连接。为了实现这一检查，服务器将会根据以下

内容进行检查。

1. 客户端连接时会提供心跳时间间隔 (Keep Alive)。
2. 如果在心跳时间间隔时长内，客户端向服务器发布了消息，则服务器会认为客户端与服务器保持连接无误。
3. 如果在心跳时间间隔时长内，客户端没有向服务端发布消息，客户端会向服务端发送pingreq信息。此信息由PubSubClient库自动发送。
4. 我们可以通过setKeepAlive函数控制心跳时间间隔时长，或者可以通过PubSubClient.h的#define MQTT_KEEPALIVE 15来设置心跳时间间隔时长。
4. 在心跳时间间隔的1.5倍时长内，如果服务端没有收到客户端信息也没有pingreq。那么服务端将会执行客户端遗嘱机制。

默认情况下，设备的心跳时间间隔时长为15秒。这是在PubSubClient.h中通过以下语句定义的：

```
#define MQTT_KEEPALIVE 15
```

若要修改keep-alive时间，可修改以上头文件，或者使用setKeepAlive函数实现。

对于PubSubClient, 遗嘱QoS允许使用0, 1。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichi-maker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

```
// 如以上MQTT服务器无法正常连接, 请前往以下页面寻找解决方案
```

```
// http://www.taichi-maker.com/public-mqtt-broker/
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
// 遗嘱设置
```

```
const char* willMsg = "CLIENT-OFFLINE"; // 遗嘱消息内容
```

```
const int willQoS = 0; // 遗嘱QoS
```

```
const bool willRetain = false; // 遗嘱保留
```

```
void setup() {
```

```
    Serial.begin(9600); // 启动串口通讯
```

```
//设置ESP8266工作模式为无线终端模式
```

```
WiFi.mode(WIFI_STA);
```

```
// 连接WiFi
```

```

connectWifi();

// 设置MQTT服务器和端口号
mqttClient.setServer(mqttServer, 1883);
mqttClient.setKeepAlive(10); // 设置心跳间隔时间

// 连接MQTT服务器
connectMQTTserver();
}

void loop() {
    // 如果开发板未能成功连接服务器，则尝试连接服务器
    if (!mqttClient.connected()) {
        connectMQTTserver();
    }

    // 处理信息以及心跳
    mqttClient.loop();
}

// 连接MQTT服务器并订阅信息
void connectMQTTserver() {

    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 建立遗嘱主题。主题名称以Taichi-Maker-为前缀，后面添加设备的MAC地址，最后

```

```

// 以“-Will”结尾，这是为确保不同ESP8266客户端的遗嘱主题名称各不相同。

String willString = "Taichi-Maker-" + WiFi.macAddress() + "-Will";

char willTopic[willString.length() + 1];

strcpy(willTopic, willString.c_str());


// 连接MQTT服务器，在连接过程中提供以下参数：

// 客户端ID，遗嘱主题，遗嘱QoS，遗嘱保留，遗嘱信息

if (mqttClient.connect(clientId.c_str(), willTopic, willQoS, willRetain,
willMsg)){

    Serial.println("MQTT Server Connected.");

    Serial.print("Server Address: ");Serial.println(mqttServer);

    Serial.print("ClientId: ");Serial.println(clientId);

    Serial.print("Will Topic: ");Serial.println(willTopic);

} else {

    Serial.print("MQTT Server Connect Failed. Client State:");

    Serial.println(mqttClient.state());

    delay(5000);

}

}

// ESP8266连接wifi

void connectWifi(){

    WiFi.begin(ssid, password);

    //等待WiFi连接,成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

}

```

```

}

Serial.println("");

Serial.println("WiFi Connected!");

Serial.println("");

}

```



示例2：利用MQTT遗嘱实现设备在线状态发布

本实例将实现本教程2-6 MQTT遗嘱这节课中的“MQTT遗嘱使用建议”示例。通过以下程序，ESP8266客户端可以利用遗嘱机制来实时的将当前在线与否状态通过服务端进行发布。也就是说，其它客户端只要订阅ESP8266客户端的遗嘱主题就可以马上了解该客户端是否在线。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : last_will_retain

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20201223

程序目的/Purpose :

本程序旨在演示如何设置客户端遗嘱机制。客户端在连接服务器时，设置遗嘱的主题和信息。

本客户端所发布的遗嘱消息为保留消息。其它客户端可通过订阅本客户端的遗嘱主题获取本

客户端是否在线的状态信息。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266->

[nodemcu-web-client/http-request/](#)

```
*****/

#include <ESP8266WiFi.h>

#include <PubSubClient.h>

// 设置wifi接入信息(请根据您的WiFi信息进行修改)

const char* ssid = "taichimaker";

const char* password = "12345678";

const char* mqttServer = "test.ranye-iot.net";

// 如以上MQTT服务器无法正常连接, 请前往以下页面寻找解决方案

// http://www.taichi-maker.com/public-mqtt-broker/

WiFiClient wifiClient;

PubSubClient mqttClient(wifiClient);

// 遗嘱设置

const char* willMsg = "CLIENT-OFFLINE"; // 遗嘱消息内容

const int willQoS = 0; // 遗嘱QoS

const bool willRetain = true; // 遗嘱保留

void setup() {

    Serial.begin(9600); // 启动串口通讯

    //设置ESP8266工作模式为无线终端模式

    WiFi.mode(WIFI_STA);

    // 连接WiFi

    connectWifi();
```

```

// 设置MQTT服务器和端口号

mqttClient.setServer(mqttServer, 1883);

mqttClient.setKeepAlive(10); // 设置心跳间隔时间


// 连接MQTT服务器

connectMQTTserver();

}


void loop() {

    // 如果开发板未能成功连接服务器，则尝试连接服务器

    if (!mqttClient.connected()) {

        connectMQTTserver();

    }


    // 处理信息以及心跳

    mqttClient.loop();

}


// 连接MQTT服务器并订阅信息

void connectMQTTserver() {

    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）

    String clientId = "esp8266-" + WiFi.macAddress();

    // 建立遗嘱主题。主题名称以Taichi-Maker-为前缀，后面添加设备的MAC地址，最后

    // 以“-Will”结尾，这是为确保不同ESP8266客户端的遗嘱主题名称各不相同。

```



```

String willString = "Taichi-Maker-" + WiFi.macAddress() + "-Will";

char willTopic[willString.length() + 1];

strcpy(willTopic, willString.c_str());


// 连接MQTT服务器，在连接过程中提供以下参数：

// 客户端ID，遗嘱主题，遗嘱QoS，遗嘱保留，遗嘱信息

if (mqttClient.connect(clientId.c_str(), willTopic, willQoS, willRetain,
willMsg)){

    Serial.println("MQTT Server Connected.");

    Serial.print("Server Address: ");Serial.println(mqttServer);

    Serial.print("ClientId: ");Serial.println(clientId);

    Serial.print("Will Topic: ");Serial.println(willTopic);

    publishOnlineStatus();    //发布在线状态

} else {

    Serial.print("MQTT Server Connect Failed. Client State:");

    Serial.println(mqttClient.state());

    delay(5000);

}

}

// 发布信息

void publishOnlineStatus() {

    // 建立遗嘱主题。主题名称以Taichi-Maker-为前缀，后面添加设备的MAC地址，最后

    // 以“-Will”结尾，这是为确保不同ESP8266客户端的遗嘱主题名称各不相同。

    String willString = "Taichi-Maker-" + WiFi.macAddress() + "-Will";

    char willTopic[willString.length() + 1];

    strcpy(willTopic, willString.c_str());

    // 建立设备在线的消息。此信息将以保留形式向遗嘱主题发布

```

```

String onlineMessageString = "CLIENT-ONLINE";

char onlineMsg[onlineMessageString.length() + 1];

strcpy(onlineMsg, onlineMessageString.c_str());


// 向遗嘱主题发布设备在线消息

if(mqttClient.publish(willTopic, onlineMsg, true)){

    Serial.print("Published Online Message:
");Serial.println(onlineMsg);

} else {

    Serial.println("Online Message Publish Failed.");

}

}

// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接,成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

        Serial.print(".");

    }

    Serial.println("");

    Serial.println("WiFi Connected!");

    Serial.println("");

}

```



假如您也喜欢太极创客，希望您将本文“[分享](#)”给更多的人，让更多朋友认识我们。

您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 (<https://space.bilibili.com/415252426/>)

2-8 MQTT用户密码认证

星期日, 三月 13, 2022 5:37 下午

已剪辑自: https://www.bilibili.com/read/cv9069916?spm_id_from=333.999.0.0

为了便于您学习本教程, 我们制作了以下视频。

在之前的课程里我曾给大家讲解了MQTT客户端连接服务端的基本过程。客户端通过CONNECT报文, 向服务端发起连接请求。CONNECT报文所包含的具体信息内容如下:

GIF

MQTT-Packet:	
CONNECT	
名称	内容
clientId	"client-1"
cleanSession	true
username (可选)	"hans"
password (可选)	"letmein"
lastWillTopic (可选)	"/hans/will"
lastWillQos (可选)	2
lastWillMessage (可选)	"unexpected exit"
lastWillRetain (可选)	false
keepAlive	60

MQTT CONNECT报文

到目前为止, 我们已经将CONNECT报文中的信息大部分讲解完毕了, 目前只剩下上图中红色方框所标注的username(用户名)和password(密码)。这里的用户名和密码是用于客户端连接服务端时进行认证需要的。

有些MQTT服务端需要客户端在连接时提供用户名和密码。只有客户端正确提供了用户名和密码后, 才能连接服务端。否则服务端将会拒绝客户端连接, 那么客户端也就无法发布和订阅消息了。

请注意, username(用户名)和password(密码)是可选的CONNECT信息。也就是说, 有些服务端开启了客户端用户密码认证, 这种服务端需要客户端在连接时正确提供认证信息才能连接。当然, 那些没有开启用户密码认证的服务端无需客户端提供用户名和密码认证信息。

在以上示例中, 我们看到此CONNECT报文中所包含的用户名是“hans”, 密码是“letmein”。

用户名和密码除了有以上功能外, 有些公用MQTT服务端也利用此信息来识别客户端属于哪一个用户, 从而对客户端进行管理。比如用户可以拥有私人主题, 这些主题只有该用户可以发布和订阅。对于私人主题, 服务端就可以利用客户端连接时的用户名和密码来判断该客户端是否有发布订阅该用户私人主题的权限。



以下示例程序实现了ESP8266连接MQTT服务端时的用户名和密码认证。

程序中的78行调用connect函数时，提供了MQTT服务端连接的用户名和密码信息。这些信息是在程序的26-28定义的。

关于本程序的详细介绍，请看本文首部的讲解。

/*****

项目名称/Project : 零基础入门学用物联网

程序名称/Program name : pub_password_auth_ranye

团队/Team : 太极创客团队 / Taichi-Maker (www.taichi-maker.com)

作者/Author : CYN0朔

日期/Date (YYYYMMDD) : 20201227

程序目的/Purpose :

本程序旨在演示如何使用PubSubClient库使用ESP8266向MQTT服务器发布信息。

发布信息中，服务器通过用户名和密码进行认证。

本示例程序为太极创客团队制作的《零基础入门学用物联网》中示例程序。

该教程为对物联网开发感兴趣的朋友所设计和制作。如需了解更多该教程的信息，请参考以下网页：

<http://www.taichi-maker.com/homepage/esp8266-nodemcu-iot/iot-c/esp8266-nodemcu-web-client/http-request/>

*****/

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
#include <Ticker.h>
```

```
// 设置wifi接入信息(请根据您的WiFi信息进行修改)
```

```
const char* ssid = "taichimaker";
```

```
const char* password = "12345678";
```

```
const char* mqttServer = "test.ranye-iot.net";
```

// 如以上MQTT服务器无法正常连接，请前往以下页面寻找解决方案

// <http://www.taichi-maker.com/public-mqtt-broker/>

// MQTT服务端连接用户名密码

```
const char* mqttUserName = "test-user";
```

```
const char* mqttPassword = "range-iot";
```

```
Ticker ticker;
```

```
WiFiClient wifiClient;
```

```
PubSubClient mqttClient(wifiClient);
```

```
int count;    // Ticker计数用变量
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    // Ticker定时对象
```

```
    ticker.attach(1, tickerCount);
```

```
    //设置ESP8266工作模式为无线终端模式
```

```
    WiFi.mode(WIFI_STA);
```

```
    // 连接WiFi
```

```
    connectWifi();
```

```
    // 设置MQTT服务器和端口号
```

```
    mqttClient.setServer(mqttServer, 1883);
```

```
    // 连接MQTT服务器
```

```
    connectMQTTServer();
```

```
}
```

```
void loop() {
```

```
    // 如果开发板未能成功连接服务器，则尝试连接服务器
```

```
    if (!mqttClient.connected()) {
```



```

        connectMQTTServer();
    }

    mqttClient.loop();
    // 每隔3秒钟发布一次信息
    if (count >= 3) {
        pubMQTTmsg();
        count = 0;
    }
}

void tickerCount() {
    count++;
}

void connectMQTTServer() {
    // 根据ESP8266的MAC地址生成客户端ID（避免与其它ESP8266的客户端ID重名）
    String clientId = "esp8266-" + WiFi.macAddress();

    // 连接MQTT服务器。此处使用了程序首部定义的用户名和密码来实现MQTT服务端认证
    if (mqttClient.connect(clientId.c_str(), mqttUserName, mqttPassword)) {
        Serial.println("MQTT Server Connected.");
        Serial.print("Server Address: ");
        Serial.println(mqttServer);
        Serial.print("ClientId: ");
        Serial.println(clientId);
    } else {
        Serial.print("MQTT Server Connect Failed. Client State:");
        Serial.println(mqttClient.state());
        delay(3000);
    }
}

```

```

    }

}

// 发布信息

void pubMQTTmsg() {

    static int value;

    // 建立发布主题。主题名称以taichi/Pub-为前缀，后面添加设备的MAC地址。

    // 这么做是为确保不同用户进行MQTT信息发布时，ESP8266客户端名称各不相同，

    String topicString = "test-user/Pub-" + WiFi.macAddress();

    char publishTopic[topicString.length() + 1];

    strcpy(publishTopic, topicString.c_str());

    // 建立发布信息。信息内容以Hello World为起始，后面添加发布次数。

    String messageString = "Hello World " + String(value++);

    char publishMsg[messageString.length() + 1];

    strcpy(publishMsg, messageString.c_str());

    // 实现ESP8266向主题发布信息

    if(mqttClient.publish(publishTopic, publishMsg)){

        Serial.println("Publish Topic:");Serial.println(publishTopic);

        Serial.println("Publish message:");Serial.println(publishMsg);

    } else {

        Serial.println("Message Publish Failed.");

    }

}

// ESP8266连接wifi

void connectWifi() {

    WiFi.begin(ssid, password);

    //等待WiFi连接, 成功连接后输出成功信息

    while (WiFi.status() != WL_CONNECTED) {

        delay(1000);

```

```
        Serial.print(".");  
    }  
  
    Serial.println("");  
  
    Serial.println("WiFi Connected!");  
  
    Serial.println("");  
}
```



假如您也喜欢太极创客，希望您将本文“**分享**”给更多的人，让更多朋友认识我们。
您的每一份**点赞与投币**，都是对我们最温暖的鼓励与支持。

本文编辑制作：CYN0朔 （<https://space.bilibili.com/415252426/>）